

The Backend Engineer's Lab Notebook

A guided-practice companion to the
Java & Spring Boot — 14-Day Speedrun Roadmap

From *reading* concepts to truly *understanding* them — then building, debugging, securing, testing, containerizing, and shipping a production Spring Boot backend, one guided day at a time.

14

DAYS

42

GUIDED LABS

110+

EXERCISES

1

CAPSTONE

This notebook belongs to: _____

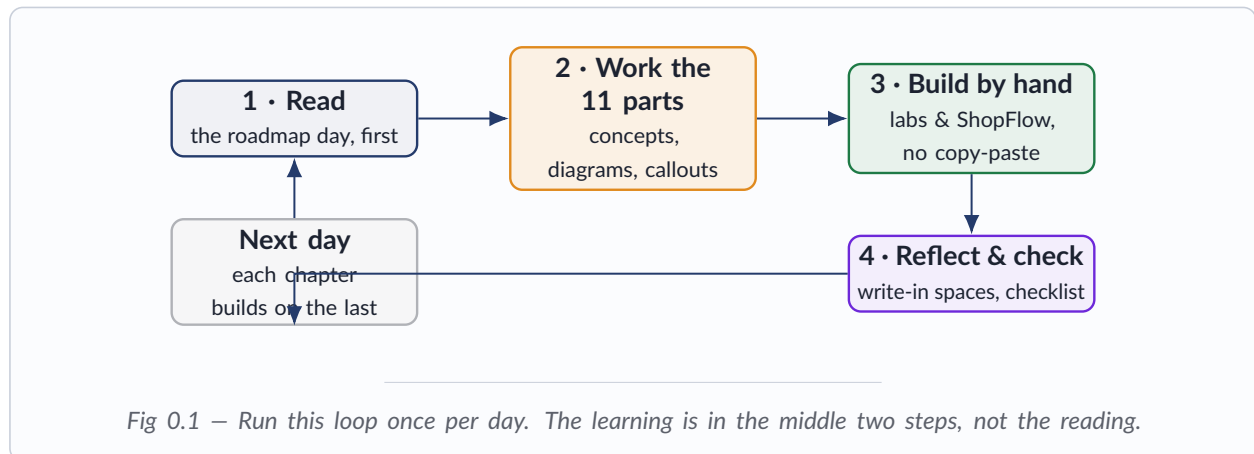
Started on: _____ Target finish: _____



How to Use This Workbook

This is not an exam book and not a second textbook. It is a **lab manual** — a place to *do the work* beside the roadmap. The roadmap tells you *what* to learn; this workbook walks you through *understanding* it, *building* it, and *reflecting* on it, so the knowledge sticks. Each chapter maps one-to-one to a day of the roadmap and always follows the same eleven-part rhythm, so you always know where you are.

The daily study loop



The eleven parts of every chapter

- | | | |
|----|----------------------------|--|
| 1 | The Big Picture | Why this exists, where it fits, how it connects to earlier days. |
| 2 | Concept Map | One picture of the whole territory and its vocabulary. |
| 3 | Guided Learning | Each idea taught progressively: what / why / how-internally / when / when-not / production / mistakes. |
| 4 | Visualizations | Redraw the key diagrams from memory to lock them in. |
| 5 | Guided Coding Lab | Build a feature step by step — the workbook guides, you write the code. |
| 6 | Thinking Exercises | Reflection prompts (not tests) that make your reasoning explicit. |
| 7 | Debugging Practice | A broken scenario worked as Observe → Hypothesise → Verify → Fix → Prevent. |
| 8 | Mini-Labs | Three graded labs: Beginner, Intermediate, Production-style. |
| 9 | Engineering Notes | Senior judgment: how experienced engineers actually think. |
| 10 | Build-Along Project | Extend ShopFlow , one production backend built across all 14 days. |
| 11 | Chapter Summary | Takeaways, pitfalls, a one-page cheat sheet, and what comes next. |

Production tip

The reflection spaces and "redraw from memory" boxes are meant to be *written in* by hand. Print the pages you are working, or annotate the PDF. Reproducing a diagram yourself is what moves it from "I recognise that" to "I can explain that in an interview."

Security warning • on copy-paste

This workbook deliberately does *not* hand you finished solutions. The labs guide you to build them yourself. If you paste code you did not type and reason through, you will feel productive and learn nothing. The struggle in Parts 5, 7 and 10 is the point.

Field Guide to the Margin Notes

Throughout the workbook, coloured callouts flag different *kinds* of thinking. Learn to read them at a glance.

How senior engineers think

How experienced engineers reason about a decision — the judgment behind the code.

Production tip

A concrete practice that holds up in real production systems.

Performance note

A latency, memory, or throughput consideration worth measuring.

Security warning

A security or correctness trap — read these twice.

Database insight

Something the database is doing underneath your ORM.

Interview insight

A question you will likely be asked, and how to answer it well.

Common misconception

A belief many beginners hold that is subtly wrong.

Behind the scenes

What the framework or runtime is actually doing behind the scenes.

And the working blocks you will act on:

Mini-Lab • Beginner / Intermediate / Production

graded practice

Self-contained labs with a Goal, Context, Requirements, Hints, Expected outcome, and Common pitfalls.

Debugging Lab • A broken app to diagnose

Symptom and evidence, then a guided Observe → Hypothesise → Verify → Fix → Prevent walkthrough.

Think it through

A reflection prompt with ruled space to write your answer by hand.

 Build-Along • ShopFlow

The one production backend you extend a little further every single day.

 The 14-Day Arc at a Glance

Each day earns its place and feeds the next. Read down the "ShopFlow milestone" column to see one real backend take shape from an empty project to a production-ready service.

Day	Topic	ShopFlow milestone you build
1	Spring Boot Internals	Base module: layered packages, typed config, profiles, <code>/ping</code> .
2	Database Mastery	Schema + constraints + composite index + migration.
3	JPA & Hibernate	Order/OrderItem/Product entities, locking, one-query listing.
4	REST API Design	Order API with DTOs, validation, errors, pagination, OpenAPI.
5	Spring Security	JWT login/refresh, BCrypt, roles, locked-down CORS.
6	Testing	Unit + Testcontainers + MockMvc coverage, incl. auth failures.
7	Clean Architecture	Refactor: validator, domain event, async email listener.
8	Docker	Multi-stage image + Compose (app + Postgres + Redis).
9	Redis & Caching	Cached catalog reads, evict-on-write, TTL.
10	Messaging (MQ)	OrderPlaced to RabbitMQ, retries, dead letter queue.
11	JVM & Concurrency	Bounded thread pool, Actuator, custom metrics.
12	CI/CD	GitHub Actions: test, build, tag by SHA, push image.
13	Production Readiness	Circuit breaker, rate limiting, liveness/readiness, Grafana.
14	Capstone	Assemble, verify, and present the whole ShopFlow backend.

 Key idea

ShopFlow is deliberately a *small* domain — roughly two entities of real complexity — carrying a *large* amount of engineering weight. That is exactly the shape of a strong take-home assignment or a well-scoped first project at a new job. Depth over breadth, every day.



Contents

Day 01 — Spring Boot Internals	6
Day 02 — Database Mastery	19
Day 03 — JPA & Hibernate Deep Dive	36
Day 04 — REST API Design	53
Day 05 — Spring Security	71
Day 06 — Testing	89
Day 07 — Clean Architecture, SOLID & Design Patterns	106
Day 08 — Docker	123
Day 09 — Redis & Caching	139
Day 10 — Messaging & Event-Driven Architecture	156
Day 11 — JVM Performance, Concurrency & Observability	173
Day 12 — CI/CD	189
Day 13 — Production Readiness	206
Day 14 — Final Project — ShopFlow	226
Appendix A — Backend Engineer Interview Cheat Sheet	242
Appendix B — The ShopFlow Build-Along Master Tracker	245
Appendix C — Glossary of Key Terms	247

Fourteen chapters • one growing project • every concept practised, not just read.

DAY 01 • FOUNDATIONS

Spring Boot Internals

How the framework actually works behind the annotations

01

🕒 Estimated study time: 8–9 hours

🎯 By the end of this day, you will be able to...

- Explain what the IoC container actually *does* at runtime — not just what `@Autowired` looks like.
- Trace the full bean lifecycle from classpath scan to destruction, and say *where* proxies are attached.
- Explain how auto-configuration decides what to wire up, and how to override it deliberately.
- Follow a single HTTP request through the Spring MVC stack, filter by filter.
- Structure configuration the way real teams do: typed properties and per-environment profiles.

1

The Big Picture

What this day solves. You already know how to sprinkle `@Service` and `@Autowired` until the app boots. This day answers the question that separates a junior from a mid-level engineer: *what is the framework actually doing between `main()` and your first HTTP response?* When something breaks at startup — a bean won't inject, a transaction silently doesn't roll back, an auto-configuration you didn't ask for changes behaviour — you cannot debug what you cannot picture.

Why backend engineers need it. Spring Boot is the JVM's default backend framework. Every production incident you will ever debug — a connection pool exhausted at 3am, a `@Cacheable` that never caches, a slow startup — is ultimately a story about beans, proxies, the request pipeline, or configuration. The mental model you build today is the substrate for all thirteen days that follow.

🔑 Key idea

Spring Boot is not magic; it is **aggressive convention plus reflection plus proxies**. Once you can name the three moving parts — the **IoC container** that builds objects, the **proxies** that wrap them, and the **servlet pipeline** that routes requests to them — the framework stops being a black box and becomes a tool you drive.

Where it fits in a Spring Boot application. Everything else in this workbook lives *inside* the container you learn about today. The persistence layer (Days 2–3) is a set of beans. Security (Day 5) is a chain of filters registered before your controllers. Caching (Day 9), async (Day 11) and resilience (Day 13) are all delivered by proxies — the exact mechanism you will study in Part 3.

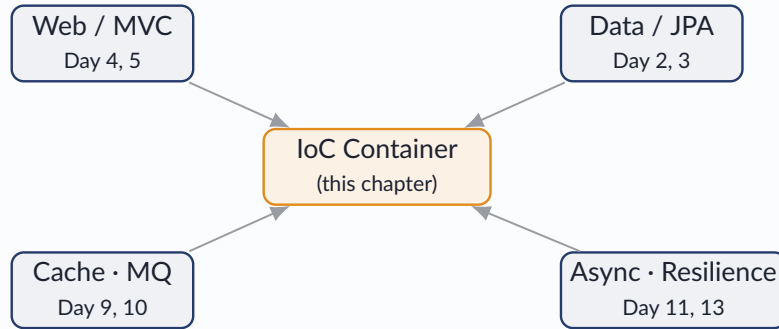


Fig 1.0 – Today you learn the container in the centre; every later day plugs a component into it.

Real company examples. Netflix built its early microservice platform (Eureka, Ribbon, Hystrix) on Spring; those ideas became Spring Cloud. Uber, Alibaba and thousands of banks run Spring Boot services at scale precisely because the container model makes wiring, testing and swapping components uniform across hundreds of teams.

2 Concept Map

Before the details, hold the whole territory in one picture. Read it left-to-right (startup) and top-to-bottom (a live request).

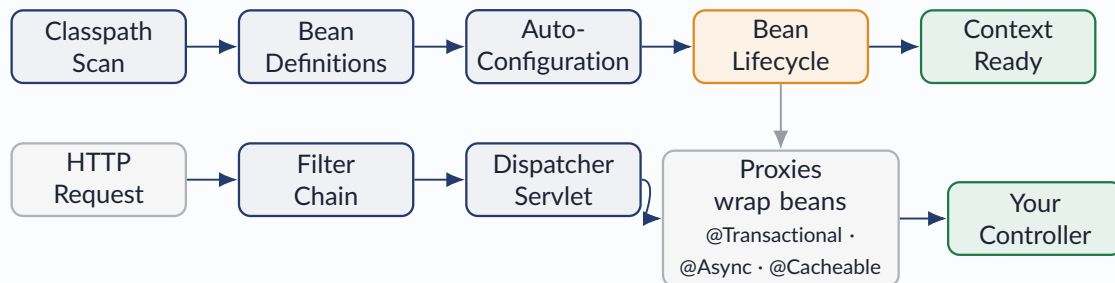


Fig 1.1 – The Day 1 concept map: startup builds the container; a request flows through the pipeline into your beans.

Terminology to anchor: **IoC** → **ApplicationContext** → **BeanDefinition** → **Bean** → **BeanPostProcessor** → **Proxy** → **DispatcherServlet** → **Controller**.

3 Guided Learning

3.1 Dependency Injection & the IoC Container

What is it? **Dependency Injection (DI)** means a class never constructs its own collaborators — they are handed to it. **Inversion of Control (IoC)** is the broader principle: your code stops controlling object creation and wiring, and a container does it instead. Spring's `ApplicationContext` is that container.

Why does it exist? Hand-wiring object graphs is repetitive, couples classes to concrete implementations, and makes testing painful — you cannot substitute a fake collaborator if the class news it up internally. DI turns wiring into a declarative concern the framework resolves once, centrally.

How does it work internally? At startup the context builds an in-memory registry (the `BeanFactory`) where every managed object is first described by a **BeanDefinition**: its class, scope, constructor arguments, dependencies and lifecycle callbacks. Crucially, *definitions are collected before any object is instantiated* — Spring resolves the dependency graph and a safe creation order first.

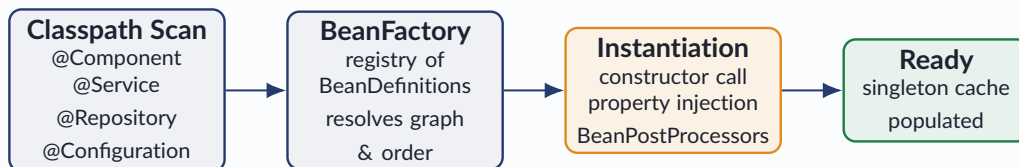


Fig 1.2 — Before your first bean exists: definitions are gathered, the graph is resolved, then objects are created.

When should I use it? Always, for collaborators between your own components (services, repositories, clients). This is the grain of the framework.

When should I not? Do not inject value objects, DTOs or entities — those are data you *new* yourself. Do not turn every helper into a bean; a stateless private method does not need to be a managed component.

Production tip

Prefer **constructor injection** over field `@Autowired`. It makes required dependencies explicit, lets fields be `final` (immutable), and — decisively — lets you construct the class in a unit test with plain `new`, no reflection or Spring context required. Every testing chapter later depends on this habit.

Common misconception

`@Autowired` is not "search the whole app for something that fits at call time." Resolution happens *once*, at context startup, against the pre-built definition graph. If it is ambiguous or missing, the application fails fast at boot — which is exactly what you want.

3.2 The Bean Lifecycle (the part most engineers cannot explain)

What is it? For every singleton, Spring runs a fixed, ordered sequence from raw object to ready bean, and a symmetric teardown on shutdown.

How does it work internally? The sequence: **instantiate** → **populate properties** (DI) → call **Aware** interfaces (`BeanNameAware`, `ApplicationContextAware`) → `postProcessBeforeInitialization` → `@PostConstruct` / `afterPropertiesSet()` → `postProcessAfterInitialization` (*this is where proxies for `@Transactional`, `@Async`, AOP are wrapped around your bean*) → bean is ready and cached → on shutdown, `@PreDestroy` / `destroy()`.

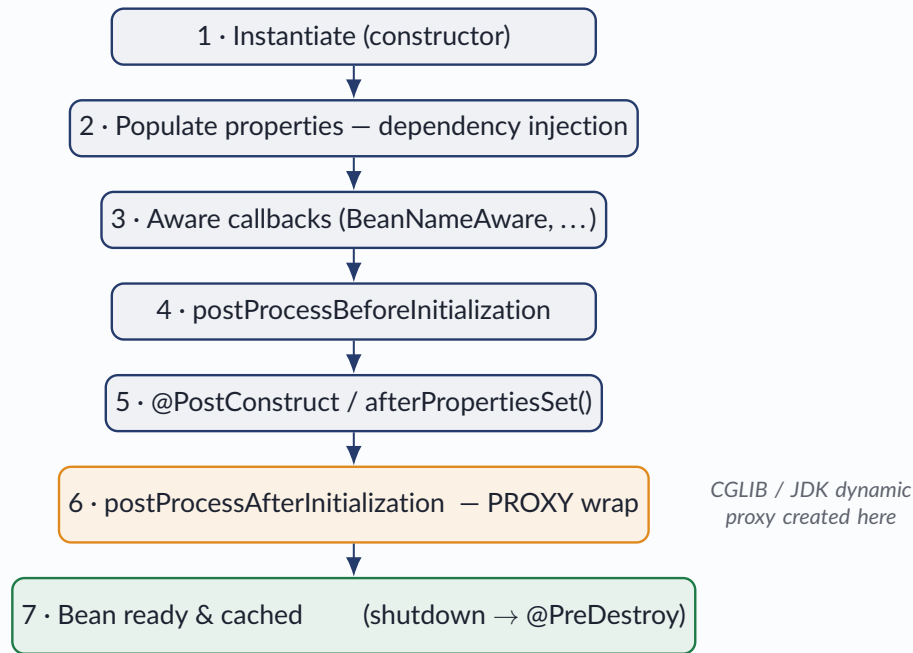


Fig 1.3 — The bean lifecycle. The object your controller calls is often a proxy created at the highlighted step, not your raw class.

👁 Behind the scenes

The object registered in the context is frequently *not* the raw class you wrote — it is a CGLIB subclass or JDK dynamic proxy wrapping it. The proxy intercepts *external* calls to add behaviour (open a transaction, check a cache, hop to another thread) before delegating to your real method.

👤 Security warning • the self-invocation trap

Because proxies only intercept calls arriving *from outside* the bean, a method calling *another method on this* bypasses the proxy entirely. That is precisely why calling a `@Transactional` method from another method in the same class *silently* skips the transaction — the advice never runs. The same breaks `@Async` and `@Cacheable`. Fix: move the annotated method to a separate bean, or inject a self-reference proxy.

3.3 Component Scanning vs. Auto-Configuration

What is it? Two different mechanisms that both create beans, constantly confused. **Component scanning** walks your own packages from `@SpringBootApplication` downward, registering anything annotated `@Component` (and `@Service`/`@Repository`/`@Controller`). **Auto-configuration** lives in `spring-boot-autoconfigure` and reacts to what is on the classpath.

How does it work internally? Auto-configuration classes are guarded by conditional annotations: `@ConditionalOnClass` (is this type on the classpath?), `@ConditionalOnMissingBean` (did you not already define your own?), `@ConditionalOnProperty`. So: "if a `DataSource` type is present and you have not declared one, create a `DataSource` from `application.yml`." This is why adding `spring-boot-starter-data-redis` can silently activate new beans — auto-config reacts to the classpath, not only to what you explicitly asked for.

💡 How senior engineers think

When a bean appears (or refuses to appear) that you did not define, a senior does not guess. They run the app with `-debug` and read the **Auto-configuration Report**: a printed list of every positive and negative condition match. "Fighting the framework" is almost always "not having read which condition fired."

3.4 Configuration Properties & Profiles

What is it? `@ConfigurationProperties` binds a group of related settings to a typed object; **profiles** let the same jar behave differently per environment.

MailProperties.java

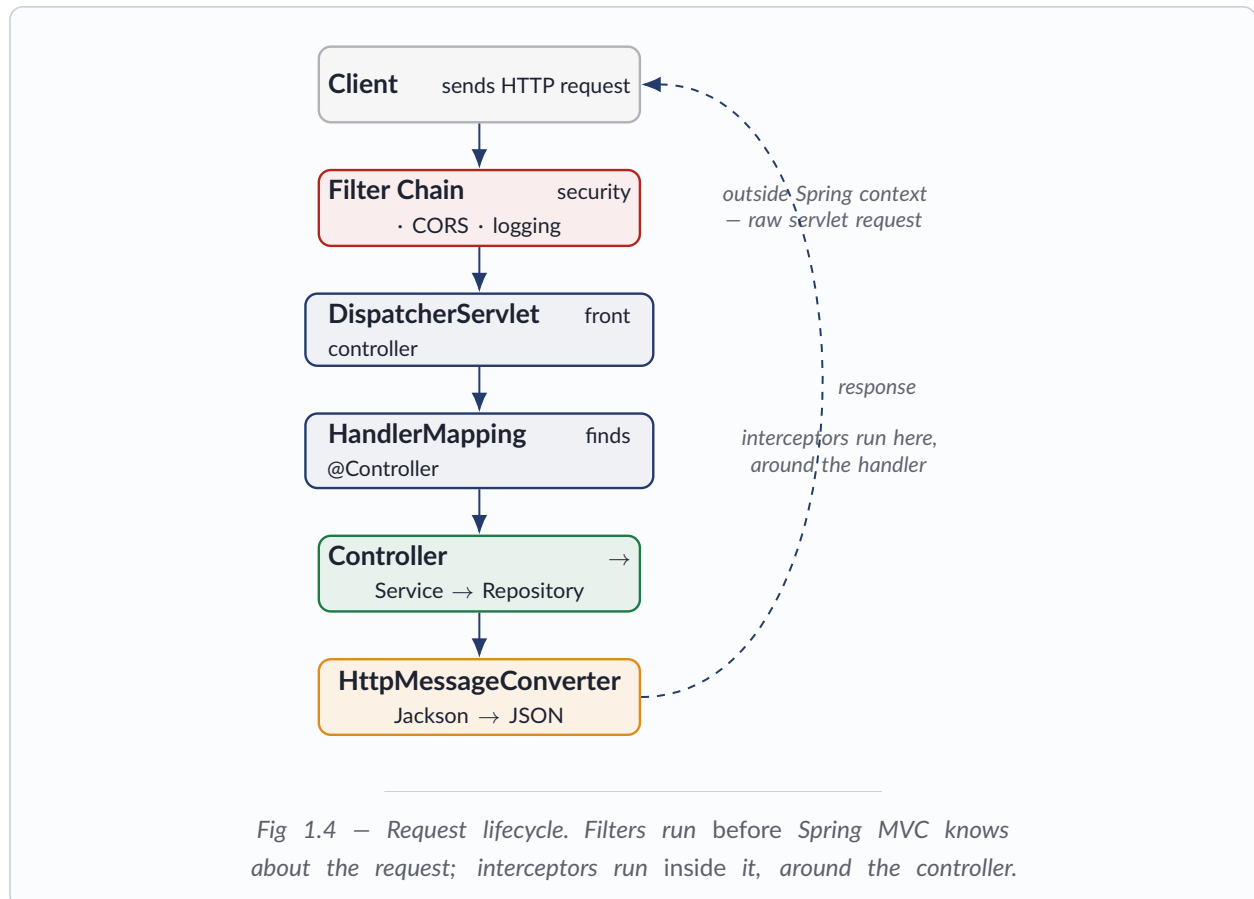
```
@ConfigurationProperties(prefix = "app.mail")
@Validated
public record MailProperties(
    @NotBlank String host,
    @Min(1) int port,
    boolean tlsEnabled) {}
```

application-prod.yml

```
app:
  mail:
    host: smtp.prod.internal
    port: 587
    tls-enabled: true
spring:
  profiles:
    active: prod # in real prod, set via env var, never hardcoded
```

Why prefer it over `@value`? Typed properties are validatable (`@Validated` fails startup on bad config — a good thing), testable in isolation, and discoverable in one class instead of scattered across the codebase as string keys.

3.5 How a Request Actually Travels Through Spring MVC



The distinction interviewers probe. **Filters** are Servlet-spec, run outside the Spring context, and see the raw `HttpServletRequest` — used for security and CORS. **Interceptors** are Spring-specific, run inside dispatch, and can see the resolved handler method. **HandlerMapping** chooses the controller; an **HttpMessageConverter** (Jackson by default) serialises the return value into the response body.

✓ Best practices — what experienced engineers do

- Constructor injection only; never field `@Autowired`. Explicit, immutable, testable.
- Keep configuration in typed `@ConfigurationProperties` classes, not scattered `@Value`.
- One `@SpringBootApplication` per module; keep component-scan scope tight in large monorepos.
- Never put logic that depends on *other* beans being fully initialised inside `@PostConstruct` — use `ApplicationRunner` instead.

✗ Common mistakes beginners make

- Calling a `@Transactional` method from inside the same class and being surprised nothing rolls back.
- Field injection everywhere, making unit construction impossible without reflection.
- Not realising beans are singletons by default, causing shared-mutable-state bugs.
- Fighting auto-configuration instead of reading which condition activated it (`-debug`).

4

Visualizations to Internalize

You have already met four diagrams; redraw two of them *from memory* in the space below — the act of reproducing them is what moves them from "recognised" to "known."

 Redraw: the bean lifecycle (7 steps) and where the proxy is attached

 Redraw: a request from Filter Chain to JSON response

5

Guided Coding Lab — Scaffold the Container You Can See

Goal of this lab: build a tiny Spring Boot module whose startup is fully observable, so the abstract lifecycle becomes something you literally watch in the console. This becomes the seed of the ShopFlow capstone.

1 Create the project with layered packages

Generate a Spring Boot 3 / Java 21 project (`start.spring.io`) with *Web* and *Validation*. Create packages `controller`, `service`, `repository`, `config`, `dto`.  **Why:** A predictable package layout is the cheapest architecture decision you will ever make — every later day drops files into these folders.

2 Write one constructor-injected service

Add a trivial `GreetingService` and a `GreetingController` that depends on it *via its constructor*. No `@Autowired` field anywhere.  **Why:** You are hard-wiring the good habit from Part 3 into muscle memory before the codebase grows.

3 Add a typed `@ConfigurationProperties` class

Bind `app.greeting.prefix` and `app.greeting.maxLength` into a validated record. Annotate the app with `@ConfigurationPropertiesScan`.  **Why:** Type-safe config that fails fast on bad input is a production reflex; prove it by starting with an invalid value and watching startup refuse.

4 Register a `BeanPostProcessor` that logs every bean

Implement `postProcessBeforeInitialization` to record `System.nanoTime()` and `postProcessAfterInitialization` to log `beanName + elapsed millis`. 💡 *Why*: This turns Fig 1.3 into observable reality — you will see the order and spot which beans are slow to initialise.

5 Add `dev` and `prod` profiles

Create `application-dev.yml` and `application-prod.yml` with a different greeting prefix; switch with `-spring.profiles.active=prod`. 💡 *Why*: "Same jar, different environment" is the principle Docker (Day 8) and CI/CD (Day 12) are built on.

6 Run with `-debug` and read the report

Find three auto-configurations that activated and name the condition that fired each. 💡 *Why*: You are practising the senior reflex: understand *why* a bean exists, don't just accept it.

🔧 Production tip

Do not paste a finished `BeanPostProcessor` from the internet. Type it, run it, and *read your own log output*. The learning is in seeing `jwtAuthFilter` initialise after `dataSource` — an ordering you will care about on Day 5.

6 Thinking Exercises

These are reflection prompts, not quiz questions. Write a sentence or two — the goal is to make your reasoning explicit.

🧠 Think it through

Why do you think Spring makes beans *singleton* by default rather than creating a new instance per injection point? What class of bug does that default invite, and why is it still the right default?

🧠 Think it through

If proxies are what make `@Transactional` work, predict what happens to a `private @Transactional` method. Why can a proxy not intercept it?

Think it through

Auto-configuration reacts to the classpath. What is the risk of that convenience in a large project with many starters, and how would you *prove* which beans a new dependency introduced?

7 Debugging Practice

🔧 Debugging Lab • The rollback that never happens

Symptom. `placeOrder()` throws halfway through, but the half-written row is still in the database after the request fails.

Evidence.

```
OrderService.java

@Service
public class OrderService {
    public void placeOrder(OrderRequest req) { // no annotation here
        validate(req);
        persistOrder(req); // internal call
    }
    @Transactional
    void persistOrder(OrderRequest req) { /* two writes, second may throw */ }
}
```

Work the method:

- ✓ **Observe.** The transaction advice never appears in the SQL log (no `BEGIN`).
- ✓ **Hypothesise.** `persistOrder` is called on `this`, so the proxy is bypassed and `@Transactional` does nothing.
- ✓ **Verify.** Move `@Transactional` to the public `placeOrder` instead, restart, watch for `BEGIN/ROLLBACK` in the log.
- ✓ **Fix.** Annotate the outer entry point, or extract `persistOrder` into its own bean called through the proxy.
- ✓ **Prevent.** Add a test that forces the second write to fail and asserts the row count is unchanged.

🔧 Debugging Lab • A bean you never wrote is answering your requests

Symptom. After adding a starter, an endpoint behaves differently and a `DataSource` exists that you did not declare.

- ✓ **Observe.** Restart with `-debug`; open the *Positive matches* section of the auto-configuration

report.

- ✓ **Hypothesise.** A `@ConditionalOnMissingBean` auto-config created the `DataSource` because you never defined one.
- ✓ **Verify.** Search the report for `DataSourceAutoConfiguration` and read the matched condition.
- ✓ **Fix.** Either accept it, or declare your own `@Bean` to win by `@ConditionalOnMissingBean` precedence.
- ✓ **Prevent.** Treat every new starter as "what beans did this add?" — check the report, don't discover it in production.

8 Mini-Labs

Mini-Lab • Beginner

25 min

Goal. Prove the singleton default with your own eyes.

Context. Two controllers both inject the same `CounterService`.

Requirements. Increment a field from each controller; expose the value.

Hints. Inject the same bean twice; log `System.identityHashCode(service)`.

Expected outcome. Identical hash codes; a single shared counter.

Common pitfalls. Adding `@Scope("prototype")` and then being confused when the count resets — that is the lesson, not a bug.

Mini-Lab • Intermediate

40 min

Goal. Make `@ConfigurationProperties` validation fail the build at startup.

Context. A `RateLimitProperties(int permitsPerSecond)` with `@Min(1)`.

Requirements. Set the value to 0 in `application.yml`; add `@Validated`.

Hints. The exception is a `BindValidationException` wrapped in a startup failure.

Expected outcome. The context refuses to start, with a clear message naming the field.

Common pitfalls. Forgetting `@Validated`, so the constraint is ignored and the app boots with bad config.

Mini-Lab • Production-style

60 min

Goal. Add startup timing observability, the way an ops-minded team would.

Context. You want to know which beans dominate cold-start time (matters for autoscaling and CI).

Requirements. Extend your `BeanPostProcessor` to accumulate per-bean init durations and print the slowest 10 on `ApplicationReadyEvent`.

Hints. Store timings in a `Map`; sort descending in an `@EventListener(ApplicationReadyEvent.class)`.

Expected outcome. A "Top 10 slowest beans" table in the log at boot.

Common pitfalls. Doing the sort in `@PostConstruct` of some bean — it runs before the context is ready and the map is incomplete.

9

Engineering Notes

A curated margin from engineers who have carried a pager. Read these as judgment, not trivia.

 How senior engineers think

Startup order is a feature, not an accident. When you can predict the order in which beans initialise, you can reason about *when* a resource (a pool, a cache, a Kafka consumer) becomes available — and therefore why a health check flickers red for the first two seconds after deploy.

 Interview insight

Expect: "Why does a `@Transactional` self-call not work?" The answer that lands is not "because proxies" — it is "because the advice lives on a proxy that only wraps *external* calls, and a `this.method()` call never touches the proxy." Say the mechanism.

 Performance note

Marking rarely-used beans `@Lazy` shortens cold start, but never make a `@Transactional` or `@Cacheable` bean lazy without thinking — the first call pays both the initialisation *and* the proxy-creation cost, which can surprise a latency-sensitive path.

 Production tip

In production, `spring.profiles.active` is an *environment variable*, injected by your platform (Day 8 Compose, Day 12 CI). Hardcoding it in `application.yml` defeats the entire "one artifact, many environments" model.

10

Build-Along Project — ShopFlow

Across the next fourteen days you build **ShopFlow**, a small but production-shaped order-management backend. Today you lay its foundation.

 Build-Along • Day 1 — the base module

- Scaffold the Spring Boot 3 / Java 21 project with layered packages: `controller`, `service`, `repository`, `config`, `dto`.
- Constructor injection everywhere — set the rule now, enforce it forever.
- One typed `@ConfigurationProperties` class (e.g. `app.shopflow.*`) with validation.
- `dev` and `prod` profiles that differ in at least one visible value.
- A `BeanPostProcessor` that logs per-bean startup timing.
- A single `GET /api/v1/ping` endpoint returning `200` — proof the pipeline is alive end-to-end.

Definition of done: the app starts, `/ping` returns `200`, and your startup log prints the bean-timing summary. This skeleton is the literal seed for Day 14.

11 Chapter Summary

Key takeaways

- ✓ The IoC container builds *definitions* first, then objects, in a resolved order.
- ✓ Your beans are often wrapped by proxies; that is how `@Transactional/@Async/@Cacheable` work.
- ✓ Self-invocation bypasses the proxy — the single most common "why doesn't my annotation work?" bug.
- ✓ Component scanning registers *your* components; auto-configuration reacts to the *classpath* via conditions.
- ✓ Filters run outside Spring; interceptors run inside — different tools for different jobs.

Things to remember

- `-debug` prints the auto-configuration report. Use it instead of guessing.
- Typed `@ConfigurationProperties` + `@Validated` makes bad config a startup failure, not a 2am surprise.

Common pitfalls (last look)

- Field injection → untestable classes. ● Self-call `@Transactional` → silent no-op.
- Hardcoded active profile → broken promotion across environments.

📄 One-Page Cheat Sheet

BeanFactory vs ApplicationContext — Context = BeanFactory + events, AOP, i18n, eager singletons.

Bean lifecycle — instantiate → inject → aware → BPP-before → `@PostConstruct` → BPP-after (proxy) → ready → `@PreDestroy`.

Proxy rule — advice only intercepts external calls; `this.method()` skips it.

Scan vs auto-config — scan = your packages; auto-config = classpath conditions (`@ConditionalOnClass/MissingBean/Property`).

Filter vs Interceptor — filter = servlet, outside Spring, raw request; interceptor = Spring, inside dispatch, sees handler.

Config — constructor injection + `@ConfigurationProperties` + profiles via env var.

👤 Interview questions to master

- › What is the difference between `BeanFactory` and `ApplicationContext`?
- › Walk me through the Spring bean lifecycle.
- › Why does calling a `@Transactional` method from within the same class not work?
- › How does Spring Boot decide which auto-configurations to apply?
- › What is the difference between a Filter and an Interceptor?
- › What are the bean scopes, and when would you use `prototype`?

Suggested further reading

- *Spring Framework Reference* — "The IoC Container" and "Web MVC" chapters.
- *Spring Boot Reference* — "Auto-configuration" and "Externalized Configuration."
- Source dive: `AbstractApplicationContext.refresh()` — the method where the whole lifecycle happens.

Key idea

What you will learn next (Day 2). Every bean you just wired eventually talks to a database. Tomorrow you drop below the ORM to raw SQL: joins, indexes, transactions and isolation — the layer where most real performance incidents are actually born.

DAY 02 • DATA LAYER

Database Mastery

02

SQL, transactions & query performance — the layer where real incidents are born

🕒 Estimated study time: 8–9 hours

🎯 By the end of this day, you will be able to...

- Write joins by thinking in sets, not loops — and spot the `LEFT JOIN` that silently degrades into an `INNER JOIN`.
- Reason about indexes like someone who has debugged a slow production query: what a B-Tree buys you, and the leftmost-prefix rule that decides whether your composite index is even used.
- Choose an isolation level under pressure by naming the exact anomaly (dirty / non-repeatable / phantom) each level prevents — and what it costs.
- Read an `EXPLAIN ANALYZE` plan and locate the bottleneck: the sequential scan where you expected an index.
- Design a normalized schema with real constraints, know when to deliberately denormalize, and paginate large lists without timing out at page 500.

1 The Big Picture

What this day solves. On Day 1 you learned that every component in a Spring Boot app is a bean living inside a container. Today you follow those beans down to where they finally touch reality: the database. This is the layer that separates engineers who *use* a database from engineers who can *explain why a query is slow at 3am*. The framework can hide the SQL behind a repository method, but it cannot hide a full-table scan, a lock held too long, or a schema that makes the right query impossible.

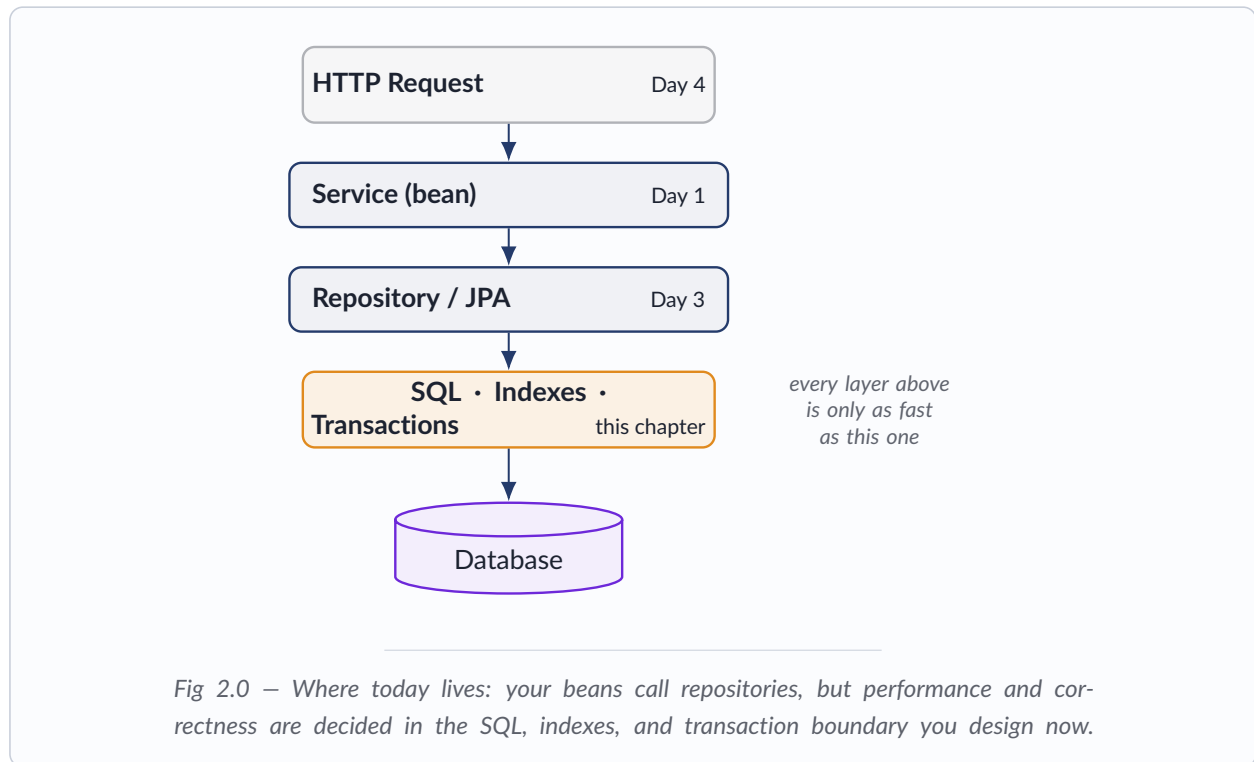
Why backend engineers need it. Most severe production incidents are, at bottom, database incidents: a connection pool exhausted because a transaction stayed open across an HTTP call; a report endpoint that offset-paginates ten million rows and times out; a "unique" email that turns out to be duplicated because the uniqueness was only enforced in Java. You cannot fix what you cannot see, and the database is where the truth of your system's performance and correctness actually lives.

🔑 Key idea

A database is not a bucket you throw objects into — it is a **set-oriented, concurrent, durable** engine with its own cost model. Three levers control almost everything: the **schema** (what shapes are even representable), the **indexes** (what the planner can do quickly), and the **transaction boundary** (what stays consistent under concurrency). Master those three and the rest is detail.

Where it fits in a Spring Boot application. Your `@Repository` beans (Day 3) sit on top of a JDBC `DataSource` that sits on top of raw SQL that sits on top of the storage engine. Every abstraction above

is only as good as your understanding of the layer below it. When JPA generates a surprising query on Day 3, or a cache goes stale on Day 9, the debugging always bottoms out in the SQL and the plan you learn to read today.



How it connects to previous chapters. Day 1's `@Transactional` was introduced as a proxy-delivered concern — today you learn *what the proxy actually opens*: a database transaction with a concrete isolation level and lock footprint. The self-invocation trap you met yesterday matters precisely because the thing being skipped is a `BEGIN/COMMIT` around real writes.

Real company examples. Stripe, GitHub and Twitter all switched their public list APIs to cursor-based pagination for the same reason you will learn today. Postgres powers Instagram and Datadog at enormous scale not because it is magic but because their engineers respect indexes, plans and vacuum. Every "we scaled the database" story is really a story about the four topics in this chapter.

2 Concept Map

Hold the whole territory in one picture before the details. The data layer has four pillars: how you *read* (joins), how you make reads *fast* (indexes & plans), how you keep data *correct under concurrency* (transactions), and how you *shape* it in the first place (schema & constraints).

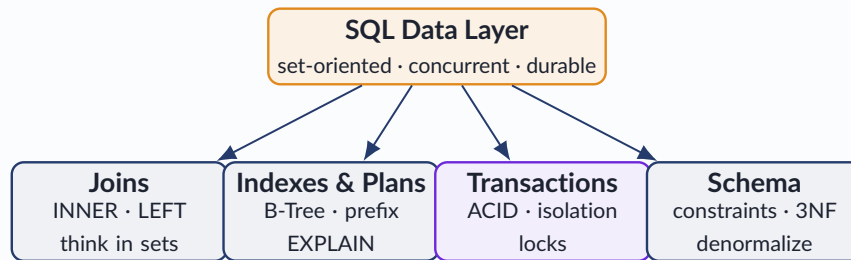


Fig 2.1 — The Day 2 concept map: four pillars sit on one engine.
Reads and speed on the left, correctness and structure on the right.

Terminology to anchor: **JOIN** → **Index** → **Leftmost-Prefix** → **Constraint** → **Transaction** → **Isolation Level** → **Execution Plan** → **Keyset Pagination**.

3 Guided Learning

3.1 Joins — Think in Sets, Not Loops

What is it? A join combines rows from two tables on a matching condition. The junior mental model is a nested loop — "for each user, find their orders." The correct mental model is **set algebra**: an **INNER JOIN** is the intersection (only rows that match on both sides), a **LEFT JOIN** keeps every row from the left set and fills unmatched right-side columns with **NULL**, a **FULL OUTER JOIN** keeps unmatched rows from both sides.

Why does it exist? Normalization (Section 3.6) deliberately splits data across tables to avoid duplication. Joins are how you put it back together at read time. The trade is intentional: cheaper, safer writes in exchange for slightly more work on reads.

How does it work internally? The planner picks a physical join strategy — **nested loop** (great when one side is tiny and the other is indexed), **hash join** (build a hash table on the smaller side, probe with the larger — great for big unsorted sets), or **merge join** (both inputs sorted, walk them in lockstep). You do not choose this directly; you *enable* good choices by indexing the join columns.

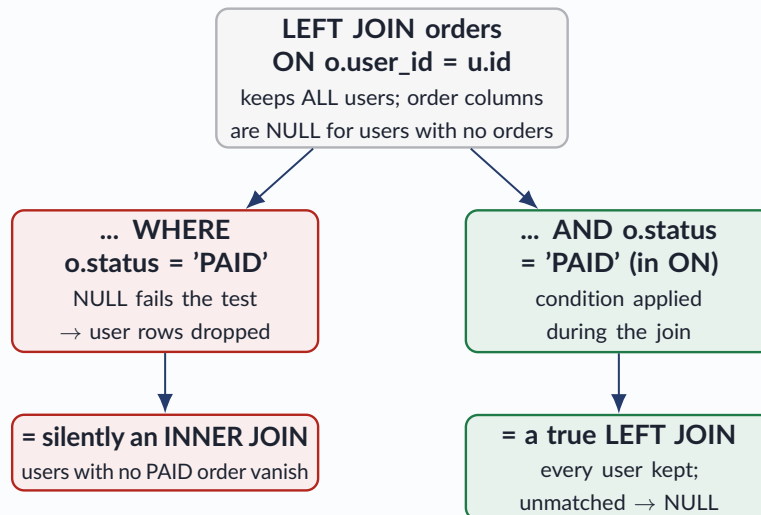


Fig 2.2 – The LEFT JOIN trap. A filter on the right table in the WHERE clause runs after the join and discards the NULL rows, silently turning your outer join back into an inner join.

the LEFT JOIN trap and its fix

```

-- TRAP: this silently behaves like an INNER JOIN
SELECT u.id, u.name, o.id AS order_id
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.status = 'PAID'; -- drops users with no PAID order!

-- FIX: push the condition into the ON clause
SELECT u.id, u.name, o.id AS order_id
FROM users u
LEFT JOIN orders o ON o.user_id = u.id AND o.status = 'PAID';
-- users with no PAID order still appear, with NULL order_id
  
```

When should I use it? LEFT JOIN whenever "rows on the left that have no match still matter" — users with zero orders, products never sold. INNER JOIN when you only care about matched pairs. Teams standardize on LEFT over RIGHT for readability: the driving table reads first.

When should I not? Do not join to then throw the extra columns away, and do not fan out a one-to-many join and then COUNT in Java — let the database aggregate. Beware the accidental cartesian product from a missing or wrong ON condition.

⚠ Common misconception

"LEFT JOIN always keeps all left rows." It keeps them *through the join* — but a later WHERE on a right-table column is evaluated after the join and will delete the NULL-filled rows. The rule: filters that belong to the outer side go in the ON clause; filters that should apply to the final result go in WHERE.

3.2 Indexes & the Leftmost-Prefix Rule — the Highest-Leverage DB Skill

What is it? An **index** is a separate, sorted data structure (almost always a **B-Tree**) that maps column values to row locations, letting the engine jump straight to matching rows instead of scanning the whole table. It is the single highest-leverage skill in this chapter: one correct index routinely turns a

2-second query into a 2-millisecond one.

Why does it exist? A table is stored roughly in insertion order (a "heap"). Finding `WHERE email = ?` in a heap means reading every row — $O(n)$. A B-Tree keeps values sorted so lookups, range scans and ordered reads become $O(\log n)$. You are trading write cost and disk space for read speed.

How does it work internally? A B-Tree is a shallow, balanced tree of sorted pages; even a billion-row table is only a handful of levels deep, so a lookup touches a few pages. Because the leaves are sorted, the same structure serves equality (`=`), ranges (`>`, `BETWEEN`), prefix `LIKE 'abc%'`, and `ORDER BY` without a separate sort. This ordering is exactly why column order in a *composite* index matters.

The leftmost-prefix rule. A composite index on `(a, b, c)` is sorted by `a`, then `b` within equal `a`, then `c`. So it can satisfy lookups on `a`, on `a+b`, or on `a+b+c` — but *never* on `b` alone or `c` alone, because those values are scattered throughout the tree. Think of a phone book sorted by last name then first name: useless for finding everyone named "James."

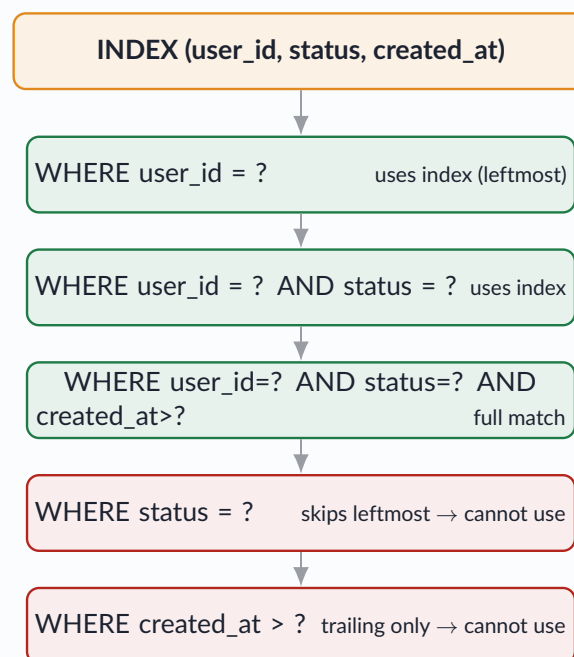


Fig 2.3 — Leftmost-prefix in action. One composite index on `(user_id, status, created_at)` serves the left column and any left-anchored prefix, but not a lone trailing column.

a composite index matched to a real query

```

-- Planned query: "list this user's orders in a status, newest first"
SELECT id, total_cents, created_at
FROM orders
WHERE user_id = 42 AND status = 'PAID'
ORDER BY created_at DESC;

-- One index serves the filter AND the sort:
CREATE INDEX idx_orders_user_status_created
ON orders (user_id, status, created_at DESC);
  
```

When should I use it? Index the columns you `WHERE` on, `JOIN` on, and `ORDER BY` on — especially every foreign key, which most databases do *not* index automatically. Design the composite index to match

your hottest query's predicate order.

When should I *not*? Do not index every column "just in case." Each index must be maintained on every `INSERT`, `UPDATE` and `DELETE`, and consumes storage and cache. Over-indexing silently taxes every write. Low-cardinality columns (a boolean `is_active`) rarely benefit from a standalone index.

Database insight

An index only helps if the planner can use it *as written*. Wrapping the indexed column in a function — `WHERE LOWER(email) = ?` — defeats a plain index on `email`, because the stored values are not lowercased. Fixes: store a normalized column, or create a matching *expression index* `ON users (LOWER(email))`. The same trap hits implicit type casts and leading-wildcard `LIKE '%abc'`.

Performance note

A **covering index** includes every column a query needs, so the engine answers from the index alone — an *Index Only Scan* — never touching the table heap. Adding the selected columns (via `INCLUDE` in Postgres, or as trailing key columns) can eliminate the random heap fetches that dominate the cost of an otherwise-indexed query.

3.3 Constraints — Push Correctness Into the Database

What is it? Constraints are rules the database enforces on every write, no matter which application, script, or human issued it: `PRIMARY KEY` (unique, non-null identity), `FOREIGN KEY` (this value must exist in another table), `UNIQUE`, `NOT NULL`, and `CHECK` (an arbitrary boolean predicate).

Why does it exist? Application-layer validation is necessary but never sufficient: code has bugs, and sooner or later a second consumer — a batch job, an admin console, a migration script — writes to the table without going through your service. A constraint is the last line of defense that no code path can bypass. It converts "we hope this is true" into "the database guarantees this is true."

constraints move correctness into the schema

```
CREATE TABLE order_items (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id    BIGINT NOT NULL REFERENCES orders(id)
              ON DELETE CASCADE,
  product_id  BIGINT NOT NULL REFERENCES products(id)
              ON DELETE RESTRICT,
  quantity    INT     NOT NULL CHECK (quantity > 0),
  unit_cents  INT     NOT NULL CHECK (unit_cents >= 0),
  UNIQUE (order_id, product_id)  -- a product appears once per order
);
CREATE INDEX idx_order_items_order ON order_items(order_id);
CREATE INDEX idx_order_items_product ON order_items(product_id);
```

How does it work internally? A `FOREIGN KEY` makes the database check the referenced row exists on insert/update, and controls what happens when the parent is deleted. `ON DELETE CASCADE` deletes the children automatically; `ON DELETE RESTRICT` (or the default `NO ACTION`) refuses the delete while children exist. This is a design decision, never a reflex: cascading from `orders` to `order_items` is sensible (an order's lines have no life of their own), but cascading a product delete that wipes historical order lines would destroy your revenue history.

Security warning • uniqueness must live in the database

Enforcing "unique email" only with a `findByEmail` check in Java is a race condition, not a constraint: two concurrent signups both read "not found," both insert, and you now have duplicate accounts. Only a `UNIQUE` index makes it atomic — the second `INSERT` fails. Validate in the app for a nice error message, but let the database be the source of truth.

How senior engineers think

Model foreign-key *delete behaviour* deliberately, table by table. A good default for anything you might need for audit or history is `RESTRICT` plus a soft-delete flag, not `CASCADE`. "Convenient" cascades are how a single mistaken `DELETE` in a support tool erases a customer's entire order history in one statement.

3.4 Transactions, ACID & Isolation Levels

What is it? A **transaction** groups statements so they succeed or fail as one unit. Its guarantees are **ACID**: **Atomicity** (all-or-nothing), **Consistency** (a valid state maps to a valid state, respecting constraints), **Isolation** (concurrent transactions do not corrupt each other's view), and **Durability** (once committed, it survives a crash).

Why does it exist? Because partial success is often worse than total failure. A money transfer that debits one account but fails to credit the other has *destroyed* money. Wrapping both writes in one transaction means the system can only move between valid states.

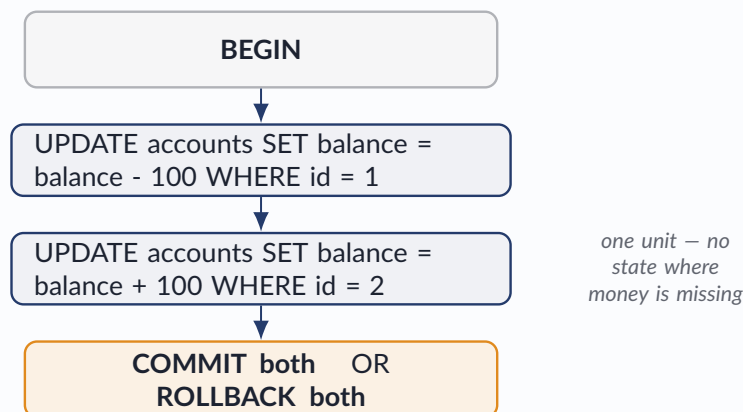


Fig 2.4 — Atomicity: a transfer is two writes in one transaction. Either both commit or both roll back — the account totals never disagree.

How does it work internally? **Isolation** is the subtle one. Perfect isolation (as if transactions ran one at a time) is expensive, so databases offer levels that trade correctness guarantees for concurrency. Each level is defined by which *anomalies* it permits:

- **Dirty read** — you read another transaction's *uncommitted* change, which may still roll back.
- **Non-repeatable read** — you read a row twice in one transaction and get different values because another transaction committed an `UPDATE` in between.
- **Phantom read** — you run the same `WHERE` twice and the second run returns *new rows* another transaction inserted.

Level	Dirty read	Non-repeatable	Phantom	Typical default
READ UNCOMMITTED	Possible	Possible	Possible	rarely used
READ COMMITTED	Prevented	Possible	Possible	PostgreSQL default
REPEATABLE READ	Prevented	Prevented	Possible*	MySQL / InnoDB default
SERIALIZABLE	Prevented	Prevented	Prevented	strictest, lowest concurrency

*Postgres implements *REPEATABLE READ* as snapshot isolation, which also blocks most phantoms; the SQL standard merely permits them.

When should I use each? Default to `READ COMMITTED` — it prevents the genuinely dangerous dirty read while keeping high concurrency. Reach for `REPEATABLE READ` or `SERIALIZABLE`, or explicit locking (Day 3's optimistic/pessimistic locking), only on specific hot paths where correctness demands it: inventory decrement, balance transfer, "book the last seat."

the boundary Spring opens for you

```
@Service
public class TransferService {
    // Day 1's proxy opens a real DB transaction around this method.
    @Transactional // default isolation = the datasource default
    public void transfer(long from, long to, int cents) {
        accounts.debit(from, cents);
        accounts.credit(to, cents); // if this throws, the debit
    }                               // is rolled back too
}
```

⚠ Common misconception

Higher isolation is not "just safer, use it everywhere." `SERIALIZABLE` makes the database detect conflicting transactions and *abort* one with a serialization error your code must catch and retry. It trades silent corruption for explicit, throughput-reducing retries — the right trade only where the data truly demands it.

⚡ Performance note

The most expensive mistake here is a *long-running transaction*. A transaction holds its locks and its snapshot until it commits; keep one open across a slow external HTTP call and you block every other writer and bloat the database's version store. Rule: open transactions late, commit early, and never do network I/O inside one.

3.5 Query Optimization & Execution Plans

What is it? `EXPLAIN` shows the planner's chosen strategy for a query; `EXPLAIN ANALYZE` actually runs it and reports *estimated vs actual* rows and time per step. It is the difference between guessing why a query is slow and knowing.

How does it work internally? The planner uses table statistics (collected by `ANALYZE` / `autovacuum`) to estimate the cost of each possible plan and picks the cheapest. When statistics are stale, its estimates diverge from reality and it can choose a bad plan — a large estimate/actual mismatch in the plan is your first clue.

reading a plan

```
EXPLAIN ANALYZE
SELECT id, total_cents FROM orders
WHERE user_id = 42 AND status = 'PAID';
```

```
-- GOOD: Index Scan using idx_orders_user_status_created
--       (actual rows=30  time=0.05..0.09)
-- BAD:  Seq Scan on orders
--       (actual rows=30  time=0.02..812.4  rows removed by filter: 4M)
-- Seq Scan on a large table + big estimate/actual gap = investigate
```

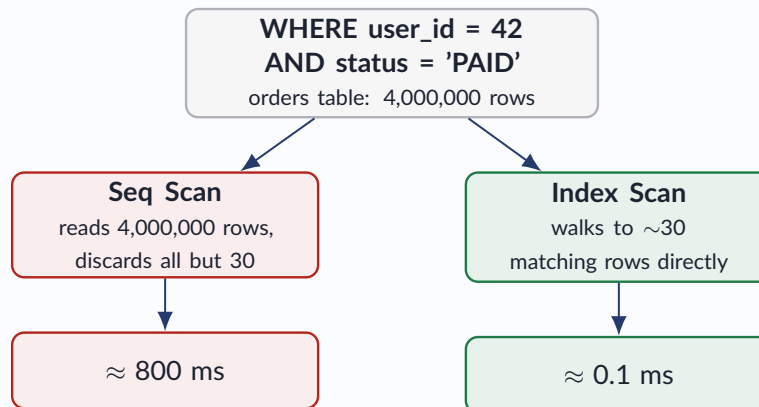


Fig 2.5 — The same query, two plans. Without a usable index the engine reads every row and filters; with one it jumps to the few matches.

When should I use it? Before optimizing anything. The most common finding in a slow-query audit is a sequential scan where you expected an index — caused by a missing index, a function on the indexed column, a type mismatch, or the table being small enough that a scan is genuinely cheaper. Always measure before you assume you need a bigger server.

💡 How senior engineers think

A senior does not "add an index and hope." They capture the query, run `EXPLAIN ANALYZE`, read which node dominates the time, form one hypothesis, change one thing, and re-run the plan. Optimization is a measurement loop, not a guessing contest — and the plan is the instrument.

3.6 Pagination & Schema Design

Pagination — why offset dies at depth. Offset pagination (`LIMIT 20 OFFSET 10000`) makes the database generate, sort, and then *throw away* the first 10,000 rows before returning 20. Cost grows with the offset, so page 500 times out even when page 1 is instant. **Keyset (cursor) pagination** instead remembers the last row seen and asks for rows *after* it, using the index directly — so it stays fast at any depth.

offset vs keyset pagination

```
-- OFFSET: scans and discards 10,000 rows every time
SELECT id, created_at FROM orders
ORDER BY created_at DESC, id DESC
LIMIT 20 OFFSET 10000; -- slow at high offsets

-- KEYSET: seeks straight to the cursor via the index
SELECT id, created_at FROM orders
WHERE (created_at, id) < (:lastCreatedAt, :lastId)
```

```
ORDER BY created_at DESC, id DESC
LIMIT 20;                                -- fast at any depth
```

Schema design — normalize by default. Normalization to third normal form (3NF) means: no repeating groups, no partial dependency on part of a composite key, and no transitive dependency (non-key columns depending on other non-key columns). The payoff is that every fact lives in exactly one place, so there are no *update anomalies* — you never have to change a customer's name in fourteen rows and miss one.

When to denormalize. Deliberately, and only for a *measured* read-heavy hot path. Storing an `order_total` column instead of summing line items on every read is a classic, valid denormalization — but you now own the job of keeping it correct on every write, and you should document why the trade was made. Denormalize as a considered optimization, never as a default shortcut.

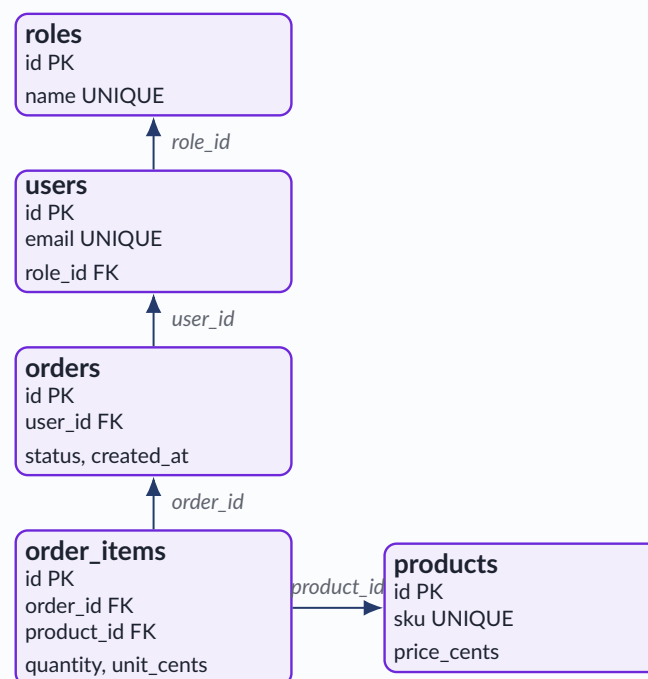


Fig 2.6 — The normalized ShopFlow schema. Each arrow is a foreign key pointing from child to parent; every FK column gets its own index.

✓ Best practices — what experienced engineers do

- Always index foreign-key columns — most databases do not do it for you.
- Run `EXPLAIN ANALYZE` before assuming a query needs a bigger server.
- Prefer keyset pagination for any list endpoint that can grow past a few thousand rows.
- Wrap multi-statement writes in one transaction, not "best-effort" sequential calls.
- Let the database enforce uniqueness, references and ranges via constraints.

✖ Common mistakes beginners make

- Adding indexes reactively to every column instead of matching them to real query patterns.
- `SELECT *` in production, pulling columns nobody needs (and defeating covering indexes).
- Not realising a long-running transaction holds locks and blocks other users.
- Offset-paginating a table with millions of rows and wondering why page 500 times out.

4 Visualizations to Internalize

Two diagrams above carry most of this chapter's judgment. Redraw them *from memory* — reproduction is what moves a diagram from "recognised" to "known."

 Redraw: the leftmost-prefix rule — which predicates the index (`user_id`, `status`, `created_at`) can and cannot serve

 Redraw: the LEFT JOIN trap — where the WHERE filter turns an outer join into an inner one, and the ON-clause fix

5 Guided Coding Lab — Script the ShopFlow Schema

Goal of this lab: turn the abstract rules above into a real, repeatable migration for ShopFlow. By the end you will have a versioned SQL file that any teammate can run to build an identical, correctly-constrained database — the foundation Day 3's JPA maps onto.

1 Add Flyway to the ShopFlow module

Add `spring-boot-starter-data-jdbc` (or JPA) and the `flyway-core` dependency, and point

`spring.flyway` at a Postgres datasource. Create the folder `src/main/resources/db/migration`.
 💡 **Why:** Flyway runs versioned SQL files on startup and records them in a `flyway_schema_history` table — so "create the schema" becomes reproducible and auditable, not a one-off script someone ran once.

2 Write V1: the reference tables (`roles`, `products`)

In `V1__create_shopflow_schema.sql`, create `roles` (`id` PK, `name` UNIQUE NOT NULL) and `products` (`id` PK, `sku` UNIQUE, `price_cents` INT with CHECK (`price_cents` >= 0)). 💡 **Why:** Reference tables have no foreign keys of their own, so they must be created first — FK targets must exist before the referencing table.

3 Add users with a role foreign key

Create `users` (`id` PK, `email` UNIQUE NOT NULL, `role_id` NOT NULL REFERENCES `roles(id)`). Add `CREATE INDEX` on `role_id`. 💡 **Why:** A foreign key guarantees every user points at a real role; indexing the FK column keeps the "users in this role" lookup and the referential-integrity check fast.

4 Add orders and order_items with CHECK constraints

Create `orders` (FK `user_id`, `status` with CHECK (`status` IN ('PENDING', 'PAID', 'SHIPPED', 'CANCELLED')), `created_at`) and `order_items` (FKs to `orders` and `products`, `quantity` CHECK (`quantity` > 0), a UNIQUE (`order_id`, `product_id`)). Choose ON DELETE CASCADE for order lines, RESTRICT for products. 💡 **Why:** You are encoding business rules — valid statuses, positive quantities, one product per order line — where no application bug can violate them.

5 Create the composite index for the planned query

Add `CREATE INDEX idx_orders_user_status_created` ON `orders` (`user_id`, `status`, `created_at` DESC) plus standalone indexes on each remaining foreign key. 💡 **Why:** This is the leftmost-prefix payoff: the same index serves "orders by user," "by user and status," and the sorted "by user and status, newest first" — your planned list query.

6 Verify with a fresh database and EXPLAIN

Drop and recreate the database, let Flyway rebuild it, seed a few rows, and run `EXPLAIN ANALYZE` on the list-orders query to confirm an *Index Scan*, not a *Seq Scan*. 💡 **Why:** "Repeatable" only counts if you prove it from an empty database — and the plan confirms your index is actually doing its job.

💡 Production tip

Never edit a migration that has already run in any shared environment — Flyway stores a checksum and will refuse to start if `v1` changed. Fix mistakes with a new `v2` migration. This immutability is the whole point: the history of your schema becomes a replayable, verifiable log.

6 Thinking Exercises

Reflection prompts, not quiz questions. Write a sentence or two to make your reasoning explicit.

💡 Think it through

You have a composite index on (`user_id`, `status`, `created_at`). A new feature needs "all orders created in the last hour, across all users," filtering on `created_at` alone. Will the existing index help? What would you add, and what is the cost of adding it?

 Think it through

Your team defaults to `READ COMMITTED`. For the "decrement inventory when an order is placed" path, what anomaly could let two orders oversell the last item, and which isolation level or locking strategy would you reach for — accepting what cost?

 Think it through

A colleague proposes storing `order_total` on the `orders` row instead of summing `order_items` each read. What class of bug does this denormalization invite, and what would you put in place before approving it?

7 Debugging Practice

 Debugging Lab • The report that gets slower every week

Symptom. An "orders history" endpoint was fast at launch. Six months later, deep pages (page 400+) time out, while page 1 is still instant.

Evidence.

the endpoint's query

```
SELECT id, created_at, total_cents FROM orders
ORDER BY created_at DESC
LIMIT 20 OFFSET 8000; -- page number x 20
```

Work the method:

- ✓ **Observe.** `EXPLAIN ANALYZE` shows the time growing with the offset; the plan sorts and discards thousands of rows before returning 20.
- ✓ **Hypothesise.** Offset pagination re-scans and throws away all skipped rows; cost is proportional to `OFFSET`, which grows as the table grows.

- ✓ **Verify.** Compare `OFFSET 0` vs `OFFSET 8000` timings — the second is orders of magnitude slower for the same 20 returned rows.
- ✓ **Fix.** Switch to keyset pagination: `WHERE (created_at, id) < (:lastSeen, :lastId) ORDER BY created_at DESC, id DESC LIMIT 20`, backed by an index on `(created_at, id)`.
- ✓ **Prevent.** Make cursor pagination the default for any growable list endpoint; treat deep `OFFSET` as a code-review smell.

🔧 Debugging Lab • The index that the planner ignores

Symptom. You added an index on `users(email)`, but login lookups still do a sequential scan and stay slow.

Evidence.

the login query

```
SELECT id, password_hash FROM users
WHERE LOWER(email) = LOWER('Alice@Shop.io');
```

- ✓ **Observe.** `EXPLAIN` shows `Seq Scan` on `users`, even though `idx_users_email` exists.
- ✓ **Hypothesise.** The index stores raw `email` values; `LOWER(email)` is a different expression, so the planner cannot match the query to the index.
- ✓ **Verify.** Run the same query as `WHERE email = 'alice@shop.io'` — the index is used, confirming the function was the blocker.
- ✓ **Fix.** Create an expression index `ON users (LOWER(email))`, or store and query a normalized lowercase `email` column with a `UNIQUE` constraint.
- ✓ **Prevent.** Never wrap an indexed column in a function in a hot `WHERE`; normalize on write instead of transforming on read.

8 Mini-Labs

🔧 Mini-Lab • Beginner

30 min

Goal. Feel the difference an index makes.

Context. A single table you can seed with a million rows.

Requirements. `INSERT 1,000,000` rows via `generate_series`; run `EXPLAIN ANALYZE` filtering on a non-indexed column, add an index, re-run.

Hints. Watch the plan node change from `Seq Scan` to `Index Scan`, and compare actual times.

Expected outcome. A dramatic drop in execution time and a visibly different plan.

Common pitfalls. Forgetting to `ANALYZE` the table after bulk insert, so the planner has stale statistics and still avoids the index.

 Mini-Lab • Intermediate

45 min

Goal. Reproduce and then prevent a non-repeatable read.**Context.** Two `psql` sessions acting as concurrent transactions on one row.**Requirements.** At `READ COMMITTED`, read a row in Tx A, commit an `UPDATE` in Tx B, re-read in Tx A — observe the value change. Repeat at `REPEATABLE READ` and observe stability.**Hints.** `BEGIN;` `SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;` before the first read.**Expected outcome.** The anomaly appears at the lower level and vanishes at the higher one.**Common pitfalls.** Auto-commit on — each statement becomes its own transaction, so there is no second read within one transaction to differ. Mini-Lab • Production-style

60 min

Goal. Convert an offset endpoint to keyset and benchmark it.**Context.** A list endpoint over a large `orders` table.**Requirements.** Implement keyset pagination with a compound cursor (`created_at`, `id`), add the matching index, and benchmark page 1 vs page 1000 against the offset version.**Hints.** Return an opaque cursor token (base64 of the last row's sort key) so clients never send raw offsets.**Expected outcome.** Keyset page 1000 is roughly as fast as page 1; offset degrades sharply.**Common pitfalls.** Sorting by a non-unique column alone — ties cause skipped or duplicated rows across pages; always include a unique tiebreaker (`id`) in both the `ORDER BY` and the cursor.**9 Engineering Notes**

Judgment from engineers who have carried a pager. Read these as reasoning, not trivia.

 How senior engineers think

The cheapest performance win in most systems is not a faster server — it is one correct composite index that matches the hottest query's predicate order. Before scaling hardware, capture the top few slow queries, read their plans, and index deliberately. Hardware hides the symptom; the index removes the cause.

 Database insight

Foreign keys are not free at write time — each insert into a child table checks the parent exists, and an unindexed FK makes a parent `DELETE` scan the whole child table to enforce the constraint. This is the concrete reason "always index your foreign keys" is a rule and not a preference.

 Interview insight

Expect: "Why is offset pagination slow at high page numbers?" The answer that lands names the mechanism: the database must generate and discard every skipped row before returning the page, so cost grows with the offset — while keyset pagination seeks straight to the cursor through the index and stays flat. Say *scan-and-discard*, not just "it's slow."

Security warning • the ORM does not exempt you from SQL

On Day 3 an ORM will write your SQL for you, but it will happily generate an N+1 query storm, a cartesian join, or a full scan. The database does not know or care that a framework generated the statement. Everything you learned today about indexes, plans and transactions is exactly what you will use to diagnose the ORM's output.

10 Build-Along Project — ShopFlow

Day 1 gave ShopFlow a spine: a layered Spring Boot 3 / Java 21 module with typed configuration, dev and prod profiles, a bean-timing `BeanPostProcessor`, and a live `GET /api/v1/ping`. Today you give it a memory — a real, constrained, versioned database.

Build-Along • Day 02 — the database foundation

- Add Flyway and a Postgres datasource to the Day 1 module; create `src/main/resources/db/migration`.
- Write `v1__create_shopflow_schema.sql` with tables `users`, `roles`, `products`, `orders`, `order_items`.
- Add every constraint: `PRIMARY KEYS`, `FOREIGN KEYS` (with deliberate `ON DELETE` behaviour), `UNIQUE` on `users.email`, `roles.name`, `products.sku`, and `CHECKS` for status, price and quantity.
- Index every foreign-key column, and add the composite index (`user_id`, `status`, `created_at DESC`) that matches the planned "list orders by user and status, newest first" query via leftmost-prefix.
- Prove repeatability: drop the database, let Flyway rebuild it on startup, seed rows, and run `EXPLAIN ANALYZE` on the list-orders query to confirm an Index Scan.

Definition of done: the app boots, Flyway applies `v1` into a clean database and records it in `flyway_schema_history`, every constraint rejects bad data, and the planned list-orders query uses the composite index (Index Scan, not Seq Scan). Day 3 maps JPA entities onto exactly this schema.

11 Chapter Summary

Key takeaways

- ✓ Think in sets: a `WHERE` on the right table silently turns a `LEFT JOIN` into an `INNER JOIN`.
- ✓ A composite index is usable only from its leftmost column inward — order the columns to match your hottest query.
- ✓ Constraints put correctness where no code path can bypass it; uniqueness in particular must live in the database.
- ✓ Default to `READ COMMITTED`; raise isolation or lock only on specific hot paths, and never hold a transaction across network I/O.

- ✓ **EXPLAIN ANALYZE** is the instrument — a `Seq Scan` on a large table, or a big estimate/actual gap, is where you start.

Things to remember

- Index every foreign key — databases rarely do it for you, and unindexed FKs make parent deletes scan the child table.
- Keyset pagination stays flat at any depth; offset pagination scans and discards everything it skips.

Common pitfalls (last look)

- Function on an indexed column (→ `Seq Scan`). • `SELECT *` in hot paths (→ no covering index).
- Enforcing uniqueness only in Java (→ race-condition duplicates). • Long transactions (→ blocked writers).

📄 One-Page Cheat Sheet

INNER vs LEFT — **INNER** = intersection; **LEFT** keeps all left rows, right filter belongs in `ON`, not `WHERE`.

Leftmost-prefix — index `(a,b,c)` serves `a`, `a+b`, `a+b+c` — never `b` or `c` alone.

FK indexes — always index foreign-key columns; unindexed FK → slow parent deletes.

Isolation — dirty < non-repeatable < phantom; default `READ COMMITTED`, raise only on hot paths.

Plans — `EXPLAIN ANALYZE`; `Seq Scan` on big table or estimate ≠ actual = investigate.

Pagination — keyset (`WHERE (k) < (:cursor)`) over offset for any growable list.

Denormalize — only for a measured read hot path, and own the write-time cost of keeping it correct.

👤 Interview questions to master

- › Explain the difference between `INNER`, `LEFT`, and `FULL OUTER JOIN` with an example.
- › What is the leftmost-prefix rule for composite indexes?
- › What is the difference between a dirty read, a non-repeatable read, and a phantom read?
- › Why would you choose `READ COMMITTED` over `SERIALIZABLE`?
- › How would you diagnose a slow query in production?
- › Why is offset pagination slow at high page numbers, and what replaces it?

Suggested further reading

- Use *The Index, Luke!* — Markus Winand's field guide to B-Tree indexing and the leftmost-prefix rule.
- PostgreSQL Documentation* — "Using EXPLAIN" and "Transaction Isolation."
- Designing Data-Intensive Applications* (Kleppmann), ch. 7 — transactions and isolation in depth.

🔑 Key idea

What you will learn next (Day 3). You now have a correct, indexed schema. Tomorrow you put an ORM on top of it: JPA and Hibernate map the beans of Day 1 onto the tables of Day 2 — and you will watch it generate the N+1 queries, lazy-loading surprises and lock strategies that only make sense once you can already read the SQL underneath.

DAY 03 • DATA LAYER

JPA & Hibernate Deep Dive

03

The persistence context, N+1, locking, and fetching under control

🕒 Estimated study time: 9 hours

🎯 By the end of this day, you will be able to...

- Explain what the **persistence context** actually does at runtime — dirty checking, identity, and the flush — not just which annotation to type.
- Diagnose and eliminate the **N+1 problem** from the SQL log, the single most common Hibernate performance bug in real codebases.
- Choose **cascade** types and **locking** strategies deliberately from the ownership and contention model, never by copy-paste.
- Control fetching explicitly with `JOIN FETCH`, `@EntityGraph`, and DTO projections instead of scattering `FetchType.EAGER`.
- Stop leaking entities out of the transaction and understand exactly why a `LazyInitializationException` happens.

1 The Big Picture

What this day solves. Yesterday you wrote SQL by hand and could see exactly what hit the database. Today an ORM writes the SQL for you — and that convenience is where careers quietly stall. The engineer who treats Hibernate as "annotations that make the database go away" ships a product page that fires 500 queries; the engineer who knows what the persistence context is doing ships one that fires two. This day is about keeping the ORM's automation *visible and under control*.

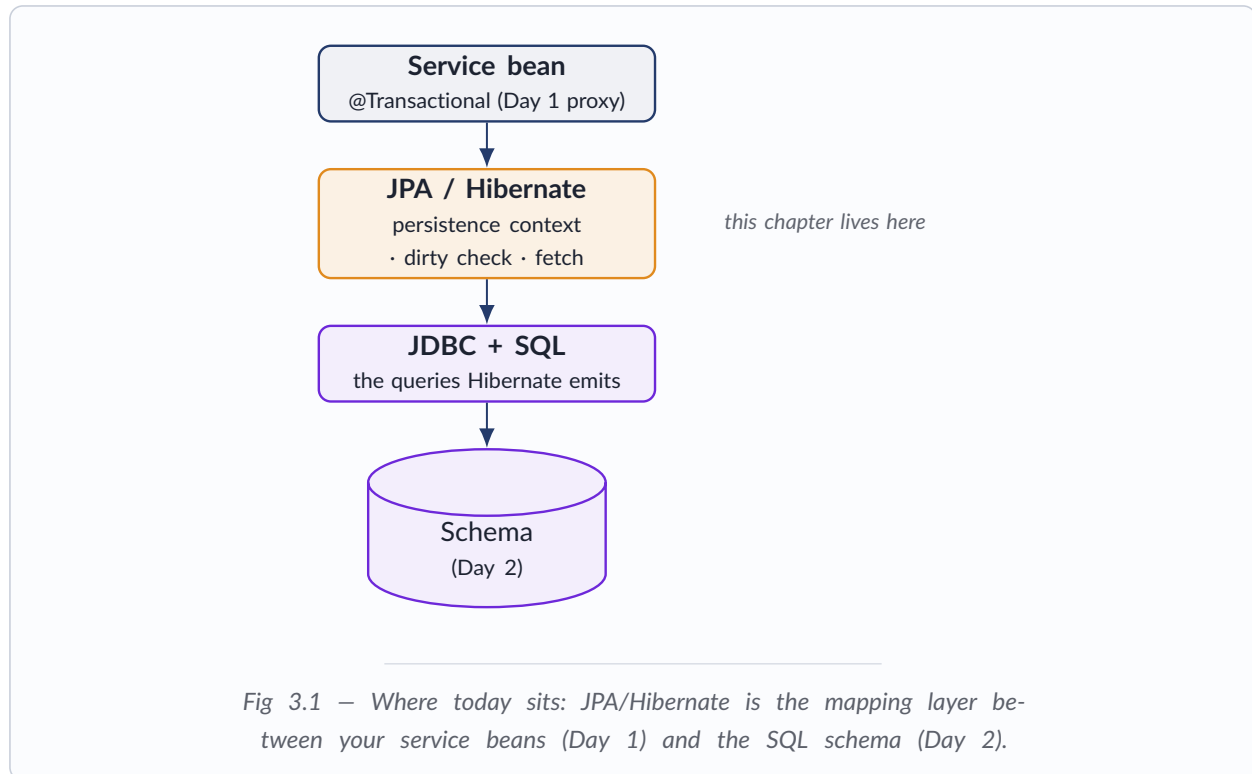
Why backend engineers need it. JPA/Hibernate is the default persistence layer on the JVM. Almost every slow endpoint, every "why is this row still here?", every `LazyInitializationException` in a stack trace, and every lost-update bug in inventory is ultimately a story about the persistence context, fetch strategy, cascade rules, or locking. These four topics are the difference between an ORM that serves you and one that ambushes you in production.

🔑 Key idea

Hibernate is not "SQL you don't have to write." It is a **unit of work**: a per-transaction workspace that tracks your objects, remembers what changed, and decides *when* and *what* SQL to emit. Master three questions — *is this entity managed?* *when does it flush?* *what does it fetch?* — and the ORM stops surprising you.

Where it fits in a Spring Boot application. The data layer sits between your service beans and the schema you designed on Day 2. A `@Transactional` service method opens a persistence context, your repository loads and mutates entities inside it, and on commit Hibernate flushes the accumulated changes as SQL. Every layer above — controllers, DTOs, the API contract on Day 4 — depends on

this layer returning the right data without leaking lazy proxies.



How it connects to previous chapters. The `EntityManager` is a Spring bean (Day 1). The transaction that scopes it is opened by the `@Transactional` proxy you studied on Day 1 — which is exactly why a self-invoked `@Transactional` method silently gets no persistence context. And the SQL Hibernate generates lands on the indexes, constraints and isolation levels you designed by hand on Day 2. Today ties those two threads together.

Real company examples. The N+1 problem is so universal it is a rite of passage: teams at every scale — from a two-person startup to banks running thousands of Spring services — have shipped a listing page that quietly issued one query per row. Hibernate has powered JVM persistence for two decades precisely because the unit-of-work model makes the common case effortless; the senior skill is knowing when the common case is silently doing something expensive.

2 Concept Map

Hold the whole territory in one picture. The persistence context is the hub; everything else — state, fetching, cascade, locking — is a rule about how that hub behaves.

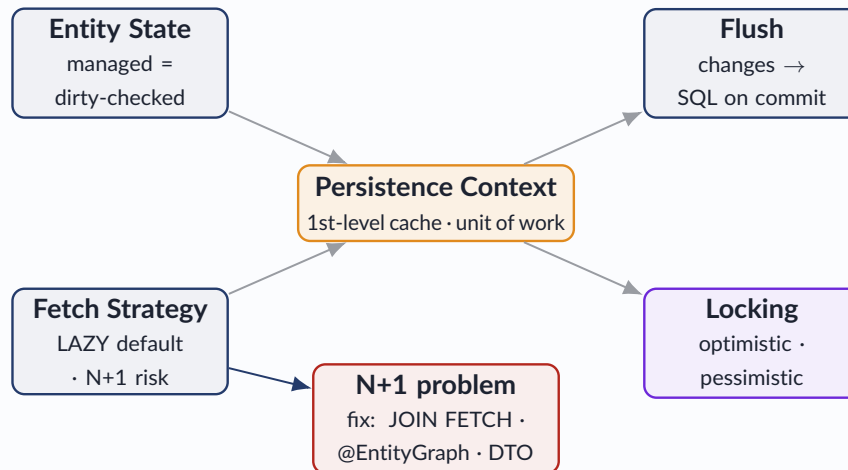


Fig 3.2 — The Day 3 concept map. The persistence context tracks entity state; fetch strategy, cascade, and locking are the rules that govern its SQL.

Terminology to anchor: **EntityManager** → **Persistence Context** → **Entity State** → **Dirty Checking** → **Flush** → **Fetch Strategy** → **N+1** → **Cascade** → **Locking**.

3

Guided Learning

3.1 Entity Lifecycle & the Persistence Context

What is it? The **persistence context** is a per-transaction workspace managed by the `EntityManager`. Every entity you load or persist inside it moves through four states: **Transient** (a plain new object nobody tracks), **Managed** (attached to the context and watched for changes), **Detached** (was managed, but the context closed or the entity was evicted), and **Removed** (marked for deletion on the next flush).

Why does it exist? It buys you three things at once: *identity* (fetching the same ID twice in one transaction returns the very same Java object, not two copies), a *first-level cache* (the second fetch skips the database entirely), and *automatic change tracking* so you never write an explicit `UPDATE`. That last property — **dirty checking** — is the one beginners find magical and seniors find obvious.

How does it work internally? When an entity becomes managed, Hibernate takes a **snapshot** of its field values. At flush time (on commit, before a query that needs current data, or on an explicit `flush()`) it compares each managed entity against its snapshot; any field that differs becomes an `UPDATE`. No snapshot difference means no SQL. This is why mutating a managed entity needs no `save()` call at all.

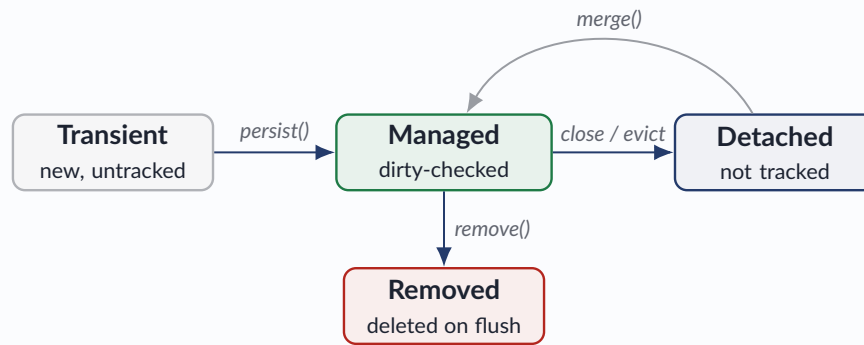


Fig 3.3 – Entity states. Only Managed entities are dirty-checked; a Detached entity's changes are silently ignored until you merge it back.

PricingService.java

```

@Transactional
public void raisePrice(Long id, BigDecimal delta) {
    Product p = productRepository.findById(id).orElseThrow();
    p.setPrice(p.getPrice().add(delta));
    // No save() call. p is MANAGED; on commit, dirty checking
    // notices price changed and emits exactly one UPDATE.
}
  
```

When should I use it? Whenever you mutate data: load inside a transaction, change fields, let the flush write it. Trust dirty checking — fighting it with manual `save()` calls just adds noise (and occasionally a redundant `UPDATE`).

When should I not? Do not rely on it once the transaction ends. A **detached** entity is a dead object as far as the ORM is concerned: setters do nothing, and touching an un-fetched lazy association throws. This is the root of the infamous `LazyInitializationException` — covered next.

👁 Behind the scenes

The persistence context is a `Map` keyed by entity type + primary key. That is the whole "first-level cache": ask for `Product 42` twice in one transaction and the second call is a map lookup, not a `SELECT`. It also means two different queries that return the same row hand you the *same* object — changes are never split across duplicate copies.

⚠ Common misconception

`save()` on a Spring Data repository does not mean "write now." For a *managed* entity it is often a no-op — the write already happens via dirty checking at flush. `save()` earns its keep for *new* (transient) entities and for re-attaching a *detached* one via `merge`. Calling it in a loop over managed entities writes nothing extra and misleads the next reader.

3.2 Lazy vs. Eager Loading, and the N+1 Problem

What is it? `FetchType.LAZY` defers loading an association until you actually touch it; `FetchType.EAGER` loads it immediately alongside the parent. The defaults trip everyone up: `@OneToMany` and `@ManyToMany` are `LAZY` by default, but `@ManyToOne` and `@OneToOne` are `EAGER` by default.

Why does it exist? You rarely need every association on every read. Lazy loading lets you pull just the parent for a summary view and only pay for children when a detail view needs them. The cost of that flexibility is the **N+1 problem**: fetch N parents with one query, then lazily touch a collection on each inside a loop, and you have silently issued $1 + N$ queries.



Fig 3.4 – The N+1 problem: one query for the list, then one more per row. Ten orders become eleven round-trips; a thousand become a thousand and one.

N+1 in the wild – and two fixes

```

// N+1 in the wild
List<Order> orders = orderRepository.findAll();           // 1 query
orders.forEach(o -> log.info("{} ", o.getItems().size())); // N queries!

// Fix #1: JOIN FETCH (JPQL) -- pull parents + children in one query
@Query("SELECT o FROM Order o JOIN FETCH o.items WHERE o.status = :s")
List<Order> findWithItems(@Param("s") OrderStatus status);

// Fix #2: Entity Graph -- declarative, reuses the derived query name
@EntityGraph(attributePaths = {"items"})
List<Order> findByStatus(OrderStatus status);

```

How do I catch it? Turn on SQL logging in dev and *count queries*. If a request that returns a list of ten prints eleven statements, you have found an N+1.

application-dev.yml – make Hibernate's SQL visible

```

spring:
  jpa:
    show-sql: true
    properties:
      hibernate:
        format_sql: true
  logging:
    level:
      org.hibernate.SQL: DEBUG
      org.hibernate.orm.jdbc.bind: TRACE # shows the bound "?" params

```


⚡ Performance note

`JOIN FETCH` on a `@OneToMany` multiplies rows: $N \text{ orders} \times M \text{ items}$ each returns $N \times M$ rows, and Hibernate de-duplicates the parents in memory. That is fine for a handful of children but a disaster for large collections — and you cannot combine it with pagination safely (Hibernate warns "*firstResult/maxResults specified with collection fetch; applying in memory*" and paginates *after* loading everything). For paginated lists, fetch IDs first, then the batch — or use a DTO projection.

❓ Why does this exist? • why "just use EAGER everywhere" is wrong

Making the association `EAGER` does not remove the $N+1$ — it hides it and makes it worse. Every query touching that entity now drags the association along whether the use case needs it or not, and an `EAGER` association *cannot* be selectively fetch-joined per query. Keep everything `LAZY` by default and fetch explicitly, per use case, with `JOIN FETCH`, `@EntityGraph`, or a projection.

Association	Default	Recommended
<code>@ManyToOne</code>	EAGER	override to LAZY
<code>@OneToOne</code>	EAGER	override to LAZY
<code>@OneToMany</code>	LAZY	keep LAZY, fetch per query
<code>@ManyToMany</code>	LAZY	keep LAZY, prefer a join entity

3.3 Cascade Types & orphanRemoval

What is it? A **cascade** propagates an `EntityManager` operation from a parent to its associations. The types are `PERSIST`, `MERGE`, `REMOVE`, `REFRESH`, `DETACH`, and `ALL` (all of them). Separately, `orphanRemoval = true` deletes a child the moment it is removed from the parent's collection — even with no `REMOVE` cascade.

Why does it exist? It encodes **ownership**. An `OrderItem` has no life outside its `Order`: persist the order, its items should persist; delete the order, its items should vanish. Cascade lets you express "this child's lifecycle belongs to this parent" in one place instead of remembering to save and delete children by hand.

How does it work internally? When you `persist` / `remove` a parent, Hibernate walks each association marked with the matching cascade and applies the same operation, respecting `orphanRemoval` for collection element removals at flush. Ownership is directional — and that direction is exactly where the danger lives.

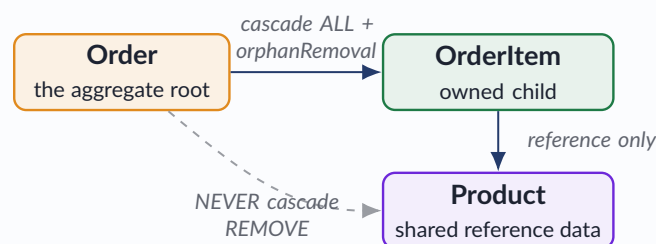


Fig 3.5 — Cascade follows ownership. `Order` OWNS its items (`cascade ALL + orphanRemoval`); it only `REFERENCES` the shared `Product` — never cascade `REMOVE` there.

Order.java – owns its items

```

@Entity
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    // Order OWNS its items: cascade all + orphanRemoval.
    @OneToMany(mappedBy = "order",
        cascade = CascadeType.ALL,
        orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>();
}

```

When should I not? Never cascade `REMOVE` (or `ALL`, which includes it) onto **shared reference data**. A `Product` is referenced by many order items across many customers; deleting an order must not delete the catalogue entry other orders still point to. That is a data-loss incident waiting for a Friday deploy.

 Security warning • the shared-entity delete trap

`cascade = CascadeType.ALL` on `OrderItem` → `Product` means "delete this order item, delete the product too." One returned order can wipe a product that a thousand other orders reference, cascading into foreign-key errors or — worse, if constraints are lax — silent data loss. Cascade *down* an ownership tree (`Order` → `OrderItem`); never cascade *across* to shared references (`OrderItem` → `Product`).

 Database insight

`orphanRemoval = true` and `CascadeType.REMOVE` are not the same thing. `REMOVE` fires when you `remove()` the *parent*. `orphanRemoval` fires when a child is *disassociated* — `order.getItems().remove(item)` — and issues a `DELETE` for that child on flush, because an owned child with no parent is meaningless. Use `orphanRemoval` for true parent-owned collections; it is what makes "remove a line item from the cart" actually delete the row.

3.4 Optimistic vs. Pessimistic Locking

What is it? Two strategies for the same danger: two transactions reading the same row, both writing, and one silently overwriting the other (a **lost update**). **Optimistic locking** assumes conflicts are rare and detects them after the fact via a `@Version` column. **Pessimistic locking** assumes conflicts are likely and prevents them up front by locking the row in the database.

How does optimistic work internally? Add a `@Version` field. Every `UPDATE` becomes ... `WHERE id = ? AND version = ?` and bumps the version. If another transaction already incremented it, the `WHERE` matches zero rows, Hibernate sees "*0 rows affected*" and throws `OptimisticLockException`. No database lock is ever held — but the loser must retry.

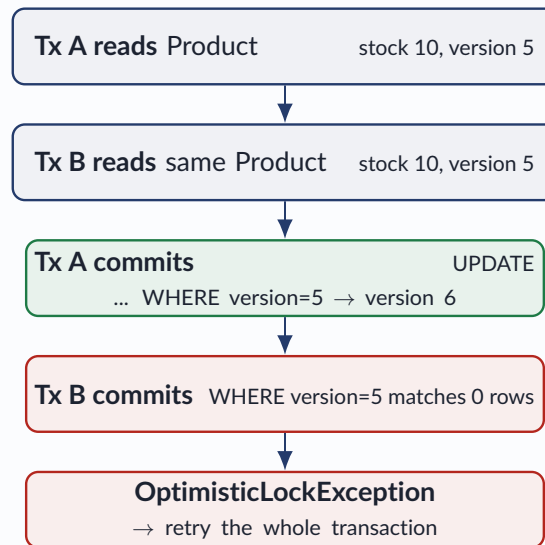


Fig 3.6 — Optimistic locking. Both read version 5; the second commit finds version already moved to 6 and fails instead of silently overwriting.

Product.java + a retrying decrement

```

@Entity @Table(name = "products")
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private int stock;

    @Version // optimistic lock guard
    private long version;
}

// In the service: retry on conflict, do not hold a DB lock.
@Retryable(retryFor = OptimisticLockingFailureException.class,
    maxAttempts = 3, backoff = @Backoff(delay = 50))
@Transactional
public void decrementStock(Long productId, int qty) {
    Product p = productRepository.findById(productId).orElseThrow();
    if (p.getStock() < qty) throw new InsufficientStockException();
    p.setStock(p.getStock() - qty); // version auto-bumped on commit
}

```

How does pessimistic work? You ask the database to lock the row on read — `SELECT ... FOR UPDATE` — via `@Lock(LockModeType.PESSIMISTIC_WRITE)`. Other transactions block until you commit. No conflict can occur because no one else can read-to-write concurrently, at the cost of throughput and the risk of deadlock.

ProductRepository.java — pessimistic variant

```

public interface ProductRepository extends JpaRepository<Product, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT p FROM Product p WHERE p.id = :id")
    Optional<Product> findByIdForUpdate(@Param("id") Long id);
}

```

}

Aspect	Optimistic (<code>@Version</code>)	Pessimistic (<code>FOR UPDATE</code>)
Assumes	conflicts are rare	conflicts are likely
DB lock held	none	row lock until commit
On conflict	throws, you retry	others block, then proceed
Throughput	high	lower (serialised)
Best for	most updates, web forms	hot rows, inventory decrement

💡 How senior engineers think

Default to optimistic; reach for pessimistic only on a proven hot row. Optimistic locking scales because it holds no locks — but it is a *contract*: the caller must handle `OptimisticLockException` with a retry, or the user just sees a 500. Wrapping the whole transaction in `@Retryable` (not just the failing line) is the piece juniors forget, because on conflict you must re-read the fresh row before re-applying the change.

3.5 JPQL, Criteria API & Entity Graphs

What is it? Three ways to say what to fetch. **JPQL** looks like SQL but operates on *entities and fields*, not tables and columns, so it is portable across databases. The **Criteria API** builds queries programmatically with compile-time type safety — ideal for dynamic filters. **Entity Graphs** declare, per query, exactly which associations to fetch eagerly — the modern, explicit alternative to entity-level `EAGER`.

When each? Fixed queries → JPQL (readable, portable). A search endpoint with six optional parameters → Criteria (string-concatenating JPQL is fragile and injection-prone). Controlling fetch depth without rewriting the query → `@EntityGraph`.

OrderRepository.java – entity graph + DTO projection

```
public interface OrderRepository extends JpaRepository<Order, Long> {

    // Fetch order + items + each item's product in ONE query.
    @EntityGraph(attributePaths = {"items", "items.product"})
    List<Order> findByCustomerEmail(String email);

    // DTO projection: read-only, no entities hydrated, no lazy risk.
    @Query("""
        SELECT new com.shopflow.dto.OrderSummary(
            o.id, o.status, COUNT(i))
        FROM Order o JOIN o.items i
        WHERE o.customerEmail = :email
        GROUP BY o.id, o.status
        """)
    List<OrderSummary> listSummaries(@Param("email") String email);
}
```

Why prefer DTO projections for lists? A read-only list endpoint does not need managed entities, dirty checking, or lazy proxies — it needs a few columns. A **projection** (a *record* or interface) selects exactly those columns in one query, skips the persistence-context bookkeeping, and — decisively — cannot throw `LazyInitializationException` because there is no lazy association to trip over.

 Interview insight

"*JOIN FETCH* vs *@EntityGraph*?" Same goal — fetch associations in one query — but different mechanism. *JOIN FETCH* is written *into* the JPQL, so it is explicit but couples fetch to that one query string. *@EntityGraph* is declarative metadata layered *onto* a derived or named query, so the same finder can be reused with different graphs. Say that both defeat N+1; the difference is where the "what to fetch" decision lives.

 Best practices — what experienced engineers do

- Default every association to *LAZY*; fetch explicitly per query with *JOIN FETCH* or *@EntityGraph*.
- Return DTO projections from read-only list endpoints, never hydrated entity graphs.
- Add *@Version* to any entity multiple users can update concurrently.
- Keep *show-sql* on in dev and *count queries* — catch N+1 before code review does.
- Cascade down ownership (*Order* → *OrderItem*); reference shared data (→ *Product*) without cascade.

 Common mistakes beginners make

- Returning entities straight from a controller and leaking *LazyInitializationException* once the session closes.
- Cascading *REMOVE* onto shared reference entities and silently deleting rows others depend on.
- Not noticing the same query firing N extra times in the log.
- Reaching for pessimistic locking everywhere "to be safe," tanking throughput.
- Calling *save()* in a loop over already-managed entities, expecting it to do something.

4 Visualizations to Internalize

Two of today's diagrams carry most of the intuition. Redraw them *from memory* — reproducing them is what turns "I recognise that" into "I can reason with that."

 Redraw: the four entity states and the transitions (*persist*, *remove*, *close/evict*, *merge*)

 Redraw: the N+1 problem as 1 + N queries, then annotate where JOIN FETCH / @EntityGraph collapse it to 1

5 Guided Coding Lab — Map & Query the Order Graph

Goal of this lab: map the ShopFlow schema (Day 2) to a JPA entity graph, then *prove* with SQL logs that you reduced an N+1 to a single query. You will watch the query count, not guess at it.

1 Map the three entities with correct fetch defaults

Create `Order`, `OrderItem`, and `Product`. Make every `@ManyToOne` explicitly `FetchType.LAZY` — override the EAGER default. Keep `@OneToMany` lazy. 💡 *Why:* The EAGER default on `@ManyToOne` is where hidden fetches are born; overriding it is the single highest-leverage line in the file.

2 Encode ownership with cascade rules

On `Order.items`, set `cascade = CascadeType.ALL` and `orphanRemoval = true`. On `OrderItem.product`, set no cascade — it only references the shared catalogue. 💡 *Why:* You are encoding the aggregate boundary: the order owns its items, but the product is shared data no order may delete.

3 Turn on SQL logging and reproduce the N+1

Enable `show-sql` and `org.hibernate.SQL: DEBUG`. Load all orders and loop over `getItems()`. Count the statements in the log. 💡 *Why:* You cannot fix what you cannot see; watching 1 + N statements scroll past is the whole lesson made concrete.

4 Collapse it with an entity graph

Add `findByCustomerEmail` with `@EntityGraph(attributePaths = {"items", "items.product"})`. Re-run and confirm the log now shows exactly one `SELECT` with joins. 💡 *Why:* This proves the fix at the layer that matters — the SQL — not just in a passing test.

5 Add @version to Product and force a conflict

Add a `@Version` field. From two threads, read the same product, decrement stock in each, commit. Catch the `OptimisticLockException` on the loser. 💡 *Why:* Seeing the second commit fail — rather than silently overwrite — is the difference between understanding locking and reciting it.

6 Return a DTO projection for the list endpoint

Write `OrderSummary(Long id, OrderStatus status, long itemCount)` and a JPQL `SELECT new ...` query. Return the DTO from the service. 💡 *Why:* A read-only list needs columns, not managed entities — and a projection can never throw `LazyInitializationException`.

7 Prove exactly one query

With the DTO endpoint and SQL logging on, hit the endpoint and assert the log shows a single statement. 💡 *Why*: "One query for the list-my-orders endpoint" is the measurable definition of done — and the exact bar Day 4's API layer will build on.

🔑 Production tip

Do not eyeball query counts by hand forever. In tests, wire a datasource proxy (e.g. `datasource-proxy` or `QuickPerf`) and `assert max queries = 1`. An N+1 that a human counts today silently returns the day someone adds a new lazy association — a test that fails on query count catches the regression in CI, not in production.

6 Thinking Exercises

Write a sentence or two for each — the goal is to make your reasoning explicit, not to find the "right" keyword.

🧠 Think it through

Dirty checking means you never call `update`. What does the persistence context have to keep in memory for that to work, and what is the cost when a transaction loads ten thousand entities it only reads? Would you still leave them all managed?

🧠 Think it through

`EAGER` loading and `JOIN FETCH` both fetch the association up front. Why does one of them still let you fix an N+1 per query while the other locks you into always fetching? Predict what happens if you fetch-join two separate collections in one JPQL query.

🧠 Think it through

Optimistic locking throws *after* the work is done; pessimistic locking blocks *before* it starts. For an inventory decrement under a flash-sale spike, which failure mode is cheaper — retrying a rejected transaction, or serialising every buyer behind a row lock? What does your answer depend on?

7 Debugging Practice

🔧 Debugging Lab • The list endpoint that fires 500 queries

Symptom. `GET /orders` is fast in tests with 3 rows but times out in staging with 500. APM shows hundreds of near-identical `SELECT` statements per request.

Evidence.

hibernate SQL log – abridged

```
select o.id, o.status from orders o;           -- the "1"
select i.* from order_items i where i.order_id=1; -- the "N"...
select i.* from order_items i where i.order_id=2;
select i.* from order_items i where i.order_id=3;
-- ... 497 more
```

Work the method:

- ✓ **Observe.** One parent query, then one child query per order — textbook `1 + N`.
- ✓ **Hypothesise.** The controller serialises `order.getItems()` for each order, triggering a lazy load per row.
- ✓ **Verify.** Count log statements for a 10-order page: 11 confirms it; 1 would be the fix.
- ✓ **Fix.** Add `@EntityGraph(attributePaths = {"items"})` to the finder, or return a DTO projection that aggregates in one query.
- ✓ **Prevent.** Add a test asserting the endpoint issues ≤ 1 query via a datasource proxy.

🔧 Debugging Lab • `LazyInitializationException` after the response starts

Symptom. A controller returns an `Order` entity; Jackson serialises it and blows up with `could not initialize proxy – no Session`.

Evidence.

OrderController.java

```
@GetMapping("/orders/{id}")
public Order get(@PathVariable Long id) {           // returns ENTITY
    return orderRepository.findById(id).orElseThrow();
} // transaction (and Session) closed here; items still LAZY
```

- ✓ **Observe.** The exception fires during JSON serialisation, *after* the service method returned.
- ✓ **Hypothesise.** The transaction closed, the entity detached, and Jackson touched the lazy `items` — with no session to load it.
- ✓ **Verify.** The stack trace runs through `getItems()` inside the Jackson converter, not inside

your service.

- ✓ **Fix.** Map to a DTO *inside* the transaction and return that — never serialise a managed entity across the layer boundary.
- ✓ **Prevent.** Ban entities from controller signatures; disable `open-in-view` so the leak fails loudly in dev, not silently in prod.

8

Mini-Labs

🔧 Mini-Lab • Beginner

25 min

Goal. Watch dirty checking issue an `UPDATE` you never asked for.

Context. A `@Transactional` method that loads a `Product` and changes its price.

Requirements. Do *not* call `save()`; enable `show-sql`; observe the `UPDATE` at commit.

Hints. Load by id, mutate a field, return; the write happens on flush.

Expected outcome. Exactly one `UPDATE` in the log, with no explicit save.

Common pitfalls. Mutating the entity *outside* the transaction — it is detached, so nothing is written.

🔧 Mini-Lab • Intermediate

45 min

Goal. Reproduce an N+1 and prove the fix from the log.

Context. `Order` with a lazy `@OneToMany` to `OrderItem`.

Requirements. Load all orders, touch `getItems()` in a loop, count queries; then add `@EntityGraph` and re-count.

Hints. With `format_sql` on, the statements are easy to tally.

Expected outcome. `1 + N` before, exactly `1` after.

Common pitfalls. "Fixing" it by making the association `EAGER` — the N+1 just moves, it does not vanish.

🔧 Mini-Lab • Production-style

70 min

Goal. Make optimistic locking safe under concurrency, with retry.

Context. `Product.stock` decremented by concurrent checkouts; `@Version` present.

Requirements. From two threads, decrement the same product; catch `OptimisticLockException` on the loser and retry with `@Retryable`, re-reading the row.

Hints. The retry must wrap the *whole* transaction and re-fetch fresh state each attempt.

Expected outcome. Both decrements succeed after a retry; stock is correct; no lost update.

Common pitfalls. Retrying with the stale entity from the failed attempt, re-applying a decrement onto an old version and looping forever.

9

Engineering Notes

Judgment from engineers who have carried a pager. Read these as reflexes, not trivia.

 How senior engineers think

The first thing a senior does with a slow endpoint is not open a profiler — it is turn on SQL logging and count queries. Ninety percent of "the ORM is slow" turns out to be "the ORM did exactly what I told it to, N times." The framework is rarely the bottleneck; the fetch plan is.

 Performance note

`spring.jpa.open-in-view` is `true` by default and it is a trap. It keeps the persistence context open for the whole request so lazy loads "just work" in the view layer — which means N+1s hide until production and database connections are held far longer than needed. Set it to `false`, force fetching to be explicit, and let leaks fail loudly in development.

 Database insight

Batch your writes. Inserting a thousand order items one `INSERT` at a time is a thousand round-trips; set `hibernate.jdbc.batch_size` (e.g. 50) and Hibernate groups them into batched statements. This does not change your code — it changes the SQL Hibernate emits under the same dirty-checking model.

 Interview insight

Expect: "Why can returning an entity from a controller throw `LazyInitializationException`?" The answer that lands: the transaction (and Hibernate Session) ends when the service method returns, the entity detaches, and the serializer then touches a lazy association with no session to load it. The fix is not "make it EAGER" — it is "map to a DTO inside the transaction."

10 Build-Along Project — ShopFlow

ShopFlow already has a base module (Day 1) and a SQL schema with a migration (Day 2). Today you map that schema to JPA and prove your fetch plan in the logs.

 Build-Along • Day 03 — the persistence layer

- Map the Day 2 schema to the `Order` → `OrderItem` → `Product` entity graph, every `@ManyToOne` explicitly `LAZY`.
- Encode ownership: `Order.items` uses `cascade = ALL + orphanRemoval = true`; `OrderItem.product` carries *no* cascade — the shared catalogue is never deleted by an order.
- Add `@Version` to `Product` for optimistic locking on `stock`, with a `@Retryable` decrement in the service.
- Add `OrderRepository.findByCustomerEmail` using `@EntityGraph(attributePaths = {"items", "items.product"})`.
- Add an `OrderSummary` DTO projection (JPQL `SELECT new ...`) for the "list my orders" endpoint.
- Turn on `show-sql`, set `open-in-view: false`, and capture the log for the list endpoint.

Definition of done: the entity graph maps the Day 2 schema, a concurrent stock decrement raises and recovers from `OptimisticLockException`, and the "list my orders" endpoint provably issues *exactly one* SQL query — verified in the Hibernate log. This persistence layer is what

Day 4's REST API will expose.

11 Chapter Summary

Key takeaways

- ✓ The persistence context is a per-transaction unit of work: identity, first-level cache, and dirty checking.
- ✓ Only *managed* entities are dirty-checked; a detached entity's changes are silently ignored.
- ✓ N+1 is $1 + N$ queries from lazily touching a collection per row — fix with `JOIN FETCH`, `@EntityGraph`, or a DTO.
- ✓ Cascade *down* ownership (`Order` → `OrderItem`); never cascade `REMOVE` across to shared data (→ `Product`).
- ✓ Optimistic locking (`@Version`) scales but needs retry; pessimistic (`FOR UPDATE`) serialises hot rows.

Things to remember

- `@ManyToOne/@OneToOne` default to `EAGER` — override them to `LAZY` deliberately.
- Never return a managed entity from a controller; map to a DTO inside the transaction.
- `open-in-view: false` makes hidden fetches fail loudly in dev instead of leaking into prod.

Common pitfalls (last look)

- `EAGER` "fix" for N+1 → hides it and forbids per-query control. • Cascade `REMOVE` to `Product` → data loss.
- Entity out of the controller → `LazyInitializationException`. • Pessimistic-everywhere → throughput collapse.

📋 One-Page Cheat Sheet

Entity states — Transient → Managed (persist) → Detached (close/evict) / Removed (remove); merge re-attaches.

Dirty checking — managed entity + field change → `UPDATE` at flush; no `save()` needed.

Fetch defaults — `@OneToMany/@ManyToMany` `LAZY`; `@ManyToOne/@OneToOne` `EAGER` — override to `LAZY`.

N+1 fix — `JOIN FETCH` (in the JPQL) or `@EntityGraph` (on the finder) or DTO projection.

Cascade rule — cascade down ownership; `orphanRemoval` deletes disassociated children; never cascade `REMOVE` to shared data.

Locking — optimistic = `@Version` + retry, no lock; pessimistic = `FOR UPDATE`, row lock, serialised.

No leaks — DTO out of controllers; `open-in-view: false`; count queries in dev.

👤 Interview questions to master

- › What is the persistence context, and how does dirty checking work?
- › What causes the N+1 problem, and how do you fix it?

- › What is the difference between `JOIN FETCH` and `@EntityGraph`?
- › Optimistic vs pessimistic locking — when would you use each?
- › How does `orphanRemoval = true` differ from `CascadeType.REMOVE`?
- › Why can returning an entity from a REST controller cause a `LazyInitializationException`?

Suggested further reading

- *Jakarta Persistence spec* — "Entity Operations" and the `EntityManager` lifecycle.
- *Hibernate ORM User Guide* — "Fetching," "Locking," and "Batching."
- Vlad Mihalcea, *High-Performance Java Persistence* — the definitive treatment of N+1, fetch plans, and locking.

Key idea

What you will learn next (Day 4). You now have entities and DTO projections that fetch exactly what they should. Tomorrow you expose them as a real REST API — resource-oriented URLs, correct HTTP status codes (including `409 Conflict` for the optimistic-lock failures you just learned to raise), validation, pagination, and an OpenAPI contract the frontend can consume without asking you a single question.

DAY 04 • API LAYER

REST API Design

04

Building APIs like a professional — DTOs, validation, errors, pagination, versioning, OpenAPI

🕒 Estimated study time: 8 hours

🎯 By the end of this day, you will be able to...

- Design resource-oriented REST APIs with correct HTTP semantics — the right method, the right status code, the right idempotency guarantee for every path.
- Build a DTO layer that decouples your public contract from your persistence model, with an explicit mapper you control.
- Enforce Bean Validation at the controller boundary and centralise every failure into one consistent error shape via `@RestControllerAdvice`.
- Implement pagination and filtering behind a stable response envelope, and choose a versioning strategy you can live with for years.
- Publish the API as a machine-readable contract with OpenAPI/Swagger so frontend, QA and gateways consume it without asking you questions.

1 The Big Picture

What this day solves. Days 1–3 gave you a container full of beans, real SQL, and JPA entities with repositories. But nothing outside your JVM can talk to any of it yet. Today you build the **edge** of the system — the thin, deliberate layer where the outside world meets your domain. The question that separates a junior from a mid-level engineer here is not *"how do I return JSON?"* (any annotation does that) but *"what exactly is my contract, and how do I keep it stable while the code behind it churns?"*

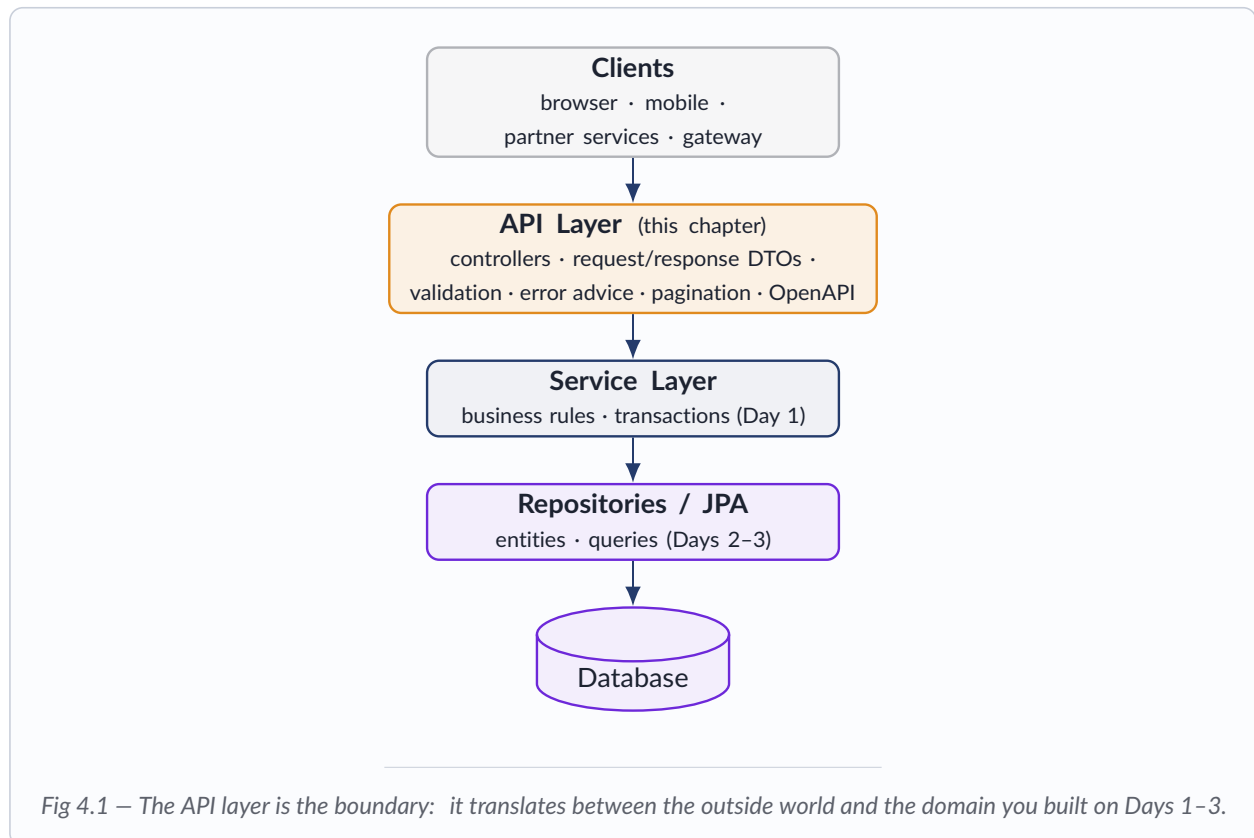
Why backend engineers need it. The API is the one part of your system other teams actually see. A frontend developer, a partner integration, a mobile app, an API gateway — they all consume the shapes and status codes you emit, and they build assumptions on top of them. A leaked entity field, an inconsistent error body, an unpaginated list endpoint: each becomes a production incident or a breaking change that ripples across every consumer. The discipline you learn today is what makes an API something people can *depend on*.

🔑 Key idea

An API is a **contract**, not a side effect of your code. Everything today serves one goal: to make that contract *explicit, consistent, and stable*. DTOs pin the shape, validation pins the inputs, a central error handler pins the failure shape, pagination pins the list shape, versioning pins the evolution, and OpenAPI writes it all down so no one has to guess.

Where it fits in a Spring Boot application. The API layer sits at the very top of the request pipeline you traced on Day 1. A request arrives through the filter chain and `DispatcherServlet`, lands on your `@RestController`, gets validated and mapped into your domain, and travels down into the service

and repository layers you already have. The response travels back up and is serialised to JSON. Everything you build today is the boundary — it owns *translation and contract*, never business rules.



How it connects to previous chapters. The proxies and transactions from Day 1 run *below* your controller. The entities and repositories from Days 2–3 are exactly what you must *not* expose — today you build the wall that keeps them private. The service layer is where logic lives; your controller stays thin.

Real company examples. Stripe’s API is famous precisely because its contract almost never breaks — additive changes only, dated versions, and an error shape identical across every endpoint. GitHub, Twilio and Shopify all publish OpenAPI specs that entire ecosystems build tooling against. The reason those APIs feel trustworthy is not clever code; it is the exact discipline in this chapter, applied relentlessly.

2

Concept Map

Hold the whole territory in one picture. The contract sits at the top; every concept below it exists to keep one part of that contract honest.

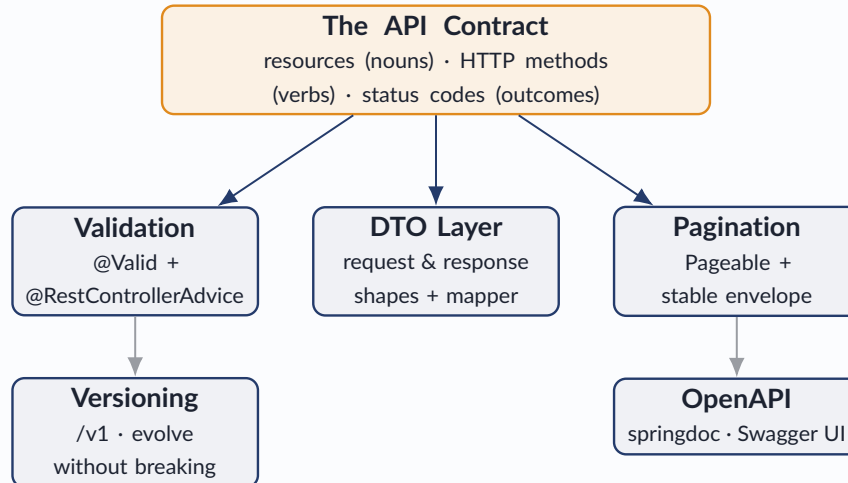


Fig 4.2 – The Day 4 concept map: a stable contract, supported by the DTO, validation, pagination, versioning and documentation concerns.

Terminology to anchor: **Resource** → **HTTP method** → **Status code** → **DTO** → **Mapper** → **Bean Validation** → **@RestControllerAdvice** → **Pageable** → **Envelope** → **Version** → **OpenAPI**.

3 Guided Learning

3.1 REST Principles & HTTP Semantics

What is it? REST models your system as a set of **resources** — nouns like *order*, *customer*, *product* — each addressed by a URL, and manipulated with a small fixed set of HTTP **methods** that supply the verb. The URL names the thing; the method says what to do to it. So it is `POST /orders`, never `POST /createOrder`; `DELETE /orders/42`, never `POST /deleteOrder?id=42`.

Why does it exist? A uniform interface is what lets caches, proxies, gateways and client libraries reason about your API *without knowing your domain*. A cache knows `GET` is safe to store; a load balancer knows `PUT` can be retried; a browser knows `POST` should warn before re-submitting. When you invent action-URLs, you throw all of that infrastructure away and force every consumer to special-case your API.

How does it work internally? Each method carries two properties that drive real behaviour. **Safe** means the call has no side effects (`GET`, `HEAD`) — infrastructure may prefetch or cache it freely. **Idempotent** means calling it *N* times leaves the server in the same state as calling it once. `GET`, `PUT`, `DELETE` are idempotent; `POST` is not. This is not academic: a client that times out on a `PUT` can safely retry, but a retried `POST` may create a second order.

Method	Safe?	Idempotent?	Use for
GET	yes	yes	read a resource or a list, never mutate
POST	no	no	create a new resource (server assigns id)
PUT	no	yes	full replace of a known resource
PATCH	no	no*	partial update of some fields
DELETE	no	yes	remove a resource

*PATCH can be made idempotent by design, but is not guaranteed to be by the spec.

⚠ Common misconception

PUT is not "update" and POST is not "create" as a rule of thumb — the real distinction is idempotency and who owns the identifier. PUT `/orders/42` means "make the resource at this exact URL look like this body" (the client already knows the id, and repeating it is harmless). POST `/orders` means "take this and create something new for me" (the server assigns the id, and repeating it makes another one). If you catch yourself doing PUT to a collection URL, you have the wrong method.

👤 Interview insight

Expect: "What is the difference between PUT and PATCH?" The landing answer names both dimensions: PUT replaces the *entire* resource and is idempotent, so a missing field means "set it to empty"; PATCH sends only the fields to change and is not required to be idempotent. Then add the consequence: sending a partial body to PUT silently wipes the fields you omitted.

3.2 Status Codes — the Contract of Outcomes

What is it? The status code is the first, most machine-readable part of your answer. Before a client parses a single byte of body, the code tells it what happened: success, client mistake, or server fault. Getting these right is the cheapest way to make an API feel professional.

Why does it exist? Consumers branch on the code. A frontend shows a validation form on 400, redirects to login on 401, shows "you can't do that" on 403, and shows "not found" on 404. If you return 200 for everything and bury the real outcome in the JSON body, every client must parse your body to know if the call even succeeded — and generic tooling (retries, alerting, gateways) is blinded entirely.

Code	Meaning	When to send it
200	OK	successful read or update with a body
201	Created	resource created — add a <code>Location</code> header
204	No Content	success with nothing to return (e.g. delete)
400	Bad Request	malformed or validation-failed input
401	Unauthorized	no or invalid credentials — who <i>are</i> you?
403	Forbidden	known identity, but not allowed
404	Not Found	the resource does not exist (or is hidden)
409	Conflict	state clash — duplicate, stale version
422	Unprocessable	well-formed but semantically invalid
500	Server Error	your bug — never the client's fault

How does it work internally? In Spring you rarely set the raw number. You either return a `ResponseEntity` (`ResponseEntity.created(uri).body(dto)` emits 201 plus the `Location` header), or you annotate a handler with `@ResponseStatus(HttpStatus.CREATED)` and return the body directly. The framework's `HttpMessageConverter` (Jackson) then serialises your object into the response.

👤 Security warning • 401 versus 403

These are the pair interviewers love because juniors swap them. 401 `Unauthorized` is misnamed — it means *unauthenticated*: the server does not know who you are, so send credentials. 403 `Forbidden` means the server *knows exactly who you are* and you still may not do this. Rule of thumb: 401 → "log in"; 403 → "logging in as someone else won't help." A subtle production choice: some APIs return 404 instead of 403 for resources a user must not even know exist, to avoid leaking their existence.

3.3 DTOs — Never Expose Your Entities

What is it? A **DTO** (Data Transfer Object) is a plain, intentional shape that crosses your API boundary — one type for the request body, another for the response. In modern Java these are *records*. They are separate from the JPA `@Entity` classes you wrote on Day 3.

Why does it exist? An entity is an internal implementation detail bound to your database schema. Exposing it directly couples your public contract to your table structure, so a harmless column rename becomes a breaking API change. Worse, entities carry things you never want on the wire: password hashes, internal flags, audit columns, and lazy-loaded associations. DTOs let you decide — field by field — exactly what the world sees.

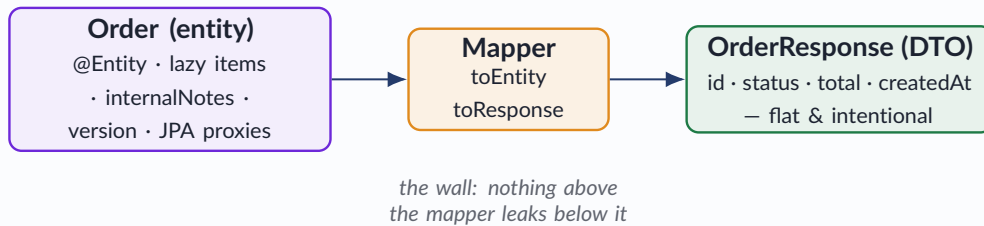


Fig 4.3 — The DTO wall: entities stay private on the persistence side; DTOs are the only shapes that cross the boundary, via an explicit mapper.

dto/OrderDtos.java

```

public record CreateOrderRequest(
    @NotNull Long customerId,
    @NotEmpty @Valid List<OrderItemRequest> items) {}

public record OrderItemRequest(
    @NotNull Long productId,
    @Positive int quantity) {}

public record OrderResponse(
    Long id, String status, BigDecimal total, Instant createdAt) {}

```

How does it work internally? A **mapper** translates in both directions: `toEntity` builds a domain object from the request, `toResponse` projects an entity into the response DTO. You can hand-write it (explicit, zero magic) or generate it with MapStruct (annotation-driven, compile-time). Start hand-written — you learn what actually crosses the boundary.

mapper/OrderMapper.java

```

@Component
public class OrderMapper {

    public Order toEntity(CreateOrderRequest req) {
        Order order = new Order(req.customerId());
        req.items().forEach(i ->
            order.addItem(i.productId(), i.quantity()));
        return order;
    }

    public OrderResponse toResponse(Order o) {
        return new OrderResponse(
            o.getId(), o.getStatus().name(), o.getTotal(), o.getCreatedAt());
    }
}

```

```
}
```

When should I use it? Always, for anything that crosses the HTTP boundary in either direction. The DTO layer is not optional ceremony; it is the seam that lets the inside and outside evolve independently.

When should I not? Do not build a DTO for purely internal method calls between your own services — that is just noise. The wall belongs at the network edge, not between every function.

Behind the scenes

When you return a JPA entity directly and Jackson serialises it, two disasters wait. First, a bidirectional association (`Order` → `items` → back to `Order`) makes Jackson recurse forever — an infinite serialisation loop or a stack overflow. Second, touching a `LAZY` collection outside the transaction throws `LazyInitializationException` deep inside the HTTP converter, producing a `500` that has nothing to do with the caller's request. DTOs make both impossible by construction.

Interview insight

"Why should you never return JPA entities directly from a controller?" Name three reasons in one breath: it couples the public contract to the database schema, it risks serialising fields you never meant to expose, and it triggers lazy-loading and cyclic-reference failures in the serialiser. DTOs are the fix for all three.

3.4 Validation & Centralised Error Handling

What is it? **Bean Validation** (Jakarta Validation) lets you declare input constraints as annotations on the request DTO — `@NotNull`, `@NotEmpty`, `@Positive`, `@Size`, `@Email` — and have the framework check them automatically. `@Valid` at the controller parameter triggers the check; a `@RestControllerAdvice` turns any failure into a consistent error body.

Why does it exist? Without it, every controller method starts with a wall of hand-written `if (req.customerId() == null) return badRequest()` checks, each returning a slightly different error shape. Declarative validation moves the rules onto the data itself, and centralised advice guarantees that every endpoint — present and future — fails the same way.

How does it work internally? When a request body fails `@Valid`, Spring throws `MethodArgumentNotValidException` *before your method body ever runs*. That exception carries a `BindingResult` listing every field error. Your `@RestControllerAdvice` catches it, extracts the field errors, and maps them into your `ErrorResponse`. Because the advice is a single bean applied across all controllers, you write the mapping once.

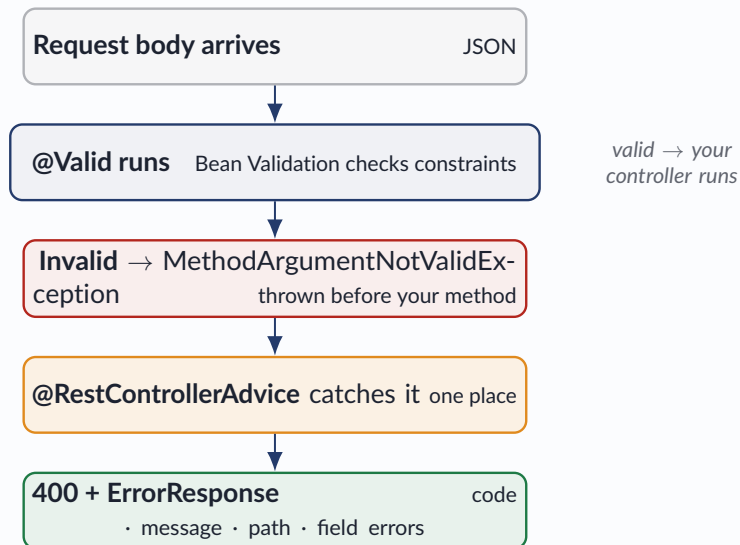


Fig 4.4 — Validation and error flow: a bad request is rejected before your logic runs; the advice shapes every failure identically.

error/ErrorResponse.java

```

public record ErrorResponse(
    String code, String message, String path,
    Instant timestamp, List<FieldError> errors) {

    public static ErrorResponse of(String code, String msg,
        String path, List<FieldError> errors) {
        return new ErrorResponse(code, msg, path, Instant.now(), errors);
    }

    public record FieldError(String field, String message) {}
}

```

error/ApiExceptionHandler.java

```

@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse onValidation(
        MethodArgumentNotValidException ex, HttpServletRequest req) {
        List<ErrorResponse.FieldError> fields = ex.getBindingResult()
            .getFieldErrors().stream()
            .map(f -> new ErrorResponse.FieldError(
                f.getField(), f.getDefaultMessage()))
            .toList();
        return ErrorResponse.of("VALIDATION_ERROR",
            "Request validation failed", req.getRequestURI(), fields);
    }

    @ExceptionHandler(OrderNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse onNotFound(

```

```

    OrderNotFoundException ex, HttpServletRequest req) {
    return ErrorResponse.of("NOT_FOUND", ex.getMessage(),
        req.getRequestURI(), List.of());
    }
}

```

The error body every endpoint now emits, whatever failed:

400 response body

```

HTTP/1.1 400 Bad Request
Content-Type: application/json

{
  "code": "VALIDATION_ERROR",
  "message": "Request validation failed",
  "path": "/api/v1/orders",
  "timestamp": "2026-07-18T10:15:30Z",
  "errors": [
    { "field": "customerId", "message": "must not be null" }
  ]
}

```

Security warning • never leak internals in errors

An error body is an attacker's map. Never let a stack trace, a SQL fragment, an entity class name, or an internal file path reach the client — each one hands out reconnaissance for free. Spring's default 500 page and an unhandled exception both leak more than you think. Your `@RestControllerAdvice` must have a catch-all `@ExceptionHandler(Exception.class)` that logs the detail *server-side* and returns a bland 500 body with a correlation id, never the exception message.

Production tip

Put validation on the DTO, and *business* rules in the service. `@NotNull` on `customerId` belongs on the record; "the customer must have an active account" belongs in `OrderService`, throwing a domain exception the advice maps to 409 or 422. Do not put business logic inside the exception handler — the advice is a translator, not a place to make decisions.

3.5 Pagination, Filtering & the Stable Envelope

What is it? A list endpoint must never return "all rows." **Pagination** returns a bounded slice — a page — plus metadata about the whole set. Spring Data gives you `Pageable` (page number, size, sort) as a controller parameter and returns a `Page<T>` from the repository.

Why does it exist? An unpaginated `getAll` is a latency and memory time-bomb: it works on 50 rows in dev and takes the service down on 5 million rows in production. It also transfers megabytes the client never renders. Pagination bounds the cost of every list call regardless of table size.

How does it work internally? You accept a `Pageable`; Spring binds `?page=0&size=20&sort=createdAt,desc` from the query string. The repository translates it into `LIMIT/OFFSET` (plus a second `COUNT` query for the total). The result is a `Page<T>` carrying content and totals. Crucially, you must *not* return the raw `Page<Entity>` — Spring's `PageImpl` JSON shape is unstable across versions and, again, would serialise entities. Wrap it in your own envelope of DTOs.

dto/PageResponse.java

```

public record PageResponse<T>(
    List<T> content, int page, int size,
    long totalElements, int totalPages,
    boolean first, boolean last) {

    public static <T> PageResponse<T> from(Page<T> p) {
        return new PageResponse<>(
            p.getContent(), p.getNumber(), p.getSize(),
            p.getTotalElements(), p.getTotalPages(),
            p.isFirst(), p.isLast());
    }
}

```

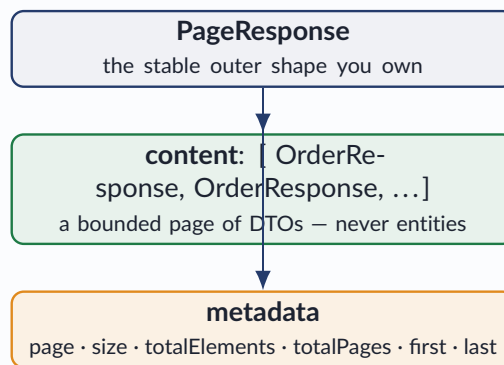


Fig 4.5 — The pagination envelope: a bounded slice of DTOs plus the metadata a client needs to page through the whole set.

What about filtering? For a couple of optional filters, plain `@RequestParam` query parameters are perfect: `?status=PENDING`. When filters multiply and must *compose* arbitrarily, reach for JPA **Specifications** (Criteria API under the hood) so that "status and customer and date-range" is built from small reusable predicates instead of exploding into `findByStatusAndCustomerAndCreatedBetween(...)` repository methods.

⚡ Performance note

OFFSET pagination degrades on deep pages: `OFFSET 100000` makes the database scan and discard 100k rows before returning 20. For large or infinite feeds, prefer **keyset (cursor) pagination** — `WHERE created_at < :lastSeen ORDER BY created_at DESC LIMIT 20` — which uses the index and stays constant-time no matter how deep the client scrolls. Offset paging is fine for admin tables of thousands; keyset is what powers social feeds of millions.

🗄 Database insight

Every paged query silently runs two SQL statements: the `SELECT ... LIMIT` for the slice and a `SELECT COUNT(*)` for `totalElements`. On a huge, heavily-filtered table that count can cost more than the page itself. If you do not actually need the exact total (an infinite-scroll UI rarely does), use `Slice<T>` instead of `Page<T>` — it fetches `size+1` rows to know if a next page exists and skips the count entirely.

3.6 Versioning & OpenAPI Documentation

What is it? Versioning is how your contract evolves without breaking existing clients. The three

strategies: **URI versioning** (`/api/v1/orders` — most visible, most common, trivially explorable in a browser), **header versioning** (`Accept: application/vnd.company.v2+json` — cleaner URLs, harder to test by hand), and query-param versioning (rare). Pick one and be consistent; URI versioning is the pragmatic default and what ShopFlow uses.

When do you actually bump the version? Only on a *breaking* change — removing a field, renaming one, changing a type, tightening validation. **Additive** changes (a new optional field, a new endpoint, a new enum value clients can ignore) do not need a new version. Over-versioning is its own failure: every version is code you must keep alive for years.

⚠ Common misconception

Cutting `v2` is not free and not a fresh start — it is a promise to run *both* `v1` and `v2` in production, often for years, until every client migrates. Teams that reach for a new version on every change drown in parallel code paths. The senior instinct is the opposite: exhaust additive, backward-compatible evolution first, and treat a version bump as a last resort with a real deprecation timeline.

What is OpenAPI? OpenAPI is a machine-readable description of your API — every path, method, parameter, request/response shape and status code. `springdoc-openapi` generates it directly from your controllers and DTOs and serves an interactive **Swagger UI**. Frontend developers, QA, and API gateways all consume this spec; an undocumented endpoint is unfinished work, not a nice-to-have.

controller annotations

```
@Operation(summary = "Create a new order")
@ApiResponses({
    @ApiResponse(responseCode = "201", description = "Order created"),
    @ApiResponse(responseCode = "400", description = "Validation failed")
})
@PostMapping
public ResponseEntity<OrderResponse> create(
    @Valid @RequestBody CreateOrderRequest req) { /* ... */ }
```

application.yml — springdoc

```
springdoc:
  swagger-ui:
    path: /swagger-ui.html
  api-docs:
    path: /v3/api-docs
```

💡 How senior engineers think

Treat the OpenAPI spec as a build artifact, not documentation you write by hand. Because springdoc derives it from your code, it can never drift out of sync — and you can export `/v3/api-docs` in CI and diff it between commits to *catch breaking changes automatically* before they ship. A contract test that fails when a field disappears is worth more than any written changelog.

✓ Best practices — what experienced engineers do

- Model resources as nouns; let the HTTP method be the verb. `POST /orders`, never `/createOrder`.
- Return `201 Created` with a `Location` header pointing at the new resource.
- Never expose entities — request and response DTOs at every boundary, mapped explicitly.
- Validate on the DTO; centralise every failure into one `ErrorResponse` shape.

- Paginate list endpoints *by default*; wrap the page in your own stable envelope.
- Version only on breaking changes; document everything in OpenAPI from day one.

✖ Common mistakes beginners make

- Using `GET` for actions that mutate state — breaks caching and safety guarantees.
- Returning `200` for everything with the real status buried in the body.
- Exposing JPA entities, causing infinite serialisation loops and lazy-init errors.
- Putting business logic inside the exception handler instead of the service layer.
- Shipping an unpaginated `getAll` that works in dev and falls over in production.

4 Visualizations to Internalize

You have met five diagrams. Redraw two *from memory* — reproducing them is what moves a diagram from "recognised" to "known."

 Redraw: the request & error flow — @Valid to a shaped 400, and the happy path to a 201

 Redraw: the DTO wall — entity, mapper, response DTO — and label what must never cross it

5 Guided Coding Lab — The Order API

Goal of this lab: expose ShopFlow's first real, production-shaped endpoint — `POST /api/v1/orders` —

with a full DTO layer, validation, a central error shape, and correct status codes. You will not paste a finished solution; you will build it step by step and watch each concern click into place.

1 Create the request and response DTOs

In the `dto` package add `CreateOrderRequest`, `OrderItemRequest`, and `OrderResponse` as records, with `@NotNull`, `@NotEmpty` and `@Positive` constraints on the request side. 💡 **Why:** The DTOs are the contract; defining them first forces you to decide exactly what crosses the boundary before any wiring exists.

2 Write the mapper

Add `OrderMapper` with `toEntity(CreateOrderRequest)` and `toResponse(Order)`. Keep it a hand-written `@Component` — no `MapStruct` yet. 💡 **Why:** Hand-writing the mapper makes the entity/DTO wall visible; you literally see every field you choose to expose.

3 Build the controller with correct status codes

Add `OrderController` mapped to `/api/v1/orders`. `create` takes `@Valid @RequestBody CreateOrderRequest` and returns `ResponseEntity.created(location).body(...)` for a 201 plus `Location`. 💡 **Why:** The 201 + `Location` pair is the single clearest signal to a client that a new resource now exists and where to find it.

4 Add the centralised error handler

Create `ErrorResponse` and `ApiExceptionHandler` (`@RestControllerAdvice`). Handle `MethodArgumentNotValidException` → 400 and `OrderNotFoundException` → 404, both returning the same shape. 💡 **Why:** One advice bean means every endpoint you ever add inherits a consistent, professional error body for free.

5 Add the paginated list endpoint

Add `GET /api/v1/orders` accepting an optional `status @RequestParam` and a `Pageable`, returning a `PageResponse<OrderResponse>` via your envelope. 💡 **Why:** Wrapping `Page<T>` in your own envelope decouples the client from Spring's internal pagination JSON, which changes between versions.

6 Wire up the not-found path

Add `GET /api/v1/orders/{id}`. In the service, throw `OrderNotFoundException` when the id is absent so the advice returns a shaped 404. 💡 **Why:** A missing resource is a 404 with your error body — not a 500, not an empty 200 — and the service, not the controller, decides it.

7 Turn on OpenAPI and read your own API

Add `springdoc-openapi-starter-webmvc-ui`, start the app, open `/swagger-ui.html`, and try every endpoint from the browser. 💡 **Why:** Reading your API as an outside consumer exposes every unclear name and missing status code before a real frontend developer ever files a ticket.

💡 Production tip

Do not test only the happy path. Send `POST /api/v1/orders` with an empty `items` list and a null `customerId`, and confirm you get a 400 with both field errors in the body. The moment your error shape survives a genuinely broken request is the moment the endpoint is actually done.

6 Thinking Exercises

Reflection prompts, not quiz questions. Write a sentence or two — make your reasoning explicit.

 Think it through

A teammate proposes `POST /api/v1/orders/{id}/cancel` to cancel an order, arguing it "reads naturally." Is `cancel` a resource or an action? What is the more RESTful alternative, and when is the action-style URL actually the pragmatic choice?

 Think it through

Your `POST /orders` is not idempotent, so a client that times out and retries may create two orders. Without changing the method, how could you make order creation safe to retry? (Hint: think about a client-supplied key.)

 Think it through

You need to add a `discountCode` field that clients may send when creating an order. Does this require `/v2`? State the rule you are applying, then describe one change to the *same* endpoint that *would* force a version bump.

7 Debugging Practice

 Debugging Lab • The infinite JSON that never ends

Symptom. `GET /api/v1/orders/1` hangs, then returns a `500` with a giant response and `StackOverflowError` in the logs. The order has line items.

Evidence.

OrderController.java

```
@GetMapping("/{id}")
public Order getById(@PathVariable Long id) { // returns the ENTITY
    return orderRepository.findById(id).orElseThrow();
}
```

Work the method:

- ✓ **Observe.** The controller returns the JPA `Order` entity, which has a `List<OrderItem>` and each `OrderItem` has a back-reference to `Order`.
- ✓ **Hypothesise.** Jackson serialises `Order` → `items` → each item's `order` → its `items` → forever.
- ✓ **Verify.** Confirm the association is bidirectional and that no DTO sits between the entity and the response.
- ✓ **Fix.** Return `OrderResponse` via the mapper; the DTO is flat and cannot cycle.
- ✓ **Prevent.** Make "controllers return DTOs, never entities" a review rule — an architecture test can even fail the build if a controller signature returns an `@Entity`.

🔧 Debugging Lab • Validation that quietly does nothing

Symptom. A `POST /api/v1/orders` with `customerId: null` and an empty `items` list returns `201 Created` instead of `400`. The bad order is persisted.

Evidence.**OrderController.java**

```
@PostMapping
public ResponseEntity<OrderResponse> create(
    @RequestBody CreateOrderRequest req) { // no @Valid
    ...
}
```

- ✓ **Observe.** The constraints on the record are present, but no validation runs — the request sails through.
- ✓ **Hypothesise.** Without `@Valid` on the parameter, Spring never invokes the validator; annotations on the DTO are inert.
- ✓ **Verify.** Add `@Valid` and resend the same body; expect `MethodArgumentNotValidException` and a `400`.
- ✓ **Fix.** Add `@Valid` before `@RequestBody`; for a nested collection, ensure the element type is also marked (`@Valid List<OrderItemRequest>`).
- ✓ **Prevent.** A test that posts a deliberately invalid body and asserts `400` — validation you never test is validation you do not have.

8**Mini-Labs**

🔧 Mini-Lab • Beginner

25 min

Goal. Make correct status codes visible.

Context. A controller returning a resource that may or may not exist.

Requirements. Return `200` on a hit, a shaped `404` on a miss, and `201 + Location` on create.

Hints. Use `ResponseEntity.created(uri)`; throw a domain exception for the miss and let the advice map it.

Expected outcome. `curl -i` shows the exact status line and `Location` header for each path.

Common pitfalls. Returning `200` with a `null` body for a miss instead of a real `404`.

🔧 Mini-Lab • Intermediate

45 min

Goal. Ship one consistent error shape across three failure types.

Context. Validation errors, not-found, and a duplicate-order conflict.

Requirements. One `@RestControllerAdvice` mapping `MethodArgumentNotValidException` → `400`, `OrderNotFoundException` → `404`, and a `DuplicateOrderException` → `409`, all returning the same `ErrorResponse`.

Hints. Give each a distinct `code` string; keep `message` human-readable and safe.

Expected outcome. Three different endpoints, three different status codes, one identical body structure.

Common pitfalls. Putting the "is this a duplicate?" decision in the advice instead of the service.

🔧 Mini-Lab • Production-style

70 min

Goal. A composable, paginated, filtered list endpoint.

Context. Ops need `GET /api/v1/orders` filterable by `status` and `customerId`, both optional, always paginated.

Requirements. Use JPA `Specifications` to compose the optional filters; return a `PageResponse<OrderResponse>`; default to `size=20, sort=createdAt, desc`.

Hints. Build each filter as a nullable `Specification` and `and()` together only the non-null ones.

Expected outcome. Any combination of the two filters works, and the envelope reports correct `totalElements/totalPages`.

Common pitfalls. Returning the raw `Page<Order>` (leaking entities and an unstable JSON shape) instead of your envelope of DTOs.

9**Engineering Notes**

Judgment from engineers who have carried a pager. Read these as reasoning, not trivia.

💡 How senior engineers think

The controller is the thinnest layer in the whole system. Its entire job is: validate, map in, call one service method, map out, choose a status code. The moment you see a loop, an `if` about business state, or a repository call in a controller, the logic has leaked upward. A thin controller is not laziness — it is what keeps your API contract independent of how the work actually gets done.

 Interview insight

"How would you design centralised exception handling in Spring?" The answer names the machinery and the boundary: a single `@RestControllerAdvice` with per-exception `@ExceptionHandler` methods, each mapping a domain exception to a status code and one shared `ErrorResponse`. Then the judgment: the advice *translates*, it never decides — business rules throw from the service; the advice only shapes the reply.

 Performance note

The cheapest API performance win is not caching — it is not sending data. A response DTO that projects only the four fields a client renders beats returning a fat entity graph on every axis: less JSON to serialise, less to transfer, fewer lazy loads, smaller GC pressure. "Return less" scales further than "return faster."

 Production tip

Give every error response a correlation id (a request-scoped UUID, also logged server-side and returned in a header). When a user reports "I got an error," that id turns a needle-in-a-haystack log search into a single grep. It costs one filter to add and pays for itself the first time production breaks.

10 Build-Along Project — ShopFlow

Days 1–3 gave ShopFlow its skeleton: a layered Spring Boot module (Day 1), raw-SQL fluency (Day 2), and JPA entities plus repositories — `Order`, `OrderItem`, and their queries (Day 3). Today you put a real, professional face on that domain: the Order API.

 Build-Along • Day 04 — the Order API

- **DTO layer.** `CreateOrderRequest`, `OrderItemRequest`, `OrderResponse` records in `dto`, plus a hand-written `OrderMapper` — entities never cross the boundary.
- **POST `/api/v1/orders`.** `@Valid @RequestBody`, delegates to `OrderService`, returns `201 Created` with a `Location` header pointing at the new order.
- **GET `/api/v1/orders?status=&page=&size=`.** Optional `status` filter and a `Pageable`, returning a `PageResponse<OrderResponse>` envelope with correct totals.
- **GET `/api/v1/orders/{id}`.** Returns the order DTO, or a shaped `404` via `OrderNotFoundException` thrown from the service.
- **Central error shape.** One `@RestControllerAdvice` mapping validation → `400`, not-found → `404`, conflict → `409`, all returning the same `ErrorResponse`.
- **OpenAPI.** Add `springdoc`, annotate the endpoints, and confirm `/swagger-ui.html` lists all three paths with their status codes.

Definition of done: all three endpoints work with correct status codes on every path (including validation `400` and not-found `404`); every response — success or failure — is a DTO with a stable shape; the list endpoint is paginated by default; and the whole API is browsable in Swagger UI. ShopFlow now has a contract the rest of the world can build on.

11 Chapter Summary

Key takeaways

- ✓ An API is a contract; DTOs, validation, error shape, pagination and versioning all exist to keep it stable.
- ✓ Resources are nouns, methods are verbs, and status codes are the machine-readable outcome — get all three right.
- ✓ Never return entities: map to DTOs to avoid schema coupling, leaked fields, and serialisation cycles.
- ✓ Validate at the boundary with `@Valid`; centralise every failure into one `@RestControllerAdvice` shape.
- ✓ Paginate lists by default behind your own envelope; version only on breaking changes; document with OpenAPI.

Things to remember

- `@Valid` is what *triggers* validation — the annotations on the DTO do nothing without it.
- `201 Created` carries a `Location` header; `404` and `400` carry your `ErrorResponse`, never a stack trace.

Common pitfalls (last look)

- Exposing entities → infinite JSON & lazy-init 500s. • Missing `@Valid` → validation silently skipped.
- Unpaginated `getAll` → production incident at scale. • Business logic in the advice → tangled failures.

📋 One-Page Cheat Sheet

Methods — GET safe+idempotent • PUT/DELETE idempotent • POST neither • PATCH partial.

Status codes — 201+Location on create • 400 validation • 401 who? • 403 not allowed • 404 missing • 409 conflict.

DTO rule — controllers return DTOs, never entities — map explicitly at the boundary.

Validation — constraints on the DTO + `@Valid` on the parameter + one `@RestControllerAdvice`.

Pagination — accept `Pageable`, wrap `Page<T>` in your own `PageResponse` envelope of DTOs.

Versioning — URI `/v1`; bump only on breaking changes; additive changes need no new version.

Docs — springdoc generates OpenAPI + Swagger UI from your controllers — an undocumented endpoint is unfinished.

👤 Interview questions to master

- › What is the difference between `PUT` and `PATCH`?
- › Why is idempotency important, and which HTTP methods are idempotent?
- › How do you version a public API without breaking existing clients?
- › How would you design centralised exception handling in Spring?
- › Why should you never return JPA entities directly from a controller?
- › 401 versus 403 — what is the actual difference?

Suggested further reading

- *Spring Framework Reference* — "Web MVC" (controllers, `ResponseEntity`, exception handling).
- *RFC 9110* — HTTP Semantics: the authoritative definition of methods and status codes.
- *springdoc-openapi* docs and the Stripe / GitHub public API references as models of contract discipline.

Key idea

What you will learn next (Day 5). Your API now speaks clearly — but it lets anyone in. Tomorrow you put a guard at the door: Spring Security, the filter chain that runs *before* your controllers, authentication versus authorisation, and how a `401` and a `403` actually get produced. The error shapes you built today are exactly what the security layer will reuse.

DAY 05 • SECURITY

Spring Security

Authentication, authorization & JWT — the filter chain, end to end

05

🕒 Estimated study time: 9 hours

🎯 By the end of this day, you will be able to...

- Explain the Security Filter Chain as an ordered pipeline of servlet filters, and say exactly *where* authentication and authorization each happen.
- Implement stateless JWT authentication end to end: registration, BCrypt storage, login, a custom validating filter, and protected routes.
- Configure modern Spring Security 6 the component way — a `SecurityFilterChain` bean, not the deprecated `WebSecurityConfigurerAdapter`.
- Apply role- and authority-based authorization deliberately, so a `CUSTOMER` sees only their own orders and an `ADMIN` manages everything.
- Reason about CORS and CSRF as *different* problems, and disable CSRF for a stateless JWT API as a decision, not a reflex.

1 The Big Picture

What this day solves. After Day 4 you have a working, documented Order API: clients can create orders, list them, fetch one by id. There is exactly one problem — *anyone on the internet can do all of that*. There is no notion of *who* is calling, and no rule about *what* they are allowed to touch. Today you close that gap. By the end, a request without a valid token gets `401`, a request from the wrong user gets `403`, and every protected handler runs only for an identity Spring has already proven.

Why backend engineers need it. Security is the one layer where a single missing check becomes a breach, a headline, and a regulatory fine. Yet most beginners treat Spring Security as an incantation — copy a config until the red errors stop, disable everything that complains. That approach ships holes. The engineers who are trusted with production are the ones who can *name* which filter authenticated the request, why CSRF is off, and what `hasRole('ADMIN')` actually compares against. This day builds that mental model.

Where it fits in a Spring Boot application. Recall Day 1: filters run *outside* the Spring MVC context, before `DispatcherServlet` ever sees the request. Spring Security is precisely a chain of those filters, inserted at the very front of the pipeline. Nothing in your controllers changes — the security chain runs first, establishes (or rejects) an identity, and only then hands the request onward to the Day 4 code you already wrote.

How it connects to previous chapters. You are wrapping, not rewriting. The controllers, services, and repositories from Days 2–4 stay exactly as they are; you add a filter in front and a few annotations on top. The `User` you authenticate becomes a JPA entity (Day 2–3), stored with a hashed password, and the login endpoint is just another controller returning JSON (Day 4).

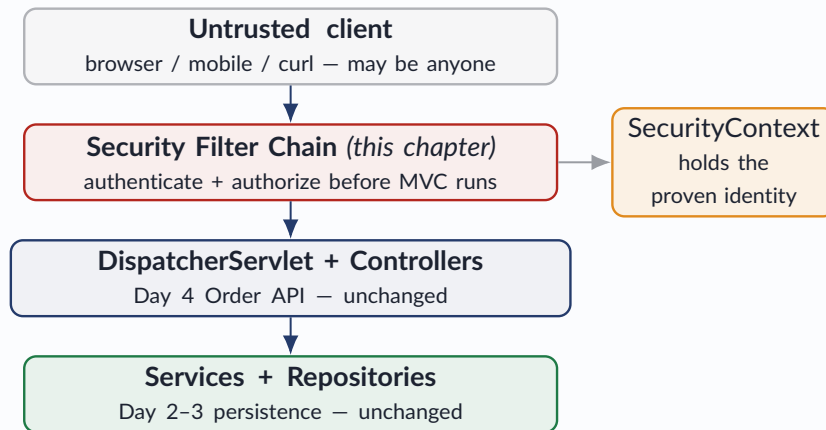


Fig 5.1 — Where it fits: the security filter chain wraps the untrusted internet around the Day 4 API you already built. Your controllers do not change.

Real company examples. Every bank, every marketplace, every SaaS built on the JVM runs some form of this chain. Netflix's Zuul edge gateways authenticate at the boundary; Auth0, Okta, and Keycloak exist to issue exactly the tokens you will validate today. When you read "sign in with Google" it is OAuth2 delegation — a cousin of what you build, covered in one paragraph in Part 3.

Key idea

Spring Security is not a wall you switch on. It is a **chain of filters** that answers two questions in order: *who are you?* (authentication) and *are you allowed?* (authorization). Everything today — JWT, BCrypt, roles, CORS — is a way of answering one of those two questions well. Hold the two questions apart and the whole framework snaps into focus.

2 Concept Map

Read the map as two lanes. The top lane is *authentication* — turning credentials into a token. The bottom lane is *authorization* — turning a token back into a proven identity and a yes/no decision on each request.

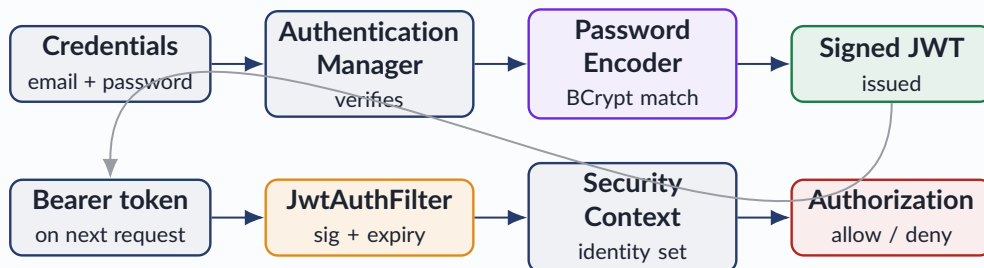


Fig 5.2 — The Day 5 concept map. Top lane: login issues a token. Bottom lane: every later request re-proves identity and is checked against the rules.

Terminology to anchor: **Filter Chain** → **AuthenticationManager** → **Authentication** → **SecurityContext** → **GrantedAuthority** → **AuthorizationFilter** → **AccessDecision**.

3 Guided Learning

3.1 Authentication vs Authorization

What is it? Two distinct questions the framework keeps deliberately separate. **Authentication** answers "who are you?" — it verifies a claimed identity against evidence (a password, a signed token). **Authorization** answers "what are you allowed to do?" — it checks, *once identity is known*, whether that identity may perform this action.

Why does it exist? Collapsing the two is the classic beginner bug: a controller that checks `if (user.isAdmin())` by hand, scattered across dozens of methods, with no single place that establishes *who the user even is*. Spring splits the concerns: an `AuthenticationManager` (and its `AuthenticationProviders`) does the first job; declarative rules — `authorizeHttpRequests` and `@PreAuthorize` — do the second.

How does it work internally? A successful authentication produces an `Authentication` object holding the **principal** (the user), their **credentials**, and a set of `GrantedAuthority` values (roles and permissions). That object is stored in the `SecurityContext`, which is bound to the current thread for the life of the request. Every later authorization check simply reads it — it never re-verifies the password.

⚠ Common misconception

"Authentication and authorization are basically the same check." They are not, and conflating them causes real holes. A valid token proves *who* you are; it says nothing about whether you may delete product 42. A system can authenticate you perfectly and still — correctly — return 403 because you lack

the authority. 401 means "I don't know who you are"; 403 means "I know exactly who you are, and the answer is no."

3.2 The Security Filter Chain

What is it? Spring Security is implemented as an ordered list of servlet `Filters`, inserted ahead of `DispatcherServlet` by a single `FilterChainProxy`. Each filter has one job; the request falls through them in a fixed order until one either rejects it or lets it reach your controller.

Why does it exist? Because security is cross-cutting. You do not want every controller to remember to check a token, translate exceptions to 401, and enforce CORS. A pipeline does each concern exactly once, in a known order, for every request – including requests to endpoints you forgot existed.

How does it work internally? The chain runs before your code. For a token-based API the filters that matter are: an entry `CorsFilter` (handles the browser's preflight and origin rules), your custom JWT filter (validates the `Authorization` header and populates the `SecurityContext`), the built-in `UsernamePasswordAuthenticationFilter` (form login – present but unused here), the `ExceptionTranslationFilter` (turns an authentication failure into 401 and an access-denied into 403), and finally the `AuthorizationFilter` (the last gate that enforces your URL rules). You insert your JWT filter *before* `UsernamePasswordAuthenticationFilter` with `addFilterBefore`.

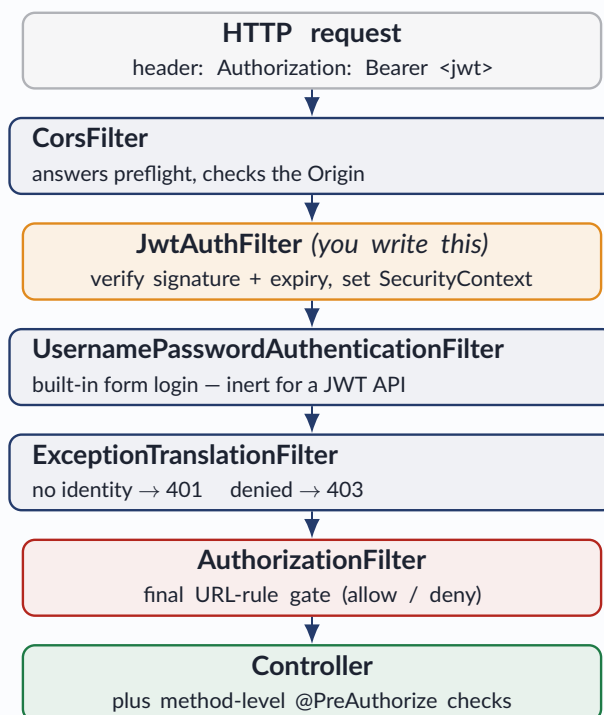


Fig 5.3 – The filter chain as a vertical pipeline. Your custom JWT filter runs early and sets the identity; the `AuthorizationFilter` is the final gate before the controller.

👁 Behind the scenes

There is not one global filter chain but a *list* of them, each guarded by a request matcher; `FilterChainProxy` picks the first whose matcher fits the URL. That is how you can have one relaxed chain for `/actuator/health` and a strict one for `/api/**`. For this workbook a single chain is enough — but knowing there can be several explains why a rule "that should apply" sometimes does not: a different chain matched first.

💡 How senior engineers think

When a request behaves unexpectedly, a senior enables `logging.level.org.springframework.security=DEBUG` and reads the printed chain: every filter that ran, in order, and the exact point a request was rejected. "Security is a black box" almost always means "I have not turned on the log that narrates it."

3.3 Password Hashing with BCrypt

What is it? BCrypt is a deliberately slow, salted, adaptive hash function purpose-built for passwords. You never store a password — you store its BCrypt hash, and at login you *compare* hashes.

Why does it exist? Plaintext passwords leak in every database breach; fast hashes like MD5 or SHA-256 are only marginally better, because an attacker can compute billions of guesses per second and precompute **rainbow tables**. BCrypt defeats both: a per-password random **salt** (embedded in the hash itself) makes rainbow tables useless, and a tunable **work factor** (cost) makes each guess expensive enough that brute force stops being economical.

How does it work internally? A BCrypt hash string encodes everything needed to verify it: the algorithm version, the cost, the 128-bit salt, and the derived hash — e.g. `$2b$12$R9h...`. Because the salt is random, hashing the same password twice yields two different strings, so you cannot compare by equality. You call `PasswordEncoder.matches(raw, stored)`, which re-extracts the salt and cost from `stored`, re-hashes `raw` the same way, and compares in constant time.

SecurityConfig.java – the encoder bean

```
@Bean
PasswordEncoder passwordEncoder() {
    // cost 12: ~250ms per hash on modern hardware.
    // Higher = safer but slower; tune to your login throughput.
    return new BCryptPasswordEncoder(12);
}
```

AuthService.java – register and verify

```
@Service
public class AuthService {
    private final UserRepository users;
    private final PasswordEncoder encoder;

    public AuthService(UserRepository users, PasswordEncoder encoder) {
        this.users = users;
        this.encoder = encoder;
    }

    public void register(RegisterRequest req) {
        if (users.existsByEmail(req.email()))
            throw new EmailAlreadyUsedException(req.email());
    }
}
```

```
// store the HASH, never the raw password
var user = new User(req.email(),
                    encoder.encode(req.password()),
                    Role.CUSTOMER);
users.save(user);
}
```

Security warning • never invent your own hashing

Do not reach for `MessageDigest`, do not concatenate a "secret pepper" and SHA-256, do not roll a clever scheme. Password hashing is a solved problem with sharp edges, and every home-grown variant loses to it. Use `BCryptPasswordEncoder` (or Argon2 via `Argon2PasswordEncoder`) and let it own the salt, the cost, and the constant-time compare.

Performance note

Cost is a dial, not a constant. BCrypt cost 12 is roughly 4× slower than cost 10. That slowness is the point at rest, but on a hot login path it caps throughput — a synchronous cost-14 encoder can bottleneck a login storm. Pick the highest cost your p99 login latency budget tolerates, and revisit it as hardware gets faster (the whole point of an *adaptive* hash).

3.4 JWT Authentication, End to End

What is it? A JWT (JSON Web Token) is a signed, self-contained credential. It has three Base64url parts joined by dots — `header.payload.signature` — where the payload carries **claims** (subject, roles, issued-at, expiry) and the signature proves the token was minted by your server and not tampered with.

Why does it exist? Traditional auth keeps a session in server memory or a shared store; every request costs a lookup, and scaling horizontally means sharing that session state across nodes. A JWT is **stateless**: the server verifies the signature and expiry with a key it already holds, learning who the user is *without any database or session lookup*. Any node can serve any request. That property is what makes JWT the default for horizontally scaled APIs.

How does it work internally? At login the server verifies the BCrypt hash, then signs a token (HMAC with a shared secret, or RSA/EC with a private key) and returns it. On every subsequent request the client sends `Authorization: Bearer <jwt>`; your filter verifies the signature with the same key, checks the expiry, reads the subject and roles, and builds an `Authentication` to place in the `SecurityContext`. No server-side state is consulted.

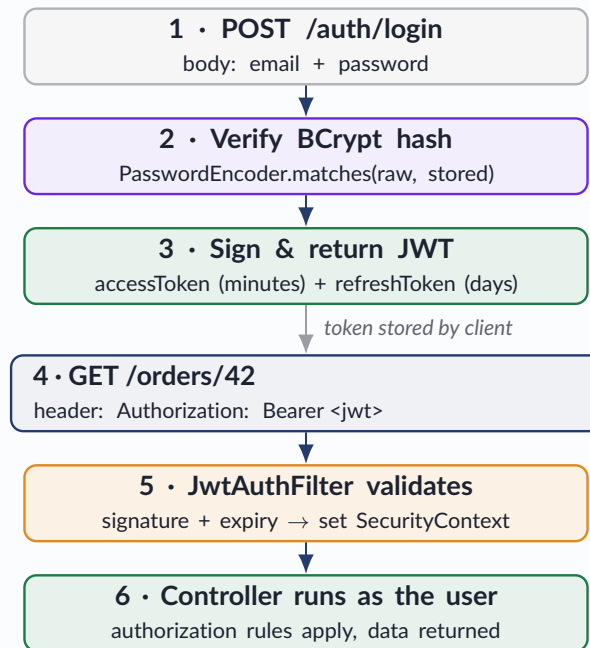


Fig 5.4 – Stateless JWT auth. Login (top) issues a signed token; every later request (bottom) re-proves identity from the token alone – no session, no lookup.

JwtService.java – issue and parse (jjwt 0.12 API)

```

@Service
public class JwtService {
    private final SecretKey key;        // >= 256-bit secret, from config
    private final long accessTtlMs;    // e.g. 15 minutes

    public String issue(User user) {
        Instant now = Instant.now();
        return Jwts.builder()
            .subject(user.getEmail())
            .claim("roles", List.of(user.getRole().name()))
            .issuedAt(Date.from(now))
            .expiration(Date.from(now.plusMillis(accessTtlMs)))
            .signWith(key)
            .compact();
    }

    public Jws<Claims> parse(String token) {
        // throws JwtException on bad signature OR expiry
        return Jwts.parser().verifyWith(key).build()
            .parseSignedClaims(token);
    }
}

```

JwtAuthFilter.java – runs once per request

```

public class JwtAuthFilter extends OncePerRequestFilter {
    private final JwtService jwt;
    private final UserDetailsService users;
    // constructor omitted
}

```

```

@Override
protected void doFilterInternal(HttpServletRequest req,
    HttpServletResponse res, FilterChain chain)
    throws ServletException, IOException {
    String header = req.getHeader("Authorization");
    if (header == null || !header.startsWith("Bearer ")) {
        chain.doFilter(req, res); // no token: let the rules decide
        return;
    }
    try {
        var claims = jwt.parse(header.substring(7)).getPayload();
        var user = users.loadUserByUsername(claims.getSubject());
        var auth = new UsernamePasswordAuthenticationToken(
            user, null, user.getAuthorities());
        SecurityContextHolder.getContext().setAuthentication(auth);
    } catch (JwtException ex) {
        // invalid/expired: leave context empty -> 401 downstream
    }
    chain.doFilter(req, res);
}
}

```

⚠ Common misconception

"A JWT is encrypted, so it is safe to put secrets in it." It is *signed*, not encrypted. The payload is Base64url – anyone holding the token can decode and read every claim in one line of code. The signature guarantees *integrity* (nobody altered it), never *confidentiality*. Never put a password, a full credit-card number, or sensitive PII in a JWT.

👤 Security warning • the revocation trade-off

The same statelessness that makes JWTs scale means you *cannot* easily revoke one before it expires – there is no server record to delete. If a token is stolen, it is valid until expiry. Mitigations: keep the access token short-lived (minutes) and pair it with a longer-lived **refresh token**; for the rare forced-logout, maintain a small server-side blocklist of token ids (`jti`) checked on each request. Choosing "short expiry + refresh" over "look up every token" is the whole design bargain.

3.5 Modern Configuration (Spring Security 6+)

What is it? In Spring Security 6 you configure the chain by declaring a `SecurityFilterChain` @Bean. The old `WebSecurityConfigurerAdapter` base class is *removed* – code that extends it will not compile against Spring Boot 3.

Why does it exist? The component-based style is composable and testable: the chain is just a bean you can inspect, and configuration reads as data (a fluent builder) rather than a set of overridden template methods. It also makes multiple chains natural – each is another bean.

How does it work internally? `HttpSecurity` is a builder; each lambda configures one aspect (CSRF, CORS, session policy, URL rules) and `.build()` produces the immutable filter chain. `addFilterBefore` splices your JWT filter into the fixed position ahead of form login.

SecurityConfig.java – the whole chain in one bean

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity           // turns on @PreAuthorize

```

```
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http,
        JwtAuthFilter jwtFilter) throws Exception {
        return http
            .csrf(csrf -> csrf.disable()) // stateless: see 3.6
            .cors(Customizer.withDefaults()) // use the CORS bean
            .sessionManagement(sm -> sm
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/v1/products/**")
                    .permitAll()
                .requestMatchers("/api/v1/products/**").hasRole("ADMIN")
                .requestMatchers("/api/v1/orders/**").authenticated()
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter,
                UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}
```

Why order matters in the rules. `authorizeHttpRequests` evaluates matchers top-to-bottom and the *first* match wins. The `GET /products` rule must come *before* the broader `/products/**` admin rule, or reads would demand `ADMIN` too. Put specific rules above general ones, and end with `anyRequest().authenticated()` so nothing is accidentally public.

Production tip

Default to closed. `anyRequest().authenticated()` as the last rule means a new endpoint you add next month is protected *by default* — you must consciously open it with `permitAll()`. The opposite habit (default open, lock down individually) guarantees that one forgotten endpoint eventually ships wide open.

3.6 Authorization: Roles, Authorities, CORS & CSRF

Roles vs authorities. A **role** is a coarse label (`ADMIN`, `CUSTOMER`); an **authority** (permission) is fine-grained (`ORDER_REFUND`). `hasRole('ADMIN')` is sugar for `hasAuthority('ROLE_ADMIN')` — Spring silently prepends `ROLE_`. Production systems bundle authorities into roles rather than sprinkling role checks everywhere.

Method-level authorization on controllers

```
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/api/v1/products/{id}")
public void delete(@PathVariable Long id) { /* admin only */ }

@PreAuthorize("hasAuthority('ORDER_REFUND')")
@PostMapping("/api/v1/orders/{id}/refund")
public void refund(@PathVariable Long id) { /* fine-grained */ }

// CUSTOMER may read only their OWN order; ADMIN reads any
@PreAuthorize("hasRole('ADMIN') or @orderGuard.owns(#id, authentication)")
@GetMapping("/api/v1/orders/{id}")
public OrderView getOrder(@PathVariable Long id) { ... }
```

CORS — a browser rule, not a firewall. CORS (Cross-Origin Resource Sharing) is enforced by the *browser*: it decides whether JavaScript loaded from `https://app.shopflow.com` may read a response from your API on a different origin. You configure which origins are trusted; you do not "fix" a CORS error by disabling it. A non-browser client (curl, Postman, your mobile app) ignores CORS entirely — so CORS is never your access control, only your browser policy.

CorsConfig.java — one trusted frontend origin

```
@Bean
CorsConfigurationSource corsConfigurationSource() {
    var cfg = new CorsConfiguration();
    cfg.setAllowedOrigins(List.of("https://app.shopflow.com"));
    cfg.setAllowedMethods(
        List.of("GET", "POST", "PUT", "PATCH", "DELETE"));
    cfg.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    cfg.setAllowCredentials(true); // NEVER combine with origin "*"
    cfg.setMaxAge(3600L); // cache preflight for 1 hour
    var src = new UrlBasedCorsConfigurationSource();
    src.registerCorsConfiguration("/**", cfg);
    return src;
}
```

CSRF — and why a JWT API disables it deliberately. CSRF (Cross-Site Request Forgery) attacks abuse an *ambient credential* the browser attaches automatically — classically a session cookie. A malicious page makes your browser fire a request to your bank; the browser helpfully includes the session cookie, and the bank cannot tell it was not you. CSRF tokens defend against that. But a JWT lives in the `Authorization` header and is attached by *your* JavaScript explicitly, never automatically by the browser. There is no ambient credential to forge, so CSRF protection guards nothing — which is why a stateless, header-token API turns it off on purpose. Disable CSRF because your auth is header-based and stateless, not because it was throwing errors.

Interview insight

Expect: "Why do JWT APIs disable CSRF?" The answer that lands names the mechanism: "CSRF exploits credentials the browser sends *automatically*, like cookies. A JWT is sent *explicitly* in a header by our own code, so there is no ambient credential for another site to ride on — CSRF protection would defend against an attack that cannot happen here." Then add the caveat: "the moment we store the JWT in a cookie, CSRF is back on the table."

Best practices — what experienced engineers do

- Keep access-token expiry short (minutes); use refresh tokens for longer sessions.
- Store the signing key outside source control — environment variable or a secrets manager.
- Whitelist CORS origins explicitly; never pair `allowCredentials(true)` with origin `*`.
- Validate the token on every request, not just at login — assume replay and theft.
- End URL rules with `anyRequest().authenticated()` so new endpoints are closed by default.

Common mistakes beginners make

- Disabling CSRF and CORS "to make it work" without knowing what each protects.
- Storing JWTs in `localStorage` and being surprised when XSS steals them.
- Putting passwords or full PII in the JWT payload — it is Base64, not encryption.

- Never rotating the signing key, or issuing tokens with no expiry at all.
- Storing `ADMIN` but writing `hasRole('ADMIN')` and wondering why every check fails.

4 Visualizations to Internalize

You have met four diagrams. Redraw two of them *from memory* below — reproducing them is what moves a diagram from "recognised" to "owned."

 Redraw: the filter chain, in order, marking where your JWT filter sits and where 401 vs 403 is decided

 Redraw: the two-phase JWT flow — login issues a token, later requests re-prove identity

5 Guided Coding Lab — Secure the Order API

Goal of this lab: take the wide-open Day 4 Order API and lock it down — registration, login issuing a JWT, a custom validating filter, role rules, and CORS for one origin. Build it step by step; do not paste a finished project.

1 Model the User and Role

Create a `User` JPA entity (`id`, `email` unique, `passwordHash`, `role`) and a `Role` enum (`CUSTOMER`, `ADMIN`). Implement `UserDetailsService` to load a user by email and map the role to `ROLE_` authorities. 💡 *Why:* Spring Security needs a `UserDetails` view of your user; wiring it now lets the framework do authentication instead of you hand-rolling it.

2 Add the BCrypt encoder and a register endpoint

Expose `POST /api/v1/auth/register`. Hash with `PasswordEncoder.encode` before saving; reject duplicate emails with `409`.  **Why:** The password must be a hash the moment it touches the database — proving the "never store plaintext" rule in code, not just in a slide.

3 Issue a JWT at login

Add `POST /api/v1/auth/login`: verify with `matches`, and on success return an access token (short TTL) and a refresh token. Load the signing secret from configuration, never a literal.  **Why:** This is the authentication half — turning verified credentials into a portable, signed identity the rest of the system trusts.

4 Write the `JwtAuthFilter`

Extend `OncePerRequestFilter`. Read the `Authorization` header, verify the token, and on success set an `Authentication` in the `SecurityContext`. On failure, leave the context empty and continue — do not throw.  **Why:** Leaving the context empty (rather than throwing) lets the chain's `ExceptionTranslationFilter` produce a clean `401`, keeping error handling in one place.

5 Declare the `SecurityFilterChain`

Write the `SecurityConfig` bean: disable CSRF, enable CORS, set `STATELESS` sessions, add your filter before `UsernamePasswordAuthenticationFilter`, and write URL rules — `/auth/**` public, `GET /products` public, other `/products/**` admin, `/orders/**` authenticated.  **Why:** This is the single source of truth for who may hit which URL; getting the rule *order* right is the lab's real lesson.

6 Add method-level ownership rules

Turn on `@EnableMethodSecurity`. Annotate the "view one order" handler so a `CUSTOMER` sees only their own order and an `ADMIN` sees any, via a small `@orderGuard.owns(...)` bean.  **Why:** URL rules are coarse; real apps need "this row belongs to this user" checks, and method security is where that lives.

7 Lock down CORS and prove it

Register the `CorsConfigurationSource` for one origin. From a disallowed origin, confirm the browser blocks the response; from the trusted origin, confirm it succeeds.  **Why:** Configuring CORS is easy; proving a bad origin is actually rejected is what turns "I think it works" into "I verified it."

Production tip

Test the whole flow with real HTTP before you trust it: register, login, copy the token, and call `GET /orders` with and without the `Authorization` header. Watch the status codes — `200` with a valid token, `401` without, `403` for the wrong user. Reading those three codes back is your end-to-end proof the chain works.

6 Thinking Exercises

Reflection prompts, not quiz questions. Write a sentence or two — the goal is to make your reasoning explicit.

Think it through

A stolen JWT is valid until it expires, because there is no server-side record to delete. Given that, argue for a specific access-token lifetime for ShopFlow. What does making it *shorter* cost,

and what does making it *longer* risk?

Think it through

Your teammate stores the JWT in `localStorage` "because it is convenient." Explain the concrete attack that makes this dangerous, and the trade-off of the `httpOnly` cookie alternative — including what it drags back into scope (hint: revisit CSRF).

Think it through

The URL rule `/orders/**` requires only `authenticated()`, yet a customer must not read another customer's order. Why is the URL rule insufficient here, and what layer must enforce the "own order" constraint instead?

7 Debugging Practice

🔧 Debugging Lab • A valid token, the right role, and still 403

Symptom. A user with role `ADMIN` logs in successfully, sends a valid token to `DELETE /products/9`, and gets `403 Forbidden`. Authentication clearly worked; authorization refuses.

Evidence.

how the authority was built

```
// in UserDetailsService
var authorities = List.of(new SimpleGrantedAuthority("ADMIN"));
// on the controller
@PreAuthorize("hasRole('ADMIN')") // expects "ROLE_ADMIN"
public void delete(@PathVariable Long id) { ... }
```

Work the method:

- ✓ **Observe.** The 401 vs 403 split proves identity is fine; the failure is in the authority comparison.
- ✓ **Hypothesise.** `hasRole('ADMIN')` checks for the authority `ROLE_ADMIN`, but the user was granted the bare authority `ADMIN` — they never match.
- ✓ **Verify.** Log the principal's authorities; you see `[ADMIN]`, not `[ROLE_ADMIN]`.
- ✓ **Fix.** Grant `new SimpleGrantedAuthority("ROLE_ADMIN")` (or use `hasAuthority('ADMIN')` consistently). Pick one convention and hold it.
- ✓ **Prevent.** Centralise the role→authority mapping in one method and add a test asserting an `ADMIN` principal carries `ROLE_ADMIN`.

🔧 Debugging Lab • Works in Postman, blocked in the browser

Symptom. The frontend at `https://app.shopflow.com` cannot call the API — the console shows a CORS error — yet the identical request from Postman returns 200.

Evidence.

browser console

```
Access to fetch at 'https://api.shopflow.com/api/v1/orders'
from origin 'https://app.shopflow.com' has been blocked by CORS
policy: No 'Access-Control-Allow-Origin' header is present.
```

- ✓ **Observe.** Postman is not a browser, so it ignores CORS; the browser enforces it. The difference tells you this is a CORS config issue, not an auth issue.
- ✓ **Hypothesise.** Either no `CorsConfigurationSource` is registered, or the browser's preflight `OPTIONS` request is being rejected by the security chain before CORS can answer it.
- ✓ **Verify.** Watch the network tab for a failing `OPTIONS` preflight, and confirm whether `.cors(Customizer.withDefaults())` is present in the chain.
- ✓ **Fix.** Register the `CorsConfigurationSource` bean with the trusted origin and enable `.cors(...)` so preflight is answered before authorization runs.
- ✓ **Prevent.** Add an integration test that sends an `OPTIONS` preflight from the allowed origin and asserts the `Access-Control-Allow-Origin` header comes back.

8 Mini-Labs

🧪 Mini-Lab • Beginner

25 min

Goal. See BCrypt's salt with your own eyes.

Context. You want proof that identical passwords hash to different strings, yet still verify.

Requirements. Encode the same password twice; print both hashes; call `matches(raw, hash)` on each.

Hints. Use `new BCryptPasswordEncoder(12)`; note the `$2b$12$` prefix encodes version and cost.

Expected outcome. Two different hash strings, both returning `true` from `matches`.

Common pitfalls. Comparing hashes with `equals` and concluding BCrypt is broken — that is the lesson: you must use `matches`.

Mini-Lab • Intermediate

45 min

Goal. Enforce role-based access on three endpoints and prove the boundary.

Context. A `CUSTOMER` must be denied admin actions; an `ADMIN` allowed.

Requirements. Add `@PreAuthorize` to create-product (ADMIN), delete-product (ADMIN), and list-my-orders (any authenticated). Log in as each role and hit all three.

Hints. Enable `@EnableMethodSecurity`; remember `hasRole('ADMIN')` looks for `ROLE_ADMIN`.

Expected outcome. A `403` for the customer on admin routes, `200` for the admin, `200` for both on their own orders.

Common pitfalls. Forgetting `@EnableMethodSecurity`, so the annotations are silently ignored and everything is allowed.

Mini-Lab • Production-style

70 min

Goal. Implement refresh-token rotation with reuse detection.

Context. Access tokens expire in minutes; a refresh endpoint issues a new pair without re-login.

Requirements. `POST /auth/refresh` validates the refresh token, issues a new access *and* a new refresh token, and invalidates the old refresh token. If an already-used refresh token is presented, revoke the whole family (theft signal).

Hints. Persist refresh tokens (id + user + used flag); rotation means each refresh is single-use.

Expected outcome. A seamless renew for honest clients; a replayed refresh token forces re-login for everyone in that chain.

Common pitfalls. Making refresh tokens as long-lived and unrotated as a password — a stolen one then never expires in practice.

9

Engineering Notes

Judgment from engineers who have carried a pager. Read these as reasoning, not trivia.

How senior engineers think

Return `404` instead of `403` for resources a user may not even know exist. Telling an attacker "that order exists, you just can't see it" (`403`) leaks structure; "no such order" (`404`) leaks nothing. Distinguish "forbidden and I'll admit it exists" from "forbidden, and I won't confirm it" per resource — it is a deliberate design call, not an accident.

Security warning • secrets never live in the jar

The signing key is the master credential of your whole auth system — anyone holding it can mint valid tokens for any user. It belongs in an environment variable or secrets manager injected at deploy time (Day 8 Compose, Day 12 CI), never in `application.yml` in git. A key committed to history is compromised even after you delete it.

 Interview insight

Expect: "How does the Spring Security filter chain process a request?" Do not list class names blindly. Narrate the journey: CORS answers preflight, your JWT filter reads the header and sets the `SecurityContext`, `ExceptionTranslationFilter` maps failures to 401/403, and `AuthorizationFilter` is the final gate. Naming the *order* and *each filter's one job* is what signals depth.

 Performance note

Stateless JWT auth removes a per-request session lookup — that is its performance gift. But if your JWT filter reloads the full `UserDetails` from the database on every request, you have quietly reintroduced the lookup you were avoiding. For hot paths, trust the roles embedded in the token (they were signed) rather than re-querying, and accept the small staleness window as the cost of the stateless bargain.

10 Build-Along Project — ShopFlow

By Day 4, ShopFlow had a working, documented Order API — but wide open: any caller could create, read, and manage every order. Today you put a lock on the door without touching the rooms behind it.

 Build-Along • Day 05 — securing the Order API with JWT

- Add a `User` entity + `Role` enum (`CUSTOMER`, `ADMIN`) and a `UserDetailsService`; store passwords with `BCryptPasswordEncoder`.
- Build `POST /api/v1/auth/register`, `/login`, and `/refresh`: register hashes the password, login issues a short-lived access token plus a refresh token, refresh rotates them.
- Write a `JwtAuthFilter` extends `OncePerRequestFilter` that validates the `Bearer` token and populates the `SecurityContext`; add it *before* `UsernamePasswordAuthenticationFilter`.
- Declare a component-based `SecurityFilterChain` (Spring Security 6, *not* `WebSecurityConfigurerAdapter`): CSRF disabled, CORS enabled, `STATELESS` sessions.
- Enforce role rules: `CUSTOMER` views only their own orders; `ADMIN` manages products and views all orders (URL rules + `@PreAuthorize` ownership check).
- Lock CORS to the single trusted origin `https://app.shopflow.com` with explicit methods and headers; keep the signing secret in an environment variable.

Definition of done: an unauthenticated call to `/api/v1/orders` returns 401; a `CUSTOMER` reading another customer's order returns 403; a `CUSTOMER` reading their own order and an `ADMIN` reading any order return 200; and a request from a disallowed origin is blocked by CORS while the trusted origin succeeds.

11 Chapter Summary

Key takeaways

- ✓ Security is a *chain of filters* run before MVC; it answers "who are you?" then "are you allowed?".

- ✓ Authentication produces an `Authentication` in the `SecurityContext`; authorization only reads it.
- ✓ BCrypt is salted and adaptive — store the hash, verify with `matches`, never store plaintext.
- ✓ A JWT is *signed, not encrypted*; it scales because it is stateless, and it is hard to revoke for the same reason.
- ✓ Configure Spring Security 6 with a `SecurityFilterChain` bean; add your JWT filter before `UsernamePasswordAuthenticationFilter`.
- ✓ CORS is a browser rule; CSRF targets ambient cookies — a header-token API disables CSRF deliberately.

Things to remember

- `hasRole('ADMIN')` matches the authority `ROLE_ADMIN` — the prefix is added for you.
- `401` = "I don't know who you are"; `403` = "I know exactly who you are, and no."
- Order your URL rules specific-first, and finish with `anyRequest().authenticated()`.

Common pitfalls (last look)

- JWT in `localStorage` → XSS token theft. • Secrets in `application.yml` → compromised for good.
- Disabling CSRF and CORS blindly → no idea what you turned off. • No expiry → tokens valid forever.

📄 One-Page Cheat Sheet

Authn vs Authz — authentication = who you are (verify credentials); authorization = what you may do (check authorities).

Filter chain — CORS → `JwtAuthFilter` → `UsernamePasswordAuthenticationFilter` → `ExceptionTranslation` → `AuthorizationFilter` → controller.

BCrypt — salted + adaptive; store hash; verify with `matches`; cost is a tunable dial.

JWT — header.payload.signature; signed not encrypted; stateless, short-lived, refresh for longevity.

Config — `SecurityFilterChain` bean; `addFilterBefore`; `STATELESS`; specific rules before general.

hasRole vs hasAuthority — `hasRole('X')` = `hasAuthority('ROLE_X')`; pick one convention and hold it.

CORS vs CSRF — CORS = browser origin policy you whitelist; CSRF = ambient-cookie attack, off for header-token APIs.

👤 Interview questions to master

- › How does the Spring Security Filter Chain process a request?
- › What problem does JWT solve that session cookies don't, and what does it give up?
- › Explain CORS vs CSRF, and why JWT APIs often disable CSRF.
- › How would you revoke a JWT before it expires?
- › What is the difference between authentication and authorization?
- › How does BCrypt protect against rainbow-table attacks?

Suggested further reading

- *Spring Security Reference* — "Architecture" (the filter chain) and "Authorization."
- RFC 7519 (JWT) and RFC 6749 (OAuth2) — read the introductions, at least.
- OWASP: the *Authentication* and *JWT* cheat sheets — concise, practical, current.

Key idea

What you will learn next (Day 6). Your API is now authenticated, authorized, and documented — but is it *correct*? Tomorrow you prove it with tests: unit tests for services, slice tests for controllers, and `@SpringBootTest` with a real database via Testcontainers — including how to authenticate a test request so your new security rules do not turn every test red.

DAY 06 • QUALITY

Testing

Unit, integration & real-world confidence — the test pyramid in practice

06

🕒 Estimated study time: 8 hours

🎯 By the end of this day, you will be able to...

- Write unit tests that isolate a single class with Mockito — and recognise the moment one quietly turns into a slow integration test.
- Stand up a real, ephemeral PostgreSQL with Testcontainers so your data-layer tests catch dialect bugs H2 hides.
- Test controllers through the real HTTP machinery with `@WebMvcTest` and `MockMvc` — status codes, validation, and the 401/403 paths Day 5 added — without booting a server.
- Decide deliberately *what to test at each layer*, and just as deliberately what not to bother testing.
- Prove a business rule and detect an N+1 with tests that assert *behaviour*, not implementation detail.

1 The Big Picture

What this day solves. For five days you have been *building* ShopFlow — a container of beans (Day 1), a persistence layer (Days 2–3), a REST API (Day 4), a JWT security chain (Day 5). Every one of those days ended with “it works on my machine.” Today answers a harder question: *how do you know it still works tomorrow, after the next change, without clicking through the app by hand every time?* Testing is the machinery that turns “I think it works” into “I can prove it works, in ninety seconds, before every commit.”

Why backend engineers need it. A backend without tests is not slower to change — it is *dangerous* to change. The business rules you encoded (an order cannot exceed available stock; a total is the sum of its line items; a `PENDING` order may move to `PAID` but never back) are invisible and fragile. One refactor silently breaks one of them, and the first person to notice is a customer or an auditor. Tests make those rules executable and self-defending. They are also how you refactor with confidence — exactly the skill Day 7 (clean architecture) will demand of you.

Where it fits in a Spring Boot application. Testing is not a layer of the application; it is a *net stretched under every layer*. Each layer you already built has a test tool matched to its shape: the service layer wants fast isolated unit tests, the repository wants a real database, the controller wants real HTTP without a real server. Picking the wrong tool for a layer is the single most common testing mistake — and the reason so many suites are both slow *and* low-confidence.

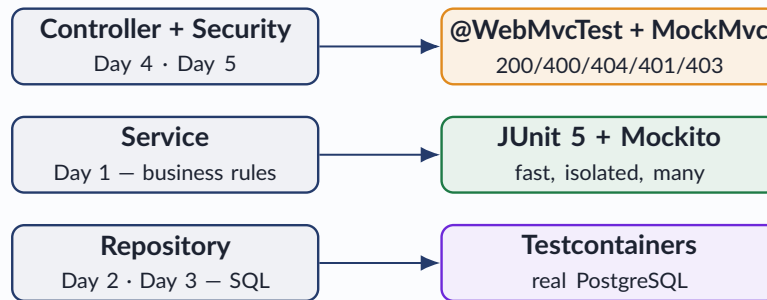


Fig 6.1 — Testing is a net under the stack you already built. Each layer has a tool matched to its shape.

How it connects to previous chapters. Every good habit from earlier days pays off here. Constructor injection (Day 1) is what lets you build a service in a test with plain `new` and hand it fake collaborators — no Spring context, no reflection. The proxies you studied (Day 1) are why a `@Transactional` rollback is worth an explicit test. The N+1 query trap (Day 3) becomes a thing you can now *catch automatically*. The JWT filter chain (Day 5) is why your controller tests must cover `401` and `403`, not just the happy path.

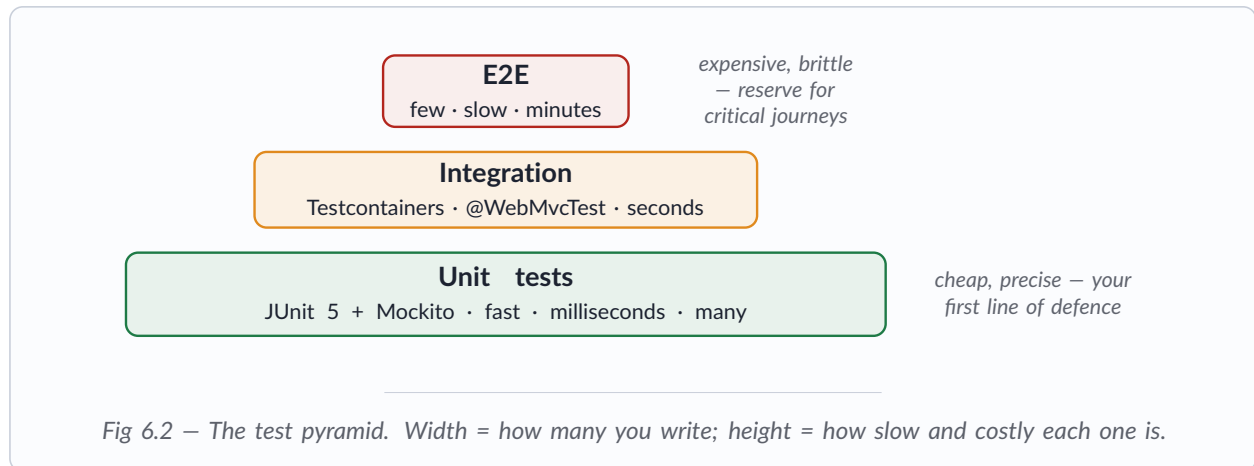
Real company examples. Google popularised the “test pyramid” vocabulary and reports engineers run unit tests thousands of times a day. Testcontainers began at a handful of teams and is now standard at companies like Spotify and Zalando precisely because “passes on H2, fails on Postgres” cost them real incidents. At scale the economics are brutal and simple: a bug caught by a unit test costs seconds; the same bug caught in production costs an on-call engineer, a rollback, and trust.

Key idea

A test suite is not a chore you add at the end — it is a **design tool** and a **safety net**. The shape of your suite (many fast unit tests, fewer integration tests, a handful of end-to-end tests) determines whether you *actually run it*. A suite you dread running protects nothing. Speed is a feature.

2 Concept Map

Hold the whole territory in one shape. Read the pyramid bottom-to-top: the wide base is cheap and fast and there is a lot of it; the narrow tip is expensive and slow and there is very little of it. Width is quantity; height is cost.



Terminology to anchor: **unit test** → **test double** (mock / stub / spy / fake) → **Mockito** → **integration test** → **Testcontainers** → **slice test** (@WebMvcTest) → **MockMvc** → **arrange-act-assert**.

3 Guided Learning

3.1 The Test Pyramid — why shape decides everything

What is it? The **test pyramid** is a rule of thumb for the *proportion* of test types in a healthy suite: a broad base of unit tests, a thinner band of integration tests, and a sliver of end-to-end tests. It is a shape you aim for, not a law.

Why does it exist? Because tests trade off along two axes: **speed** and **fidelity**. A unit test runs in a millisecond but only proves one class behaves in isolation. An end-to-end test proves the real system works but takes minutes, needs the whole stack up, and fails for a dozen unrelated reasons (flaky network, slow container, a shared test database someone else mutated). If you invert the pyramid — many slow broad tests, few fast ones — the suite becomes so slow and flaky that engineers stop running it. A suite nobody runs has negative value: it costs maintenance and provides no protection.

How does it work internally? Feedback speed compounds. Suppose a change breaks a business rule. With a fast unit test you learn in two seconds, at your desk, from a precise message pointing at one method. With only an end-to-end test you learn ten minutes later, in CI, from a failure that says “expected 200 got 500” with no hint which of forty classes is at fault. The pyramid is really a statement about *where you want failures to surface*: as close to the cause, and as early, as possible.

💡 How senior engineers think

The pyramid is a default, not a dogma. Some mature teams deliberately run a “testing trophy” shape — fewer pure unit tests, a fat middle of integration tests — because for a thin CRUD service most bugs live in wiring and SQL, not in branching logic. The point is never “obey the shape.” It is: *spend your test budget where your bugs actually are*, and keep the fast tests fast enough to run constantly.

3.2 Unit Testing with JUnit 5 & Mockito

What is it? A **unit test** exercises one class — the **class under test** — while replacing its collaborators with **test doubles**. **JUnit 5** is the test runner and assertion framework; **Mockito** is the library that fabricates the fake collaborators.

Why does it exist? You want to test that `OrderService.placeOrder` throws `OutOfStockException` when stock is zero — *without* a database, a web server, or a real `ProductRepository`. Isolation buys you speed (no I/O) and precision (a failure can only mean the service logic is wrong, because everything else is a controlled fake).

How does it work internally? `@Mock` asks Mockito to generate, at runtime, a subclass of the collaborator whose every method returns a harmless default (`null`, `0`, empty `Optional`) until you say otherwise. Mockito builds that subclass with ByteBuddy — the same bytecode trick Spring uses for proxies. `when(mock.method()).thenReturn(x)` records a **stub**: “when this is called, answer x.” `verify(mock).method()` checks an **interaction** happened. `@InjectMocks` constructs the real class under test and pushes the mocks in — trying constructor injection first, which is exactly why the Day 1 habit matters.

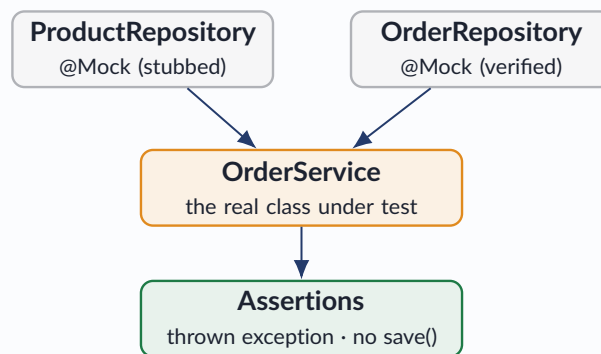


Fig 6.3 — Unit isolation: real service in the middle, fabricated collaborators around it, assertions on the outputs.

OrderServiceTest.java

```

@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock OrderRepository orderRepository;
    @Mock ProductRepository productRepository;
    @InjectMocks OrderService orderService;

    @Test
    void throwsWhenProductOutOfStock() {
        // arrange: a product that exists but has zero stock
        when(productRepository.findById(1L))
            .thenReturn(Optional.of(new Product(1L, "Widget", 0)));

        // act + assert: the rule fires
        assertThatThrownBy(() ->
            orderService.placeOrder(new OrderRequest(1L, 5))
        ).isInstanceOf(OutOfStockException.class);

        // and nothing was written
        verify(orderRepository, never()).save(any());
    }
}
  
```

When should I use it? For every branch of real logic: the out-of-stock throw, the totals calculation, each legal and illegal status transition, boundary values (ordering exactly the remaining stock, ordering zero). This is where the bulk of your tests should live.

When should I not? Do not unit-test framework behaviour (that Spring injects a bean, that Jackson serialises a field), getters and setters, or a method that only delegates. And the moment your “unit” test needs a real database or the Spring context to pass, it is no longer a unit test — move it down to the integration band and stop pretending.

Production tip

Use **AssertJ** (`assertThat(...).isEqualTo(...)`, `assertThatThrownBy(...)`) rather than raw `assertEquals`. Its fluent chains read like a sentence, produce far better failure messages (“expected total 40 but was 35” instead of “expected 40 was 35” with no context), and its collection assertions (`containsExactly`, `extracting`) turn three lines of loop into one readable line. It ships inside `spring-boot-starter-test`.

Common misconception

“More mocking means a better unit test.” The opposite is usually true. If a test mocks so much that it only asserts *that mocks were called in a certain order*, it tests your implementation’s internal wiring, not its behaviour — so it breaks on every refactor and catches no real bug. Mock only what you must to gain isolation (I/O, collaborators you do not own). Prefer asserting on *return values and thrown exceptions* over `verify(...)` on interactions.

Behind the scenes

A “mock” is only one kind of **test double**. A **stub** returns canned answers (`thenReturn`). A **mock** additionally lets you assert interactions (`verify`). A **spy** wraps a *real* object and intercepts only the methods you override. A **fake** is a lightweight working implementation (an in-memory map standing in for a repository). Mockito’s `@Mock` gives you a stub-and-mock hybrid; reach for a real object or a fake when behaviour, not interaction, is what you care about.

3.3 Integration Testing with Testcontainers

What is it? An **integration test** exercises several real components together — your repository against a real database, your mappings against real SQL. **Testcontainers** is a library that starts a throwaway Docker container (a real PostgreSQL) for the test run and tears it down afterwards.

Why does it exist? The old shortcut was an in-memory H2 database: fast, no Docker, boots instantly. The problem is that H2 is *not* PostgreSQL. Its “PostgreSQL compatibility mode” still differs on the things that bite you in production: `JSONB` and array columns, identity/sequence behaviour, `ON CONFLICT` upserts, `RETURNING`, case-folding of unquoted identifiers, `ILIKE`, and window-function edge cases. A query that is green on H2 can throw at runtime on real Postgres. Testcontainers removes the entire category of “works on the fake database” bugs by testing against the real engine.

How does it work internally? Testcontainers talks to your local Docker daemon, pulls `postgres:16-alpine` (cached after the first run), and starts a container on a random free port. Because the port is random, you cannot hard-code the JDBC URL — so `@DynamicPropertySource` feeds the live URL, username and password into Spring’s `Environment` *before* the context starts, and the app connects to the ephemeral database as if it were production.

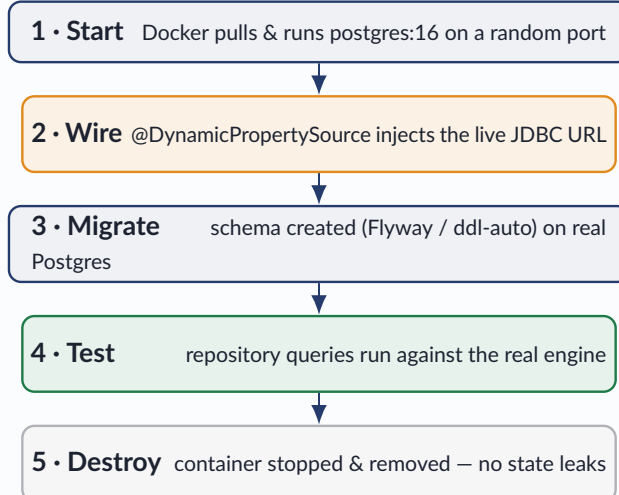


Fig 6.4 — The Testcontainers lifecycle: a real database is born for the test run and dies with it.

OrderRepositoryIT.java

```

@DataJpaTest
@Testcontainers
@AutoConfigureTestDatabase(replace = NONE) // do NOT swap in H2
class OrderRepositoryIT {

    @Container
    static PostgreSQLContainer<> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine");

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired OrderRepository orderRepository;

    @Test
    void savesAndReloadsOrder() {
        Order saved = orderRepository.save(new PendingOrder());
        assertThat(orderRepository.findById(saved.getId())).isPresent();
    }
}
  
```

Catching the N+1 (Day 3) automatically. The most valuable repository test proves a query does *not* explode into one query per row. Hibernate exposes a live statement counter; assert on it. This turns the abstract “avoid N+1” rule into a regression test that fails the day someone drops a `JOIN FETCH`.

NPlusOneIT.java

```

@Test
void loadingOrdersWithItemsRunsOneQuery() {
    var stats = entityManagerFactory
        .unwrap(SessionFactory.class).getStatistics();
}
  
```

```
stats.setStatisticsEnabled(true);
stats.clear();

List<Order> orders = orderRepository.findAllWithItems(); // JOIN FETCH
orders.forEach(o -> o.getItems().size()); // touch the collection

// one SELECT total, not 1 + N
assertThat(stats.getPrepareStatementCount()).isEqualTo(1);
}
```

Database insight

The whole reason to prefer a real Postgres here is fidelity. H2 will happily accept SQL that Postgres rejects and vice-versa, and it silently “fixes” things Postgres is strict about — identifier case, transaction isolation defaults, type coercion. Testing persistence on H2 is like testing a bridge with a cardboard model: it tells you the shape is roughly right, not that it holds weight.

Performance note

Starting a container costs a second or two. Do not pay it per test. Make the container `static` so one instance is shared across all methods in the class, and enable Testcontainers’ **reusable containers** (or the Singleton-container pattern) so the same database is reused across test classes in a run. Better still: run all your Testcontainers ITs in one Gradle/Maven task so the JVM and Docker warm-up amortise. Speed is what keeps you running them.

3.4 Controller Testing with `@WebMvcTest` & `MockMvc`

What is it? `@WebMvcTest` is a **slice test**: it boots only the web layer — your controller, the JSON converters, validation, exception handlers, and (if it is on the classpath) the Spring Security filter chain — and nothing else. `MockMvc` then drives it by simulating HTTP requests *in-process*, with no real server or socket.

Why does it exist? You want to prove that `POST /api/v1/orders` returns `400` for an invalid body, `404` for a missing product, `201` on success, and — since Day 5 — `401` without a token and `403` for the wrong role. A full `@SpringBootTest` could do this but boots the entire context (database, security, every bean) and runs an order of magnitude slower. The slice gives you real HTTP semantics at unit-test speed by mocking the service layer with `@MockBean`.

How does it work internally? `@WebMvcTest(OrderController.class)` tells Spring to register only that controller and the MVC infrastructure, and to leave `@Service/@Repository` beans out. You supply the collaborators the controller needs as `@MockBean OrderService` — a Mockito mock *registered in the context* so the controller can be wired. `MockMvc.perform(...)` builds a `MockHttpServletRequest`, pushes it through the real `DispatcherServlet` and filter chain, and hands you the response to assert on.

@WebMvcTest
 loads: controller · filters ·
 validation · Jackson · security
 service = @MockBean

fast — run many

@SpringBootTest
 loads: the whole context —
 every bean, DB, security
 real service · slow · rare

slow — a handful only

Fig 6.5 — The slice (left) loads only the web layer and mocks the service; the full context (right) loads everything.

OrderControllerTest.java

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired MockMvc mockMvc;
    @MockBean OrderService orderService;

    @Test
    @WithMockUser(roles = "CUSTOMER") // authenticated
    void returns400OnInvalidBody() throws Exception {
        mockMvc.perform(post("/api/v1/orders")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{}")) // fails @Valid
            .andExpect(status().isBadRequest());
    }

    @Test
    void returns401WhenNoToken() throws Exception { // no @WithMockUser
        mockMvc.perform(post("/api/v1/orders")
            .contentType(MediaType.APPLICATION_JSON)
            .content(validBody()))
            .andExpect(status().isUnauthorized());
    }

    @Test
    @WithMockUser(roles = "CUSTOMER") // lacks ADMIN
    void returns403OnAdminOnlyEndpoint() throws Exception {
        mockMvc.perform(delete("/api/v1/orders/1"))
            .andExpect(status().isForbidden());
    }
}
```

Security warning • 401 is not 403

Test both, and know the difference cold. 401 Unauthorized means “I do not know who you are” — no token, or an invalid/expired one; the Day 5 `ExceptionHandler` produces it. 403 Forbidden means “I know who you are, and you may not do this” — a valid token but the wrong role. A suite that only checks the happy path will happily ship an endpoint that returns 200 to anonymous callers. To exercise security inside the slice you need `spring-security-test` on the classpath and `@WithMockUser` (or `SecurityMockMvcRequestPostProcessors.jwt()` for token-shaped auth).

3.5 What (and what NOT) to test at each layer

The one table to internalise. Match the tool to the layer, and know what each layer is for. Testing business rules with a full `@SpringBootTest` is slow and imprecise; testing SQL with a mock proves

nothing about the SQL.

Layer	Tool	What you actually test
Service	JUnit 5 + Mockito	Business rules, edge cases, thrown exceptions
Repository	Testcontainers (real Postgres)	Query correctness, constraints, N+1
Controller	@WebMvcTest + MockMvc	Status codes, validation, JSON shape, auth
Full flow	@SpringBootTest + Testcontainers	A few critical journeys only (checkout, login)

What NOT to test, ever. Getters, setters, and `equals/hashCode` generated by Lombok or the compiler. That the framework injects a bean. That Jackson serialises a public field. Third-party library internals. Trivial delegation methods. Every one of these is either guaranteed by the platform or so trivial that a test costs more to maintain than the bug it could catch is worth.

✓ Best practices — what experienced engineers do

- Name tests by behaviour: `throwsWhenProductOutOfStock`, never `test1`. The name is the spec.
- One logical assertion focus per test; use AssertJ for readable failures.
- Prefer Testcontainers over H2 for anything that touches real SQL semantics.
- Keep full-context `@SpringBootTest` tests rare — they are slow; use slices and units by default.
- Follow **arrange-act-assert**; keep each test independent and order-free.

✗ Common mistakes beginners make

- Mocking so much the test only verifies mocks were called, not real behaviour.
- Using H2 for integration tests and missing real Postgres-specific bugs.
- Testing getters/setters and framework code instead of business logic.
- Flaky tests from shared mutable state, real network calls, or clock/time-of-day dependence.

4

Visualizations to Internalize

You met five diagrams; reproduce two *from memory* below. Redrawing is what moves a picture from “recognised” to “known.”

 Redraw: the test pyramid, with speed/cost on one axis and quantity on the other

✎ Redraw: the Testcontainers lifecycle, from “start container” to “destroy”

One consolidating picture — the feedback loop that makes the pyramid worth its shape:



cost of a caught bug rises left → right; run the cheap ones constantly

Fig 6.6 — The feedback spectrum: the faster the test, the tighter the loop, the earlier the bug dies.

5 Guided Coding Lab — Testing the Order API

Goal of this lab: give ShopFlow's order-placement flow real coverage — unit tests for every rule, a Testcontainers repository test, and `@WebMvcTest` tests including the Day 5 auth paths. Guide yourself through the steps; do not paste a finished suite. The learning is in watching each test go red, then green.

1 Add the test dependencies

`spring-boot-starter-test` already brings JUnit 5, Mockito and AssertJ. Add the Testcontainers BOM plus `testcontainers:postgresql` and `testcontainers:junit-jupiter`, and `spring-security-test` for the auth tests. 💡 *Why:* Getting the dependency set right once means every later test just works; `spring-security-test` is the piece juniors always forget when their 401 test refuses to compile.

2 Unit-test the out-of-stock rule

With `@ExtendWith(MockitoExtension.class)`, `@Mock` the two repositories and `@InjectMocks` the service. Stub a zero-stock product and assert `placeOrder` throws, and that `save` was *never* called. 💡 *Why:* This is the canonical isolated unit test — no database, no context, sub-millisecond — and it locks down a rule that protects real inventory.

3 Unit-test totals and status transitions

Add tests: the order total equals the sum of `quantity * unitPrice` across line items; a `PENDING` order may transition to `PAID`; a `PAID` order may *not* transition back to `PENDING` (expect an exception). 💡 *Why:* State machines and money are where silent bugs hide; each branch deserves its own named test so a failure tells you exactly which rule broke.

4 Write the Testcontainers repository test

Use `@DataJpaTest` with `@AutoConfigureTestDatabase(replace = NONE)` and a static `PostgreSQLContainer`, wired via `@DynamicPropertySource`. Save an order with items, reload it, and assert the graph came back intact. 💡 *Why:* This proves your JPA mappings work against the real engine you deploy on — not against H2's approximation of it.

5 Prove there is no N+1

Enable Hibernate statistics, call your `findAllWithItems()` (the `JOIN FETCH` query from Day 3), touch each order's items, and assert the prepared-statement count is exactly one. 💡 *Why:* An N+1 is invisible until production load makes it a fire; this test makes the regression fail in CI the moment someone removes the fetch join.

6 Slice-test the controller for 200/400/404

With `@WebMvcTest(OrderController.class)` and `@MockBean OrderService`, cover: a valid request returns 201/200; an empty body fails validation with 400; a service that throws “product not found” maps to 404. 💡 *Why:* You are testing real HTTP semantics — status codes, validation wiring, exception-to-status mapping — without paying for a full context.

7 Add the auth paths (401 & 403)

Add a test with no `@WithMockUser` that expects 401, and one with a `CUSTOMER` role hitting an admin-only endpoint that expects 403. 💡 *Why:* Day 5 added a security filter chain; an API test suite that ignores it will ship an endpoint open to the world and never notice.

🔧 Production tip

Measure your suite. Run it with timing on (`mvn test` surfaces per-class times; Gradle's build scan ranks them) and find the slowest tests. Nine times out of ten the slow ones are accidental `@SpringBootTest` runs that should have been slices or units. Speeding up the slowest 5% often halves total suite time — and a suite that runs in seconds is a suite you run before every commit.

6 Thinking Exercises

Reflection prompts, not quiz questions. Write a sentence or two — the goal is to make your reasoning explicit.

🧠 Think it through

A colleague says “we have 95% code coverage, so we are well tested.” Coverage measures which lines *ran* during tests, not whether anything was *asserted*. Describe a test suite with high coverage that catches almost no bugs. What would you measure instead?

 Think it through

Your `OrderService` unit test needs a real `OrderRepository` to pass, so you `@Autowired` it and add `@SpringBootTest`. It still passes. What did you lose by doing this, and at what point should you have moved the test to a different layer of the pyramid?

 Think it through

Testcontainers gives you a real Postgres but starts a container. Where is the boundary between “worth the seconds” and “too slow to keep”? How would you decide which handful of flows deserve a full `@SpringBootTest` + Testcontainers end-to-end test?

7

Debugging Practice

 Debugging Lab • The green test that proves nothing

Symptom. `placeOrderSavesOrder` passes, yet in production orders are silently not persisted. The test is green; the feature is broken.

Evidence.

```
OrderServiceTest.java

@Test
void placeOrderSavesOrder() {
    when(productRepository.findById(1L))
        .thenReturn(Optional.of(new Product(1L, "Widget", 100)));
    when(orderRepository.save(any())).thenReturn(new Order(1L));

    Order result = orderService.placeOrder(new OrderRequest(1L, 2));

    verify(orderRepository).save(any()); // the only assertion
}
```

Work the method:

- ✓ **Observe.** The test only asserts that `save` was *called* with `any()` — it never checks *what* was saved, nor the returned order.
- ✓ **Hypothesise.** The service builds an order with the wrong total or a null status; the mock

accepts `any()` and returns a canned object, so the bug is invisible to the test.

- ✓ **Verify.** Replace `any()` with an `ArgumentCaptor<Order>`; inspect the captured order's total and status. Assert on the *returned* order too.
- ✓ **Fix.** Assert real values: `captured.getTotal()` equals the expected sum, `captured.getStatus()` is `PENDING`.
- ✓ **Prevent.** Adopt the rule “assert on state and return values, not just interactions.” Treat a test whose only assertion is `verify(...)` as a smell to review.

🔧 Debugging Lab • Green on H2, red on Postgres

Symptom. The repository test passes in CI (which uses H2) but the same query throws `column ... does not exist` the moment it runs against real Postgres in staging.

Evidence.

the failing query

```
-- entity mapped a camelCase column without quoting
SELECT orderDate FROM orders WHERE ... -- Postgres folds to orderdate
-- H2 (default) preserved case and matched; Postgres did not
```

- ✓ **Observe.** H2 and Postgres disagree on identifier case-folding; H2 accepted the mixed-case column, Postgres lower-folded it and found nothing.
- ✓ **Hypothesise.** The whole class of dialect bugs (case, JSONB, sequences, `ON CONFLICT`) is hidden because the test database is not the production database.
- ✓ **Verify.** Switch the repository test to Testcontainers Postgres with `@AutoConfigureTestDatabase(replace = NONE)` and re-run — the bug now reproduces locally.
- ✓ **Fix.** Correct the mapping/column naming (or an explicit `@Column(name = ...)`), and let the real engine validate it.
- ✓ **Prevent.** Ban H2 for persistence tests. Make Testcontainers the default so “passes in CI” means “passes on the real engine.”

8 Mini-Labs

🔧 Mini-Lab • Beginner

25 min

Goal. Write your first behaviour-named unit test with Mockito.

Context. `OrderService.placeOrder` must reject a quantity that exceeds stock.

Requirements. `@Mock` the repositories, `@InjectMocks` the service, stub a low-stock product, assert the exception, and `verify(save, never())`.

Hints. Use `assertThatThrownBy(...).isInstanceOf(OutOfStockException.class)`; no Spring context, no `@SpringBootTest`.

Expected outcome. A sub-millisecond test that fails if the rule is removed.

Common pitfalls. Adding `@SpringBootTest` “to be safe” and turning a unit test into a slow context test.

Mini-Lab • Intermediate

45 min

Goal. Prove a query does not N+1, against real Postgres.

Context. `findAllWithItems()` should load orders and their items in one query.

Requirements. `@DataJpaTest` + a static `PostgreSQLContainer` + `@DynamicPropertySource`; enable Hibernate statistics and assert the statement count is 1.

Hints. Set `@AutoConfigureTestDatabase(replace = NONE)` so H2 is not substituted; clear stats before the call.

Expected outcome. The test passes with a `JOIN FETCH` and fails the instant you revert to a lazy mapping.

Common pitfalls. Forgetting to touch the collection (so lazy loading never triggers) and getting a false pass.

Mini-Lab • Production-style

70 min

Goal. Cover the full Order API surface at the web layer, auth included.

Context. `POST /api/v1/orders` and `DELETE /api/v1/orders/{id}` (admin only).

Requirements. With `@WebMvcTest` + `@MockBean OrderService` + `spring-security-test`, assert 201 (valid), 400 (invalid body), 404 (service throws not-found), 401 (no user), 403 (CUSTOMER calling the admin delete).

Hints. `@WithMockUser(roles = "ADMIN")` for the allowed delete; omit it for 401; use it with CUSTOMER for 403.

Expected outcome. Five named tests, each pinned to one status code, running in well under a second total.

Common pitfalls. Omitting `spring-security-test`, so `@WithMockUser` does nothing and every request is anonymous.

9 Engineering Notes

Judgment from engineers who have carried a pager. Read these as mindset, not trivia.

💡 How senior engineers think

The first question a senior asks of a failing test is not “how do I make it green?” but “is the test wrong, or is the code wrong?” A test that is hard to write is usually telling you the code is hard to use — too many collaborators, hidden state, a method doing three jobs. Painful tests are design feedback. Listen to them before Day 7 makes you refactor anyway.

👤 Interview insight

Expect: “What is the difference between `@Mock` and `@MockBean`?” `@Mock` (Mockito) is a plain fake object with no Spring involvement — for pure unit tests. `@MockBean` (Spring Boot) creates a mock *and registers it in the application context*, replacing the real bean — for slice/context tests like `@WebMvcTest` where the framework must wire it. Say “one is a bare object, the other is a bean in the context.”

⚡ Performance note

Test suite speed is a production concern, not a nicety. A slow suite lengthens every developer's inner loop and lengthens CI, which lengthens the time-to-recover during an incident when you need to ship

a fix fast. Budget it: aim for the unit tests to finish in a couple of seconds and the whole suite in the low minutes. When it creeps, hunt down the accidental full-context tests — they are almost always the culprit.

Production tip

Make tests deterministic. The two great sources of flakiness are **shared mutable state** (a static field, a reused database row, test-order dependence) and **real-world coupling** (the wall clock, random values, live network calls). Inject a `clock` so you can freeze time; seed randomness; never let a unit test touch the network. A test that fails one run in fifty erodes trust in the whole suite — people start re-running until green, which defeats the point.

10 Build-Along Project — ShopFlow

ShopFlow is now a secured Order API: a bean container (Day 1), a JPA persistence layer over PostgreSQL (Days 2–3), a REST controller (Day 4), and a JWT security chain that answers 401/403 (Day 5). Today you give all of it a real safety net so the next five days of change cannot silently break it.

Build-Along • Day 06 — real coverage for the Order API

- **Service unit tests (JUnit 5 + Mockito)** for every order-placement rule: out-of-stock rejection, correct total from line items, and each legal/illegal status transition (`PENDING` → `PAID`, and the forbidden reverse). Mock the repositories; assert on state and thrown exceptions, not just interactions.
- **A Testcontainers PostgreSQL repository test** that saves and reloads an order graph on the real engine, plus an N+1 guard: enable Hibernate statistics and assert the `JOIN FETCH` query (Day 3) runs exactly one statement.
- **@WebMvcTest + MockMvc tests** for the Order API covering 200/201, 400 (validation), and 404 (not found) — with the service @MockBean-ed.
- **Auth-path controller tests** covering the Day 5 additions: 401 for an anonymous request and 403 for an authenticated `CUSTOMER` hitting an admin-only endpoint, using `spring-security-test`.
- **No H2 anywhere near persistence** — `@AutoConfigureTestDatabase(replace = NONE)` so integration tests hit real Postgres.

Definition of done: `mvn test` runs green in under two minutes; every order-placement business rule has a named unit test; one Testcontainers test proves no N+1; and the Order API controller tests assert 200/400/404 and 401/403. From here, any refactor that breaks a rule turns the build red before it ever reaches a reviewer.

11 Chapter Summary

Key takeaways

- ✓ The pyramid is about *speed of feedback*: many fast unit tests, fewer integration tests, a sliver of

E2E.

- ✓ Unit tests isolate one class with Mockito; assert behaviour (return values, exceptions), not mock interactions.
- ✓ Testcontainers gives a real Postgres so you catch dialect bugs H2 hides — never test persistence on H2.
- ✓ `@WebMvcTest` + `MockMvc` test real HTTP (status, validation, JSON, auth) without booting a server.
- ✓ Cover the auth paths: 401 = “who are you?”, 403 = “you may not.” A happy-path-only suite ships open endpoints.

Things to remember

- `@Mock` is a bare object; `@MockBean` is a mock *registered in the Spring context*.
- A test whose only assertion is `verify(...)` usually tests wiring, not behaviour — add a real assertion.
- Make containers `static` and reused; the slowest tests in a suite are almost always accidental full-context ones.

Common pitfalls (last look)

- Over-mocking → tests that pass while the feature is broken.
- H2 for integration → “green in CI, red in production.” • Skipping 401/403 → an API open to anyone.

📄 One-Page Cheat Sheet

Test pyramid — many fast unit · fewer integration · a few E2E; width = quantity, height = cost.

Unit test — `@ExtendWith(MockitoExtension.class)` + `@Mock` + `@InjectMocks`; assert state/exceptions.

@Mock vs @MockBean — bare Mockito object vs a mock registered in the Spring context.

Testcontainers — real ephemeral Postgres via `@Container` + `@DynamicPropertySource`; `replace = NONE`.

N+1 guard — Hibernate statistics; assert `getPreparedStatementCount() == 1` after touching the collection.

@WebMvcTest — web slice only; `@MockBean` the service; `MockMvc` drives real HTTP in-process.

Auth in tests — `spring-security-test` + `@WithMockUser`; test 401 (no user) and 403 (wrong role).

👤 Interview questions to master

- › What is the difference between `@Mock` and `@MockBean`?
- › Why use Testcontainers instead of an in-memory database?
- › What does `@WebMvcTest` load compared to `@SpringBootTest`?
- › How do you test that a transaction rolls back correctly on exception?
- › What makes a good unit test versus a brittle one?

Suggested further reading

- *JUnit 5 User Guide* — lifecycle, parameterised tests, and extensions.

- *Mockito documentation* — argument captors, `BDDMockito`, and when *not* to mock.
- *Testcontainers for Java* — reusable containers and the Singleton-container pattern.
- *Spring Boot Reference* — “Testing” chapter: test slices, `@MockBean`, and `@DynamicPropertySource`.

Key idea

What you will learn next (Day 7). You now have a safety net — and a safety net is what makes fearless refactoring possible. Tomorrow you use it: SOLID principles, layered vs hexagonal architecture, and the design patterns Spring already leans on. With today's tests guarding every rule, you can reshape ShopFlow's internals aggressively and let the green build tell you nothing broke.

DAY 07 • ARCHITECTURE

Clean Architecture, SOLID & Design Patterns

Structure that survives change — and the patterns Spring already uses

🕒 Estimated study time: 8 hours

🎯 By the end of this day, you will be able to...

- Apply each SOLID principle as a concrete Spring decision — a class boundary, an interface, a strategy — not as an abstract slogan.
- Diagnose a class by its **coupling** and **cohesion**, the two forces every SOLID rule is really optimising.
- Choose between **Layered** and **Hexagonal** architecture on evidence, and explain what each one costs.
- Name the design patterns Spring itself is built from, so you extend the framework along its grain instead of against it.
- Refactor a tangled "God service" into focused collaborators using a validator, the Strategy pattern, and domain events.

1 The Big Picture

What this day solves. By now your ShopFlow app works: it persists, it authenticates, it has tests. But "works today" and "still works after six months of features" are different properties. This day is about the second one. It answers a question that has nothing to do with any single annotation: *when a new requirement arrives, how much of the existing code do I have to open and risk breaking?* Good architecture is simply the discipline that keeps that number small.

Why backend engineers need it. Every codebase decays toward a big ball of mud unless someone actively fights it. The fight is not glamorous — it is a hundred small decisions about where a responsibility lives, what depends on what, and which interface stands between two things that change at different rates. Engineers who cannot make those decisions produce code that is correct on Tuesday and unmaintainable by the following quarter. This day gives you the vocabulary (SOLID, coupling, cohesion, ports) and the reflexes (extract, invert, decouple) to keep a system soft — *software*, literally, meaning changeable.

Where it fits in a Spring Boot application. Architecture is not a layer you add; it is the shape of the layers you already have. The `controller` → `service` → `repository` split you scaffolded on Day 1 is an architectural choice. Today you make that choice deliberate: you decide what each layer is allowed to know, which direction dependencies point, and where to insert an interface so that a change on one side does not ripple to the other.

How it connects to previous chapters. You have been practising these patterns without naming them. Day 1's **Dependency Injection** is the **Dependency Inversion** principle in action; its proxies are the **Proxy** pattern. Day 2–3's `Repository` interfaces are the textbook `Repository` pattern and

your first real dependency-inversion boundary. Day 4 told you to keep controllers thin — that is the Single Responsibility Principle. Day 5's security filter chain is a **Decorator**/chain. Day 6's tests only became easy because your classes depend on interfaces you can mock. Today those scattered instincts become one coherent framework.

Key idea

Architecture is the art of **managing dependencies so that change stays local**. Two questions decide almost everything: *who depends on whom* (point dependencies at stable abstractions, never at volatile details) and *what changes together* (put those things in one class, and only those). SOLID, layering, and patterns are all tactics for those two questions.

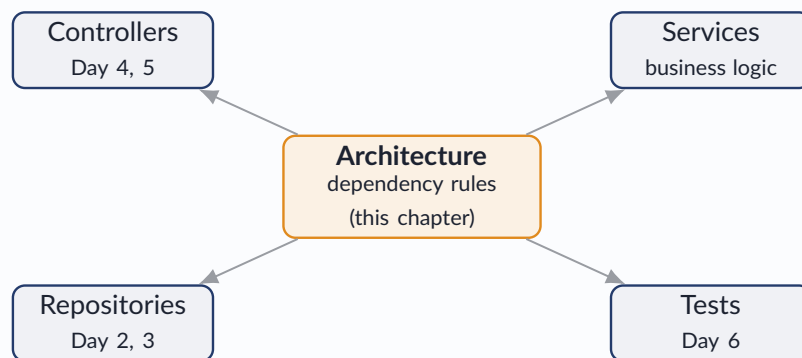


Fig 7.0 — Architecture is not a new box; it governs how the boxes you already have depend on each other, and it is what makes every later day cheap to add.

Real company examples. Uber documented its **Domain-Oriented Microservice Architecture** (DOMA) after its service count exploded, precisely to re-impose layering and clear boundaries. Netflix leans on ports-and-adapters thinking to keep domain logic independent of the AWS services underneath it. Banks adopt Hexagonal architecture so a decade of regulatory rules can outlive three generations of persistence technology. And Spring itself is a masterclass in the Gang-of-Four patterns — once you can see them, you stop fighting the framework.

2

Concept Map

Hold the whole territory in one picture. The centre is the goal — change that stays local. Everything around it is a tactic that serves that goal.

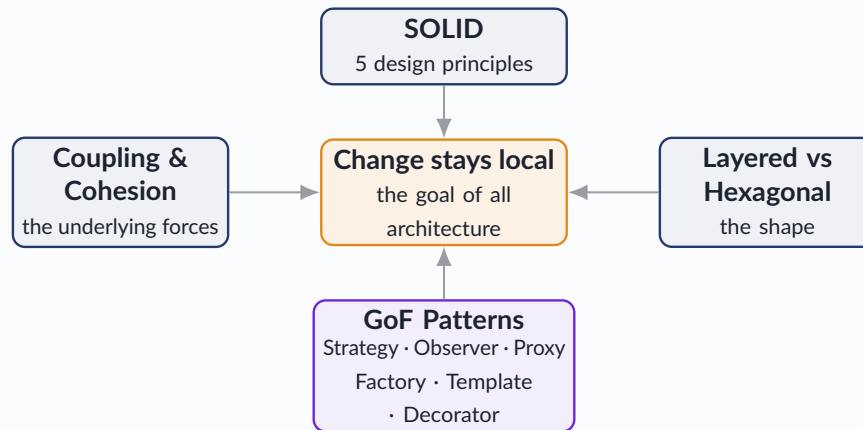


Fig 7.1 – The Day 7 concept map: SOLID and patterns are tactics; coupling and cohesion are the forces; layering is the overall shape.

Terminology to anchor: **Coupling** → **Cohesion** → **SOLID** → **Dependency Inversion** → **Port** → **Adapter** → **Strategy** → **Domain Event**.

3 Guided Learning

3.1 SOLID, translated into Spring reality

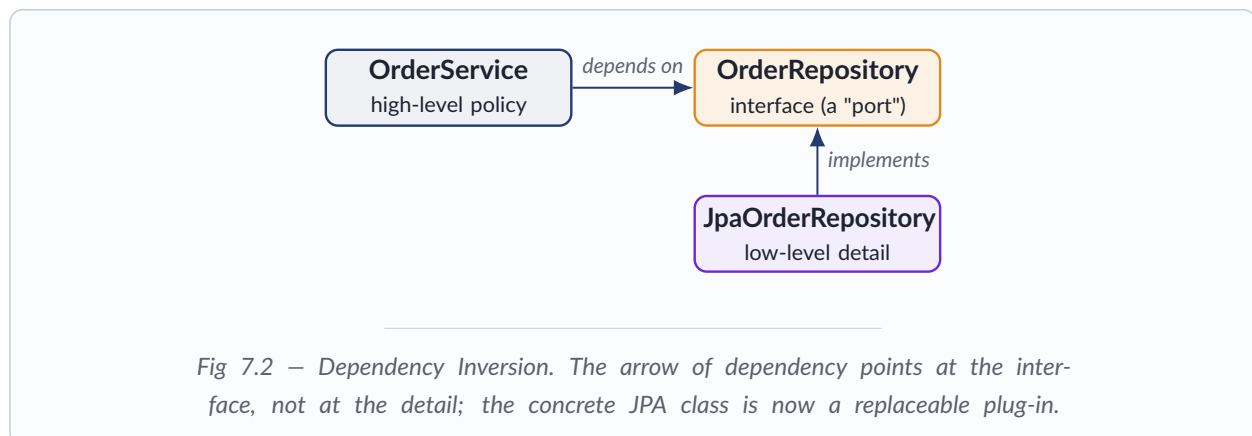
What is it? **SOLID** is five principles for arranging classes so a system stays changeable. They are usually taught as abstractions; in a Spring codebase each one is a concrete, physical decision — a class you split, an interface you introduce, an `if/else` you delete.

Principle	What it means in a Spring codebase
Single Responsibility	A class has one reason to change. <code>OrderService</code> places orders — it does not also format emails, compute tax tables, and talk to Stripe. Those are separate reasons to change, so separate classes.
Open/Closed	Add a new payment provider by writing a new <code>PaymentStrategy</code> implementation, not by editing an <code>if (provider == ...)</code> chain that already works.
Liskov Substitution	Every <code>PaymentStrategy</code> must be usable through the interface without surprising the caller — no implementation that throws on a method the contract promises.
Interface Segregation	Prefer several small role interfaces over one fat one. A read-only client should not depend on <code>save()</code> and <code>delete()</code> it never calls.
Dependency Inversion	Services depend on repository <i>interfaces</i> , not on <code>JdbcTemplate</code> or Hibernate directly. This is what DI has been doing for you since Day 1.

Why does it exist? Each letter targets a specific pain. SRP fights the "I touched the tax code and broke email" class of bug. Open/Closed fights the merge-conflict-magnet file everyone edits. DIP fights the "we can't test this without a real database" trap. They are not aesthetics; they are defences against known failure modes.

How does it work internally — the one that matters most? Dependency Inversion is the keystone, so make it concrete. A naive design has the high-level policy (the service) depend on a low-level

detail (the database). DIP inverts the arrow: both depend on an interface the *service* owns. The database becomes a plug-in.



When should I use it? SRP and DIP: essentially always — they are the grain of a Spring app. Open/Closed and Strategy: the moment you write your *second* branch of an `if/else` that selects behaviour by type. That second branch is the signal to reach for an interface.

When should I not? Do not invent an interface with exactly one implementation that will never have a second, purely to "follow DIP" — that is ceremony. Do not split a class into three because a checklist said "one responsibility" when the three parts genuinely change together. SOLID is a set of trade-offs to reason about, not commandments to obey.

⚠ Common misconception

"Single Responsibility means a class should do only one thing." It does not — a class can have many methods. It means a class should have one *reason to change*, i.e. one stakeholder or axis of change. `OrderService` can have `place`, `cancel`, and `refund`; they all change when order *policy* changes. Email formatting changes when *marketing* changes — a different reason, a different class.

👤 Interview insight

Expect: "Explain SOLID with a Spring example for each letter." The answer that lands names a physical thing, not a definition: "O — I add a `PaymentStrategy` bean instead of editing a switch; D — my service is constructor-injected with a `PaymentRepository` interface, so a test passes a fake." Concrete beats abstract every time.

3.2 Coupling & Cohesion — the forces SOLID is really tuning

What is it? **Coupling** measures how much one module depends on the internals of another — change one, must you change the other? **Cohesion** measures how strongly the parts inside one module belong together. The eternal goal, older than SOLID by decades, is *low coupling, high cohesion*.

Why does it exist? SOLID, patterns, and layering are all instruments; coupling and cohesion are the two dials they turn. If you can only remember one thing from this chapter, remember the two dials: whenever you propose a refactor, ask "does this lower coupling or raise cohesion?" If it does neither, it is probably just moving code around.

How does it work internally? Coupling has flavours, from loose to tight: depending on an *interface* (loosest — you only know a contract), on a *concrete class*, on its *internal fields*, or on a shared mutable *global* (tightest). Every step tighter means a change is more likely to force a change elsewhere. Cohesion is the mirror image: a class where every method touches the same fields is highly cohesive;

a class split across unrelated field-groups is a candidate to split into two.

💡 How senior engineers think

When a senior reviews a pull request, they are not really reading logic first — they are reading *dependencies*. "Why does the controller import a JPA type? Why does this domain object know about HTTP status codes?" A misplaced import is a coupling smell you can catch in seconds, long before you understand the algorithm.

3.3 Layered vs Hexagonal Architecture

What is it? These are two shapes for a whole application. **Layered** architecture stacks `Controller` → `Service` → `Repository`, each layer depending only on the one below. **Hexagonal** (also called **Ports & Adapters**) puts a framework-ignorant *domain core* in the middle; everything external — REST, the database, a message queue — is an *adapter* that plugs into a *port* (an interface the core owns).

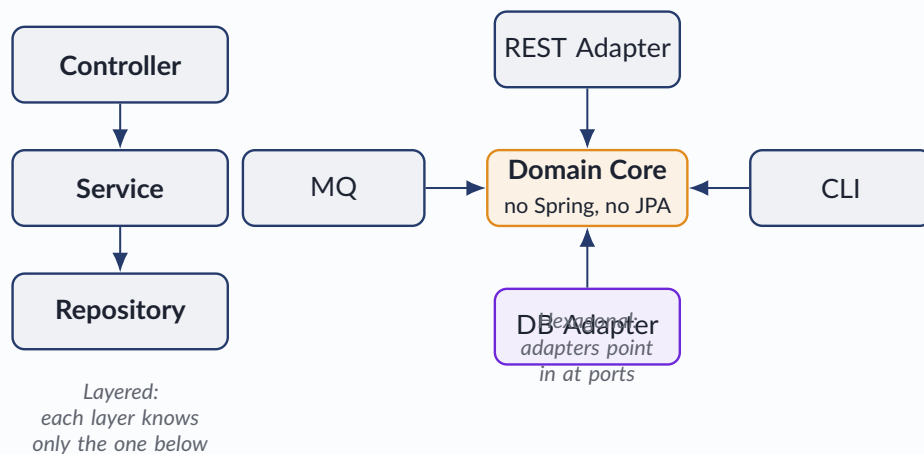


Fig 7.3 — Layered stacks dependencies downward; Hexagonal points every external dependency inward at a framework-free core.

How does it work internally? In Layered, dependencies flow one way (down) and the `Service` still imports Spring and, indirectly, persistence types — pragmatic and totally fine for most apps. In Hexagonal, the dependency rule is stricter: *nothing points outward from the core*. The domain defines an `OrderRepository` port; the JPA adapter implements it. You could delete Spring Data and rewrite the adapter without the domain noticing. That isolation is the entire value proposition — and its entire cost, because it demands more interfaces, mapping between domain and persistence models, and discipline.

When should I use each? Default to Layered. It matches Spring's grain, every engineer already understands it, and for a CRUD-shaped service it is exactly right. Reach for Hexagonal when the *domain logic is genuinely complex and long-lived* — rich rules that must outlast today's frameworks, or a core you want to test with zero Spring context. Insurance pricing, trading, logistics routing: yes. A product-catalogue CRUD: no.

When should I not? Applying Hexagonal or full DDD to a simple CRUD app is the most common over-engineering mistake juniors make after they read one influential blog post. Three interfaces and two mapper classes to save a row to a table is indirection with no payoff — and every future reader pays the tax of tracing through it.

🔑 Production tip

You do not have to choose globally. A single ShopFlow deployment can keep 90% of its modules Layered and make *only* the pricing/promotions engine Hexagonal, because that is the one place with rules worth protecting. Architecture is per-module judgement, not a religion applied uniformly.

❓ Why does this exist?

Hexagonal exists because the original sin of layered apps was letting the database define the domain. Teams modelled their business as tables, and a schema migration became a domain rewrite. Ports & Adapters flips ownership: the domain defines what it needs, and persistence conforms to it. You pay for that inversion in boilerplate; you buy freedom from framework churn.

3.4 The Design Patterns Spring Already Uses

What is it? Spring is not just *compatible* with the Gang-of-Four patterns — it is *built* from them. Learning to recognise them turns the framework from a bag of annotations into a coherent design you can extend idiomatically.

Pattern	Where Spring uses it
Proxy	AOP — <code>@Transactional</code> , <code>@Async</code> , <code>@Cacheable</code> all work by wrapping your bean in a proxy (Day 1).
Factory	<code>BeanFactory</code> and <code>ApplicationContext</code> manufacture and hand you fully wired objects.
Singleton	The default bean scope — one shared instance per context.
Template Method	<code>JdbcTemplate</code> , <code>RestTemplate</code> , <code>RabbitTemplate</code> — the framework runs the boilerplate, you supply the varying part in a callback.
Strategy	Swap a <code>PasswordEncoder</code> , a <code>HttpMessageConverter</code> , an <code>AuthenticationProvider</code> — interchangeable algorithms behind one interface.
Observer	<code>ApplicationEventPublisher</code> plus <code>@EventListener</code> — publish an event, decoupled listeners react.
Decorator / Chain	The <code>Filter</code> chain and <code>HandlerInterceptor</code> chain (Day 5) wrap each other, adding behaviour layer by layer.

👁 Behind the scenes

When you write `JdbcTemplate.query(sql, rowMapper)`, you are supplying the "variable part" of a Template Method: Spring owns connection handling, statement lifecycle, and exception translation (the invariant boilerplate), and calls back into your `RowMapper` for the one thing only you know — how to turn a row into an object. Every `*Template` class in Spring is this same deal.

Why does it matter? Because it tells you *how* to extend Spring. Need custom auth? Do not fork the filter chain — add a Strategy (an `AuthenticationProvider`). Need a side effect after an order? Do not thread a call through five layers — publish an Observer event. Recognising the pattern points you at the intended extension point.

3.5 Refactoring the God Service (Strategy + Domain Events)

What is it? The single most common real-world refactor: a 300-line method that validates, applies business rules, orchestrates persistence, *and* sends an email — all inline. It works, and it is a nightmare. Here is the "before", compressed:

OrderService.java – the God method (before)

```
@Service
public class OrderService {
    public Order placeOrder(OrderRequest req) {
        // 1. validation (40 lines of if-throw)
        if (req.items().isEmpty()) throw new BadRequest("empty");
        if (req.total() <= 0) throw new BadRequest("total");
        // ... 30 more lines ...
        // 2. payment (a growing if/else by provider)
        if (req.provider().equals("stripe")) { /* ... */ }
        else if (req.provider().equals("paypal")) { /* ... */ }
        // 3. persistence
        Order order = repository.save(toEntity(req));
        // 4. side effect: send email INLINE, inside the tx
        emailClient.sendConfirmation(order); // slow + can fail
        return order;
    }
}
```

Why is it a problem? Four reasons to change live in one method (SRP violated). The payment *if/else* grows every time a provider is added (Open/Closed violated). And the email call sits *inside the transaction*: if the SMTP server hiccups, the whole order rolls back — a mail outage becomes an ordering outage. That last one is not a style issue; it is a production incident waiting to happen.

How do we fix it? Three moves. **(1)** Extract validation into an `OrderValidator`. **(2)** Replace the payment *if/else* with the Strategy pattern. **(3)** Replace the inline email with a *domain event* that an asynchronous listener handles *after* the transaction commits.

OrderValidator.java – one responsibility

```
@Component
public class OrderValidator {
    public void validate(OrderRequest req) {
        if (req.items().isEmpty())
            throw new InvalidOrderException("Order has no items");
        if (req.total() <= 0)
            throw new InvalidOrderException("Total must be positive");
        // all validation rules, in one cohesive place
    }
}
```

For payment, define one interface and let each provider be its own bean. Spring can inject *all* implementations as a `Map` keyed by bean name — no *if/else*, and a new provider is a new file:

PaymentStrategy.java – Open/Closed via Strategy

```
public interface PaymentStrategy {
    PaymentResult charge(Order order);
}

@Component("stripe")
class StripePayment implements PaymentStrategy {
    public PaymentResult charge(Order order) { /* ... */ }
}

@Component("paypal")
class PayPalPayment implements PaymentStrategy {
```



```
public PaymentResult charge(Order order) { /* ... */ }
}
```

PaymentService.java – resolve strategy by name

```
@Service
public class PaymentService {
    private final Map<String, PaymentStrategy> strategies;
    // Spring injects EVERY PaymentStrategy bean, keyed by name
    public PaymentService(Map<String, PaymentStrategy> strategies) {
        this.strategies = strategies;
    }
    public PaymentResult pay(String provider, Order order) {
        PaymentStrategy s = strategies.get(provider);
        if (s == null) throw new UnknownProviderException(provider);
        return s.charge(order);
    }
}
```

👁 Behind the scenes

Injecting `Map<String, PaymentStrategy>` is a Spring superpower most juniors have never seen: the container finds *every* bean implementing the interface and gives you a map from bean name to instance. Adding a third provider is now genuinely zero edits to existing code — the definition of Open/Closed. This is the Strategy pattern and the Factory pattern shaking hands.

The event, and why it must fire after commit. The naive fix publishes an event and lets a listener send the email — but a plain `@EventListener` runs *synchronously, inside the caller's transaction*. If it throws, it still rolls back the order. Worse, if you send the email and the transaction *then* rolls back for another reason, you have emailed a customer about an order that does not exist. The correct tool is `@TransactionalEventListener` bound to the `AFTER_COMMIT` phase, made non-blocking with `@Async`.

Order placement – publish, don't call

```
@Service
public class OrderService {
    private final OrderValidator validator;
    private final PaymentService payments;
    private final OrderRepository repository;
    private final ApplicationEventPublisher events;
    // constructor injection omitted for brevity

    @Transactional
    public Order placeOrder(OrderRequest req) {
        validator.validate(req);
        payments.pay(req.provider(), toEntity(req));
        Order order = repository.save(toEntity(req));
        events.publishEvent(new OrderPlacedEvent(order.getId()));
        return order; // email is NOT sent here
    }
}
```

OrderNotificationListener.java – after commit, off-thread

```
@Component
public class OrderNotificationListener {
    private final EmailClient email;
```

```

@Async
@TransactionalEventListener(phase = AFTER_COMMIT)
public void onOrderPlaced(OrderPlacedEvent e) {
    email.sendConfirmation(e.orderId());
    // runs only if the tx committed; a mail failure
    // no longer rolls back the order
}
}

```

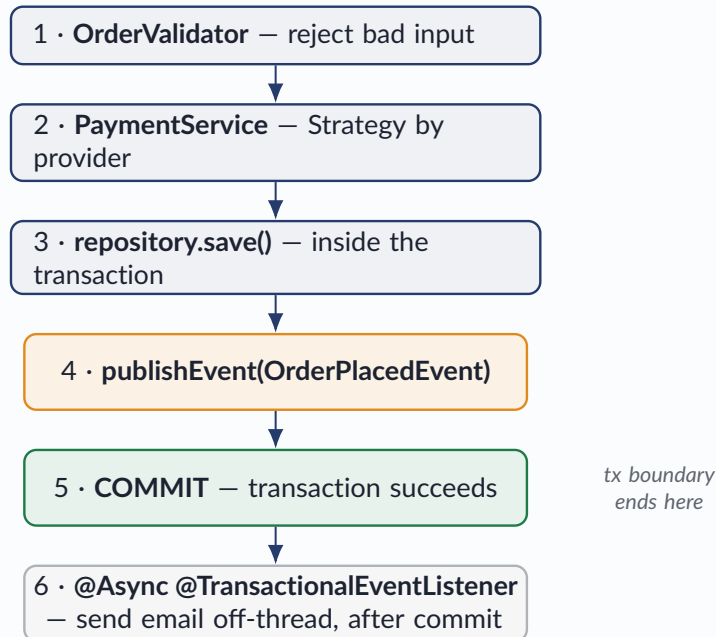


Fig 7.4 — After the refactor: the transaction commits the order, then — and only then — an async listener sends the email on a separate thread.

⚡ Performance note

The email now runs on a separate thread *after* the response has effectively been decided, so the customer's request no longer waits on SMTP. But "async" is not free: you need a real thread pool (configure `@EnableAsync` with a bounded `ThreadPoolTaskExecutor`, not the default unbounded one) and an `AsyncUncaughtExceptionHandler`, because an exception on an async void method vanishes silently otherwise.

✓ Best practices — what experienced engineers do

- Depend on interfaces at service boundaries — even with one implementation — so mocking (Day 6) and swapping stay trivial.
- Replace type-selecting `if/else` chains with the Strategy pattern the moment a second branch appears.
- Use domain events (`ApplicationEventPublisher`) to decouple side effects from the main transaction.
- Keep controllers thin: translate HTTP to a service call, map the result back, nothing more.

- Default to Layered; justify Hexagonal with real, named domain complexity.

✖ Common mistakes beginners make

- Applying Hexagonal/DDD ceremony to a simple CRUD app — indirection with no payoff.
- Letting business logic leak into controllers, or worse, into JPA entities.
- Copy-pasting the "God service" shape because that is what the codebase already does.
- Sending email (or any slow/failable side effect) synchronously inside the order transaction.
- Treating SOLID as dogma rather than trade-offs to reason about case by case.

4 Visualizations to Internalize

Two diagrams carry this chapter. Redraw them from memory — reproducing a diagram is what moves it from "recognised" to "owned."

 Redraw: Layered (Controller → Service → Repository) beside Hexagonal (adapters pointing into a core)

 Redraw: the refactored order flow — validate, pay, save, publish, COMMIT, then async email after commit

5 Guided Coding Lab — Break the God Service Apart

Goal of this lab: take a deliberately tangled `placeOrder` method and refactor it into focused collaborators, ending with an order flow whose side effects are decoupled from its transaction.

This is exactly the ShopFlow increment you will finalise in Part 10.

1 Reproduce the smell

Write (or open) an `OrderService.placeOrder` that does validation, a payment `if/else`, a `save`, and an inline `emailClient.send` in one method. Do not fix it yet — name the four responsibilities out loud first. 💡 *Why*: You cannot refactor what you cannot see; explicitly labelling the four reasons-to-change is the diagnosis that drives every following step.

2 Extract the validator

Move all validation into an `OrderValidator @Component` with a single `validate(OrderRequest)` method that throws a domain exception. Inject it into the service. 💡 *Why*: This is Single Responsibility made physical — validation now changes independently of order orchestration, and becomes unit-testable in isolation.

3 Introduce the PaymentStrategy interface

Define `PaymentStrategy` with a `charge(order)` method and implement two beans, `@Component("stripe")` and `@Component("paypal")`. Delete the `if/else`. 💡 *Why*: You are converting an Open/Closed violation into an extension point — the third provider will be a new file, not an edit to a working one.

4 Resolve the strategy by a Map injection

Inject `Map<String, PaymentStrategy>` into a `PaymentService` and look up the provider by key. Handle the unknown-provider case explicitly. 💡 *Why*: Letting Spring hand you every implementation keyed by bean name is the idiomatic `Factory+Strategy` combination — no registry to maintain by hand.

5 Publish a domain event instead of calling email

Create an `OrderPlacedEvent(orderId)` record. In `placeOrder`, replace the email call with `events.publishEvent(...)`. The service no longer knows email exists. 💡 *Why*: This is the Observer pattern: it inverts the dependency so the order flow depends on nothing downstream, and new reactions (analytics, inventory) attach without touching it.

6 Consume the event after commit, asynchronously

Add a listener annotated `@Async` and `@TransactionalEventListener(phase = AFTER_COMMIT)`. Enable `@EnableAsync` with a bounded executor. 💡 *Why*: After-commit guarantees you never email about an order that rolled back; async guarantees a slow mail server never slows — or fails — the customer's request.

7 Prove the decoupling

Force `emailClient.send` to throw. Confirm the order still commits and the endpoint still returns 200 — only a log line records the failed email. 💡 *Why*: The whole point of the refactor is that a side-effect failure is now contained; a test that proves it is what stops a future engineer from re-coupling them.

🔧 Production tip

Do not delete the God method until the new path is green under test. Refactor *behind* the existing tests from Day 6: they are your safety net proving behaviour is unchanged. A refactor without tests is just a rewrite with extra confidence and no evidence.

6 Thinking Exercises

Reflection prompts, not quizzes. Write a sentence or two — the value is in making your reasoning explicit.

 Think it through

"One responsibility" is famously slippery. For your `OrderService`, list the distinct *reasons it might change* (a pricing rule, a new provider, a schema tweak, a new notification channel). Which of those belong in the service, and which are really separate stakeholders wearing a trench coat?

 Think it through

Hexagonal architecture buys you independence from frameworks at the cost of mapping code and extra interfaces. For ShopFlow's order module specifically, is that trade worth it today? What concrete future event would flip your answer from "no" to "yes"?

 Think it through

Events decouple the publisher from its listeners — but they also make the flow harder to follow, because "what happens after an order is placed?" is no longer answerable by reading one method. When does that loss of traceability outweigh the decoupling, and how would you mitigate it (naming, docs, tracing)?

7

Debugging Practice

 Debugging Lab • A mail outage is taking down order placement

Symptom. Every time the SMTP provider has a slow spell, `POST /orders` starts returning 500 and orders fail to save — even though payment succeeded.

Evidence.

OrderService.java

```
@Transactional
public Order placeOrder(OrderRequest req) {
    Order order = repository.save(toEntity(req));
    emailClient.sendConfirmation(order); // throws on SMTP timeout
    return order;
}
```

Work the method:

- ✓ **Observe.** The 500s correlate exactly with mail-server latency; the stack trace ends in `emailClient.sendConfirmation`.
- ✓ **Hypothesise.** The email runs inside the transaction, so its exception propagates and rolls the `save` back — a side effect is coupled to the core.
- ✓ **Verify.** Temporarily wrap the email in a try/catch that logs and swallows; the 500s stop, confirming email was the cause.
- ✓ **Fix.** Publish an `OrderPlacedEvent` and move the email into an `@Async @TransactionalEventListener(AFTER_COMMIT)` listener so it runs after commit, off-thread.
- ✓ **Prevent.** Add a test that makes `emailClient` throw and asserts the order still commits and the endpoint returns 200.

🔧 Debugging Lab • Spring refuses to start: expected one `PaymentStrategy`, found two

Symptom. After adding a second payment provider, the app fails at startup with `NoUniqueBeanDefinitionException: expected single matching bean but found 2: stripe, paypal`.

Evidence.**PaymentService.java**

```
@Service
public class PaymentService {
    // injecting the interface directly -- but there are now TWO beans
    private final PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) {
        this.strategy = strategy;
    }
}
```

- ✓ **Observe.** Startup fails immediately (fail-fast), naming both candidate beans — `stripe` and `paypal`.
- ✓ **Hypothesise.** You asked for a single `PaymentStrategy`, but two implementations satisfy the type, so the container cannot choose.
- ✓ **Verify.** Temporarily delete one implementation; the app starts — proving ambiguity, not a missing bean, is the issue.
- ✓ **Fix.** Inject `Map<String, PaymentStrategy>` (all of them, keyed by name) and select by provider at call time — the intended Strategy design.
- ✓ **Prevent.** Treat "more than one implementation of an interface" as a design decision: choose Map injection, a `@Qualifier`, or a `@Primary`, on purpose.

8 Mini-Labs

Mini-Lab • Beginner

25 min

Goal. Extract a cohesive validator from a fat method.

Context. A `registerUser` method mixes 30 lines of field validation with the actual save.

Requirements. Move all validation into a `UserValidator @Component`; the service calls `validator.validate(req)` then saves.

Hints. The validator should have no dependency on the repository — it only inspects the request.

Expected outcome. The service method shrinks to a few lines; the validator is unit-testable with plain `new`.

Common pitfalls. Injecting the repository into the validator "just in case" — that recouples them and defeats the split.

Mini-Lab • Intermediate

40 min

Goal. Implement the Strategy pattern for two payment providers behind one interface.

Context. Callers currently branch on a `provider` string with an `if/else`.

Requirements. Define `PaymentStrategy`; implement `stripe` and `paypal` beans; resolve via `Map<String, PaymentStrategy>`; add a third provider with zero edits to existing classes.

Hints. The bean name (`@Component("stripe")`) becomes the map key.

Expected outcome. Adding "applepay" is one new file; the `if/else` is gone.

Common pitfalls. Injecting a single `PaymentStrategy` and hitting `NoUniqueBeanDefinitionException` — inject the `Map` instead.

Mini-Lab • Production-style

60 min

Goal. Fully decouple order confirmation email from the order transaction.

Context. Email currently runs inline and occasionally rolls orders back.

Requirements. Publish `OrderPlacedEvent`; handle it in an `@Async @TransactionalEventListener(AFTER_COMMIT)`; configure `@EnableAsync` with a bounded `ThreadPoolTaskExecutor` and an `AsyncUncaughtExceptionHandler`.

Hints. Verify ordering by logging the commit and the email on different threads.

Expected outcome. A thrown email exception leaves the order committed and the response at 200; the failure appears only in logs.

Common pitfalls. Using a plain `@EventListener` (runs in-transaction) or the default async executor (unbounded — a mail backlog can exhaust threads).

9 Engineering Notes

Judgment from engineers who have carried a pager. Read these as reasoning, not trivia.

 How senior engineers think

The best architecture is the one your team can still change quickly a year from now — not the one with the most patterns. Seniors optimise for the *next* engineer's ability to make a change safely, which usually means fewer, clearer boundaries rather than more. "Could a new hire find where to add a feature in five minutes?" is a better architecture metric than any diagram.

 Interview insight

Expect: "What is the difference between coupling and cohesion?" A crisp answer: "Coupling is *between* modules — how much a change here forces a change there; you want it low. Cohesion is *within* a module — how much its parts belong together; you want it high. SOLID is mostly a toolkit for pushing both dials in the right direction."

 Performance note

Domain events decouple, but at scale they can hide cost: fifteen listeners on `OrderPlacedEvent`, each doing a database write, turns one logical action into sixteen. When a flow feels slow, count the listeners — observability tools that don't show event fan-out will hide it from you. Decoupling is a design win, not a free lunch.

 Production tip

When you inherit a God service, do not rewrite it — *strangle* it. Add tests that pin current behaviour, then extract one collaborator at a time, keeping the suite green after each step. A big-bang rewrite of a critical path is how teams ship a month of regressions; incremental extraction is how they ship none.

10 Build-Along Project — ShopFlow

ShopFlow is persistent (Days 2–3), exposed over REST (Day 4), secured (Day 5), and tested (Day 6). But its `placeOrder` flow has quietly grown into a God service: it validates, selects a payment provider with an `if/else`, saves, and sends a confirmation email — all in one transactional method, so a mail outage can roll back real orders. Today you make it clean.

 Build-Along • Day 7 — refactor order placement toward clean layering

- Extract all order validation into a dedicated `OrderValidator @Component`; the service calls it first and nothing else validates.
- Replace the payment `if/else` with a `PaymentStrategy` interface and per-provider beans, resolved through a `Map<String, PaymentStrategy>` injection (Open/Closed by construction).
- Keep the controller thin and depending only on the service interface — no business logic, no JPA types above the service line.
- Publish an `OrderPlacedEvent` via `ApplicationEventPublisher` on successful placement; the service must not know that email exists.
- Move "send confirmation email" into an `@Async @TransactionalEventListener(phase = AFTER_COMMIT)` listener, backed by a bounded executor and an uncaught-exception handler.
- Add a test that forces the email to fail and asserts the order still commits and the endpoint returns 200.

Definition of done: `placeOrder` reads as five clear steps (validate → pay → save → publish → return), a new payment provider needs zero edits to existing classes, and a simulated mail-server failure no longer rolls back an order. ShopFlow is now structured to survive the next seven days of features.

11 Chapter Summary

Key takeaways

- ✓ Architecture is dependency management: point dependencies at stable abstractions, and put things that change together in one place.
- ✓ SOLID is five tactics for low coupling and high cohesion — concrete class splits and interfaces, not slogans.
- ✓ Layered is the right default; Hexagonal earns its boilerplate only when domain complexity justifies protecting the core.
- ✓ Spring is built from GoF patterns; recognising them shows you the intended extension points.
- ✓ Strategy replaces type-switching `if/else`; domain events decouple side effects from the transaction that triggers them.

Things to remember

- `Map<String, PaymentStrategy>` injection gives you every implementation keyed by bean name — Strategy without a hand-written registry.
- `@TransactionalEventListener(AFTER_COMMIT) + @Async` is the safe way to run a side effect only after a successful commit, off the request thread.

Common pitfalls (last look)

- Inline email inside a transaction → a mail outage becomes an ordering outage.
- Hexagonal on a CRUD app → indirection with no payoff. • Fat interface → forced dummy methods.
- Injecting a single bean where two implementations exist → `NoUniqueBeanDefinitionException` at startup.

📄 One-Page Cheat Sheet

SRP — One reason to change per class — separate stakeholders, separate classes.

Open/Closed — Extend by adding a Strategy bean, not by editing a working `if/else`.

Dependency Inversion — Depend on an interface the high-level module owns; the detail implements it.

Coupling / Cohesion — Low between modules, high within — the two dials SOLID tunes.

Layered vs Hexagonal — Layered by default; Hexagonal only for complex, long-lived domains.

Patterns in Spring — Proxy · Factory · Singleton · Template Method · Strategy · Observer · Decorator.

Events — `publishEvent + @TransactionalEventListener(AFTER_COMMIT) + @Async` = decoupled side

effect.

Interview questions to master

- › Explain SOLID with a Spring Boot example for each letter.
- › When would you choose Hexagonal architecture over a simple layered approach?
- › Name three design patterns Spring itself uses internally, and where.
- › How would you refactor a service method that has grown to 300 lines?
- › What is the difference between coupling and cohesion?
- › Why should a confirmation email be sent after commit rather than inside the transaction?

Suggested further reading

- Robert C. Martin, *Clean Architecture* — the dependency rule and SOLID in depth.
- Alistair Cockburn, *Hexagonal Architecture* (the original "Ports & Adapters" article).
- *Spring Framework Reference* — "Application Events" and the `@TransactionalEventListener` docs.
- Uber Engineering, *Domain-Oriented Microservice Architecture* — layering at real scale.

Key idea

What you will learn next (Day 8). Your ShopFlow app is now cleanly structured and self-contained — exactly the property that makes it portable. Tomorrow you package it into a Docker image and stand up its dependencies with Compose, turning "runs on my machine" into "runs identically anywhere," the foundation for the CI/CD and deployment days that follow.

DAY 08 • INFRASTRUCTURE

Docker

08

Containerizing the backend — images, multi-stage builds, Compose & networking

 Estimated study time: 7–8 hours By the end of this day, you will be able to...

- Explain what a container actually is at the kernel level — and why it is not a lightweight virtual machine.
- Reason about images as stacked, cached layers, and order a `Dockerfile` so rebuilds are fast.
- Write a production-grade *multi-stage* Dockerfile for a Spring Boot app: a JDK build stage, a slim JRE runtime, a non-root user.
- Bring up the app plus PostgreSQL and Redis with one `docker compose up`, wired by *service name*, not `localhost`.
- Configure the same image for every environment through environment variables, and persist data with named volumes.

1 The Big Picture

What this day solves. For seven days your app has run one way: `./mvnw spring-boot:run` on your laptop, against a database you installed by hand. That works until it has to run somewhere else — a teammate's machine, a CI runner, a staging server, production. Every one of those has a different JDK patch level, a different libc, a different Postgres, a different set of ports already taken. The phrase “*but it works on my machine*” is the sound of that gap. Today you close it: you package ShopFlow — app plus its exact runtime — into an image that runs *identically* everywhere.

Why backend engineers need it. Containers are the unit of deployment for modern backends. CI builds one, a registry stores it, Kubernetes or ECS schedules it, autoscaling replicates it. If you cannot write a correct Dockerfile you cannot ship; if you write a careless one you ship a 900 MB image running as root with your build tools and, occasionally, your secrets baked in. The difference between those two outcomes is entirely today's material.

 Key idea

A container is not a small virtual machine. It is one or more ordinary Linux processes that the kernel has been told to **isolate** — given their own view of the filesystem, network, and process table — while still sharing the *host's* kernel. That single fact explains everything else: why containers start in milliseconds, why an image is just a stack of files, and why “it runs the same everywhere” as long as the kernel is Linux.

Where it fits in a Spring Boot application. Nothing about your Java code changes today. The layered app from Days 1–4, the persistence from Days 2–3, the security from Day 5, the tests from Day 7 — all of it stays exactly as written. What changes is the *boundary around* the app: how it is built into an

artifact, how it finds its database, how it reads its configuration. Docker sits underneath everything and becomes the substrate for CI/CD (Day 12) and any orchestration that follows.

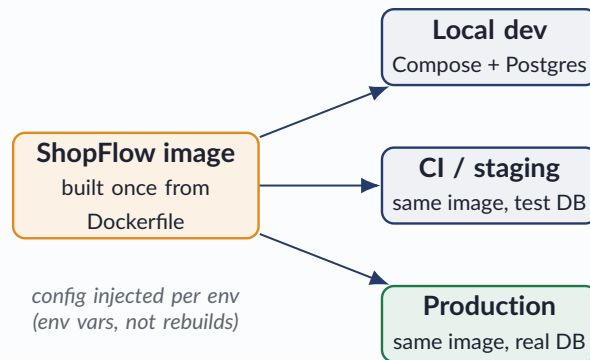


Fig 8.1 — The same immutable image, built once, promoted unchanged across every environment; only the injected configuration differs.

Real company examples. Google runs everything in containers and generates billions of container starts a week; the ideas in Docker descend directly from Google's internal *Borg*. Netflix bakes images in CI and promotes the exact bytes to production. Every managed platform you will meet — AWS ECS/Fargate, Google Cloud Run, Fly.io, Heroku's newer stack — takes an image as its input. Learn the image and you can deploy anywhere.

2 Concept Map

Hold the whole territory in one picture. A **Dockerfile** is a recipe; building it produces an **image** (a stack of read-only layers); running the image produces a **container** (a live process with a thin writable layer on top); **Compose** runs several containers together on a private network.

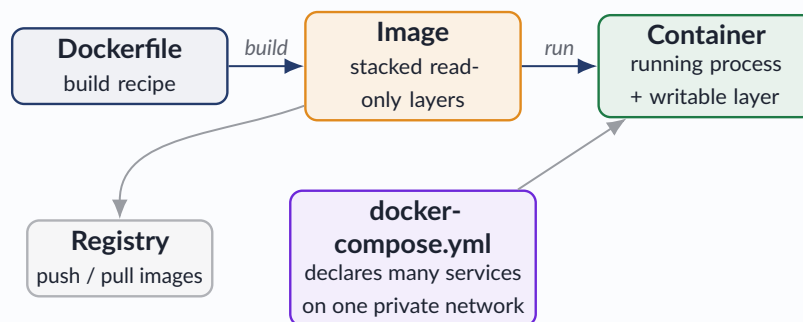


Fig 8.2 — The Day 8 concept map: recipe becomes image becomes container; Compose orchestrates several on one network.

Terminology to anchor: **Dockerfile** → **layer** → **image** → **registry** → **container** → **volume** → **Compose service** → **user-defined network**.

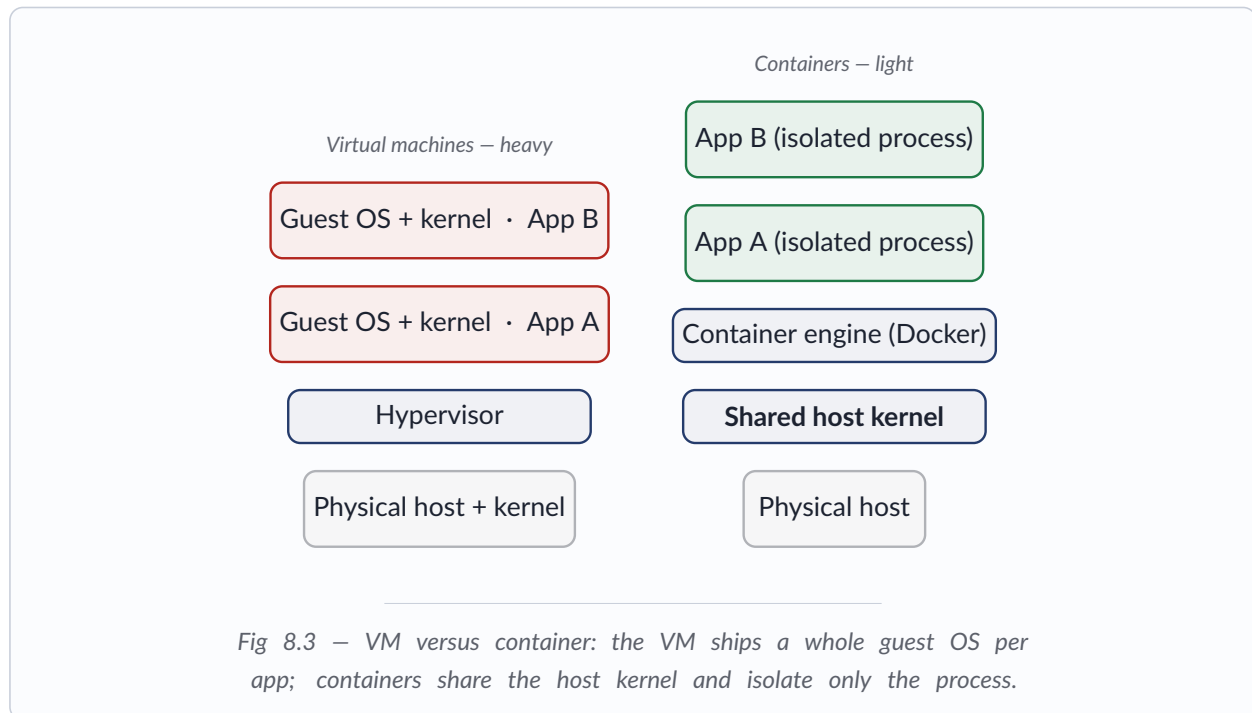
3 Guided Learning

3.1 What a Container Actually Is (and Why It Is Not a VM)

What is it? A **container** is a normal process — your `java -jar app.jar` — that the Linux kernel runs with a restricted, private view of the system. It sees its own root filesystem, its own network interface, its own process IDs, but it is scheduled by the same kernel as every other process on the host.

Why does it exist? We wanted the isolation and reproducibility of a virtual machine without paying for a whole guest operating system per app. A VM virtualises *hardware* and boots a full guest kernel; that costs gigabytes of RAM and tens of seconds to start. A container virtualises the *operating-system interface* instead, so ten containers share one kernel and start in milliseconds.

How does it work internally? Two Linux kernel features do the heavy lifting. **Namespaces** give a process a private view of a resource — there are namespaces for the filesystem mount table, the network stack, process IDs, users, and more. **Control groups (cgroups)** limit and account for how much CPU, memory, and I/O a process may consume. Docker is, at heart, a friendly way to launch a process wrapped in the right set of namespaces and cgroups, using an image as its root filesystem.



When should I use it? For essentially every backend service you deploy. Uniform build, uniform run, trivial local parity, and it is what every orchestrator expects.

When should I not? When you genuinely need a *different kernel* (a Windows app on a Linux host), or hard security isolation between mutually hostile tenants — containers share the kernel, so a kernel exploit escapes them. Those cases want VMs or micro-VM sandboxes (Firecracker, gVisor).

⚠ Common misconception

“Alpine images are tiny because Alpine is a lightweight kernel.” There is no Alpine kernel in the image at all — every container uses the host’s kernel. Alpine is small because its *userland* (its `libc` and base tools) is small. In fact Alpine uses `musl libc` rather than `glibc`, which occasionally breaks native libraries; a common gotcha with certain JVM or DNS behaviours.

3.2 Images, Layers & the Build Cache

What is it? An **image** is a stack of read-only **layers**. Each instruction in a Dockerfile that changes the filesystem (`COPY`, `RUN`, `ADD`) produces one new layer — a diff on top of the layers below it. The image is those diffs stacked by a union filesystem; a running container adds one thin *writable* layer on top.

Why does it exist? Layers are shared and cached. If two images start `FROM eclipse-temurin:21-jre-alpine`, that base layer is stored once on disk and downloaded once over the network. And during a build, Docker reuses a layer's cached result if its instruction and inputs are unchanged — so a rebuild after a one-line code change can skip re-downloading every dependency.

How does it work internally? The build cache is keyed, in order, on each instruction and the content it touches. Docker walks the Dockerfile top to bottom; for each step it checks whether it has a cached layer for “this instruction, applied to this parent, over these input files.” The moment one step misses, *every step after it is rebuilt* — the cache invalidates forward. This is the single most important thing to understand about writing a fast Dockerfile.

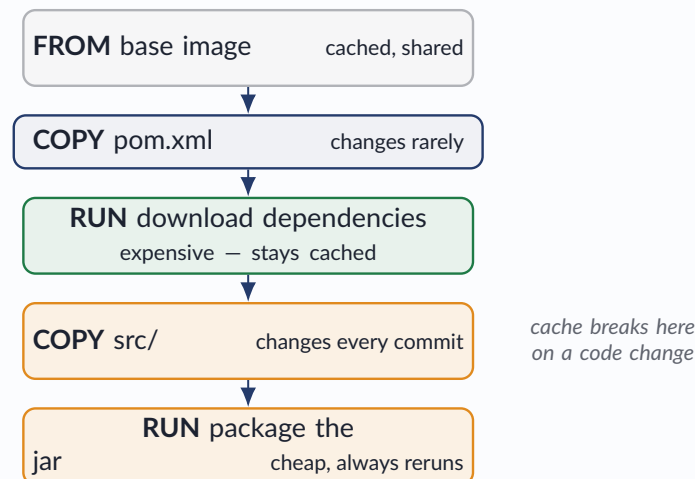


Fig 8.4 — Layer cache invalidation flows downward: copy rarely-changing dependencies before frequently-changing source so the expensive step stays cached.

When should I use it? Always order instructions *stable-to-volatile*: base image, then dependency manifests, then the dependency download, then application source last. That way an edit to a controller invalidates only the last two cheap layers, not the multi-minute dependency resolution.

⚡ Performance note

Each `RUN` makes a layer, and a layer's size is the *net* of files it added — deleting a file in a *later* `RUN` does not shrink the earlier layer, it just hides the file. That is why “`RUN apt-get install ...`” and its cleanup must live in the *same* `RUN`, joined with `&&`. In multi-stage builds (next topic) this matters far less, because you copy only the finished jar into a fresh final stage.

3.3 The Multi-Stage Dockerfile

What is it? A **multi-stage build** writes several `FROM` sections in one Dockerfile. Earlier stages do work you need but do not want to ship (compiling, running Maven); a final stage starts from a clean, minimal base and copies *only the finished artifact* out of an earlier stage with `COPY --from=`.

Why does it exist? To build with a fat toolchain and run with a lean one. Compiling a Spring Boot app needs a full JDK plus Maven plus your entire dependency cache — hundreds of megabytes you must never ship to production. The runtime needs only a JRE and one jar. Before multi-stage builds people maintained two Dockerfiles or brittle shell scripts; now it is one file, and the toolchain simply never reaches the final image.

Dockerfile — ShopFlow, production-grade

```
# ----- Stage 1: build -----
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app

# copy only what the dependency resolution needs, first
COPY .mvn/ .mvn
COPY mvnw pom.xml ./
RUN ./mvnw dependency:go-offline -B

# now the source; edits here don't bust the dependency layer
COPY src ./src
RUN ./mvnw clean package -DskipTests -B

# ----- Stage 2: runtime -----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# create and switch to an unprivileged user
RUN addgroup -S spring && adduser -S spring -G spring
USER spring

# copy ONLY the finished jar out of the build stage
COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

How does it work internally? Each stage is an independent image built in the same pass. `COPY --from=build` reaches into the *build* stage's filesystem and lifts out the path you name. When the build finishes, Docker tags and keeps only the *last* stage; the JDK/Maven stage is discarded (though its layers stay in the build cache for next time). The final image is essentially: Alpine JRE base + one jar.

💡 How senior engineers think

Notice `USER spring` sits *before* the `COPY` and `ENTRYPOINT`. Running as a non-root user is not decoration — if an attacker achieves remote code execution inside the container, a root process can attempt to exploit the kernel from a much stronger position than an unprivileged one can. Many platforms (OpenShift, hardened Kubernetes) will *refuse* to run a container that declares no non-root user. Set it once, here, forever.

When should I use it? For every compiled-language service. It is the default, not an optimisation.

When should I not? Rarely — but if you already produce the jar in CI *before* the Docker build, your final Dockerfile can be a single stage that just copies that pre-built jar in. The principle survives: the runtime image contains only the JRE and the jar.

👁 Behind the scenes

Spring Boot supports **layered jars**: the fat jar is internally split into dependencies, snapshot dependencies, and your application classes, so Docker can cache the (large, stable) dependency layer separately from the (small, volatile) application layer. Extract with `java -Djarmode=layertools -jar app.jar extract` in the build stage and `COPY` each layer in order. For a first production image the simple single-jar copy above is completely fine; reach for layered jars when image push time becomes a bottleneck.

3.4 Docker Compose & Service-Name Networking

What is it? **Docker Compose** declares a set of containers — “services” — and their wiring in one `docker-compose.yml`, brought up together with `docker compose up`. For local development it is how you run ShopFlow’s whole world (app, database, cache) with one command.

Why does it exist? A real backend is never one container. You need Postgres and Redis alongside the app, on a network where they can find each other, with volumes for data and a repeatable teardown. Doing that by hand with `docker run` flags is error-prone; Compose makes it a file you commit.

docker-compose.yml

```
services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/shopflow
      SPRING_DATASOURCE_USERNAME: shopflow
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
      SPRING_DATA_REDIS_HOST: redis
      SPRING_PROFILES_ACTIVE: docker
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started

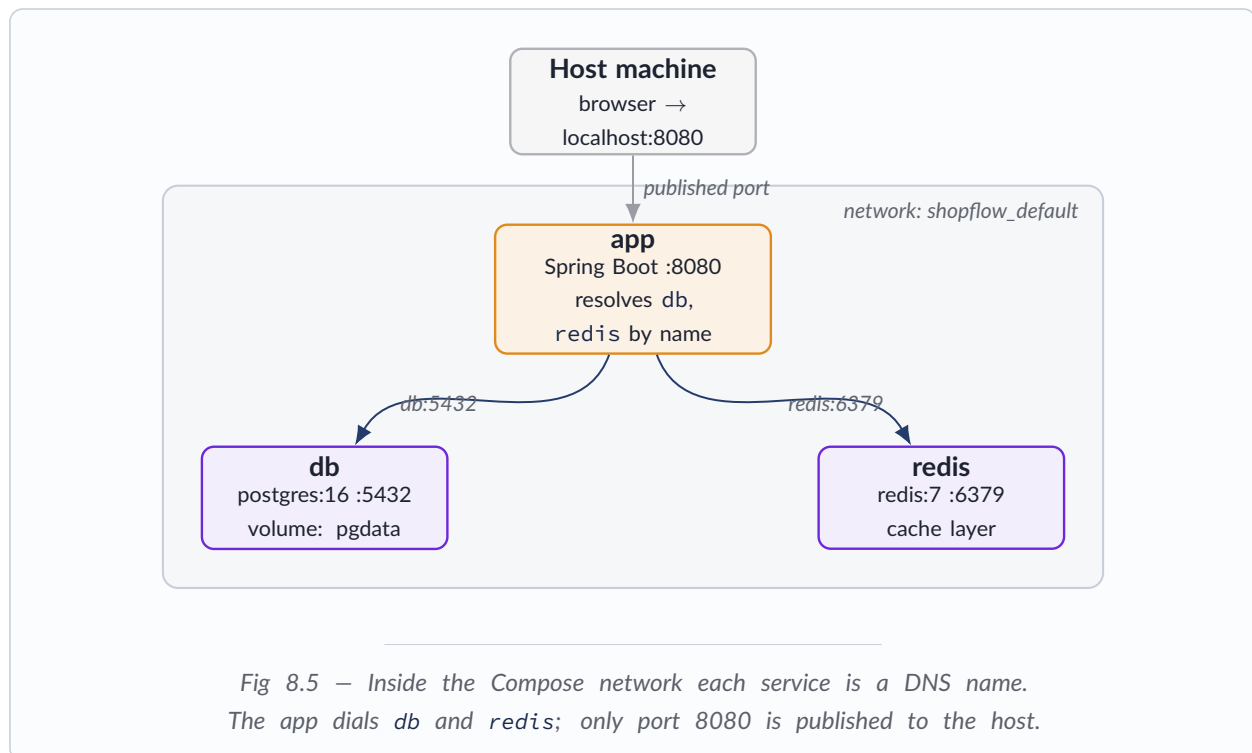
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: shopflow
      POSTGRES_USER: shopflow
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shopflow"]
      interval: 5s
      timeout: 3s
      retries: 5

  redis:
    image: redis:7-alpine

volumes:
  pgdata:
```

How does it work internally? Compose creates a **user-defined bridge network** for the project and

attaches every service to it. On that network Docker runs an embedded DNS server, and *each service is reachable by its service name*. So the app reaches Postgres at the host `db` and Redis at `redis` — those names resolve to the containers' current internal IPs. This is why the datasource URL is `jdbc:postgresql://db:5432/...` and never `localhost`.



Why `depends_on` with a condition? Plain `depends_on` only controls *start order* — it starts the DB container first, but does not wait for Postgres to be *ready to accept connections*. Postgres takes a second or two to initialise; the app can win the race and crash on “connection refused.” A `healthcheck` plus `condition: service_healthy` makes Compose wait until `pg_isready` passes before starting the app.

Production tip

`depends_on` is a *local-development* convenience, not a production guarantee. In Kubernetes there is no `depends_on`; the platform restarts your pod until its dependencies are reachable. So even with a healthcheck, your app must still tolerate its database being briefly unavailable — connection-pool retries and a readiness probe are the real fix. Compose ordering just makes local runs pleasant.

3.5 Environment Configuration, Secrets & Named Volumes

What is it? The same image must behave differently per environment without being rebuilt. Docker’s answer is **environment variables**: values injected at `run` time, read by the process at startup. Spring Boot maps them onto its properties automatically.

How does it work internally? Spring Boot’s **relaxed binding** turns an environment variable like `SPRING_DATASOURCE_URL` into the property `spring.datasource.url`: uppercase, dots and dashes become underscores. So you never edit `application.yml` to point at the containerised database — you set `SPRING_DATASOURCE_URL` in Compose and Spring picks it up, overriding the file. This is the exact mechanism behind Day 1’s “one artifact, many environments” promise.

.env – git-ignored, local only

```
# Read automatically by docker compose for ${...} substitution.
# NEVER commit this file. Commit a .env.example with blanks instead.
DB_PASSWORD=local-dev-only-password
```

Why never bake secrets into the image? An image is a stack of layers anyone who can pull it can inspect — `docker history` and a layer unpack reveal every `ENV` and every copied file. A password written into the Dockerfile is a password published to your registry. Inject secrets at run time instead: a git-ignored `.env` locally, and a real secrets manager (AWS Secrets Manager, Vault, Kubernetes Secrets) in production — the subject of Day 12.

👤 Security warning • secrets leak through layers and history

A secret “removed” in a later Dockerfile instruction still lives in the earlier layer that added it. `docker history --no-trunc` and simple layer extraction expose it. The only safe secret is one that was *never* written to a layer: pass it as a runtime environment variable or mount it, and keep `.env` out of version control and out of the build context (via `.dockerignore`).

Why named volumes? A container’s writable layer is *ephemeral* — `docker compose down` and it is gone, taking the database with it. A **named volume** (pgdata above) is storage that Docker manages *outside* any single container’s lifecycle, mounted into Postgres’s data directory. It survives restarts and recreates, and is the correct way to persist stateful data locally.

🗄 Database insight

Bind-mounting a host directory into Postgres (`./data:/var/lib/postgresql/data`) works on Linux but causes permission and performance grief on Docker Desktop for Mac/Windows, where the data crosses a virtualisation boundary. Prefer a *named volume* for database data — Docker keeps it inside its own managed storage where the database’s `uid` owns the files cleanly.

You also want a `.dockerignore`, the build-context twin of `.gitignore`. Before a build, Docker sends the whole build context (your project directory) to the daemon; without exclusions that means `.git`, `target/`, IDE folders — often hundreds of megabytes — copied on every build and, worse, potentially copied *into* the image.

.dockerignore

```
.git
target/
*.iml
.idea/
.env
**/*.log
Dockerfile
docker-compose.yml
```

✓ Best practices — what experienced engineers do

- Multi-stage always: build with the JDK, ship only a JRE and the jar.
- Run as a declared non-root user (`USER spring`).
- Pin base image tags (`postgres:16-alpine`), never `latest`.
- Order the Dockerfile stable-to-volatile so the dependency layer stays cached.

- Named volumes for anything that must survive `compose down`; a `.dockerignore` to keep the context lean.

✖ Common mistakes beginners make

- Copying the whole project (`.git`, `target/`) in with no `.dockerignore`.
- Hardcoding `localhost` as the DB host in Compose instead of the service name `db`.
- `latest` tags everywhere, making builds non-reproducible.
- Running the app as root; baking passwords into `ENV` or the image.
- Assuming `depends_on` waits for the database to be *ready*.

4 Visualizations to Internalize

You have met five diagrams. Reproducing two of them from memory is what converts “I recognised that” into “I can draw that at a whiteboard in an interview.”

 Redraw: the VM stack vs the container stack — mark exactly where the kernel is shared

 Redraw: the Compose network — three services, which names resolve, which port is published to the host

5 Guided Coding Lab — Containerize ShopFlow

Goal of this lab: take the ShopFlow app exactly as it stands after Day 7 and give it a production-grade image and a one-command local stack. You will *watch* the image shrink and the services find each

other by name.

1 Baseline a naive single-stage image

Write a throwaway `Dockerfile.naive` that does `FROM eclipse-temurin:21-jdk`, copies everything, and runs `./mvnw package`. Build it and record the size with `docker images`. 💡 *Why*: You need a “before” number so the multi-stage win is a measurement, not a claim — expect roughly 700–900 MB.

2 Write the multi-stage Dockerfile

Create the real `Dockerfile` from Topic 3.3: JDK build stage, dependency layer before source, JRE-alpine runtime, non-root `spring` user, `COPY --from=build`. 💡 *Why*: This is the artifact every later day and every deploy target consumes; getting the stage boundary right is the whole point.

3 Add a `.dockerignore` and measure the context

Add the `.dockerignore` from Topic 3.5. Rebuild and watch the “transferring context” size in the build output drop. 💡 *Why*: A lean context builds faster and prevents `.git` or a stray `.env` from ever entering an image layer.

4 Compare the two images

Run `docker images shopflow` for both tags side by side. The multi-stage image should be a small fraction of the naive one. 💡 *Why*: Seeing 800 MB collapse to under 200 MB is the moment multi-stage stops being abstract.

5 Write `docker-compose.yml` for app + db + redis

Use the Compose file from Topic 3.4: service-name networking, `${DB_PASSWORD}` from a git-ignored `.env`, a `pgdata` named volume, a Postgres healthcheck gating the app. 💡 *Why*: This is the local world your teammates will `git clone` && `docker compose up` into on day one.

6 Bring it up and prove networking

Run `docker compose up --build`. Watch the app wait for the DB healthcheck, then connect to `db:5432`. Hit `localhost:8080` from your browser. 💡 *Why*: You are confirming the two hardest-won facts of the day: readiness gating works, and `db` resolves without `localhost`.

7 Prove the volume persists

Insert a row, run `docker compose down` (not `-v`), then `docker compose up` again and confirm the row is still there. 💡 *Why*: This is the difference between a demo and a database: state must outlive the container.

🔑 Production tip

Do not copy image sizes from this page — run the two builds and read *your own* `docker images` output. The number depends on your dependency set, and the habit of measuring (not assuming) image size is exactly what keeps production images from quietly bloating over a year of commits.

6 Thinking Exercises

Write a sentence or two for each — the goal is to make your reasoning explicit, not to be right on the first try.

 Think it through

Your image ships in 40 seconds after a code-only change but takes 6 minutes after you edit `pom.xml`. Explain, in terms of layer-cache invalidation, why editing the dependency manifest is so much more expensive — and what that tells you about instruction ordering.

 Think it through

Compose lets the app reach Postgres at the hostname `db`. From first principles, what makes `db` resolve to an IP address inside the network, and why would the very same URL fail if you ran the app with a bare `docker run` on the default bridge?

 Think it through

A teammate argues “non-root inside a container is pointless — it is isolated anyway.” Give the strongest counter-argument, and name one thing running as root inside the container makes materially easier for an attacker who has already achieved code execution.

7 Debugging Practice

 Debugging Lab • The app can't reach the database — “Connection refused: localhost:5432”

Symptom. Under Compose the app crashes at startup; the log shows it trying `localhost:5432` and getting connection refused, even though the `db` container is clearly running.

Evidence.

docker-compose.yml (excerpt)

```
app:
  build: .
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://localhost:5432/shopflow
  depends_on: [db]
```

Work the method:

- ✓ **Observe.** The URL host is `localhost`. Inside the app container, `localhost` is *that container itself*, where nothing listens on 5432.
- ✓ **Hypothesise.** On a Compose network each service is reachable by its *service name*, so the DB is at host `db`, not `localhost`.
- ✓ **Verify.** `docker compose exec app getent hosts db` resolves to an IP; `getent hosts localhost` points back at the app.
- ✓ **Fix.** Change the URL to `jdbc:postgresql://db:5432/shopflow`.
- ✓ **Prevent.** Never hardcode `localhost` in container config; use service names and inject them as environment variables.

🔧 Debugging Lab • Intermittent boot failure — the app wins the race against Postgres

Symptom. `docker compose up` succeeds maybe half the time. When it fails, the app log shows a connection error in the first second, then the container exits; the DB comes up healthy moments later.

- ✓ **Observe.** `depends_on: [db]` only orders *startup*; it does not wait for Postgres to accept connections, and Postgres needs a second or two to initialise.
- ✓ **Hypothesise.** The app is racing the database: sometimes it connects, sometimes it tries before Postgres is listening.
- ✓ **Verify.** `docker compose logs db` shows “database system is ready to accept connections” *after* the app’s failed attempt.
- ✓ **Fix.** Add a `healthcheck` to `db` and gate the app with `depends_on: db: condition: service_healthy`.
- ✓ **Prevent.** Beyond Compose, make the app itself resilient — connection-pool retries and a readiness probe — so it tolerates a briefly-absent database in any environment.

8 Mini-Labs**🔧 Mini-Lab • Beginner**

25 min

Goal. Feel the multi-stage size win as a measured number.

Context. You have a working Spring Boot jar and a single-stage image already.

Requirements. Write the multi-stage Dockerfile; build both; record both sizes.

Hints. `docker images` shows a `SIZE` column; tag them `shopflow:naive` and `shopflow:multi`.

Expected outcome. The multi-stage image is a small fraction of the single-stage one.

Common pitfalls. Forgetting `COPY --from=build`, so the final stage has no jar; or leaving `-jdk` instead of `-jre` in the runtime stage.

Mini-Lab • Intermediate

45 min

Goal. Stand up app + Postgres + Redis with one command and correct wiring.

Context. The app reads its datasource and Redis host from environment variables.

Requirements. Compose file with service-name networking, a `pgdata` named volume, and `${DB_PASSWORD}` sourced from a git-ignored `.env`.

Hints. Verify name resolution with `docker compose exec app getent hosts db`.

Expected outcome. One `docker compose up` brings the whole stack live; the app connects to `db` and `redis` by name.

Common pitfalls. Using `localhost` for the DB host; committing the `.env`; publishing 5432 to the host when nothing outside needs it.

Mini-Lab • Production-style

75 min

Goal. Harden the image and stack the way a platform team would review it.

Context. You are preparing this image to be promoted unchanged from CI to production.

Requirements. Non-root user; `.dockerignore`; pinned base tags; a Postgres `healthcheck` gating the app; a `HEALTHCHECK` or Actuator readiness endpoint; container memory limits.

Hints. Point the JVM at container limits with `-XX:MaxRAMPercentage`; `expose /actuator/health` for the healthcheck.

Expected outcome. `docker compose up` yields a stack where the app starts only after the DB is healthy and reports healthy itself.

Common pitfalls. A healthcheck that curls an endpoint the app secures behind auth; setting a heap larger than the container's memory limit and getting OOM-killed.

9 Engineering Notes

A curated margin from engineers who have shipped and debugged images at scale. Read these as judgment, not trivia.

How senior engineers think

The JVM did not understand cgroup memory limits until the container-awareness fixes that are standard on Java 11+. On a modern JDK the JVM reads the container's limit, but you should still set `-XX:MaxRAMPercentage` (say 75) rather than a fixed `-Xmx` — then the same image sizes its heap correctly whether it runs with 512 MB or 4 GB, which is exactly the “one image, many environments” property you want.

Interview insight

Expect: “What is the difference between an image and a container?” The answer that lands: an image is an immutable, versioned stack of read-only layers — a template; a container is a running instance of that image with a thin *writable* layer on top. Many containers can run from one image; the image never changes. Say “read-only layers” and “writable layer” and you have shown you know the mechanism, not just the metaphor.

⚡ Performance note

Pin base images by *digest*, not just tag, when reproducibility matters: `FROM eclipse-temurin:21-jre-alpine@sha256:...` guarantees the identical bytes months later, whereas the `21-jre-alpine` tag is re-pointed at new patch builds over time. Tags are convenient for humans; digests are what make a build truly deterministic in CI.

🔑 Production tip

Add a `HEALTHCHECK` that hits your Actuator readiness endpoint so the orchestrator knows when the app is genuinely ready, not merely started. But keep the readiness path unauthenticated or the healthcheck itself will report unhealthy forever — a classic self-inflicted outage where the platform kills a perfectly healthy app because it cannot get past your own security filter.

10 Build-Along Project — ShopFlow

By Day 7, ShopFlow is a clean-layered, tested, secured Spring Boot app that you run locally against a hand-installed database. Today you make it portable: one image, one command, service-name networking, no secrets on disk.

🔧 Build-Along • Day 08 — containerize the whole stack

- Add a production-grade *multi-stage* Dockerfile: JDK build stage with the dependency layer before source, JRE-alpine runtime, a non-root `spring` user, `COPY --from=build` of only the jar.
- Add a `.dockerignore` excluding `.git`, `target/`, IDE files, `.env`, and logs so the build context stays lean and secret-free.
- Write `docker-compose.yml` bringing up `app` + `db` (`postgres:16-alpine`) + `redis` (`redis:7-alpine`) on one network, addressed by service name (`db`, `redis` — never `localhost`).
- Configure everything through environment variables (`SPRING_DATASOURCE_URL`, `SPRING_DATA_REDIS_HOST`, `SPRING_PROFILES_ACTIVE=docker`); `source ${DB_PASSWORD}` from a git-ignored `.env`.
- Persist the database with a named `pgdata` volume, and gate the app on a Postgres healthcheck so startup is race-free.

Definition of done: a fresh clone runs the entire stack with a single `docker compose up --build`; the app reaches Postgres and Redis by service name; inserted data survives `docker compose down` and a fresh `up`; and no secret appears in the image, the Dockerfile, or a committed file. This image is the exact artifact Day 12 will build in CI and promote to production.

11 Chapter Summary**Key takeaways**

- ✓ A container is an isolated *process* sharing the host kernel via namespaces and cgroups — not a VM.
- ✓ An image is a stack of cached, read-only layers; the cache invalidates *downward*, so order

Dockerfiles stable-to-volatile.

- ✓ Multi-stage builds compile with a JDK and ship only a JRE plus the jar, as a non-root user.
- ✓ On a Compose network, services reach each other by *service name*; `localhost` means “this container.”
- ✓ Configure per environment with environment variables and persist state with named volumes; keep secrets out of every layer.

Things to remember

- `depends_on` orders startup; only a `healthcheck` with `condition: service_healthy` waits for readiness.
- `docker history` exposes every `ENV` and layer — a baked-in secret is a published secret.

Common pitfalls (last look)

- `localhost` DB host in Compose → connection refused. • No `.dockerignore` → bloated context and leaked files.
- `latest` tags → non-reproducible builds. • Root user + baked secrets → avoidable risk.

📄 One-Page Cheat Sheet

Image vs container — image = immutable read-only layers (template); container = running instance + thin writable layer.

Multi-stage — `FROM jdk AS build ...FROM jre; COPY --from=build` only the jar; toolchain never ships.

Layer cache — keyed per instruction + inputs; first miss rebuilds everything after it — order stable-to-volatile.

Compose networking — services reach each other by service name via embedded DNS; `localhost` = this container.

depends_on — orders start; add a `healthcheck` + `condition: service_healthy` to wait for readiness.

Config & secrets — env vars per environment (relaxed binding); never bake secrets — inject at run time.

Persistence — named volume (`pgdata`) survives `compose down`; the writable layer does not.

👤 Interview questions to master

- › Why use a multi-stage Dockerfile instead of a single-stage build?
- › How do containers on the same Compose network talk to each other?
- › What is the difference between a Docker image and a container?
- › How would you persist database data across container restarts?
- › Why avoid the `latest` tag in production?
- › Why run the container as a non-root user, and how do you set that up?

Suggested further reading

- *Docker docs* — “Multi-stage builds” and “Networking in Compose.”

- *Spring Boot Reference* — “Container Images” (buildpacks, layered jars) and “Externalized Configuration.”
- *Google* — “Best practices for building containers” (non-root, minimal base, reproducibility).

 Key idea

What you will learn next (Day 09). Your app now runs beside a real Redis container. Tomorrow you put it to work: caching. You will learn where a cache belongs in the request path, how `@Cacheable` is delivered by the very proxies you met on Day 1, and how to avoid the stampedes and stale-data traps that make caching subtle in production.

DAY 09 • PERFORMANCE

Redis & Caching

09

Serving reads without hammering the database — cache-aside, TTL, eviction

🕒 Estimated study time: 6–7 hours

🎯 By the end of this day, you will be able to...

- Decide *when* caching genuinely helps and when it only buys you a new consistency problem to debug.
- Explain the cache-aside pattern precisely — who owns the logic, what a hit and a miss actually do, and why the write-back step matters.
- Implement Spring's cache abstraction correctly with Redis: `@Cacheable` on reads, `@CacheEvict` on every write path, and a deliberately configured cache manager.
- Choose a sane TTL and a Redis eviction policy for real data, instead of caching "forever" and inheriting eventual staleness.
- Prove a cache works by measuring its *hit ratio* under load, and recognise the low-ratio smell that means you are paying serialization cost for nothing.

1 The Big Picture

What this day solves. Your ShopFlow catalog endpoint answers the same question thousands of times a minute: "give me product 42." Every one of those calls opens a connection, plans a query, reads pages off disk or the buffer pool, serialises rows, and hands them back — tens of milliseconds of work to return data that did not change since the last request. Caching is the discipline of *not doing expensive work you already did*. Today you put a fast in-memory layer in front of the database so repeated reads cost sub-millisecond lookups instead of full query round-trips.

Why backend engineers need it. The database is almost always the hardest thing in your stack to scale. You can add stateless app servers behind a load balancer in minutes; you cannot casually add a primary database. Caching is how teams survive traffic they could never afford to serve from the database directly — and it is where a specific, dangerous class of bug lives: **stale data**. The engineer who reaches for a cache without a plan for invalidation ships a system that is fast *and wrong*. This day is as much about knowing when *not* to cache as about the annotations.

🔑 Key idea

A cache is a **bet that a read will repeat before the data changes**. When that bet pays off you turn a database query into a memory lookup. When it does not — data that changes every request, or a write path that forgets to invalidate — you have added latency, memory, serialization cost, and a consistency bug, for zero benefit. Caching is a trade, not a free win.

Where it fits in a Spring Boot application. Caching lives *between* your service layer and everything expensive behind it. It is delivered by the exact proxy mechanism you studied on Day 1: `@Cacheable`

is advice wrapped around your bean, checked *before* your method body runs. It leans on the Redis container you added to the Compose stack on Day 8. And it sits directly on top of the JPA repositories from Days 2–3 — the cache's whole job is to let those repositories rest.

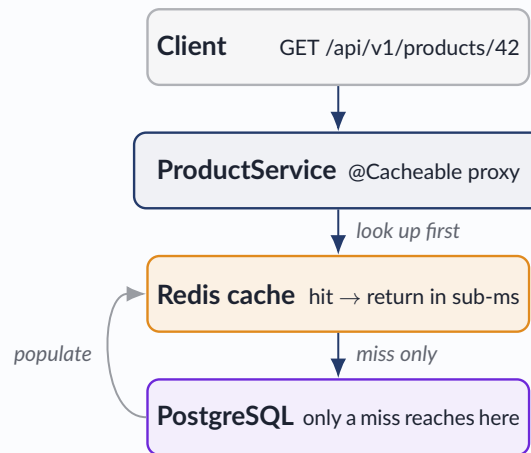
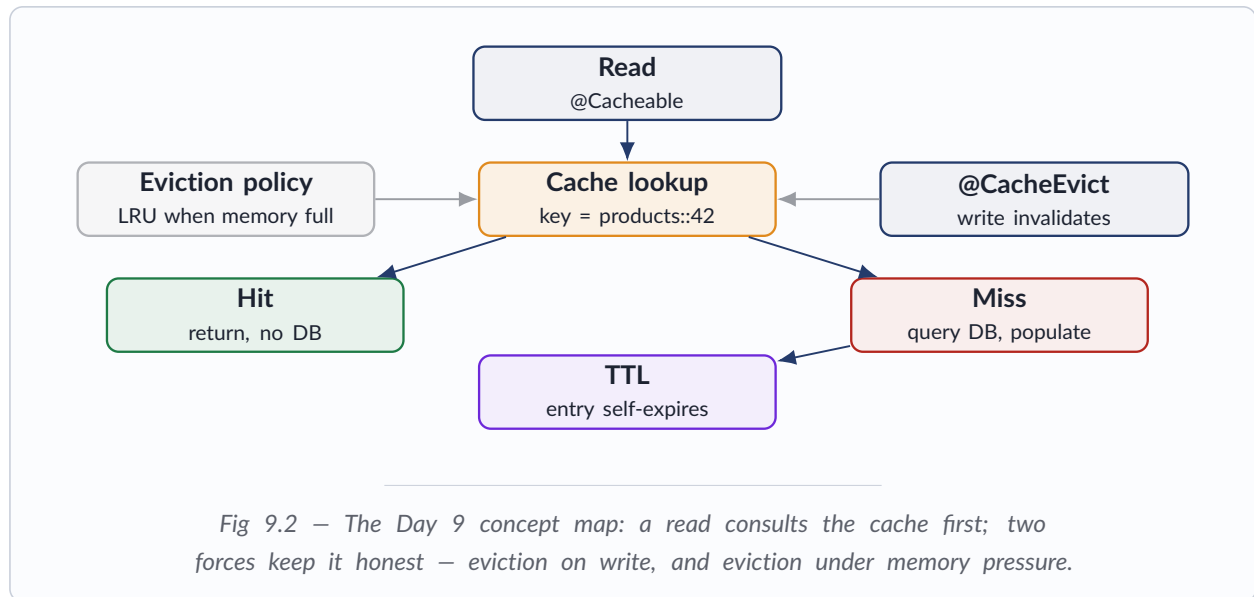


Fig 9.1 — Where caching fits: a read layer in front of the database that absorbs repeated reads, so only misses reach the repository.

Real company examples. Reddit famously caches rendered comment trees and listing pages in Redis/memcached because the same hot threads are read millions of times between edits. Twitter's timeline delivery is a giant fan-out cache problem. Stack Overflow serves the vast majority of its traffic from a small number of cache servers, keeping the SQL Server tier almost idle for reads. In every case the shape is identical to your ShopFlow catalog: read-heavy, change-rarely data that would crush the database if served fresh every time.

2 Concept Map

Hold the whole territory in one picture before the details. Read it top-to-bottom (a single read) and notice the two callouts hanging off it — the write path and the memory limit — because those are where caches go wrong.



Terminology to anchor: **cache-aside** → **hit / miss** → **@Cacheable** → **key** → **TTL** → **@CacheEvict** → **eviction policy** → **hit ratio**.

3 Guided Learning

3.1 Why caching exists – the read-amplification problem

What is it? A **cache** is a small, fast store that holds copies of data whose authoritative home is somewhere slower. **Redis** is the industry-standard choice: a single-threaded, in-memory key-value store with rich data structures, native **TTL** (time-to-live) support, and optional persistence. You talk to it over the network in microseconds.

Why does it exist? Because reads massively outnumber writes for most catalog-shaped data, and the database is the resource you cannot cheaply multiply. A product's price is written maybe once a week and read a million times before it changes. Serving those million reads from the database means a million query plans, a million buffer-pool touches, a million serializations – to return bytes that were identical each time. Caching collapses that to one database read followed by 999,999 memory lookups.

How does it work internally? Redis keeps its entire dataset in RAM, indexed by a hash table, so a `GET` is an $O(1)$ memory read with no disk, no query planner, no join. Your app serialises a value once (say, a product as JSON), stores it under a key, and every later read deserialises from memory. The latency difference is not marginal: a Postgres round-trip is typically 1–20 ms; a Redis `GET` on the same network is 0.1–0.5 ms, and the database is freed to do work only it can do – writes and cache misses.

⚡ Performance note

The headline win is often not the per-request latency – it is *database load shed*. If 95% of catalog reads become cache hits, your database sees a twentieth of the read traffic. That is frequently the difference between one database instance and a painful, expensive sharding project. Measure both: p99 latency and queries-per-second at the database.

When should I use it? Read-heavy, change-rarely, tolerant-of-slight-staleness data: product catalogs, configuration, reference tables, expensive computed aggregates, rendered fragments.

When should I not? Data that changes on nearly every read (a live counter you also write constantly), data where *any* staleness is unacceptable and you have no clean invalidation hook, or data so cheap to fetch that serialization to Redis costs more than the query it replaces.

3.2 The cache-aside pattern

What is it? **Cache-aside** (also called lazy loading) means *the application*, not the database, owns the caching logic. On a read the app looks in the cache first; on a hit it returns the cached copy; on a miss it queries the database, writes the result back into the cache with a TTL, and returns it. The cache is populated *on demand*, by misses.

Why does it exist? It is the simplest pattern that gives you the two properties you want: the cache only ever holds data someone actually asked for (no wasted memory pre-loading cold data), and the database remains the single source of truth. Other patterns exist — read-through, write-through, write-behind — but cache-aside is the default in Spring applications because `@Cacheable` implements exactly this flow for you.

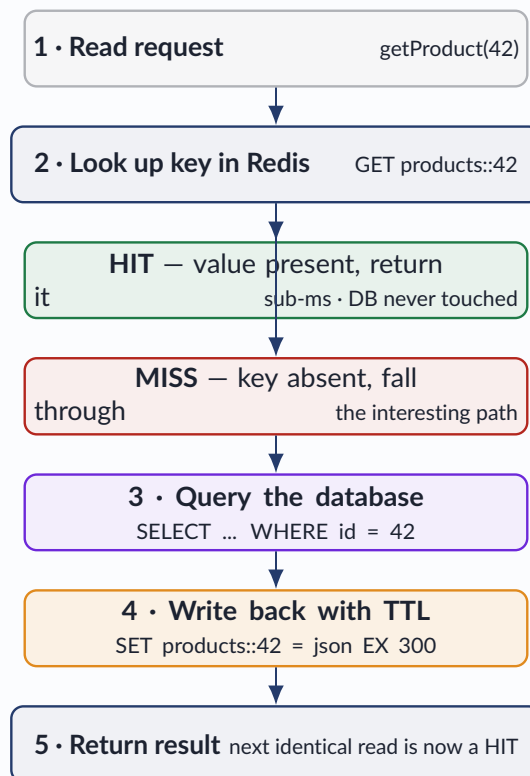


Fig 9.3 — Cache-aside read: only a miss touches the database, and the result is written back with a TTL so the next identical read is a hit.

⚠ Common misconception

Cache-aside does *not* mean the cache and database update together atomically. On a miss there is a tiny window between "read from DB" and "write to cache" where a concurrent writer could change the row — so a cache can briefly hold a value that is already one version behind. For catalog data with a short

TTL this is fine. If it is not fine for your data, that is a strong signal you should not be caching it this way.

3.3 Spring's cache abstraction — @Cacheable, @CacheEvict, @CachePut

What is it? Spring gives you a declarative caching layer that is *provider-agnostic*: you annotate methods and Spring wires the cache-aside flow around them, whether the backing store is Redis, Caffeine, or a plain `ConcurrentHashMap`. You switch providers by changing configuration, not code.

How does it work internally? Exactly like `@Transactional` from Day 1: a **proxy**. When you annotate a bean method with `@Cacheable`, Spring wraps that bean in a proxy at the `postProcessAfterInitialization` step of the bean lifecycle. Calls from *outside* the bean hit the proxy first; the proxy computes a key, checks the cache, and only invokes your real method body on a miss.

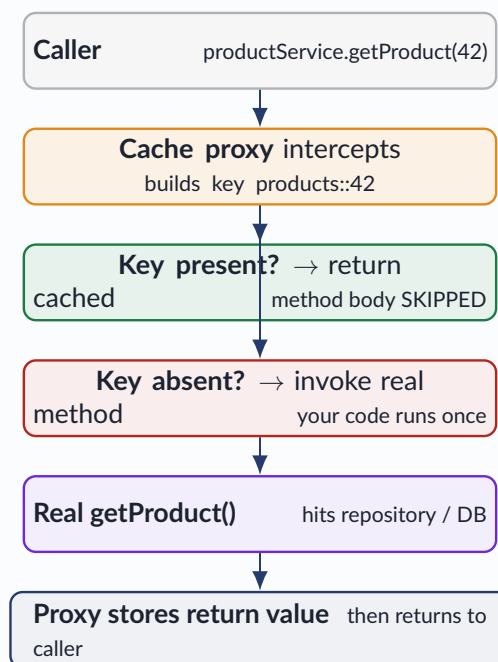


Fig 9.4 — `@Cacheable` is proxy advice: on a hit your method body never runs. This is the same interception mechanism as `@Transactional` (Day 1).

ProductService.java

```

@Service
public class ProductService {
    private final ProductRepository repo;
    public ProductService(ProductRepository repo) { this.repo = repo; }

    // On a hit, the method body below NEVER executes.
    @Cacheable(value = "products", key = "#id")
    public ProductResponse getProduct(Long id) {
        return repo.findById(id)
            .map(ProductResponse::from)
            .orElseThrow(() -> new ProductNotFoundException(id));
    }

    // The write path MUST invalidate the cached copy.
  
```

```

@CacheEvict(value = "products", key = "#id")
public ProductResponse updateProduct(Long id, UpdateProductRequest req) {
    Product p = repo.findById(id).orElseThrow();
    p.applyUpdate(req);
    return ProductResponse.from(repo.save(p));
}

@CacheEvict(value = "products", key = "#id")
public void deleteProduct(Long id) {
    repo.deleteById(id);
}
}

```

The three annotations, sharply distinguished. `@Cacheable` is read-through: check first, run the method only on a miss, store the result. `@CacheEvict` removes an entry — use it on every write path so the next read repopulates from the database. `@CachePut` *always* runs the method and overwrites the cached value with its return — it is "update and refresh the cache in one step," useful when your update method already returns the fresh entity and you would rather refresh than invalidate.

Annotation	Runs method?	Touches cache	Use when
<code>@Cacheable</code>	only on miss	read + write-back	Read paths you want to serve from cache.
<code>@CacheEvict</code>	always	removes entry	Any write (update/delete) that makes the cached copy wrong.
<code>@CachePut</code>	always	overwrites entry	Update methods that return the fresh value and should refresh, not invalidate.

Security warning • the self-invocation trap, again

Because `@Cacheable` is proxy advice, the Day 1 self-invocation rule applies verbatim: a method calling *another* `@Cacheable` method on *this* bypasses the proxy, so the cache is never consulted and the method runs every time. If a controller calls `service.getProduct()` the proxy intercepts; if `getProduct()` internally calls `this.getProductInternal()` where the annotation lives, the advice never fires. Keep cached methods as external entry points, or split them into a separate bean.

Production tip

Spend real thought on the `key`. The default key is derived from *all* method parameters, which is rarely what you want. Use SpEL to name it explicitly: `key = "#id"`. Two different methods writing the same logical entity must produce the *same* key, or an evict on one will not clear the entry cached by the other — a classic source of "I evicted it but it is still stale."

3.4 Configuring the Redis cache manager

What is it? Spring Boot will auto-configure a `RedisCacheManager` the moment `spring-boot-starter-data-redis` is on the classpath and `spring.cache.type=redis` is set — another Day 1 auto-configuration reacting to the classpath. But the defaults serialise with JDK serialization and cache "forever," neither of which you want in production. You override the manager deliberately.

How does it work internally? A `RedisCacheManager` owns one `RedisCacheConfiguration` per named cache. That config carries the TTL, the key prefix, the null-handling policy, and — crucially — the

serializer that turns your objects into bytes and back. Choose JSON serialization so the values are human-readable in `redis-cli` and portable across app restarts and language boundaries.

CacheConfig.java

```
@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory cf) {
        RedisCacheConfiguration base = RedisCacheConfiguration
            .defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(5))           // default TTL for all caches
            .disableCachingNullValues()                // do not cache "not found"
            .serializeValuesWith(SerializationPair.fromSerializer(
                new GenericJackson2JsonRedisSerializer()));

        return RedisCacheManager.builder(cf)
            .cacheDefaults(base)
            .withCacheConfiguration("products",
                base.entryTtl(Duration.ofMinutes(5))) // per-cache override
            .enableStatistics()                        // hit/miss counters
            .build();
    }
}
```

application.yml

```
spring:
  cache:
    type: redis
  data:
    redis:
      host: redis           # the service name from Day 8 docker-compose
      port: 6379
      timeout: 2s          # fail fast if Redis is unreachable
```

Database insight

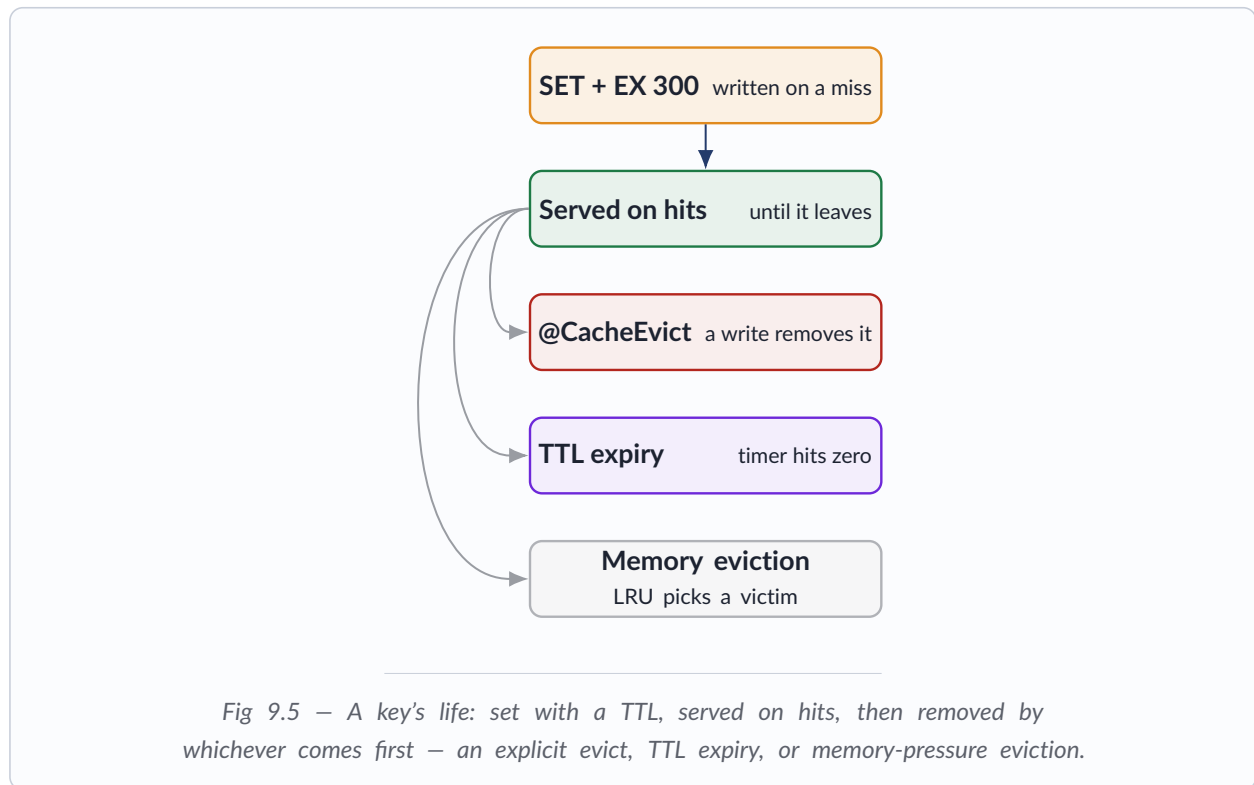
`disableCachingNullValues()` is a small decision with big consequences. If you *do* cache nulls, a lookup for a missing id stores an empty value for the whole TTL — which protects the database against a burst of requests for a non-existent product (a cheap defence against **cache penetration**). If you do *not*, every request for a missing id falls through to the database. Neither is universally right; the point is to choose on purpose, knowing the trade.

3.5 TTL and eviction — two different ways entries leave the cache

What is it? Entries leave a Redis cache for two independent reasons. **TTL expiry**: every key you set carries a countdown, and Redis deletes it when the timer hits zero — this is *time-based* and per-key. **Eviction**: when Redis hits its `maxmemory` limit, it forcibly removes keys to make room — this is *memory-pressure-based* and governed by a policy you configure. Do not confuse them; a key can vanish because its TTL expired or because Redis was full and chose it as the victim.

Why does TTL matter? Because caching "forever" guarantees eventual staleness. Even with perfect `@CacheEvict` on every write path, a bug, a missed code path, or an out-of-band database change (a DBA runs an `UPDATE`) will eventually leave a stale entry. A TTL is your **backstop**: it caps how long any

staleness can possibly last. Choose it by how tolerant the data is — a product price might happily tolerate 5 minutes; a user's own just-edited profile might need seconds or immediate eviction.



How does eviction work internally? When `used_memory` approaches `maxmemory`, Redis applies its `maxmemory-policy` to decide which keys to drop. The policies differ on *which keys are candidates* (all keys, or only those with a TTL set) and *how a victim is chosen* (least-recently-used, least-frequently-used, random, or by nearest expiry).

maxmemory-policy	Behaviour when memory is full
noeviction	Reject writes with an error. Safe for a datastore, dangerous for a cache — writes start failing.
allkeys-lru	Evict the least-recently-used key among <i>all</i> keys. The sensible default for a pure cache.
allkeys-lfu	Evict the least-frequently-used key. Better when a stable hot set dominates.
volatile-lru	Evict LRU, but <i>only</i> among keys that have a TTL. Keeps TTL-less keys pinned.
volatile-ttl	Evict the key with the shortest remaining TTL first.

redis.conf — configure deliberately, not on defaults

```
maxmemory 512mb
maxmemory-policy allkeys-lru
```

💡 How senior engineers think

The default policy in many Redis builds is `noeviction`. For a *cache* that is a trap: the day your working set outgrows memory, Redis stops accepting writes and your cache-aside populate step throws instead of degrading gracefully. A senior sets `allkeys-lru` (or `allkeys-lfu`) explicitly and sizes `maxmemory` below the container limit, so eviction — not the OOM killer — reclaims memory.

3.6 The consistency problem — when caching hurts

What is it? The moment you keep a second copy of data, you own a **consistency problem**: the cached copy and the database can disagree. Every technique this day teaches — TTLs, `@CacheEvict`, `@CachePut` — exists to bound that disagreement. Caching does not remove the problem; it makes it *your* problem to manage deliberately.

When does caching actively hurt? Four cases worth memorising. (1) *Write-heavy data*: if a key is evicted almost as often as it is read, you pay serialization and invalidation cost for near-zero hit rate. (2) *Low cardinality of repeats*: if every request asks for a different key (a per-user, per-second unique query), nothing is ever read twice before it expires — the hit ratio floors at zero. (3) *Correctness-critical reads*: inventory counts at checkout, account balances — serving a stale value here is a bug with a dollar cost, and a 5-minute TTL is unacceptable. (4) *Huge or lazy-loaded objects*: caching a JPA entity with lazy associations either explodes into a giant payload or throws a serialization error when Jackson touches an uninitialised proxy.

⚠ Common misconception

"Add a cache to make it faster" is not a strategy. A cache with a 5% hit ratio is *slower* than no cache: every read now pays a Redis round-trip *and* a database query on the 95% of misses. If you cannot articulate why a given read will repeat before its data changes, you do not yet have a caching candidate — you have a guess.

⚡ Performance note

Cache raw entities and you invite two failures: the lazy-association serialization blow-up, and cache-wide invalidation churn on any field change. Prefer caching a purpose-built, immutable **response DTO** (e.g. `ProductResponse`) — it serialises cleanly to JSON, carries only the fields the reader needs, and is decoupled from your persistence model.

3.7 Proving it works — the cache hit ratio

What is it? The **hit ratio** is $\text{hits} / (\text{hits} + \text{misses})$ — the fraction of reads served from cache without touching the database. It is the single number that tells you whether a cache is earning its keep. A ratio above roughly 80–90% on catalog data is healthy; a ratio in the single digits means you are caching the wrong thing.

How do I measure it? Two ways. Redis itself tracks `keyspace_hits` and `keyspace_misses` server-wide, visible in `redis-cli INFO stats`. Spring's `RedisCacheManager` tracks per-cache statistics once you call `.enableStatistics()` — readable through the `RedisCache` at runtime, which lets you log a per-cache ratio line under load.

inspect Redis directly

```
$ redis-cli
127.0.0.1:6379> KEYS products:*
1) "products:42"
127.0.0.1:6379> TTL products:42
(integer) 287
127.0.0.1:6379> GET products:42
{"id":42,"name":"Aeron Chair","price":1395.00}
127.0.0.1:6379> INFO stats
keyspace_hits:18342
keyspace_misses:271
```

✓ Best practices — what experienced engineers do

- Always set an explicit TTL. It is your backstop against every invalidation bug you have not found yet.
- Evict on *every* write path (update *and* delete). Never rely on TTL alone for correctness-sensitive data.
- Cache immutable response DTOs and computed aggregates — not raw entities you will mutate.
- Set `maxmemory` and `allkeys-lru` deliberately; never ship a cache on `noeviction`.
- Log the hit ratio. A cache you do not measure is a cache you cannot trust.

✗ Common mistakes beginners make

- Caching data that changes nearly every request — zero benefit, pure added latency and complexity.
- Forgetting `@CacheEvict` on the update path, serving stale data indefinitely (the #1 bug).
- Caching entities with lazy associations — serialization errors or enormous payloads.
- Treating Redis as the source of truth, with no plan for a cold or wiped cache.
- Calling a `@Cacheable` method via `this` and wondering why it never caches.

4 Visualizations to Internalize

You have met five diagrams. Redraw two *from memory* — reproducing them is what moves them from "recognised" to "known."

 Redraw: the cache-aside read flow — hit path, miss path, and the write-back-with-TTL step

 Redraw: the three ways a key leaves the cache — `@CacheEvict`, TTL expiry, memory-pressure eviction

5 Guided Coding Lab — Cache the Product Catalog

Goal of this lab: put a Redis-backed cache in front of ShopFlow's product read endpoint, prove it works with `redis-cli`, then deliberately break and fix consistency. Guide yourself through the steps — do not paste a finished solution.

1 Enable caching and point Spring at Redis

Add `spring-boot-starter-data-redis`, annotate a config class with `@EnableCaching`, and set `spring.cache.type=redis` with the `redis` host from your Day 8 Compose stack. 💡 **Why:** Caching is off until `@EnableCaching` registers the proxy infrastructure — without it, your annotations are silently inert.

2 Annotate the read with `@Cacheable`

Put `@Cacheable(value = "products", key = "#id")` on `getProduct(Long id)`. Return an immutable `ProductResponse` DTO, never the JPA entity. 💡 **Why:** The DTO serialises cleanly to JSON and dodges the lazy-association blow-up; the explicit key makes eviction predictable.

3 Configure the cache manager and TTL

Define a `RedisCacheManager` bean with a 5-minute `entryTtl`, JSON value serialization, and `.enableStatistics()`. 💡 **Why:** Defaults use JDK serialization and no TTL — both are production hazards you override once, here.

4 Prove the hit with `redis-cli`

Call the endpoint twice. Run `redis-cli KEYS 'products:*'`, then `TTL` and `GET` on the key. Confirm the value is readable JSON and the TTL counts down. 💡 **Why:** Seeing the key, its countdown, and its payload turns "I added an annotation" into "I can observe the cache working."

5 Break consistency on purpose

Update the product *without* `@CacheEvict`. Read it again and watch the stale price come back for the full TTL. This is the #1 caching bug, reproduced deliberately. 💡 **Why:** You cannot reliably prevent a bug you have never watched happen — reproduce it once so you recognise it forever.

6 Fix it with `@CacheEvict`

Add `@CacheEvict(value = "products", key = "#id")` to `updateProduct` and `deleteProduct`. Repeat the update-then-read; confirm the fresh value now appears immediately. 💡 **Why:** Eviction on the write path is the correctness half of caching; the TTL is only the backstop.

7 Log the hit ratio under load

Fire a burst of reads (a loop or a small load tool). On a schedule, read the `RedisCache` statistics and log `hits`, `misses`, and the computed ratio. 💡 **Why:** A hit ratio near your read-repeat rate proves the cache earns its cost; a low ratio tells you to stop caching this endpoint.

🔧 Production tip

Do not benchmark on a warm cache and call it a win. Measure *cold* (first requests, all misses) and *warm* (steady state) separately, and record database queries-per-second in both. The number that justifies the cache to your team is usually "database read QPS fell by 20x," not "p50 dropped 3 ms."

6 Thinking Exercises

Reflection prompts, not quiz questions. Write a sentence or two to make your reasoning explicit.

Think it through

Your catalog read has a 5-minute TTL and correct `@CacheEvict` on writes. A colleague argues the TTL is now pointless — "we evict on every write, so the cache is always correct." Where is the hole in that argument? Name a concrete way a stale entry survives despite perfect eviction code.

Think it through

You add caching to an endpoint and the hit ratio settles at 4%. Before deleting the cache, what would you check to decide whether the problem is the *key design*, the *TTL*, or the fact that this data simply should not be cached at all?

Think it through

Caching a JPA entity with lazy associations tends to fail. Explain the mechanism: what does the JSON serializer encounter when it walks a lazy field, and why does a purpose-built response DTO avoid the problem entirely?

7

Debugging Practice

Debugging Lab • The price that will not update

Symptom. An admin updates a product's price; the write succeeds and the database row is correct, but the public endpoint keeps returning the old price for several minutes.

Evidence.

ProductService.java

```
@Cacheable(value = "products", key = "#id")
public ProductResponse getProduct(Long id) { ... }

// no cache annotation here
public ProductResponse updateProduct(Long id, UpdateProductRequest req) {
    Product p = repo.findById(id).orElseThrow();
    p.applyUpdate(req);
    return ProductResponse.from(repo.save(p)); // DB is correct...
}
```

Work the method:

- ✓ **Observe.** `redis-cli GET products::42` still shows the old price; TTL is a large positive number.
- ✓ **Hypothesise.** The write path never invalidates the cache, so reads keep hitting the stale entry until the TTL expires.
- ✓ **Verify.** Update, then immediately `GET products::42` in `redis-cli`; the old value is still there — eviction never ran.
- ✓ **Fix.** Add `@CacheEvict(value = "products", key = "#id")` to `updateProduct` (and `deleteProduct`).
- ✓ **Prevent.** A test that updates then reads and asserts the fresh value; treat "every write path evicts" as a review checklist item.

🔧 Debugging Lab • The cache that never caches

Symptom. You added `@Cacheable` but `redis-cli KEYS 'products::*'` is always empty and the database is queried on every read.

Evidence.

ProductService.java

```
public ProductResponse getProduct(Long id) {
    return this.loadProduct(id); // internal call on this
}
@Cacheable(value = "products", key = "#id")
public ProductResponse loadProduct(Long id) { ... }
```

- ✓ **Observe.** No keys appear in Redis; enabling cache debug logging shows the advice never fires.
- ✓ **Hypothesise.** `loadProduct` is called via `this`, so the call bypasses the proxy and the `@Cacheable` advice never runs (the Day 1 self-invocation trap).
- ✓ **Verify.** Have the controller call `loadProduct` directly (an external call) and watch a key appear in Redis.
- ✓ **Fix.** Make the cached method the external entry point, or move it to a separate bean injected as a collaborator.
- ✓ **Prevent.** Remember: all Spring annotation magic — `@Transactional`, `@Async`, `@Cacheable` — is proxy-based and dies on self-invocation.

8 Mini-Labs

Mini-Lab • Beginner

25 min

Goal. See a hit and a miss with your own eyes.

Context. A single `@Cacheable` read endpoint backed by Redis.

Requirements. Log a line inside the method body; call the endpoint twice for the same id.

Hints. On a hit the method body does not run — so your log line prints only once, on the miss.

Expected outcome. First call logs and is slow; second call is fast and silent; `redis-cli` shows the key.

Common pitfalls. Forgetting `@EnableCaching`, so the annotation is inert and the body runs both times.

Mini-Lab • Intermediate

45 min

Goal. Prove eviction fixes staleness.

Context. A read and an update path for the same product.

Requirements. First reproduce stale data with no `@CacheEvict`; then add it and show the fix.

Hints. Watch `redis-cli GET` before and after the update in each version.

Expected outcome. Version one serves stale for the TTL; version two returns fresh immediately after update.

Common pitfalls. Evicting with a different `key` expression than the read used, so the wrong entry is cleared.

Mini-Lab • Production-style

70 min

Goal. Instrument and defend the cache like an on-call team would.

Context. A read-heavy catalog endpoint under a simulated load loop.

Requirements. Enable cache statistics; log `hits`, `misses`, and the hit ratio on a schedule; set `maxmemory` and `allkeys-lru`; confirm eviction happens under pressure instead of write failures.

Hints. Read the `RedisCache` statistics; use `redis-cli INFO memory` to watch `used_memory` approach `maxmemory`.

Expected outcome. A steady hit-ratio log line and evictions (not errors) once memory fills.

Common pitfalls. Leaving the default `noeviction` policy, so writes start failing under load instead of degrading.

9 Engineering Notes

Judgment from engineers who have carried a pager, not trivia.

How senior engineers think

Before adding any cache, a senior asks one question: "what is the invalidation story?" If the answer is "TTL only," the data had better tolerate up to a full TTL of staleness. If it must be fresh, the answer must include an explicit evict on every write path — and if you cannot enumerate those paths, you are not ready to cache this data yet.

 Interview insight

Expect: "When would caching actually hurt performance?" The answer that lands names the mechanism: on a low hit ratio, every read pays a Redis round-trip *plus* the database query on each miss, so you have strictly added latency. Then give the four smells — write-heavy data, unique-per-request keys, correctness-critical reads, and lazy-loaded entities.

 Performance note

Beware the **thundering herd**: when a hot key expires, thousands of concurrent requests all miss at once and stampede the database to repopulate it. Mitigations include slightly randomised TTLs (jitter) so keys do not expire in lockstep, and request coalescing so only one caller does the expensive reload while the rest wait. On a truly hot key this matters more than the average latency.

 Security warning • cache your caches, not your secrets

Redis in a default Compose setup is unauthenticated and unencrypted. Never cache secrets, tokens, or PII you would not want readable via `redis-cli`. And remember the cache is a shared surface — a poisoned or attacker-influenced key can serve bad data to every reader until it expires. In production, set a Redis password, restrict network access, and think about what a compromised cache could leak.

10 Build-Along Project — ShopFlow

ShopFlow already runs as a Compose stack (Day 8) with Postgres and a Redis container available, layered services and JPA repositories from earlier days, and constructor injection throughout. Today you make its read-heavy catalog endpoint fast — correctly.

 Build-Along • Day 09 — cache the product catalog, evict on write

- Add `spring-boot-starter-data-redis`; enable caching with `@EnableCaching` on a config class.
- Put `@Cacheable(value = "products", key = "#id")` on the catalog read, returning an immutable `ProductResponse` DTO (never the entity).
- Add `@CacheEvict(value = "products", key = "#id")` to both `updateProduct` and `deleteProduct` — the correctness half.
- Define a `RedisCacheManager` bean: 5-minute TTL, JSON value serialization, `disableCachingNullValues`, `enableStatistics`.
- Configure Redis with `maxmemory` and `allkeys-lru` so it evicts under pressure instead of rejecting writes.
- Add a scheduled log line reporting cache hits, misses, and the hit ratio, and prove it climbs under a read loop.

Definition of done: the catalog read is served from Redis (verify with `redis-cli KEYS` and TTL), an update immediately reflects the new value (eviction works), and the hit-ratio log line shows a healthy ratio under load.

11 Chapter Summary

Key takeaways

- ✓ Caching is a bet that a read repeats before the data changes — it trades a consistency problem for speed.
- ✓ Cache-aside: the application checks the cache, and only a miss touches the database, then writes back with a TTL.
- ✓ `@Cacheable/@CacheEvict/@CachePut` are proxy advice — and die on self-invocation, exactly like `@Transactional`.
- ✓ TTL (time-based) and eviction policy (memory-based) are two independent ways an entry leaves the cache.
- ✓ The hit ratio is the number that proves a cache earns its keep; a low ratio means stop caching that read.

Things to remember

- Every cache entry needs an explicit TTL; "forever" guarantees eventual staleness.
- Every write path must evict; forgetting it is the single most common caching bug.
- Set `allkeys-lru` and `maxmemory` deliberately — never ship a cache on `noeviction`.

Common pitfalls (last look)

- Missing `@CacheEvict` → stale reads for a full TTL. • Self-call `@Cacheable` → never caches.
- Caching lazy entities → serialization blow-ups. • Low hit ratio → pure added latency.

📄 One-Page Cheat Sheet

Cache-aside — app checks cache; miss → query DB → write back with TTL → return.

@Cacheable — run method only on a miss, store the result; key it explicitly with SpEL.

@CacheEvict / @CachePut — evict removes on write; put always runs and overwrites the entry.

TTL vs eviction — TTL = per-key timer; eviction = policy when `maxmemory` is hit (`allkeys-lru`).

Proxy rule — caching is proxy advice — `this.method()` bypasses it and never caches.

Hit ratio — $\text{hits} / (\text{hits} + \text{misses})$; low ratio = you are caching the wrong read.

Don't cache — write-heavy, unique-per-request, correctness-critical, or lazy-loaded entities.

👤 Interview questions to master

- › Explain the cache-aside pattern — who owns the logic, and what happens on a hit versus a miss?
- › What is the risk of caching without a TTL?
- › How do you keep a cache consistent with the database when a row is updated?
- › When would caching actually *hurt* performance?
- › What is the difference between TTL expiry and a Redis eviction policy?
- › Why does a `@Cacheable` method called from within the same class not cache?

Suggested further reading

- *Spring Framework Reference* — "Cache Abstraction," including SpEL key generation and conditional caching.
- *Redis documentation* — "Key eviction" and the `maxmemory-policy` options.
- *AWS / caching literature* — cache-aside vs read-through vs write-through, and the thundering-herd problem.

 Key idea

What you will learn next (Day 10). Caching absorbs reads; it does nothing for the slow, write-side work that makes a request wait — sending email, generating invoices, calling third parties. Tomorrow you move that work off the request thread with a message queue, trading synchronous latency for asynchronous throughput.

DAY 10 • DISTRIBUTED SYSTEMS

Messaging & Event-Driven Architecture10

RabbitMQ, Kafka, retries & the dead letter queue

 Estimated study time: 8 hours By the end of this day, you will be able to...

- Decide *when* to replace a direct synchronous call with asynchronous messaging – and when messaging is the wrong tool.
- Explain the RabbitMQ broker model (exchange → binding → queue) and the Kafka log model, and pick correctly for a given access pattern.
- Implement a producer and a consumer with Spring AMQP, including a bounded retry policy with exponential backoff.
- Route messages that fail persistently to a Dead Letter Queue so failures are visible and inspectable, never silent.
- Design consumers to be **idempotent** under at-least-once delivery, so a redelivered message never charges a card or sends an email twice.

1 The Big Picture

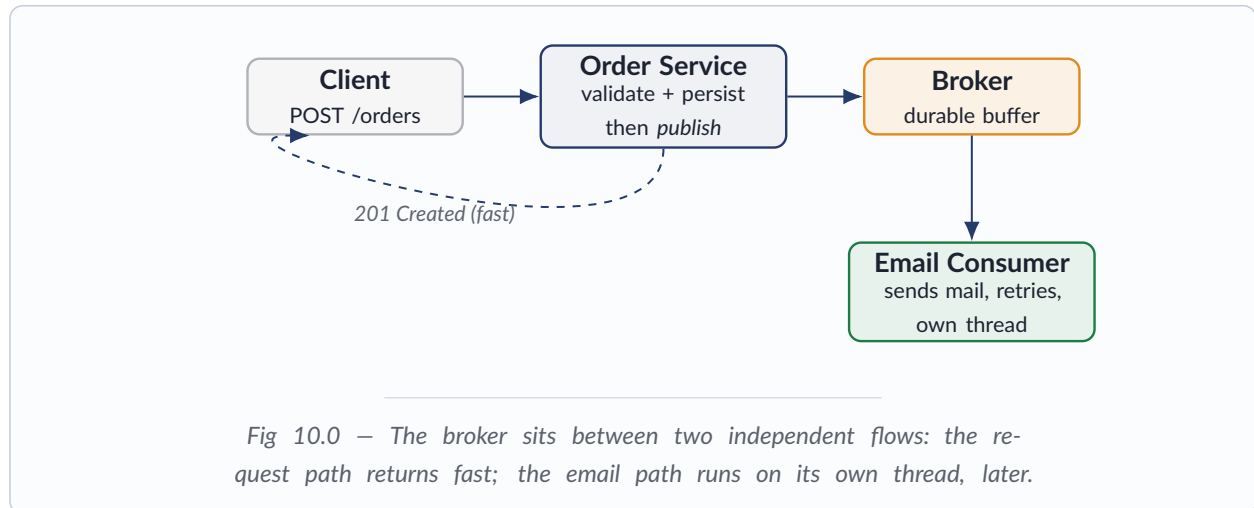
What this day solves. Until now every part of ShopFlow that needed to happen when an order was placed happened *inline*, on the request thread: validate, persist, then send the confirmation email. That works until the email provider is slow or down – and now the customer waits three seconds for a spinner, or the whole `placeOrder` call fails because a mail server hiccuped. This day answers the question that turns a monolith into a *distributed system*: how do you let one part of the application tell another part "this happened" without waiting for it, without being coupled to it, and without losing the message if the other part is temporarily broken?

Why backend engineers need it. Every backend of meaningful size is eventually a set of services (or modules) that must react to each other's events: an order triggers an email, a payment triggers a receipt, a signup triggers a welcome flow. Wiring these with direct calls makes the slowest and least reliable link determine your entire response time and failure rate. **Messaging** is how the industry decouples them. Knowing *when* to use it, and how to make it reliable, is a defining mid-to-senior skill – and it is where a lot of production incidents are born.

 Key idea

A message broker is a **buffer with delivery guarantees** placed between a producer and a consumer. The producer's job ends when the broker has durably accepted the message; the consumer processes it later, at its own pace, and can fail and retry without the producer ever knowing. That one indirection buys you decoupling, resilience, and independent scaling – and costs you the certainties of a synchronous call: ordering, exactly-once, and an immediate answer.

Where it fits in a Spring Boot application. Messaging sits at the *edges* of your service, alongside the database. Your controller (Day 4) and service layer still do the synchronous work that must return an answer — "is this order valid, did it persist?" — and then hand off the *consequences* of that work ("tell the customer") to a broker. The consumer is just another Spring bean, wrapped by a listener-container proxy, driven by the same lifecycle you learned on Day 1.



How it connects to previous chapters. On Day 7 you already decoupled the email using an in-process Spring `ApplicationEvent` — a publisher and an `@EventListener` in the *same* JVM. That was the right first step, and it taught the shape. But an in-process event dies with the process: if the JVM crashes between "order saved" and "email sent," the email is lost forever, because the event lived only in memory. Today you move that same publish/subscribe pattern onto a *real* broker so the message survives a crash, survives a slow consumer, and can be retried. Day 9's caching stays exactly as it was; messaging is orthogonal to it.

Real company examples. Uber's dispatch, receipts and analytics pipelines are event-driven at their core. Netflix moves billions of events a day through Kafka for its data platform. Banks use RabbitMQ for transactional workflow queues where each task must be processed exactly once by exactly one worker. Almost every "we send you an email / push / receipt after you do X" feature you have ever used is a message on a queue.

2

Concept Map

Hold the whole territory in one picture before the details. A message is *published* by a producer, *routed* by the broker to one or more queues, *consumed* by a worker, and — if the worker keeps failing — *dead-lettered* for a human to inspect.

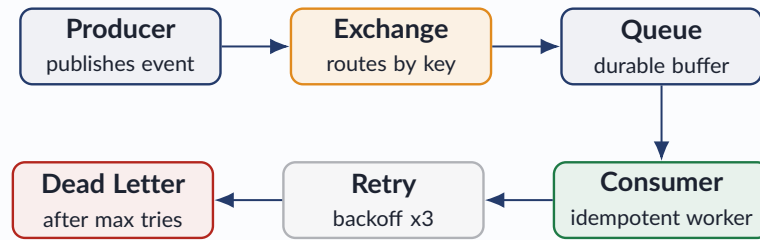


Fig 10.1 – The Day 10 concept map: publish, route, consume, retry, and dead-letter. Idempotency guards the consumer against duplicates.

Terminology to anchor: **Producer** → **Exchange** → **Binding** → **Queue** → **Consumer** → **Ack/Nack** → **Retry** → **Dead Letter Queue** → **Idempotency**.

3 Guided Learning

3.1 Why Go Async At All

What is it? Asynchronous messaging replaces a direct in-process or in-request call (`emailService.send(...)`, which blocks until it returns) with a *fire-and-hand-off*: the producer writes a message to a broker and returns immediately. Someone else reads it later.

Why does it exist? Consider the classic synchronous chain: order service calls payment, which calls email, all inline on one thread. Three problems compound. First, **latency adds up** — the customer waits for the sum of every hop. Second, **failure couples** — if the email server is down, the order fails, even though the order itself was perfectly valid. Third, **scaling couples** — you cannot scale the slow email work independently of the fast order work. Event-driven architecture cuts all three: the order service publishes `OrderPlaced` and moves on; email, analytics and inventory each react on their own thread, at their own pace, and a failure in one does not block the others.

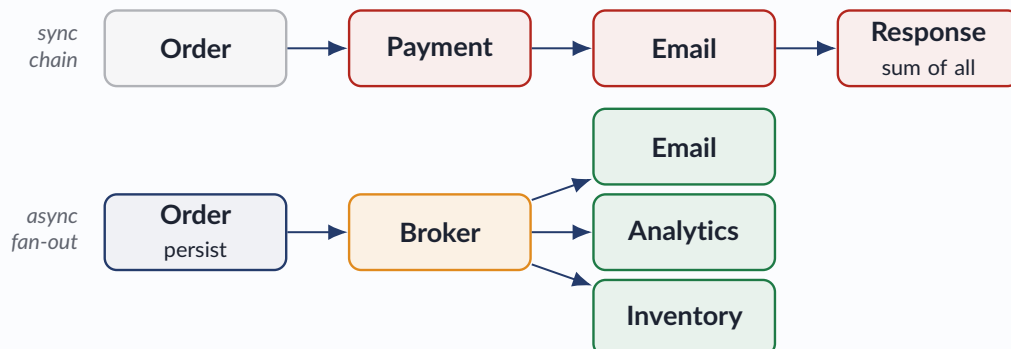


Fig 10.2 – Synchronous (top): total time and failure are the sum of the chain. Async (bottom): the request returns after persist; consumers fan out independently.

When should I use it? When the work is a *consequence* of the main action, not part of deciding whether the main action succeeds. Sending a confirmation email, updating a search index, emitting

an analytics event, warming a cache — none of these need to finish before you tell the customer "order placed." Also use it to smooth load spikes: the queue absorbs a burst and the consumer drains it at a sustainable rate.

When should I *not*? When the caller genuinely needs the answer to proceed. "Is this coupon valid?" "Is this seat still available?" "Did the payment authorise?" — these are *questions*, and a question needs a synchronous reply. Pushing them onto a queue just to look modern gives you all the complexity of distributed systems with none of the benefit, and forces you to invent a fragile request/response-over-messaging protocol to get the answer back.

⚠ Common misconception

"Async is always faster." It is not — the *total* work is the same or slightly more (you added a broker hop and serialization). What changes is *perceived* latency for the caller and *isolation* of failures. If you measured end-to-end time from click to email-delivered, async is often marginally slower. You trade a little total time for a lot of resilience and responsiveness.

3.2 The Broker's Anatomy — Exchanges, Bindings, Queues

What is it? In RabbitMQ a producer never publishes to a queue directly. It publishes to an **exchange** with a **routing key**. The exchange applies its type's rules to route a copy of the message into zero or more **queues** it is **bound** to. Consumers read from queues. This indirection is what lets you add a new consumer (analytics) later without the producer ever changing.

How does it work internally? The exchange *type* decides the routing rule: a **direct** exchange delivers to queues whose binding key exactly equals the routing key; a **topic** exchange matches wildcard patterns like `order.*`; a **fanout** exchange ignores the key and copies to every bound queue. A message accepted by the broker is held in the queue — in memory and, if the queue is **durable** and the message **persistent**, on disk — until a consumer **acknowledges** it. Until the ack arrives, the broker considers the message unprocessed and will redeliver it if the consumer dies.



Fig 10.3 — One publish, two queues. The exchange routes by key; adding the analytics queue needs no change to the producer.

👁 Behind the scenes

The **acknowledgement** is the heart of reliability. In the default **AUTO** ack mode, Spring AMQP acks a message only after your `@RabbitListener` method returns *normally*. If the method throws, the message is **nacked** and (depending on config) requeued or dead-lettered. This is why "at-least-once" is the natural default: the broker will not forget a message until someone confirms success, so a crash mid-processing means redelivery, not loss.

3.3 RabbitMQ vs Kafka — Pick Based On The Actual Problem

What is it? They are both called "message systems," but they are different data structures wearing similar clothes. RabbitMQ is a **smart broker with dumb consumers**: it routes, tracks per-message acknowledgement, and *deletes a message once it is consumed*. Kafka is a **dumb broker with smart consumers**: it is an append-only, partitioned **commit log** that *retains every message for a configured time regardless of consumption*, and each consumer tracks its own read position (offset).

How does it work internally? In RabbitMQ, a message is a transient thing en route to a worker; once acked, it is gone. Ordering is per-queue, and competing consumers on one queue share the load (a work queue). In Kafka, a topic is split into **partitions**; each partition is an ordered, immutable log. A message's key decides its partition, so ordering is guaranteed only *within* a partition. A brand-new consumer group can start reading from offset zero and **replay** the entire history — something RabbitMQ simply cannot do, because the messages were deleted on consumption.

Dimension	RabbitMQ	Kafka
Model	Broker with smart routing (exchanges/queues)	Distributed, partitioned commit log
Best for	Task queues, RPC-style workflows, complex routing	High-throughput streaming, event sourcing, replay
Retention	Deleted once consumed (by default)	Retained for a configured period regardless of consumption
Ordering	Per-queue	Per-partition
Consumer model	Broker pushes; competing workers share a queue	Consumer pulls; tracks its own offset
Typical line	"Send this email," "process this job"	"Every order-placed event, replayable by any new consumer"

When should I use RabbitMQ? When each message is a *task* to be done once by one worker, when you need rich routing (topic/header-based), and when the natural lifecycle is "do it, ack it, forget it." ShopFlow's confirmation email is exactly this: one order-placed event, one email, done. That is why this chapter builds on RabbitMQ.

When should I use Kafka? When you need *throughput* (hundreds of thousands of events a second), *replay* (a new fraud-detection service wants to re-read the last month of orders), or *event sourcing* (the log is the source of truth). If your answer to "what if we add a new consumer next year that needs the old events?" is "it must see history," you want Kafka's retention.

💡 How senior engineers think

The junior framing is "Kafka is newer/faster, so default to Kafka." The senior framing is "what is the *shape* of my data access?" A task that must be done once by one worker and then forgotten is a RabbitMQ queue. A stream of facts that many independent consumers read at their own pace, possibly replaying history, is a Kafka log. Choosing the log for a task queue means you now hand-roll "has this been processed?" tracking that Rabbit gave you for free; choosing the queue for a replayable stream means the data you will want next quarter is already deleted.

3.4 Retries, Backoff, and the Dead Letter Queue

What is it? A consumer will sometimes fail to process a message: the mail server had a brief blip, the database held a momentary lock, a downstream API timed out. These are **transient** failures — the same message would probably succeed if tried again in a second. A **retry policy** re-attempts processing a bounded number of times, ideally with **exponential backoff** (wait 1s, then 2s, then 4s) so you do not hammer a struggling dependency. When the retries are *exhausted*, the message is routed to a **Dead Letter Queue** (DLQ) — a separate holding queue for messages that could not be processed — instead of being retried forever or silently dropped.

Why does it exist? Two failure modes must be told apart. A **transient** failure clears on its own; blind retry fixes it. A **poison message** — malformed payload, a bug that throws on this specific input, a permanently deleted referenced record — will *never* succeed. If you retry a poison message forever, it sits at the head of the queue and **blocks every message behind it** (head-of-line blocking), starving the whole system for one bad input. The DLQ is the escape valve: after a bounded number of tries, move the poison message aside so the queue keeps flowing, and so a human (or an alert) can inspect exactly what failed and why.

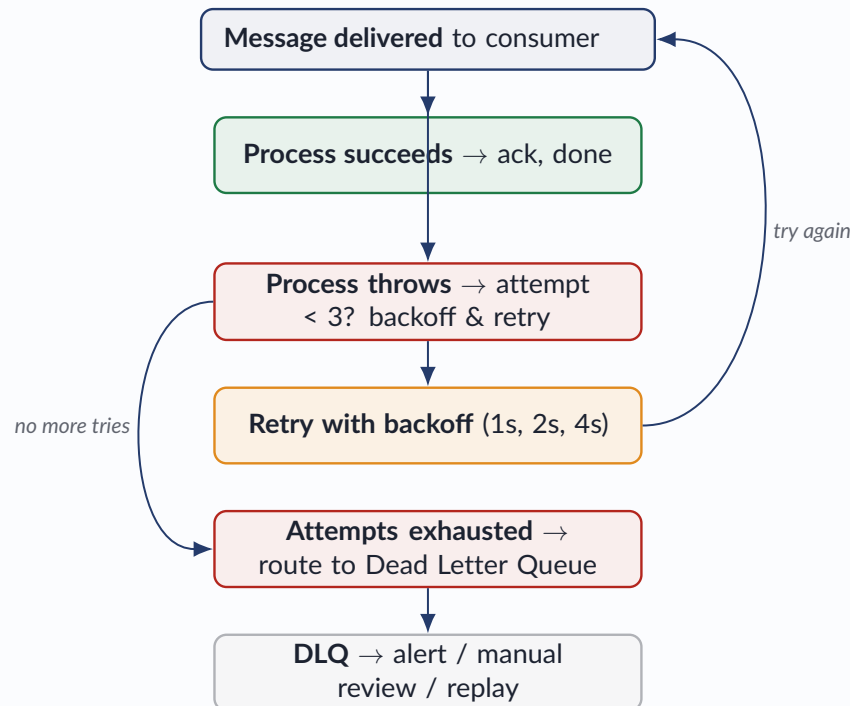


Fig 10.4 — The retry-then-dead-letter flow. Transient failures resolve on retry; persistent failures are parked in the DLQ for inspection, never lost, never blocking.

How does it work internally? In Spring AMQP the simplest retry lives *in the listener container*: when your `@RabbitListener` throws, a `RetryTemplate` catches it and re-invokes your method after a backoff, up to `max-attempts`. If all attempts fail, the container gives up and, because you set `default-requeue-rejected: false`, the message is *nacked without requeue* — which, on a queue configured with a `x-dead-letter-exchange`, routes it to the DLQ. A common production alternative is **broker-side retry via TTL**: the message goes to a "wait" queue with a time-to-live and a dead-letter binding back to the work queue, so the backoff happens in the broker rather than by blocking a consumer thread.

⚡ Performance note

In-container retry blocks the consumer thread during the backoff sleep. With `max-attempts: 3` and a max interval of a few seconds that is fine. But do not set a 5-minute backoff on an in-container retry — you would tie up a worker thread doing nothing for five minutes, throttling your whole consumer. Long backoffs belong in a broker-side TTL/DLX delay queue, where the wait costs no thread.

Security warning • never requeue a poison message forever

The single most damaging misconfiguration here is leaving `default-requeue-rejected` at its default of `true` with *no* retry limit. A message that always throws is nacked, requeued at the head, redelivered, throws again — an infinite hot loop that pins a CPU, spams your logs, and blocks the queue. Always bound the attempts and always give the queue a dead-letter target.

3.5 Delivery Guarantees & Idempotency

What is it? A **delivery guarantee** is the contract for how many times a message may reach your consumer. **At-most-once** (fire and forget) can lose messages if the consumer dies before processing — acceptable only for disposable data like metrics samples. **At-least-once** is the default for any reliable setup: the broker keeps the message until you ack success, so a crash mid-processing means the message is *redelivered*. **Exactly-once** is the holy grail and is genuinely expensive; in practice it is *approximated* by making an at-least-once consumer **idempotent**, so processing the same message twice has the same effect as processing it once.

Why does it exist? The moment you cross a network you cannot have both "never lose a message" and "never deliver twice" without heavy coordination. Consider: the consumer processes the message, sends the ack, and the ack is lost on the wire. The broker never heard the ack, so it redelivers. Your consumer now sees the same message a second time. At-least-once accepts this reality; the burden shifts to you to make a duplicate harmless.

How does it work internally? An **idempotent** consumer records which messages it has already handled and skips repeats. The standard technique is a **processed-message table**: each event carries a unique message ID; before doing the work, the consumer tries to insert that ID into a `processed_messages` table with a unique constraint. If the insert succeeds, this is the first time — do the work. If it violates the unique constraint, this is a duplicate — ack and skip. Because the insert and the work commit in the same transaction, a crash between them redelivers and safely retries; a successful duplicate is caught by the constraint.

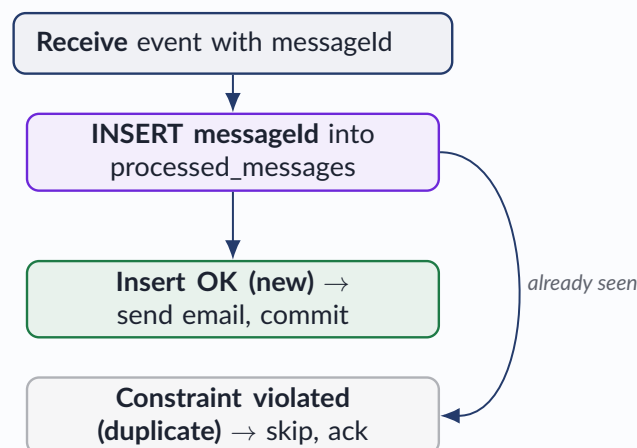


Fig 10.5 — Idempotent consume: the unique constraint on the message ID is the guard that turns "at-least-once" into "effectively once."

⚠ Common misconception

"We use exactly-once delivery, so we do not need idempotency." Almost no real system has true end-to-end exactly-once across a broker and an external side effect like sending an email. Even Kafka's "exactly-once semantics" applies to reads-and-writes *within* Kafka, not to the SMTP call your consumer makes. Treat idempotency as mandatory: assume redelivery will happen, and make it harmless. The consumer that assumes it will never see a duplicate is the consumer that double-charges a customer.

👤 Interview insight

Expect: "*How do you handle duplicate message delivery?*" The answer that lands is not "we use exactly-once." It is: "at-least-once is the realistic guarantee, so I make the consumer idempotent — dedupe on a unique message ID, typically via a unique constraint on a processed-messages table inside the same transaction as the side effect, so a redelivery is caught and skipped." Name the mechanism.

✓ Best practices — what experienced engineers do

- Make consumers idempotent (dedupe by message ID) — assume at-least-once delivery, always.
- Always configure a DLQ and a bounded retry count; never let a poison message block or loop forever.
- Keep payloads small and versioned; a schema field lets consumers evolve without breaking on old messages.
- Propagate a correlation ID from the producing request into the message headers, so you can trace one event across service logs.
- Publish only *after* the source-of-truth write commits, so you never announce an order that was rolled back.

✗ Common mistakes beginners make

- Assuming exactly-once delivery and not handling duplicates — the classic double-email, double-charge bug.
- No retry limit, so a bad message loops forever and starves the queue.
- Using messaging for something that needs a synchronous answer ("is this coupon valid?").
- Ignoring ordering requirements when they genuinely matter (state transitions arriving out of order).
- Fat, unversioned payloads that break every consumer the day you add a field.

4 Visualizations to Internalize

You have met five diagrams. Redraw two *from memory* — reproducing them is what moves them from "recognised" to "known."

 Redraw: the retry-then-dead-letter flow (Fig 10.4), labelling where backoff happens and where the DLQ catches poison messages

 Redraw: the idempotent-consume path (Fig 10.5) – show the unique constraint deciding "new" vs "duplicate"

5**Guided Coding Lab — Publish, Consume, Retry, DLQ**

Goal of this lab: take ShopFlow's confirmation email off the request thread and onto RabbitMQ. You will publish an `OrderPlacedEvent`, consume it in a listener that sends the email, add a bounded retry with backoff, route exhausted messages to a DLQ, and make the consumer idempotent. Guide yourself through it — do not paste a finished solution.

1 Run a broker and add the starter

Start RabbitMQ locally (`docker run -p 5672:5672 -p 15672:15672 rabbitmq:3-management`) and add `spring-boot-starter-amqp`. Open the management UI at `localhost:15672` (guest/guest).  **Why:** Seeing the exchanges, queues and message counts in the UI turns the abstract broker into something you can watch fill and drain — indispensable for debugging.

2 Declare the topology as beans

In a `RabbitConfig`, declare a durable work queue `orders.placed` whose arguments set `x-dead-letter-exchange` to `orders.dlx`, a dead-letter queue `orders.placed.dlx`, the exchanges, and the bindings.  **Why:** Declaring topology in code (not by hand in the UI) makes it reproducible across every environment and versioned with the app — the same "one artifact, many environments" principle as Day 1's profiles.

3 Publish after the commit

In the order service, after the order row commits, call a `RabbitTemplate` to `convertAndSend` an

`OrderPlacedEvent` (`orderId`, `email`, `total`, and a generated `messageId`) to `orders.exchange` with routing key `order.placed`. 💡 **Why:** Publishing after commit guarantees you never send a confirmation for an order that was rolled back — the message must reflect committed truth, not an in-flight transaction.

4 Consume and send the email

Write an `@RabbitListener(queues = "orders.placed")` method that deserialises the event and calls your Day 7 email service. Configure a `Jackson2JsonMessageConverter` so the payload is JSON, not Java serialization. 💡 **Why:** JSON payloads are language-agnostic and human-readable in the UI; Java serialization couples producer and consumer to the same class bytes and is a known security risk.

5 Add bounded retry with backoff

In `application.yml` enable listener retry: `max-attempts: 3, initial-interval: 1000, multiplier: 2.0`, and `default-requeue-rejected: false`. Throw a `RuntimeException` from the consumer on purpose and watch three attempts in the log. 💡 **Why:** You are proving the transient-failure path by hand: three spaced attempts, then a nack — exactly the behaviour a real flaky mail server would exercise for you.

6 Verify the DLQ catches the poison message

Keep the consumer throwing so all retries fail. Watch the message land in `orders.placed.dlq` in the UI. Inspect its headers — `x-death` records why and how many times it failed. 💡 **Why:** The DLQ is only useful if you can trust it catches what you think it does; verifying it now means you will not discover a leak during a real incident.

7 Make the consumer idempotent

Add a `processed_messages` table with a unique `message_id` column. At the top of the consumer, insert the id; on a duplicate-key exception, log and return (ack without re-sending). Redeliver the same message and confirm only one email is sent. 💡 **Why:** This is the line that makes at-least-once safe: the unique constraint, not your hope, guarantees the email goes out exactly once.

🔑 Production tip

Do not test the DLQ only in theory. In the management UI, publish a deliberately malformed message straight to `orders.placed`, watch it fail every retry, and confirm it lands in the DLQ with a readable `x-death` header. The five minutes you spend proving the safety net works is worth more than any amount of reading about it.

RabbitConfig.java

```
@Configuration
public class RabbitConfig {

    @Bean
    Queue ordersQueue() {
        return QueueBuilder.durable("orders.placed")
            .withArgument("x-dead-letter-exchange", "orders.dlx")
            .withArgument("x-dead-letter-routing-key", "order.dead")
            .build();
    }

    @Bean
    Queue deadLetterQueue() {
        return QueueBuilder.durable("orders.placed.dlq").build();
    }

    @Bean
```

```

TopicExchange ordersExchange() { return new TopicExchange("orders.exchange"); }

@Bean
DirectExchange deadLetterExchange() { return new DirectExchange("orders.dlx"); }

@Bean
Binding workBinding(Queue ordersQueue, TopicExchange ordersExchange) {
    return BindingBuilder.bind(ordersQueue).to(ordersExchange).with("order.*");
}

@Bean
Binding dlqBinding(Queue deadLetterQueue, DirectExchange deadLetterExchange) {
    return BindingBuilder.bind(deadLetterQueue).to(deadLetterExchange).with("order.dead");
}

@Bean
Jackson2JsonMessageConverter converter() {
    return new Jackson2JsonMessageConverter();
}
}

```

OrderEmailConsumer.java

```

@Component
public class OrderEmailConsumer {

    private final ProcessedMessageRepo processed;
    private final EmailService email;

    public OrderEmailConsumer(ProcessedMessageRepo processed, EmailService email) {
        this.processed = processed;
        this.email = email;
    }

    @RabbitListener(queues = "orders.placed")
    @Transactional
    public void onOrderPlaced(OrderPlacedEvent event) {
        // idempotency guard: unique constraint on message_id
        if (!processed.tryClaim(event.messageId())) {
            return; // duplicate delivery -> ack and skip
        }
        email.sendConfirmation(event.email(), event.orderId(), event.total());
        // throwing here triggers retry; after max-attempts -> DLQ
    }
}

```

application.yml

```

spring:
  rabbitmq:
    listener:
      simple:
        acknowledge-mode: auto
        default-requeue-rejected: false # nack -> DLQ, do not requeue
        retry:
          enabled: true
          max-attempts: 3 # 1 initial + 2 retries
          initial-interval: 1000ms
          multiplier: 2.0 # 1s, 2s backoff

```

```
max-interval: 10000ms
```

6 Thinking Exercises

These are reflection prompts, not quiz questions. Write a sentence or two — the goal is to make your reasoning explicit.

Think it through

ShopFlow publishes `OrderPlacedEvent` *after* the order row commits. What could go wrong if the process crashes in the tiny window between the commit and the publish? What pattern (hint: outbox) would close that gap, and what does it cost you?

Think it through

Your idempotency guard dedupes on `messageId`. Suppose instead you deduped on `orderId`. Describe a legitimate scenario where the *same* order should produce two messages, and explain why choosing the wrong dedupe key would silently drop the second one.

Think it through

The email consumer retries three times over a few seconds, then dead-letters. If the mail provider is down for a full hour, all those messages pile up in the DLQ. Is that the right behaviour, or would a longer broker-side delay have been better? What is the trade-off between failing fast to the DLQ and holding messages for a slow retry?

7 Debugging Practice

🔧 Debugging Lab • Customers receive two confirmation emails

Symptom. Occasionally — not always — a customer gets the same order-confirmation email twice, seconds apart. The order itself is fine; only the email duplicates.

Evidence.

OrderEmailConsumer.java

```
@RabbitListener(queues = "orders.placed")
public void onOrderPlaced(OrderPlacedEvent event) {
    email.sendConfirmation(event.email(), event.orderId(), event.total());
    // no dedupe; ack happens after this returns
}
```

Work the method:

- ✓ **Observe.** The mail log shows two sends with the same `orderId` and `messageId`, milliseconds apart, both after a broker connection blip.
- ✓ **Hypothesise.** Delivery is at-least-once: the consumer processed and sent, but the ack was lost (connection dropped), so the broker redelivered the same message.
- ✓ **Verify.** Reproduce by killing the connection right after the email send but before the ack; confirm a redelivery arrives with the identical `messageId`.
- ✓ **Fix.** Add the idempotency guard: claim `messageId` in `processed_messages` under a unique constraint before sending; on duplicate key, skip and ack.
- ✓ **Prevent.** Make idempotency a checklist item for every new consumer; add a test that delivers the same event twice and asserts a single email.

🔧 Debugging Lab • The queue stops draining and one message is stuck

Symptom. The `orders.placed` queue depth climbs and never falls. Logs show the same stack trace repeating every second forever. New orders' emails never go out.

- ✓ **Observe.** In the management UI the "unacked" count is 1 and the same `NullPointerException` loops in the log with an unchanging `messageId`.
- ✓ **Hypothesise.** A poison message throws on every attempt, and with `default-requeue-rejected: true` and no retry limit it is requeued at the head, blocking everything behind it (head-of-line blocking).
- ✓ **Verify.** Inspect the stuck message payload; confirm it is malformed (e.g. `email` is null) so it will *never* succeed.
- ✓ **Fix.** Set `default-requeue-rejected: false` and bound `max-attempts`, and give the queue an `x-dead-letter-exchange` so the poison message is parked in the DLQ instead of looping.
- ✓ **Prevent.** Never deploy a queue without a DLQ and a retry bound; alert on DLQ depth > 0 so poison messages surface immediately.

 Mini-Lab • Beginner

25 min

Goal. Watch at-least-once delivery redeliver a message with your own eyes.

Context. A single `@RabbitListener` that logs the `messageId` and then throws.

Requirements. Enable retry with `max-attempts: 3`; publish one message; read the log.

Hints. Log the delivery attempt count; the same `messageId` appears three times before the nack.

Expected outcome. Three spaced log lines for one publish, then the message leaves the queue.

Common pitfalls. Forgetting to enable retry, so you see only one attempt and conclude retries "don't work."

 Mini-Lab • Intermediate

45 min

Goal. Make a consumer idempotent with a processed-message table.

Context. An email consumer that must send exactly one mail per order even under redelivery.

Requirements. Add a `processed_messages(message_id UNIQUE)` table; claim the id before sending; on duplicate-key, skip.

Hints. Do the claim and the side effect in one `@Transactional` method so a crash between them safely retries.

Expected outcome. Delivering the same event twice sends exactly one email; the second is skipped.

Common pitfalls. Checking "does the id exist?" then inserting in two steps — a race lets two threads both pass the check. Rely on the unique constraint, not a pre-read.

 Mini-Lab • Production-style

70 min

Goal. Prove the DLQ safety net end to end and alert on it.

Context. You want confidence that persistently failing messages are parked, not lost, and that someone is told.

Requirements. Configure `orders.placed` with a DLX; force a poison message through all retries; confirm it lands in `orders.placed.dlq`; expose the DLQ depth as a metric and log a warning when it rises.

Hints. Read the dead-lettered message's `x-death` header — it records the reason, original queue, and count.

Expected outcome. The poison message sits in the DLQ with a readable `x-death`; the work queue keeps flowing; DLQ depth is observable.

Common pitfalls. Declaring the DLX but never binding a queue to it — the message is dead-lettered into the void and silently lost, the exact failure the DLQ was meant to prevent.

9

Engineering Notes

A curated margin from engineers who have carried a pager. Read these as judgment, not trivia.

 How senior engineers think

The moment you introduce a broker you have a *distributed* system, and the dual-write problem arrives: you must both commit the order to the database *and* publish the event, and there is no single transaction across both. If you publish inside the DB transaction and it rolls back, you announced a phantom order; if you publish after commit and crash, you lost the event. The mature answer is the **transactional outbox**:

write the event to an `outbox` table in the same transaction as the order, and a separate relay publishes committed outbox rows to the broker. Know that this problem exists before it bites you in production.

⚡ Performance note

Prefetch count is the throughput dial most teams never touch. By default a consumer may fetch many unacked messages at once; if one is slow, the rest wait behind it. For slow, side-effect-heavy work like sending email, a low prefetch (1–10) spreads messages evenly across worker instances and avoids one consumer hoarding a batch it cannot drain. Measure before and after — the right value depends entirely on per-message processing time.

👤 Interview insight

Expect: "When would you choose Kafka over RabbitMQ?" A crisp answer names the access pattern, not the brand: "Kafka when I need high-throughput streaming, retention, and *replay* — a new consumer re-reading history. RabbitMQ when each message is a task done once by one worker with rich routing, and deleted on ack. ShopFlow's confirmation email is a task queue, so RabbitMQ; our clickstream that three teams re-process is a log, so Kafka."

👤 Security warning • do not put secrets or PII in message payloads carelessly

Messages are durable and often replayed, logged, and inspected in a management UI. A payload carrying a full credit-card number or password is now sitting on disk in the broker and in every DLQ dump. Keep payloads to identifiers and non-sensitive fields; let the consumer fetch sensitive data from the source of truth by ID. Treat the broker as a place data lingers, not a transient wire.

10 Build-Along Project — ShopFlow

On Day 7 ShopFlow's order-confirmation email was decoupled with an in-process `ApplicationEvent`; Day 9 added caching. Today you move that email onto a real broker so it survives a crash, retries transient failures, and never silently loses a confirmation.

🔧 Build-Along • Day 10 — order email over RabbitMQ

- Add `spring-boot-starter-amqp` and declare the topology in a `RabbitConfig`: a durable `orders.placed` queue with an `x-dead-letter-exchange`, the `orders.dlx` dead-letter exchange, an `orders.placed.dlq`, the exchanges and bindings.
- Replace the Day 7 in-process listener's transport: after the order commits, publish an `OrderPlacedEvent` (`orderId`, `email`, `total`, generated `messageId`) to `orders.exchange` with key `order.placed`, using a `Jackson2JsonMessageConverter`.
- Build an `@RabbitListener` consumer that sends the confirmation email via the existing Day 7 email service.
- Configure bounded retry with backoff (`max-attempts: 3`, `multiplier: 2.0`, `default-requeue-rejected: false`); on exhaustion the message routes to the DLQ.
- Make the consumer idempotent: a `processed_messages` table with a unique `message_id`, claimed in the same transaction as the send, so redelivery never double-emails.
- Verify: force a permanently failing send and confirm the message lands in `orders.placed.dlq`

with a readable `x-death`; redeliver a success twice and confirm exactly one email.

Definition of done: placing an order returns immediately; the confirmation email is sent by a separate consumer; a transient failure retries and succeeds; a persistent failure lands in the DLQ (not lost, not looping); and a redelivered message sends the email exactly once.

11 Chapter Summary

Key takeaways

- ✓ A broker is a durable buffer between producer and consumer; it buys decoupling and resilience, and costs you synchronous certainties.
- ✓ Use messaging for *consequences* of an action, not for questions that need an answer to proceed.
- ✓ RabbitMQ = smart routing, delete-on-consume, task queues. Kafka = partitioned log, retention, replay, streaming. Pick by access pattern.
- ✓ Retry transient failures with bounded backoff; route exhausted (poison) messages to a Dead Letter Queue so they are visible, not silent.
- ✓ Delivery is realistically at-least-once, so make consumers idempotent — dedupe on a unique message ID.

Things to remember

- `default-requeue-rejected: false` plus a bounded `max-attempts` plus an `x-dead-letter-exchange` is the minimum safe consumer config.
- Publish after commit (or use a transactional outbox); never announce an order that might roll back.

Common pitfalls (last look)

- Assuming exactly-once → duplicate emails and double charges. ● No retry limit → poison message loops forever.
- DLX declared but no bound DLQ → dead-lettered into the void. ● Messaging a question that needed a synchronous answer.

📄 One-Page Cheat Sheet

Exchange types — direct = exact key; topic = wildcard `order.*`; fanout = every bound queue.

Ack model — AUTO acks on normal return; a thrown listener nacks → retry or DLQ.

DLQ trio — `x-dead-letter-exchange` on the queue + a DLX + a bound DLQ. Miss one and messages vanish.

Retry — `max-attempts`, `initial-interval`, `multiplier`; keep backoff short for in-container retry.

Delivery — at-most-once (lossy), at-least-once (default, dedupe), exactly-once (approximate via idempotency).

Idempotency — unique constraint on `message_id` in the same transaction as the side effect.

Rabbit vs Kafka — task done once + routing = Rabbit; throughput + retention + replay = Kafka.

 Interview questions to master

- › When would you choose Kafka over RabbitMQ?
- › What is a dead letter queue and why does it matter?
- › How do you handle duplicate message delivery?
- › What is the trade-off between synchronous and event-driven service communication?
- › What delivery guarantees exist, and which is realistic in production?
- › What is the dual-write problem and how does a transactional outbox solve it?

Suggested further reading

- *Spring AMQP Reference* — "Message Listener Containers," retry, and dead-lettering.
- *RabbitMQ documentation* — exchanges, bindings, and the dead-letter-exchange guide.
- *Designing Data-Intensive Applications* (Kleppmann) — Ch. 11, "Stream Processing," for the log vs queue distinction.
- *microservices.io* — the Transactional Outbox and Saga patterns.

 Key idea

What you will learn next (Day 11). You just moved work off the request thread and onto a broker. Tomorrow you stay on the theme of doing work in the background but *inside* one JVM: `@Async`, thread pools, `CompletableFuture`, and scheduled tasks — when in-process concurrency is enough and when you genuinely need the broker you built today.

DAY 11 • PERFORMANCE

JVM Performance, Concurrency & Observability

Memory, garbage collection, thread pools, async & metrics

🕒 Estimated study time: 8 hours

🎯 By the end of this day, you will be able to...

- Draw the JVM heap from memory — Eden, survivor spaces, old generation, Metaspace — and use that picture to reason about *why* an `OutOfMemoryError` happened.
- Explain generational garbage collection and pick a collector (G1, ZGC) for a given latency budget.
- Size a thread pool deliberately for CPU-bound versus I/O-bound work, and configure a `ThreadPoolTaskExecutor` instead of trusting an unbounded default.
- Compose independent downstream calls concurrently with `CompletableFuture.thenCombine`, and *always* attach a failure handler.
- Instrument a service with Actuator and Micrometer so health, metrics and a Prometheus scrape endpoint exist from day one, not as an afterthought.

1 The Big Picture

What this day solves. Up to now you have made ShopFlow *correct*: it wires beans, persists orders, secures endpoints, caches reads and publishes events to RabbitMQ. This day makes it *fast, resilient under load, and visible*. When a service falls over at 3am it is almost never because the logic was wrong — it is because the heap filled up, a thread pool starved, an async exception vanished silently, or nobody could see the metric that would have predicted it an hour earlier. Today you build the mental models and the instrumentation to prevent all four.

Why backend engineers need it. Performance and observability are the line between "it works on my laptop" and "it survives Black Friday." Every senior engineer can tell you, without a debugger, roughly *where* memory goes, *how many* threads a request consumes, and *which* metric to look at first. These are not exotic skills — they are the daily vocabulary of on-call. The JVM gives you extraordinary observability for free; most teams simply never learn to read it.

🔑 Key idea

A running service is three finite resources fighting for room: **heap** (memory the GC manages), **threads** (units of concurrency), and **connections** (to databases, brokers, downstreams). Almost every production incident is one of these three being exhausted. Today you learn to picture all three, bound them on purpose, and expose a metric for each.

Where it fits in a Spring Boot application. The container from Day 1 runs your beans; the async work you schedule today runs those beans *on a different thread* via the same proxy mechanism (`@Async` is proxied exactly like `@Transactional`). The RabbitMQ publish from Day 10 becomes a background

task on a pool you sized. The metrics you export today feed the monitoring stack you assemble on Day 13.

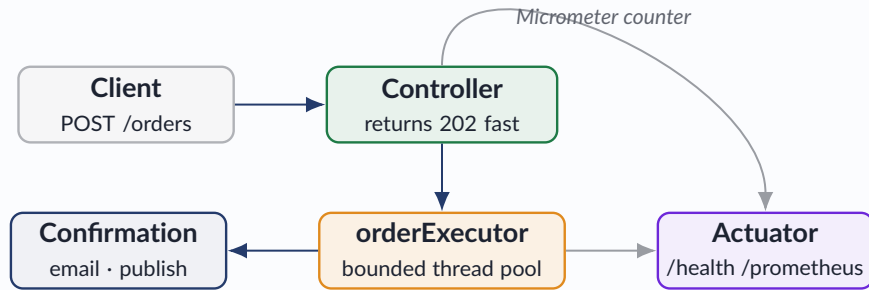


Fig 11.1 – Where today fits: a fast synchronous response, real work offloaded to a bounded pool, and every layer observed.

How it connects to previous chapters. Day 1's proxies are the delivery mechanism for `@Async`. Day 9's cache and Day 10's broker are both downstream calls you will now run *concurrently* instead of one after another. And the "fail fast at startup" reflex from Day 1 has a runtime sibling here: "observe continuously in production."

Real company examples. Netflix wrote and open-sourced much of the JVM-tuning and async tooling the industry now takes for granted; their services routinely tune G1 and ZGC for tail latency. Every company running Prometheus + Grafana – which is most of them – is scraping a `/actuator/prometheus` endpoint produced by Micrometer, the exact facade you wire up today.

2 Concept Map

Hold the whole day in one picture. The left half is *memory* (where objects live and die), the middle is *concurrency* (how work is scheduled onto threads), and the right is *observability* (how you see any of it).

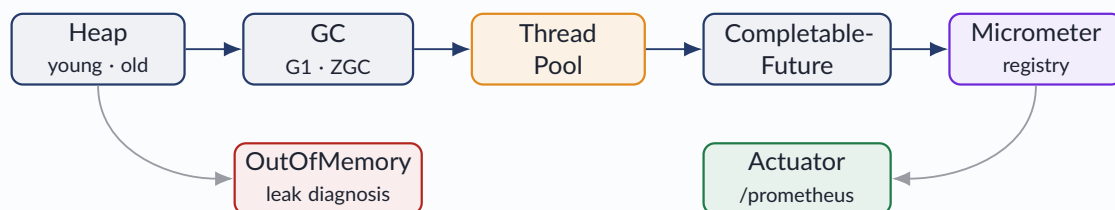


Fig 11.2 – The Day 11 concept map: memory feeds concurrency; both are made visible by metrics.

Terminology to anchor: **Heap** → **Young/Old Gen** → **GC** → **Thread Pool** → **Executor** → **CompletableFuture** → **Micrometer** → **Prometheus**.

3 Guided Learning

3.1 The JVM Heap — where your objects actually live

What is it? The **heap** is the region of memory where every object you `new` is allocated. The JVM splits it into a **Young Generation** (further divided into **Eden** and two **Survivor** spaces, `S0` and `S1`) and an **Old Generation** (also called Tenured). Separately, **Metaspace** — which is *not* on the heap — holds class metadata.

Why does it exist? The split exists because of one empirical observation, the **weak generational hypothesis**: *most objects die young*. A request creates hundreds of short-lived objects — DTOs, strings, intermediate collections — that are garbage before the response is sent. A few objects (caches, connection pools, the Spring context itself) live for the whole process. Treating those two populations differently lets the collector be fast for the common case.

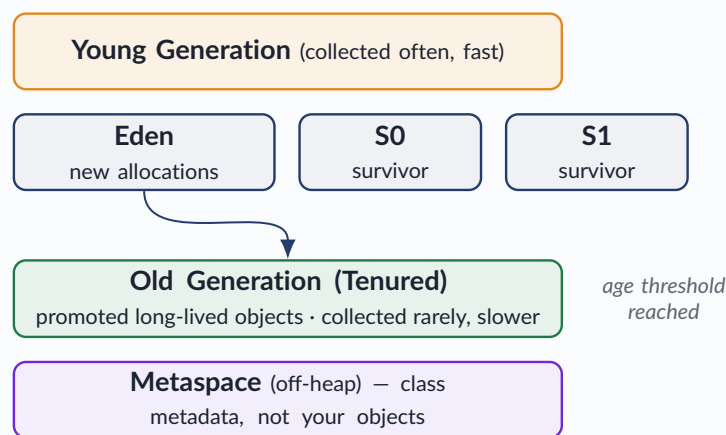


Fig 11.3 — The heap: new objects land in Eden; survivors are copied between S0/S1; long-lived objects are promoted to Old. Metaspace is off-heap.

How does it work internally? New objects go into Eden. When Eden fills, a **minor GC** runs: live objects are *copied* to a survivor space and everything else in Eden is reclaimed in one sweep (copying is why young GC is cheap — dead objects cost nothing to collect). Each survivor round increments an object's **age**; once age crosses a threshold (often 15), the object is **promoted** to the Old Generation. Old-gen fills far more slowly and is collected by a **major** (or, in G1, **mixed**) GC, which is more expensive.

When should I care? When you set `-Xmx` (max heap) and `-Xms` (initial heap). A container with a 512MB limit that runs the JVM with the default heap can be killed by the kernel OOM-killer even though the JVM never threw — because the JVM sized its heap against the host, not the cgroup. Modern JDKs are container-aware, but you should still set limits explicitly.

👁 Behind the scenes

"Stop-the-world" (STW) means the JVM pauses every application thread to collect safely. A minor GC pauses for a few milliseconds; a full GC on a large old generation can pause for hundreds of milliseconds or more. Tail latency spikes that correlate with GC logs are one of the most common "the app randomly freezes" incidents — and they are invisible unless you enabled GC logging.

3.2 Garbage Collectors — choosing for a latency budget

What is it? A **garbage collector** is the subsystem that finds unreachable objects and reclaims their memory. You do not call it; you *configure* which algorithm runs and give it a goal.

How does it work internally? Every collector starts from **GC roots** (thread stacks, static fields, JNI references) and marks everything reachable; the rest is garbage. They differ in *how* they do this while minimising the pause:

Collector	Best for	Character
Serial	tiny heaps, CLI tools	single-threaded, simplest
Parallel	batch / throughput jobs	maximises throughput, longer pauses
G1	general server apps	default since JDK 9; region-based; pause target
ZGC / Shenandoah	latency-critical	sub-millisecond pauses, huge heaps

When should I use which? **G1** (Garbage-First) is the right default for almost every Spring Boot service — it divides the heap into regions and aims to meet a *pause-time target* (`-XX:MaxGCPauseMillis=200`). Reach for **ZGC** (`-XX:+UseZGC`) only when you have measured that GC pauses are your tail-latency problem and you can afford slightly lower throughput for consistently tiny pauses.

When should I not tune it? Before you have GC logs. Changing collectors by intuition is the classic junior mistake. Turn on logging first: `-Xlog:gc*:file=gc.log`. Then, and only then, decide.

⚡ Performance note

The single highest-value flag in production is `-XX:+HeapDumpOnOutOfMemoryError` together with `-XX:HeapDumpPath`. It costs nothing until the moment you need it, and then it hands you the exact object graph that filled the heap — the difference between a five-minute diagnosis and a week of guessing.

run-shopflow.sh – production JVM flags

```
java -Xms512m -Xmx512m \
  -XX:+UseG1GC \
  -XX:MaxGCPauseMillis=200 \
  -XX:+HeapDumpOnOutOfMemoryError \
  -XX:HeapDumpPath=/var/log/shopflow/heapdump.hprof \
  -Xlog:gc*:file=/var/log/shopflow/gc.log:time,uptime:filecount=5 \
  -jar shopflow.jar
```

3.3 Diagnosing an OutOfMemoryError & memory leaks

What is it? An `OutOfMemoryError: Java heap space` means the collector ran, reclaimed everything it could, and *still* could not find room for a new allocation. It is a symptom, not a cause.

Why does it exist? (in a language with GC) Because "garbage collected" does not mean "leak-proof." A Java memory leak is not a lost pointer like in C — it is an object that is *still reachable* from a GC root but will never be used again. The collector cannot reclaim it because, as far as it can prove, you might still use it.

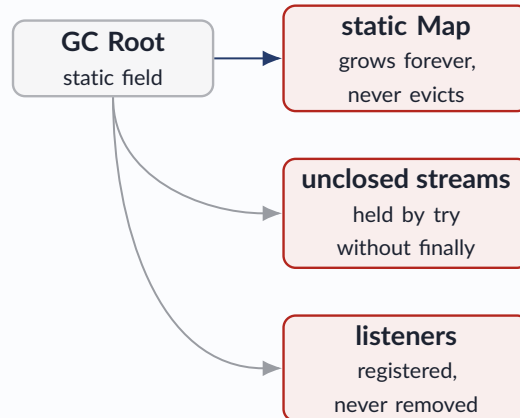


Fig 11.4 – The three classic leak shapes: an ever-growing static collection, an unclosed resource, and a listener never unregistered.

How do I diagnose it? Work the evidence in order. (1) Confirm the shape from the heap dump – open it in **Eclipse MAT** or VisualVM and read the *dominator tree*: the object retaining the most memory is almost always your leak. (2) Watch `jvm.memory.used` over time; a healthy service saw-tooths (GC reclaims); a leaking one climbs a staircase that never comes back down. (3) Correlate with a deploy – leaks usually start at a code change.

💡 How senior engineers think

A leak is a *reachability* bug, so the fix is always "stop holding the reference," never "add more heap." Bumping `-Xmx` to make an `OutOfMemoryError` go away buys you a slower, larger crash later – and hides the actual defect. The heap dump tells you which field holds the reference; go delete that reference.

⚠️ Common misconception

"Java has garbage collection, so it can't leak memory." Wrong, and dangerously so. GC reclaims *unreachable* objects. A `static List` that you keep adding to, an unbounded in-memory cache (a real risk after Day 9 if you skip eviction), or `ThreadLocals` never cleared on a pooled thread will all grow until the heap is gone. GC is a floor, not a guarantee.

3.4 Thread Pools – never spawn raw threads

What is it? A **thread pool** is a fixed set of worker threads plus a queue of pending tasks. You submit work; a free worker picks it up; when none is free, the task waits in the queue. In Java the interface is `ExecutorService`; in Spring the friendly wrapper is `ThreadPoolTaskExecutor`.

Why does it exist? Threads are expensive: each costs roughly 1MB of stack, plus OS scheduling overhead, and creating one is not free. "One new `Thread` per request" scales beautifully in a demo and collapses under load – ten thousand concurrent requests become ten thousand threads, several gigabytes of stack, and a machine thrashing on context switches. A pool **bounds** concurrency to a number you chose on purpose.

How does it work internally? This is the detail almost everyone gets wrong. A `ThreadPoolExecutor` does *not* grow to `maxPoolSize` when it gets busy. The real order is: fill `corePoolSize` threads first; then *queue* tasks until the queue is full; only when the queue is full does it create threads up to `maxPoolSize`; when *both* are full it invokes the **rejection policy**.

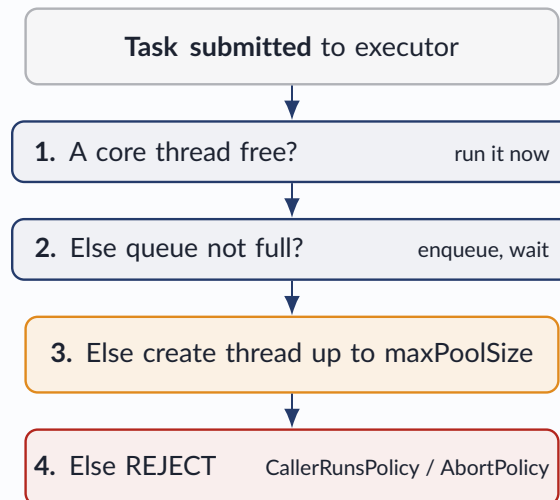


Fig 11.5 – A submitted task's path through a pool: core threads, then the queue, then extra threads, then rejection. Most people wrongly assume the queue comes last.

How do I size it? Two workload shapes, two formulas:

- **CPU-bound** (hashing, compression, in-memory computation): pool size $\approx$ number of cores. More threads than cores just adds context-switching; the CPUs are already saturated. Rule of thumb: `cores` **OR** `cores + 1`.
- **I/O-bound** (calling a database, an HTTP downstream, RabbitMQ): threads spend most of their time *waiting*, so you want many more than cores. A useful starting point is Little's Law: `threads = cores * (1 + waitTime / computeTime)`. If a task waits 90% of the time on 8 cores, that is roughly $8 \times 10 = 80$ threads.

AsyncConfig.java

```

@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean("orderExecutor")
    public ThreadPoolTaskExecutor orderExecutor() {
        var ex = new ThreadPoolTaskExecutor();
        ex.setCorePoolSize(8);           // kept alive always (I/O-bound work)
        ex.setMaxPoolSize(16);          // ceiling under burst
        ex.setQueueCapacity(100);        // backlog before growing to max
        ex.setThreadNamePrefix("order-async-");
        // Backpressure: run on the caller instead of dropping work.
        ex.setRejectedExecutionHandler(
            new ThreadPoolExecutor.CallerRunsPolicy());
        ex.initialize();
        return ex;
    }
}

```

Production tip

Set a `ThreadNamePrefix`. When you take a thread dump at 3am, `order-async-7` tells you instantly which pool is stuck; `pool-2-thread-7` tells you nothing. Naming your pools is the cheapest observability you will ever add.

Security warning • the unbounded default

If you annotate a method `@Async` *without* naming an executor and never define one, Spring falls back to a `SimpleAsyncTaskExecutor` — which is *not a pool at all*. It creates a brand-new thread for every single call, unbounded. Under load it does exactly what raw `new Thread()` does: exhausts memory. Always name your executor, and always define it.

3.5 CompletableFuture — composing async work

What is it? `CompletableFuture<T>` is a handle to a value that will exist *later*. Unlike the old `Future`, it is *composable*: you can chain transformations (`thenApply`), combine two futures (`thenCombine`), and handle failure (`exceptionally`) — all without blocking a thread until you actually need the answer.

Why does it exist? To turn *sequential waiting* into *concurrent waiting*. If a request needs inventory from one service and pricing from another, doing them one after another costs `inventoryTime + pricingTime`. Running them concurrently costs `max(inventoryTime, pricingTime)`. For two 100ms calls that is 200ms versus 100ms — a free halving of latency on every request.

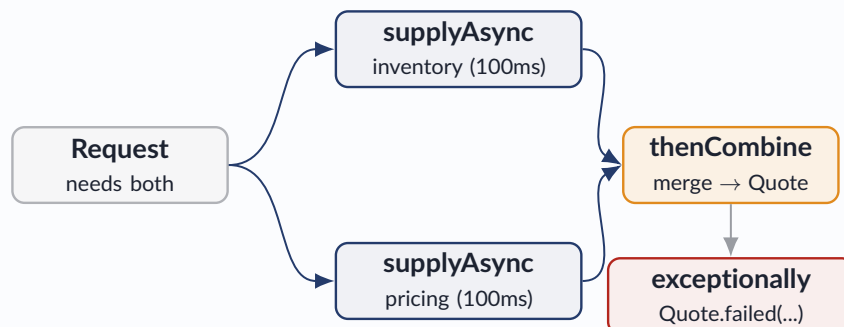


Fig 11.6 — `thenCombine` forks two independent calls onto the pool, waits for both, then merges. The `exceptionally` branch catches either failure.

QuoteService.java

```

public OrderQuote quote(long id) {
    var inv = CompletableFuture.supplyAsync(
        () -> inventoryClient.check(id), orderExecutor);
    var price = CompletableFuture.supplyAsync(
        () -> pricingClient.get(id), orderExecutor);

    return inv
        .thenCombine(price, (i, p) -> new OrderQuote(i, p))
        .exceptionally(ex -> OrderQuote.failed(ex.getMessage()))
        .join(); // block ONCE, here -- not on each call
}
  
```

How does it work internally? Each stage runs on a thread from an `Executor` — *always pass your own*,

or the work lands on the shared `ForkJoinPool.commonPool()`, which is sized to your CPU count and shared with every parallel stream in the JVM. `thenCombine` registers a callback that fires when *both* inputs complete. `join()` is the one place you block, and you do it once at the end.

When should I *not* use it? When the calls are *dependent* (B needs A's result) — then there is nothing to parallelise, and `thenCompose` (flat-map for futures) expresses the sequential dependency more honestly. Also do not reach for it to parallelise CPU-bound loops; a parallel stream or a sized pool is clearer.

⚠ Common misconception

An unhandled exception in a `CompletableFuture` chain does *not* crash your app or print a stack trace by default — it is stored inside the future and only surfaces if you call `join()/get()` or attach `exceptionally/handle`. Forget the handler and the failure vanishes silently: the email is never sent and nobody ever knows. Every chain needs an explicit failure path.

3.6 Observability — Actuator & Micrometer

What is it? **Spring Boot Actuator** exposes operational HTTP endpoints — `/actuator/health`, `/actuator/metrics`, `/actuator/prometheus` — describing the live state of the app. **Micrometer** is the vendor-neutral metrics *facade* behind it (think SLF4J, but for metrics): you write to one API, and a registry ships the data to Prometheus, Datadog, CloudWatch, whichever you configure.

Why does it exist? Because a service you cannot see is a service you cannot operate. Health endpoints let Kubernetes and load balancers know whether to route traffic; metrics let you alert *before* users notice; the Prometheus endpoint is the standard scrape target the whole cloud-native ecosystem speaks.

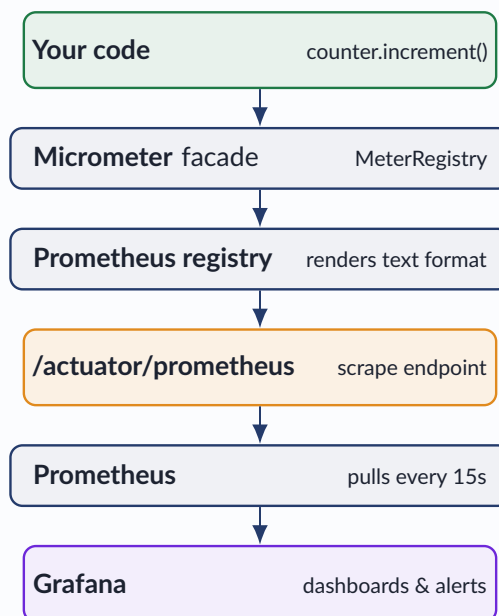


Fig 11.7 — Metrics flow: your code writes to Micrometer; the Prometheus registry renders text at `/actuator/prometheus`; Prometheus scrapes it; Grafana draws it.

How does it work internally? Add `spring-boot-starter-actuator` and `micrometer-registry-prometheus` and the registry auto-configures (Day 1 conditions at work). Micrometer ships dozens of

meters for free — `jvm.memory.used`, `jvm.gc.pause`, `executor.active` (your pool!), `http.server.requests`. You add domain meters on top.

OrderMetrics.java

```
@Service
public class OrderMetrics {
    private final Counter ordersPlaced;

    public OrderMetrics(MeterRegistry registry) {
        this.ordersPlaced = Counter.builder("shopflow.orders.placed")
            .description("Total orders placed")
            .tag("channel", "web")
            .register(registry);
    }

    public void recordOrder() { ordersPlaced.increment(); }
}
```

Counter vs the rest. A **Counter** only goes up (orders placed, errors); a **Gauge** samples a current value (queue depth, active threads); a **Timer** records duration *and* count together (request latency). "Orders per minute" is a Counter — Prometheus computes the per-minute rate at query time with `rate(shopflow_orders_placed_total[1m])`. You never compute the rate yourself; you just count.

💡 How senior engineers think

Expose `health` and `prometheus` on *day one* of a service, not after the first incident. The endpoints cost almost nothing and mean that when something does go wrong, the data already exists. Retrofitting observability during an outage is like installing smoke detectors while the kitchen is on fire.

✅ Best practices — what experienced engineers do

- Size every pool deliberately from measured workload (CPU vs I/O), never a copied default.
- Give every executor a `ThreadNamePrefix` so thread dumps are readable.
- Every async chain has an explicit `exceptionally/handle` failure path.
- Set `-XX:+HeapDumpOnOutOfMemoryError` and GC logging in production from the start.
- Expose `/actuator/health` and `/actuator/prometheus` before you need them.

❌ Common mistakes beginners make

- `@Async` with no configured executor — silently unbounded thread creation.
- Blocking calls inside a "concurrent" stream, gaining nothing but complexity.
- Swallowing exceptions in a `CompletableFuture` chain with no handler.
- Raising `-Xmx` to "fix" a leak instead of finding the retained reference.
- Tuning the garbage collector by intuition, before ever reading a GC log.

4 Visualizations to Internalize

Two of today's diagrams carry most of the day's meaning. Redraw them from memory — the act of

reproducing a diagram is what promotes it from "recognised" to "owned."

 Redraw: the heap (Eden, S0, S1, Old, Metaspace) with the promotion arrow

 Redraw: a task's four-step path through a thread pool (core → queue → max → reject)

5 Guided Coding Lab — Make the Confirmation Path Async & Observable

Goal of this lab: take ShopFlow's order-confirmation work off the request thread, run two downstream lookups concurrently, and prove the whole thing is visible through Actuator. You guide the design; do not paste a finished solution.

1 Add the dependencies

Add `spring-boot-starter-actuator` and `micrometer-registry-prometheus` to the build. Restart and hit `/actuator` to see which endpoints are live by default.  **Why:** Observability is a dependency decision — with the registry on the classpath, auto-configuration (Day 1) wires the Prometheus endpoint for you.

2 Define a bounded `orderExecutor`

Create the `AsyncConfig` from Part 3 with explicit core/max/queue and a `ThreadNamePrefix`. Add `@EnableAsync` on it.  **Why:** This is the one bean that stops "async" from meaning "unbounded thread creation" — everything else in this lab runs on it.

3 Make confirmation `@Async`

Annotate `sendConfirmation(Order)` with `@Async("orderExecutor")` and return `CompletableFuture<Void>`. Log the current thread name inside it.  **Why:** Seeing `order-async-3` in the log (not `http-nio-8080`) is your proof the work actually hopped threads.

4 Run two lookups with `thenCombine`

Fetch inventory and pricing concurrently via `supplyAsync(..., orderExecutor)`, combine into an `OrderQuote`, and attach `exceptionally`.  **Why:** You are converting sequential waiting into concurrent waiting — and forcing yourself to write the failure path that most people forget.

5 Add the orders-placed counter

Wire the `OrderMetrics` counter and call `recordOrder()` when an order is persisted. Confirm `shopflow_orders_placed_total` appears at `/actuator/prometheus`.  **Why:** A domain metric is what lets you graph "orders per minute" tomorrow — business signal, not just JVM internals.

6 Expose the endpoints deliberately

In `application.yml`, set `management.endpoints.web.exposure.include` to `health,info,metrics,prometheus`. Scrape it with `curl`.  **Why:** Actuator hides most endpoints by default for safety; you opt in explicitly, so you know exactly what is public.

7 Watch the pool under load

Fire concurrent requests (hey or ab) and watch `executor.active` and `executor.queued` at `/actuator/metrics`.  **Why:** You will see the queue fill and the active count climb — Fig 11.5 becomes a live graph instead of a static picture.

Production tip

Do not just trust that the work went async — log `Thread.currentThread().getName()` on both the controller and inside `sendConfirmation`. Two different names is your receipt. One name means the proxy was bypassed (self-invocation, exactly the Day 1 trap) and you are still running synchronously.

6 Thinking Exercises

Reflection prompts, not quiz questions. Write a sentence or two to make your reasoning explicit.

Think it through

Generational GC exists because "most objects die young." Think of a request in ShopFlow: name three objects it creates that die within milliseconds, and one that lives for the whole process. How does that split justify collecting the young generation far more often than the old?

Think it through

You have an 8-core machine. One endpoint hashes passwords (pure CPU); another calls three slow HTTP downstreams (mostly waiting). Why would sizing both pools to 8 threads be wrong for at least one of them? Work out a defensible size for each.

 Think it through

An unhandled exception in a `CompletableFuture` chain "fails silently." Predict, step by step, what a user sees when the confirmation email throws inside an async chain with no `exceptionally`. Where does the exception actually go, and why is that worse than a loud crash?

7 Debugging Practice

 Debugging Lab • The heap that only ever climbs

Symptom. ShopFlow runs fine for a day, then throws `OutOfMemoryError: Java heap space` and restarts. The graph of `jvm.memory.used` is a staircase that never comes down — no saw-tooth.

Evidence.

```
NotificationRegistry.java
@Service
public class NotificationRegistry {
    // never evicted, never bounded
    private static final List<Order> SENT = new ArrayList<>();

    public void remember(Order o) { SENT.add(o); } // grows forever
}
```

Work the method:

- ✓ **Observe.** `jvm.memory.used` climbs monotonically; GC runs but reclaims almost nothing; a heap dump was written by `-XX:+HeapDumpOnOutOfMemoryError`.
- ✓ **Hypothesise.** A reachable-but-unused collection is retaining every `Order` ever seen — a static `List` that only ever grows.
- ✓ **Verify.** Open the dump in Eclipse MAT; the dominator tree shows `NotificationRegistry.SENT` retaining 90% of the heap.
- ✓ **Fix.** Remove the static accumulation, or bound it (a size-capped cache with eviction); persist to the database if the history is genuinely needed.
- ✓ **Prevent.** Ban unbounded static collections in review; add an alert on sustained `jvm.memory.used` growth over a rolling window.

Debugging Lab • The async email that never arrives

Symptom. Orders succeed and return 202, but customers report no confirmation email. No error appears in the logs — nothing at all.

Evidence.

ConfirmationService.java

```
@Async("orderExecutor")
public CompletableFuture<Void> send(Order o) {
    var body = CompletableFuture.supplyAsync(
        () -> template.render(o), orderExecutor);
    // renders, then sends -- but nothing catches a failure
    return body.thenAccept(b -> mailClient.send(o.email(), b));
}
```

- ✓ **Observe.** 202 returns instantly; `mailClient` is never hit for some orders; logs are empty — suspiciously so.
- ✓ **Hypothesise.** `template.render` (or `mailClient.send`) throws; with no `exceptionally/handle`, the exception is captured inside the future and never logged.
- ✓ **Verify.** Attach `.exceptionally(ex -> { log.error("confirm failed", ex); return null; })` and reproduce — the stack trace finally appears.
- ✓ **Fix.** Add the failure handler permanently; on failure, record a metric and enqueue a retry rather than dropping the email.
- ✓ **Prevent.** Make "every chain ends in `exceptionally` or `handle`" a review rule; add a `shopflow.confirmations.failed` counter and alert on it.

8 Mini-Labs

Mini-Lab • Beginner

25 min

Goal. See a thread pool's four-step behaviour with your own eyes.

Context. A deliberately tiny pool: core 2, max 2, queue 2.

Requirements. Submit 10 tasks that each sleep 1s; log start/end with the thread name.

Hints. Use a `ThreadPoolTaskExecutor` and `CallerRunsPolicy`; count how many run on the caller.

Expected outcome. Two tasks run at once, two wait in the queue, and the rest run *on the calling thread* — backpressure made visible.

Common pitfalls. Assuming max threads spawn immediately; they only appear after the queue is full.

Mini-Lab • Intermediate

40 min

Goal. Halve a request's latency with `thenCombine`.

Context. An endpoint that needs inventory and pricing, each a 200ms simulated call.

Requirements. Implement it sequentially first, time it, then convert to two `supplyAsync` calls combined with `thenCombine`; time again.

Hints. Pass `orderExecutor` explicitly to both; add `exceptionally` before `join`.

Expected outcome. Sequential 400ms, concurrent 200ms; a failure in either call yields a graceful fallback, not a hang.

Common pitfalls. Forgetting the executor argument, so work runs on the shared `commonPool`; omitting the handler and losing the error.

🔧 Mini-Lab • Production-style

60 min

Goal. Ship the "orders per minute" metric end to end.

Context. Product wants a live orders-per-minute graph; ops wants a pool-saturation alert.

Requirements. Add the `shopflow.orders.placed` counter, expose `/actuator/prometheus`, scrape it, and write the PromQL `rate(shopflow_orders_placed_total[1m]) * 60`.

Hints. Also chart `executor.active / executor.pool.max` for the pool; tag the counter by channel.

Expected outcome. A rate that tracks real order volume, and a saturation ratio you can alert on above 0.8.

Common pitfalls. Computing the rate in Java instead of letting Prometheus do it; forgetting the `_total` suffix Prometheus adds to counters.

9 Engineering Notes

Judgment from engineers who have carried a pager. Read these as hard-won reflexes.

⚡ Performance note

"Profile before you optimise" is not a platitude — it is the whole discipline. Attach **Java Flight Recorder** (`-XX:StartFlightRecording`) or `async-profiler`, capture a real load, and read where time and allocation actually go. The bottleneck is almost never where intuition points; optimising a guessed hot path wastes more time than it ever saves.

👤 Interview insight

Expect: *"What's the difference between concurrency and parallelism?"* The crisp answer: **concurrency** is *structuring* work as independent tasks that can make progress in overlapping time windows; **parallelism** is *physically* running them at the same instant on multiple cores. A single-core machine can be concurrent (interleaving) but not parallel. Thread pools give you concurrency; multiple cores turn some of it into parallelism.

💡 How senior engineers think

Backpressure is a design choice you make on purpose. When a pool is full you can drop the task (`AbortPolicy`), silently discard it (`DiscardPolicy`), or slow the producer by running the work on its own thread (`CallerRunsPolicy`). For order confirmations, `CallerRunsPolicy` is usually right — it throttles intake instead of losing a customer's email. Choosing "just make the queue bigger" only moves the cliff further out.

🔧 Production tip

Micrometer meters are cheap, but *tags* are not free: every distinct tag combination is a separate time series. Never tag a counter with something unbounded like an order ID or user ID — you will create millions of series and melt Prometheus (this is called a "cardinality explosion"). Tag with bounded dimensions: channel, region, status.

10 Build-Along Project — ShopFlow

ShopFlow already persists orders (Days 2–3), secures them (Day 5), caches reads (Day 9) and publishes an `OrderPlaced` event to RabbitMQ (Day 10). Today you make that async path *correct* and *observable* — bounded, resilient, and measured.

🔧 Build-Along • Day 11 — async made correct & observable

- Define a dedicated, explicitly-sized `ThreadPoolTaskExecutor` named `orderExecutor` (core/-max/queue set on purpose, named threads, `CallerRunsPolicy`) — never the unbounded default.
- Move order-confirmation work (render + email + the Day 10 RabbitMQ publish) onto that pool via `@Async("orderExecutor")`, returning `CompletableFuture<Void>`.
- Fetch inventory and pricing concurrently with `CompletableFuture.thenCombine`, guarded by an `exceptionally` fallback, so one slow downstream can't hang the request.
- Add `spring-boot-starter-actuator` + `micrometer-registry-prometheus`; expose health, metrics and prometheus, with health details shown only when authorised (Day 5).
- Add a `Micrometer Counter` `shopflow.orders.placed` incremented on each persisted order; verify `rate(...[1m])` yields orders-per-minute.
- Set production JVM flags: `-Xms/-Xmx` pinned, `-XX:+UseG1GC`, heap dump on OOM, GC logging.

Definition of done: placing an order returns fast; confirmation runs on `order-async-*` threads (proven by the logged thread name); `/actuator/prometheus` shows `shopflow_orders_placed_total`, `jvm_memory_used_bytes` and `executor_active_threads`; and a forced downstream failure returns a graceful fallback quote instead of hanging.

11 Chapter Summary

Key takeaways

- ✓ The heap is split young/old because most objects die young; young GC is cheap copying, old GC is rarer and costlier.
- ✓ An `OutOfMemoryError` is a reachability bug — find the retained reference, don't raise `-Xmx`.
- ✓ A thread pool fills core threads, then the queue, then grows to max, then rejects — in that order.
- ✓ Size CPU-bound pools near core count; size I/O-bound pools much higher because threads mostly wait.
- ✓ `thenCombine` runs independent calls concurrently; every chain needs an `exceptionally/handle`.
- ✓ Actuator + Micrometer give health, JVM metrics and a Prometheus endpoint — expose them from day one.

Things to remember

- Always name your executor and your threads; an unnamed `@Async` silently uses an unbounded default.

- Pass your own `Executor` to `supplyAsync`, or work lands on the shared `commonPool`.
- Counters only go up; let Prometheus compute the rate — and never tag with unbounded cardinality.

Common pitfalls (last look)

- Unbounded `@Async` default → thread exhaustion under load. • Missing `exceptionally` → silent async failure.
- Raising `-Xmx` to hide a leak → a slower, bigger crash later.

📄 One-Page Cheat Sheet

Heap layout — Young (Eden + S0/S1) → promote by age → Old; Metaspace is off-heap.

GC choice — G1 = sane default (pause target); ZGC = sub-ms pauses; log first, tune second.

OOM — reachability bug — read the heap-dump dominator tree, delete the reference.

Pool order — core → queue → max → reject; $CPU \approx cores$, $I/O \approx cores \times (1 + wait/compute)$.

CompletableFuture — `supplyAsync` + `thenCombine` + `exceptionally`; pass your executor; `join` once.

Metrics — Counter (up-only) · Gauge (sample) · Timer (duration+count); rate at query time.

Actuator — expose `health,metrics,prometheus`; micrometer ships JVM & pool meters free.

👤 Interview questions to master

- › Explain Young vs Old generation and why generational GC exists.
- › How would you size a thread pool for an I/O-bound service?
- › What happens if you don't handle an exception in a `CompletableFuture` chain?
- › What is the difference between concurrency and parallelism?
- › How would you diagnose a memory leak in a running Spring Boot app?
- › When would you choose ZGC over G1, and how would you justify it with data?

Suggested further reading

- *Java Performance* (Scott Oaks) — the JVM memory and GC chapters.
- *Spring Boot Reference* — "Production-ready Features" (Actuator) and Micrometer docs.
- The G1 and ZGC tuning guides in the OpenJDK documentation; `jdk.jfr` for Flight Recorder.

🔑 Key idea

What you will learn next (Day 12). You can now build a service that is fast and observable — but shipping it by hand does not scale. Tomorrow you automate the path from commit to running container: build, test, package and deploy through CI/CD, so the "one artifact, many environments" promise from Day 1 finally comes true on every push.

DAY 12 • DELIVERY

CI/CD

12

Shipping safely and often — pipelines, artifacts & secrets

🕒 Estimated study time: 7 hours

🎯 By the end of this day, you will be able to...

- Build a GitHub Actions pipeline that runs the full test suite on every pull request and blocks the merge when it is red.
- Explain and apply *build once, promote the same artifact* — and say precisely what drifts when you rebuild per environment.
- Distinguish Continuous Delivery from Continuous Deployment, and place a manual approval gate in exactly the right spot.
- Manage secrets and per-environment configuration safely: injected from the CI secret store at run time, never committed, never logged.
- Add quality gates — static analysis and dependency-CVE scanning — that fail the build before a vulnerable change can merge.

1 The Big Picture

What this day solves. You have a service that builds, tests, containerises and reports its own health. But right now you are the pipeline: you run `./mvnw verify` on your laptop, you type `docker build`, you remember to push the image, you SSH somewhere and restart it. That is slow, it is inconsistent between engineers, and it is exactly where a Friday-evening outage is born. Today you replace the human with an automated pipeline that does the same steps the same way on every change — and refuses to ship when something is wrong.

Why backend engineers need it. The single largest predictor of whether a team ships reliably is not talent — it is the *deployment pipeline*. Small, frequent, automatically-verified changes fail small and recover fast; big, manual, quarterly releases fail catastrophically. CI/CD is how a backend engineer turns "it works on my machine" into "it is proven to work, byte-for-byte, in production." Every incident you will ever post-mortem traces back through this pipeline: which commit, which build, which artifact, which gate did or did not catch it.

🔑 Key idea

CI/CD is not a tool you install; it is a **contract about how change reaches production**. The contract has three clauses: **every change is automatically verified** (CI), **every verified change produces one immutable artifact** (build once), and **that same artifact is promoted — never rebuilt — through each environment** (CD). Learn the contract and the specific tool (GitHub Actions, Jenkins, GitLab CI) becomes an implementation detail.

Where it fits in a Spring Boot application. CI/CD sits *around* everything you have built, not inside it.

It consumes the tests from your earlier days, the `Dockerfile` from Day 8, and the metrics and health endpoints from Day 11 — and it produces a tagged image in a registry plus a running deployment. Nothing in your application code changes; what changes is that a machine now performs the ritual of verifying and shipping it, identically, every time.

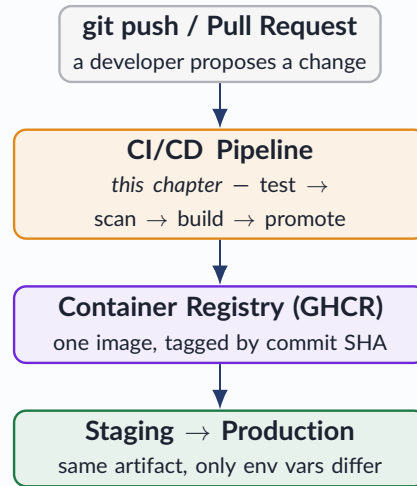


Fig 12.1 — CI/CD wraps the whole application: it consumes your code and tests, emits one artifact, and promotes it into each environment.

How it connects to previous chapters. Day 1 taught *one jar, many environments* via profiles and environment variables; today that promise finally pays off, because the pipeline builds the jar once and lets the environment supply the differences. Day 8 gave you a container image; today the pipeline builds and tags it deterministically. Day 11 gave you observability; today it becomes the thing you watch during and after a deploy to know it worked.

Real company examples. Amazon deploys to production thousands of times per day — a number only possible because every deploy runs through an automated pipeline with automated verification and rollback. GitHub, GitLab and Netflix built their entire engineering culture around "merge to main is safe because the pipeline guarantees it." The 2018–2023 DORA research (the "Accelerate" studies) put numbers on it: elite teams deploy on-demand and recover from incidents in under an hour, and the mechanism is always a trustworthy pipeline.

2 Concept Map

Read this top-to-bottom as one change's journey from a developer's laptop to production. Every arrow is a place the pipeline can *stop* the change if it is not good enough.

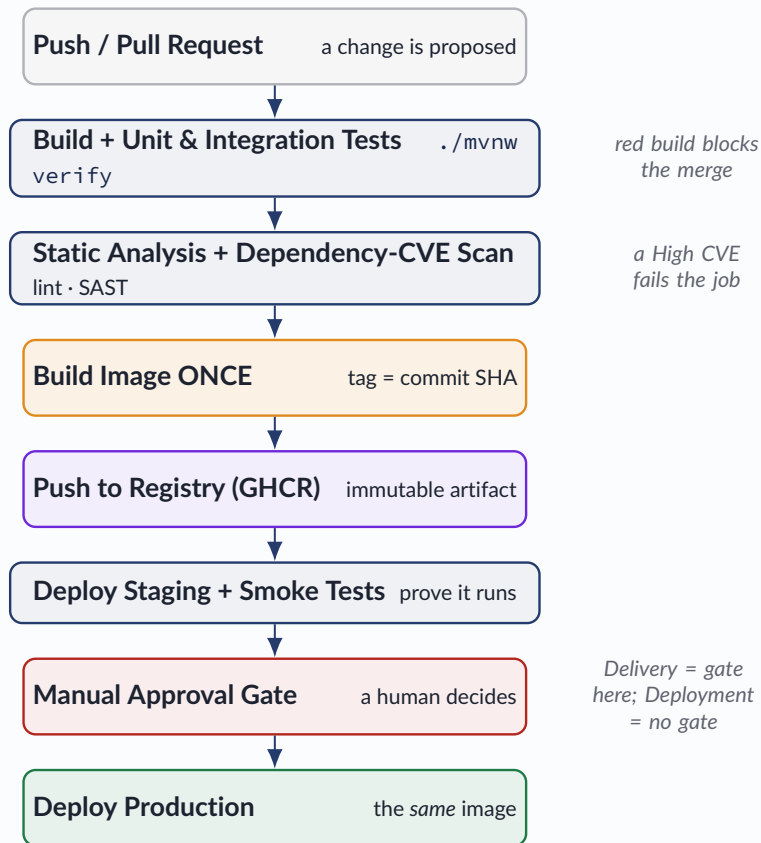


Fig 12.2 — The Day 12 concept map: a single change flows down; each stage is a gate that can halt it. The image is built exactly once.

Terminology to anchor: **Continuous Integration** → **Pipeline** → **Job** → **Runner** → **Artifact** → **Registry** → **Promotion** → **Approval Gate** → **Continuous Delivery / Deployment**.

3 Guided Learning

3.1 Why CI/CD Exists: Integration Pain and the Release Cliff

What is it? **Continuous Integration (CI)** means every push is automatically built and tested, so integration problems surface within minutes of the commit that caused them. **Continuous Delivery** extends this so that every build which passes is *ready to release* at the push of a button. **Continuous Deployment** removes even the button — a green build goes to production on its own.

Why does it exist? Before CI, teams practised "integration week": everyone merged months of divergent work at once, then spent days untangling conflicts and regressions. And before CD, a release was a rare, manual, all-hands, terrifying event — the "release cliff" — which meant releases were batched large, which made them riskier still, a vicious circle. CI/CD breaks the circle by making the safe thing (small, frequent, verified change) also the easy thing.

How does it work internally? A **trigger** (a push or pull request) fires an event; the CI system spins up a clean, disposable machine; it checks out the exact commit, runs your build and tests, and reports

a pass/fail status back onto the commit. **Branch protection** then refuses to merge a pull request whose required status is not green. That single mechanism — a required, automated status check — is what mechanically prevents "merging past a red build."

When should I use it? Always, from the second commit of any project that more than one person will touch, or that will run anywhere other than your laptop. The cost of a basic pipeline is an afternoon; the cost of not having one compounds forever.

When should I not? A throwaway spike or a one-file script does not need a pipeline. And Continuous Deployment (fully automatic to prod) is a poor fit until your test suite and observability are genuinely trustworthy — otherwise you are just automating the delivery of bugs.

⚠ Common misconception

"CI just means running the tests on a server." Running tests is necessary but not sufficient. CI is the discipline of *integrating small changes frequently into a shared main branch*, each one automatically verified. A pipeline that runs tests on a branch nobody merges for three weeks gives you the machinery of CI with none of its benefit — the integration pain you were avoiding is still waiting for you at merge time.

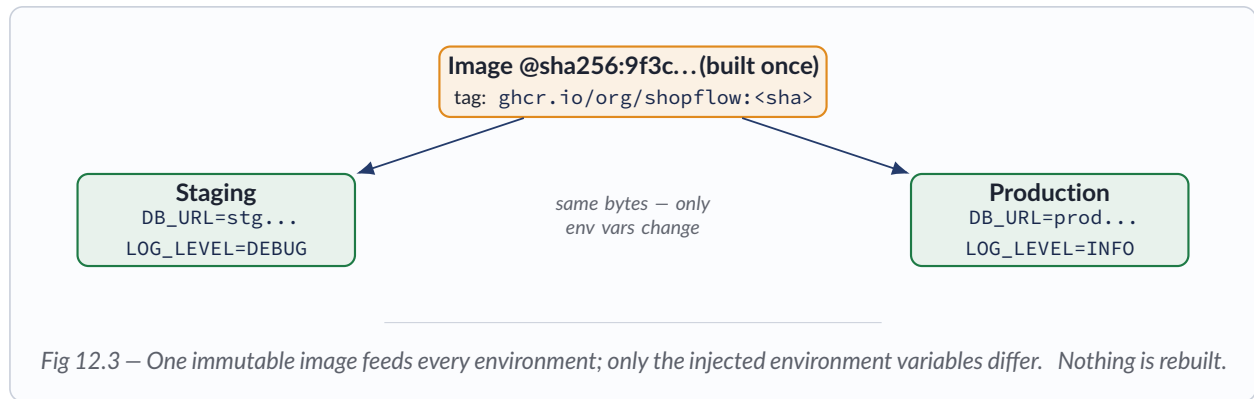
Aspect	Continuous Delivery	Continuous Deployment
Release to prod	one click / manual approval	fully automatic on green build
Human gate	yes, before production	none
Prerequisite	trustworthy tests	trustworthy tests <i>and</i> strong observability + fast rollback
Good default for	most teams, regulated domains	mature teams with high test confidence

3.2 Build Once, Promote the Same Artifact

What is it? The pipeline builds your deployable **artifact** — for a containerised Spring Boot app, a Docker image — *exactly once*, then ships those identical bytes to staging, then (unchanged) to production. You never run `docker build` again for a later environment.

Why does it exist? Because a rebuild is never guaranteed to be the same. Between your staging build and your prod build, a transitive dependency can publish a new patch, a base image tag like `eclipse-temurin:21` can move to a newer digest, a Maven mirror can return a different jar, the build clock changes, and a flaky test can pass this time. So "the same source" does not mean "the same binary." Promoting one artifact eliminates that entire class of "it worked in staging but not prod" mysteries.

How does it work internally? The build produces an image identified by a content-addressed **digest** (`sha256:...`). You additionally *tag* it with the git commit SHA so a human can trace image → commit. Promotion is then just "tell environment N to run the image at this digest." Configuration that must differ between environments (database URL, feature flags, log level) is supplied at run time via environment variables — the Day 1 and Day 8 lesson — so the artifact itself carries no environment-specific baggage.



👁 Behind the scenes

An image tag is a mutable pointer; a digest is not. `shopflow:latest` can point at three different images this week — which is why deploying `latest` to production is how you end up unable to answer "what is actually running?" Tagging by commit SHA (and pinning deploys to the immutable `sha256` digest) makes every running container traceable to one line of code.

🔑 Production tip

Never deploy the `latest` tag to any environment you care about. Tag with the commit SHA (`ghcr.io/org/shopflow:$GITHUB_SHA`) so the image name itself answers "which commit is this?", and record the resolved `sha256` digest in your deploy log so a rollback is "point back at the previous digest," not "rebuild an old commit and hope."

3.3 GitHub Actions, Anatomy First

What is it? GitHub Actions runs a **workflow** — a YAML file in `.github/workflows/` — in response to repository **events**. A workflow contains **jobs**; each job runs on a fresh **runner** (a disposable VM); each job is a sequence of **steps**; a step either runs a shell command or invokes a reusable **action**.

How does it work internally? On an event, GitHub schedules your jobs onto ephemeral runners. By default jobs run *in parallel* on *separate* machines that share nothing; you declare ordering with `needs:`, and pass files between jobs by uploading/downloading **artifacts** or restoring a dependency **cache**. Within a job, steps run sequentially and *do* share the runner's filesystem and a per-job `GITHUB_TOKEN`. When the job ends the runner is destroyed — so every run starts from a clean, reproducible state, which is the whole point.

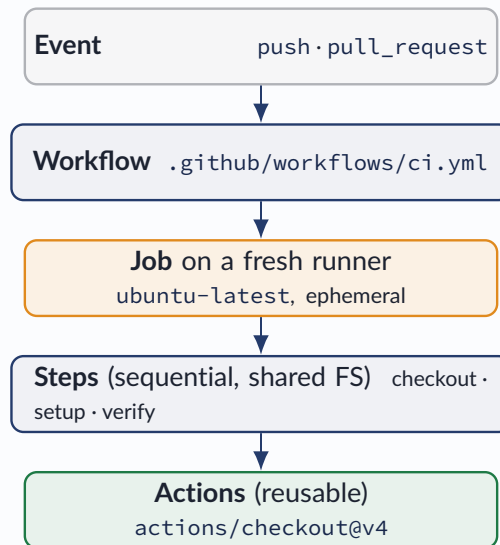


Fig 12.4 — GitHub Actions anatomy. Each job gets a brand-new runner; nothing survives between jobs except artifacts and caches you explicitly persist.

How does it work in practice? Here is the workhorse workflow: test on every pull request, and on merge to `main` additionally build and push the image. Read the annotations top to bottom.

`.github/workflows/ci.yml`

```

name: CI
on:
  pull_request:      # run on every PR
  push:
    branches: [ main ] # and on merge to main
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: maven # restore ~/.m2 between runs
      - run: ./mvnw --batch-mode verify # compile + unit + integration
  build-push:
    needs: build-test # only if tests passed
    if: github.ref == 'refs/heads/main' # only on main, not PRs
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write # allow pushing to GHCR
    steps:
      - uses: actions/checkout@v4
      - name: Log in to GHCR
        run: echo "${{ secrets.GITHUB_TOKEN }}" \
          | docker login ghcr.io -u ${github.actor} --password-stdin
      - name: Build & push image (tagged by commit SHA)
        run: |
  
```

```
IMAGE=ghcr.io/${{ github.repository }}:${{ github.sha }}
docker build -t "$IMAGE" .
docker push "$IMAGE"
```

The two decisions that matter here. The `needs: build-test` makes the push job wait for — and depend on — a green test job. The `if: github.ref == 'refs/heads/main'` means pull requests are tested but never build or push an image, so a contributor's branch can never publish an artifact. Those two lines encode the entire "verify first, publish only trusted commits" policy.

💡 How senior engineers think

Ephemeral runners are a security and correctness feature, not just a convenience. Because the machine is destroyed after each job, a compromised dependency in one run cannot linger to affect the next, and "works on the runner" genuinely means "works from a clean checkout." Seniors distrust any pipeline that runs on a long-lived, hand-maintained server precisely because state accumulates there — the pipeline slowly becomes another pet you cannot reproduce.

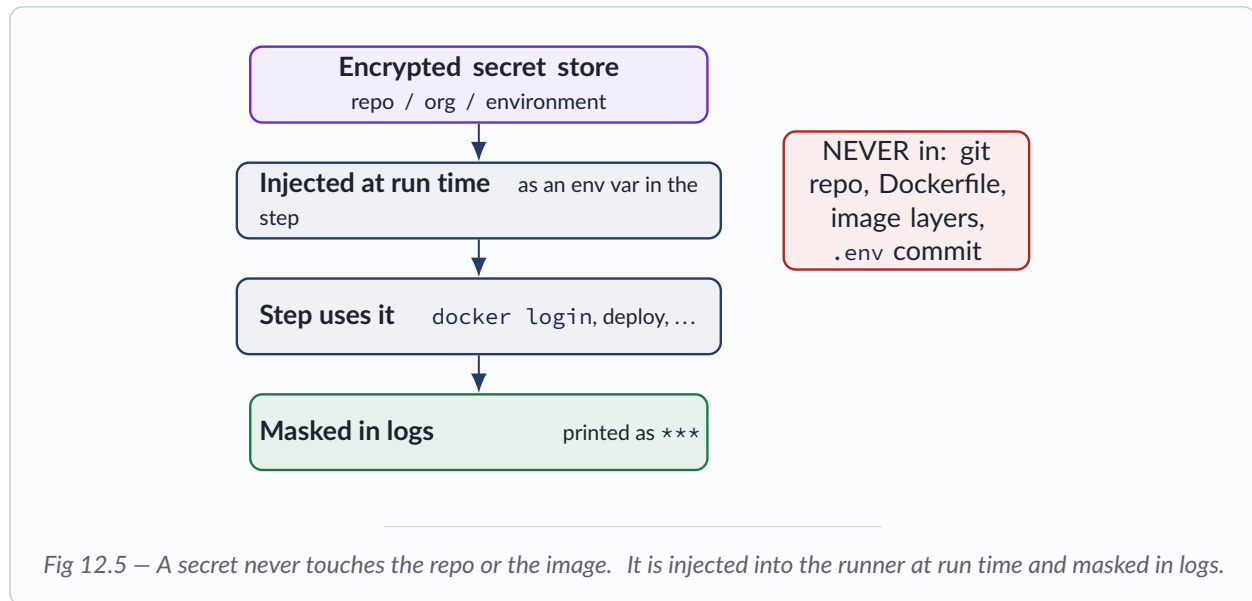
⚡ Performance note

The slowest part of most Java pipelines is re-downloading dependencies. Caching `/.m2` (via `setup-java`'s `cache: maven`) and using Docker layer caching turns a five-minute cold build into a sub-minute warm one. But cache is an *optimisation, never a correctness input*: a nightly job should occasionally build with the cache disabled, so a "works only because of a stale cached artifact" bug cannot hide forever.

3.4 Secrets & Environment Configuration Across Stages

What is it? A **secret** is any credential the pipeline needs but must never appear in the repository: a registry token, a cloud key, a database password. CI providers offer an **encrypted secret store**; you reference a secret by name and the value is injected at run time and never written to disk in your repo.

How does it work internally? In GitHub Actions the `secrets` context resolves `${{ secrets.NAME }}` at run time from the encrypted store (scoped to the repo, the organisation, or a specific **environment**). The runner substitutes the value into the step's environment and *automatically masks* it in logs — any exact match of the secret's value is replaced with `***`. The value lives only in the runner's memory for the life of the job, then vanishes with the runner.



Per-environment configuration. The artifact is one image, so everything that differs between staging and production is a variable, not a rebuild. Non-secret differences (log level, feature flags) live in the deploy manifest; secret differences (the prod DB password) live in that environment's secret scope. GitHub **Environments** let you attach both the variables and the secrets – and the approval rules – to a named environment like `production`.

Security warning • secrets do not belong in logs, forks, or images

Three ways beginners leak a secret. **(1)** `echo`-ing it for "debugging" – masking catches exact matches but not base64 or partial prints, so never `echo` a secret at all. **(2)** Baking it into the image with `ENV API_KEY=...` or `ARG` – it is then in a layer anyone who pulls the image can read with `docker history`. **(3)** Exposing secrets to workflows triggered by *forked* pull requests: on a plain `pull_request` from a fork, GitHub withholds secrets for exactly this reason – never "fix" that by switching to `pull_request_target`, which runs untrusted code *with* your secrets.

Production tip

The strongest secret is no long-lived secret. For deploys to AWS/GCP/Azure, use **OIDC**: the workflow requests a short-lived identity token (`permissions: id-token: write`) that the cloud trusts for a few minutes, so there is no static key to leak or rotate. Long-lived tokens in the secret store are fine for a registry; they are a liability for cloud infrastructure.

3.5 Quality Gates: Static Analysis, CVEs, and the Approval Gate

What is it? A **quality gate** is a pipeline step that can *fail the build* on grounds other than a failing test: a style/bug pattern from static analysis, a known vulnerability in a dependency, or – for production – a human who has not yet clicked approve.

Why does it exist? Tests prove your code does what you meant; they do not catch a `log4j`-class vulnerability in a library you transitively depend on, or a resource leak a linter would flag, or a change that simply should not go to prod at 5pm on a Friday. Gates catch the failure modes tests structurally cannot.

How does it work internally? **Static analysis** (SpotBugs, PMD, Checkstyle) and **SAST** (GitHub CodeQL) parse your code without running it and flag suspect patterns. **Dependency scanning** (OWASP Dependency-Check, Trivy, Dependabot, Snyk) reads your lockfile, matches every dependency and

its transitives against CVE databases, and fails the job if a vulnerability at or above a chosen severity is found. Each is just another job whose non-zero exit blocks the merge via branch protection.

scan-job snippet

```
cve-scan:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Scan dependencies for CVEs (fail on High)
      uses: aquasecurity/trivy-action@master
      with:
        scan-type: fs
        severity: HIGH,CRITICAL
        exit-code: '1'      # non-zero -> the build fails
```

The manual approval gate. For a production deploy you want a human in the loop. In GitHub this is an **Environment** with *required reviewers*: a job that targets `environment: production` pauses and waits for an approver before its steps run. This is the exact line between Continuous Delivery (the gate exists) and Continuous Deployment (the gate is removed).

deploy-prod job

```
deploy-prod:
  needs: [ build-push ]
  runs-on: ubuntu-latest
  environment: production # <- requires reviewer approval
  steps:
    - name: Promote the tested image
      run: ./deploy.sh ghcr.io/${{ github.repository }}:${{ github.sha }}
```

Database insight

Database migrations are the sharpest edge in any deploy pipeline, because the database — unlike your stateless image — cannot be rolled back by "point at the old digest." Gate migrations too: run them as an explicit, ordered step (Flyway/Liquibase) *before* the app rolls out, and keep every migration backward-compatible with the previous app version, so the old and new image can briefly run against the same schema during a rolling deploy.

Interview insight

Expect: "What would you include as a gate before allowing a deploy to production?" A strong answer names layers, not one thing: green tests (required status check), passing static analysis, a dependency-CVE scan with a severity threshold, a successful staging deploy with smoke tests, and a human approval on the production environment — plus a rollback plan defined *before* the deploy, not during the incident.

3.6 Jenkins — Why It Is Still Around

What is it? Jenkins is the open-source automation server that predates hosted CI. A `Jenkinsfile` (Groovy, declarative or scripted) expresses the same idea — stages of checkout, build, test, deploy — but runs on **agents** you host and maintain yourself.

Jenkinsfile (declarative)

```
pipeline {
  agent any
```

```

stages {
  stage('Build & Test') { steps { sh './mvnw verify' } }
  stage('Image')        { steps { sh 'docker build -t app:$GIT_COMMIT .' } }
  stage('Deploy')      {
    when { branch 'main' }
    steps { sh './deploy.sh app:$GIT_COMMIT' }
  }
}

```

When should I use it? When the hosted model does not fit: on-prem or air-gapped infrastructure, heavy existing plugin/pipeline investment, or approval and audit workflows that predate and outlive any single SaaS. Jenkins trades operational overhead (you patch it, you scale the agents, you secure it) for total control.

When should I not? For a new project on GitHub with cloud deploys, a hosted runner (Actions, GitLab CI) removes an entire server you would otherwise own. The concepts transfer either way — the same stages, the same "build once, promote," the same secret discipline — so learn the model, and the tool is interchangeable.

✓ Best practices — what experienced engineers do

- Fail the pipeline on any test failure — and enforce it as a *required status check* so a red build cannot be merged.
- Build the image once; promote that exact artifact (by digest) through every environment.
- Store secrets in the CI provider's encrypted store; prefer short-lived OIDC tokens for cloud access.
- Run static analysis and a dependency-CVE scan as blocking gates, not advisory reports.
- Define the rollback path (previous digest) *before* you deploy, and keep migrations backward-compatible.

✗ Common mistakes beginners make

- Skipping or disabling tests in CI "to make the pipeline faster."
- Rebuilding the image separately for each environment, inviting silent drift.
- Committing a `.env` file with real credentials, or baking secrets into an image layer.
- Deploying the `latest` tag, so nobody can say which commit is in production.
- No rollback plan — deployment is one-directional with no fast revert.

4 Visualizations to Internalize

Redraw these two from memory — reproducing them is what moves the pipeline from "I read it" to "I could design it."

 Redraw: the full pipeline from Push/PR to Production, marking every gate that can stop a change and where the image is built

 Redraw: one artifact promoted to staging and prod – show what is identical and what differs

5

Guided Coding Lab – Ship ShopFlow Through a Pipeline

Goal of this lab: give ShopFlow a real delivery pipeline – tested on every pull request, built once and pushed on merge, and promoted to production only behind a human gate. You will not paste a finished workflow; you will grow it stage by stage and watch each gate work.

1 Add the PR test workflow

Create `.github/workflows/ci.yml` with an `on: pull_request` trigger and one `build-test` job that checks out, sets up Temurin 21 with Maven caching, and runs `./mvnw -batch-mode verify`. Open a pull request and watch the check run. 💡 **Why:** Every later stage depends on a trustworthy "did the tests pass?" signal; you build that signal first.

2 Make the check required

In the repo's branch-protection settings for `main`, mark the `build-test` check as required. Push a commit that breaks a test and confirm the merge button is disabled. 💡 **Why:** A pipeline that *reports* red without *blocking* the merge is theatre – branch protection is what turns it into a gate.

3 Add the static-analysis and CVE gates

Add a `cve-scan` job (e.g. Trivy filesystem scan) configured to fail on `HIGH,CRITICAL`, and wire your static analysis (SpotBugs/Checkstyle) into `verify`. Introduce a dependency with a known CVE and watch the build go red. 💡 **Why:** You are proving the gate catches a class of failure your tests never could – a vulnerable library.

4 Build once and push on merge, tagged by SHA

Add a `build-push` job with `needs: build-test` and `if: github.ref == 'refs/heads/main'` that logs in to GHCR with `GITHUB_TOKEN`, builds the image, and pushes it tagged `ghcr.io/<org>/shopflow:${{github.sha}}`. 💡 *Why:* This is "build once": the artifact is created exactly here, on a trusted commit, and never rebuilt downstream.

5 Deploy to staging and smoke-test the artifact

Add a `deploy-staging` job (`needs: build-push`) that runs the pushed image and hits its `/actor/health` endpoint (Day 11) as a smoke test, failing if it is not UP. 💡 *Why:* Promotion is only trustworthy if you prove the exact artifact actually boots and serves before you consider production.

6 Add the production approval gate

Create a GitHub `production` environment with a required reviewer, and a `deploy-prod` job that targets `environment: production` and deploys *the same* image digest from staging. 💡 *Why:* This single line is the difference between Continuous Delivery and Continuous Deployment — put the human exactly here.

7 Wire secrets and verify nothing leaks

Move the registry/deploy credentials into the environment's secret store, reference them via the `secrets` context, and grep the run logs to confirm every value shows as `***`. 💡 *Why:* The habit you are hard-wiring: a credential is injected at run time and masked — it never lives in the repo, the image, or the logs.

💡 Production tip

Do not deploy to a real cloud in this lab; a `deploy.sh` that just echos "would deploy <digest> to <env>" is enough to exercise every gate. The learning is in seeing the pipeline *stop* at a red test, a High CVE, and the approval prompt — not in provisioning infrastructure.

6 Thinking Exercises

💡 Think it through

Your teammate proposes rebuilding the image in the production deploy job "so prod always has a fresh build." Name three specific things that could differ between that rebuild and the image you tested in staging — and explain why "same Dockerfile, same commit" does not guarantee "same bytes."

💡 Think it through

Continuous Deployment removes the human approval gate. Under what concrete conditions (about your tests, your observability, and your rollback speed) would you *trust* that automation — and what is the first thing that would make you put the gate back?

 Think it through

A secret is masked in logs, so a colleague argues it is safe to `echo` it for debugging. Explain why masking is a last line of defence and not a licence — give one way a masked secret can still leak.

7 Debugging Practice

 Debugging Lab • The deploy that "succeeded" but shipped the wrong code

Symptom. The pipeline is green and the deploy job reports success, but production is still serving last week's behaviour. Nobody can say which commit is live.

Evidence.

deploy.yml (excerpt)

```
- run: docker build -t ghcr.io/org/shopflow:latest .
- run: docker push ghcr.io/org/shopflow:latest
- run: ./deploy.sh ghcr.io/org/shopflow:latest # prod pulls "latest"
```

Work the method:

- ✓ **Observe.** Every stage tags and pulls `latest`; the running container's image ID does not match the newest pushed digest.
- ✓ **Hypothesise.** `latest` is a mutable pointer; the node cached an older `latest` and `deploy.sh` did not force a re-pull, so it re-ran the stale image.
- ✓ **Verify.** Compare the `sha256` digest the deploy actually pulled against the digest built in this run; they differ.
- ✓ **Fix.** Tag by commit SHA and deploy by the immutable digest, never by `latest`.
- ✓ **Prevent.** Make the deploy log the resolved digest, and add a smoke check that asserts a `/actuator/info` build-commit matches the expected SHA.

 Debugging Lab • Forked pull request fails — "secret is empty"

Symptom. An external contributor's pull request fails at `docker login` with an empty password, though the same job works on internal branches.

- ✓ **Observe.** `${ secrets.GHCR_TOKEN }` resolves to an empty string only on PRs from forks.

- ✓ **Hypothesise.** GitHub deliberately withholds secrets from `pull_request` runs triggered by forks, because the PR contains untrusted code.
- ✓ **Verify.** The same commit pushed to an internal branch has the secret and succeeds.
- ✓ **Fix.** Do not need secrets on PR validation — test and scan on PRs (no secrets), and build/push only on `push` to `main`, where secrets are available.
- ✓ **Prevent.** Never "solve" it with `pull_request_target`, which would run the fork's untrusted code with your secrets — a classic supply-chain foot-gun.

8 Mini-Labs

Mini-Lab • Beginner

25 min

Goal. Make a red test block a merge.

Context. A repo with a trivial passing test and a protected `main`.

Requirements. Add a `pull_request` workflow running `./mvnw verify`; mark it required; open a PR that fails the test.

Hints. Branch protection lives in Settings → Branches; the check name must match the job.

Expected outcome. The merge button is disabled with "required check failing."

Common pitfalls. Reporting the check but not marking it *required* — the merge stays allowed.

Mini-Lab • Intermediate

40 min

Goal. Build once and tag by commit SHA.

Context. The Day 8 Dockerfile and a GHCR-enabled repo.

Requirements. On `push` to `main`, build the image, tag it `ghcr.io/<org>/shopflow:${{ github.sha }}`, push, and print the resolved `sha256` digest.

Hints. Grant `packages: write`; log in with `GITHUB_TOKEN`; PRs must *not* run this job.

Expected outcome. A SHA-tagged image in GHCR and its digest in the log.

Common pitfalls. Also tagging `latest` and then deploying `latest` — defeating traceability.

Mini-Lab • Production-style

60 min

Goal. A gated promotion: staging auto, production on approval.

Context. A pushed SHA-tagged image and two GitHub Environments.

Requirements. Auto-deploy to `staging` with a health smoke test; then a `production` job requiring a reviewer that promotes the *same* digest; CVE scan gates the whole chain.

Hints. Use `needs:` to order jobs; `environment: production` enforces the reviewer.

Expected outcome. The run pauses for approval before prod; approving promotes the identical artifact.

Common pitfalls. Re-building or re-tagging between staging and prod, so "the same artifact" is a lie.

9 Engineering Notes

💡 How senior engineers think

The question a senior asks of any pipeline is "what does green actually guarantee?" If green means "unit tests passed" but not "the image boots" or "no critical CVEs," then green is a comforting lie. Design the gates so that a green main branch is genuinely a promise you would stake a deploy on — and treat a broken main as a stop-the-line, fix-it-now event, not background noise.

👤 Interview insight

Expect: *"Why promote the same artifact instead of rebuilding per environment?"* The answer that lands names the mechanism: a rebuild can pull a moved base-image digest or a newer transitive dependency, so identical source can yield a different binary — and then staging no longer tested what production runs. Promoting one digest makes staging a real test of the prod artifact.

🛡️ Security warning • the pipeline is a supply-chain attack surface

Your CI has write access to your registry and, often, your cloud. Treat third-party actions like dependencies: pin them to a full commit SHA (not a moving tag), grant each job the least `permissions` it needs, and never expose secrets to untrusted (forked) code. A compromised action with your credentials is a breach, not a bug.

⚡ Performance note

Fast pipelines get run; slow ones get skipped. Parallelise independent jobs, cache dependencies, and put the cheapest gates first (compile, unit tests) so a broken change fails in seconds, not after a ten-minute integration suite. But never trade away correctness for speed — disabling a flaky test instead of fixing it is borrowing against a future outage.

10 Build-Along Project — ShopFlow

ShopFlow is already observable (Day 11) and containerised (Day 8), configured entirely through environment variables (Day 1). Today you automate its delivery so no human ever hand-builds or hand-ships it again.

🔧 Build-Along • Day 12 — an automated delivery pipeline

- Add `.github/workflows/ci.yml`: on every pull request, check out, set up Temurin 21 (Maven cache), and run `./mvnw -batch-mode verify`; mark this check *required* on `main`.
- Add a dependency-CVE scan and static-analysis gate that fail the build on High/Critical findings.
- On merge to `main`, build the Docker image *once* and push it to GHCR tagged with `${{ github.sha }}`; log the resolved `sha256` digest.
- Auto-deploy that image to a `staging` environment and smoke-test `/actuator/health` before proceeding.
- Add a `production` GitHub Environment with a required reviewer; the prod job promotes the *same* digest — no rebuild.
- Inject every credential from the environment's secret store via the `secrets` context; confirm all values appear as `***` in logs and that no `.env` is committed.

Definition of done: a pull request runs tests and scans and cannot merge while red; a merge to `main` publishes one SHA-tagged image, deploys it to staging, and pauses for human approval before promoting the identical artifact to production — with no secret anywhere in the repository, image, or logs.

11 Chapter Summary

Key takeaways

- ✓ CI verifies every change automatically; CD promotes verified changes; the pipeline is a contract, not a tool.
- ✓ Build the artifact once and promote that exact digest — rebuilding per environment invites silent drift.
- ✓ Continuous Delivery keeps a human approval gate before prod; Continuous Deployment removes it.
- ✓ Secrets come from the encrypted store at run time, masked in logs, never in the repo, image, or a committed `.env`.
- ✓ Gates (tests, static analysis, CVE scan, staging smoke test, approval) define what "green" is allowed to mean.

Things to remember

- A required status check — not just a reported one — is what actually blocks a red merge.
- Tag by commit SHA and deploy by immutable digest; `latest` destroys traceability and rollback.

Common pitfalls (last look)

- Rebuilding per environment → "worked in staging, broke in prod." • Secret in an image layer → readable via `docker history`.
- Deploying `latest` → nobody knows what is live. • No rollback plan → a one-way door.

📋 One-Page Cheat Sheet

CI vs CD — CI = auto build+test every change; Delivery = one-click release; Deployment = auto release, no gate.

Build once — one image, tagged by commit SHA, pinned by `sha256` digest; promote it unchanged.

Config split — same artifact; env vars differ per environment; secrets per environment scope.

Secrets — `${{ secrets.X }}` injected at run time, masked in logs; never committed or baked into an image.

Gate — required tests + static analysis + CVE scan + staging smoke test + human approval on prod.

Runners — ephemeral VMs; jobs parallel and stateless; order with `needs:`, share files via artifacts/cache.

Jenkins — same stages, self-hosted agents, Groovy `Jenkinsfile`; control at the cost of operational overhead.

 Interview questions to master

- › What is the difference between Continuous Delivery and Continuous Deployment?
- › How do you manage secrets in a CI/CD pipeline?
- › Why should the same artifact be promoted across environments instead of rebuilt?
- › What would you include as a gate before allowing a deploy to production?
- › Why do CI runners being ephemeral matter for correctness and security?
- › How does Jenkins differ from GitHub Actions, and when would you still choose it?

Suggested further reading

- *Continuous Delivery* (Humble & Farley) — the canonical text; chapters on the deployment pipeline and build-once.
- GitHub Actions docs — "Workflow syntax," "Encrypted secrets," and "Using environments for deployment."
- The DORA / *Accelerate* research — the four key metrics and why pipelines drive them.

 Key idea

What you will learn next (Day 13). You can now ship safely and often — but shipping is not the same as staying up. Tomorrow you make ShopFlow **resilient**: timeouts, retries, circuit breakers and bulkheads, so that when a downstream dependency fails (and it will), your service degrades gracefully instead of collapsing. Delivery gets the code out; resilience keeps it alive once it is there.

DAY 13 • OPERATIONS

Production Readiness

13

Observability & resilience — what "production-ready" actually means

🕒 Estimated study time: 8 hours

🎯 By the end of this day, you will be able to...

- Instrument a service across the three pillars — structured logs with correlation IDs, Micrometer metrics, and traces — and say what each one answers at 3am.
- Separate *liveness* from *readiness* and wire readiness into the Compose healthcheck, so a briefly-degraded dependency never triggers a needless restart.
- Wrap an external call with a Resilience4j circuit breaker, retry and fallback — and explain every state transition and the correct composition order.
- Rate-limit public endpoints with a token bucket in Redis, returning 429 with a `Retry-After` header, and reason about *which* key you throttle on.
- Stand up Prometheus + Grafana and build a RED dashboard — request rate, error rate, p95 latency — that a human actually watches.

1 The Big Picture

What this day solves. Twelve days in, ShopFlow *runs*. It compiles, the tests are green, CI ships an image (Day 12), and it emits a handful of metrics (Day 11). None of that means it is **production-ready**. Production-ready is a different, harder claim: when a downstream payment provider goes dark at 02:40, does ShopFlow fail one feature gracefully — or does it exhaust its thread pool and take the whole checkout flow down with it? When a user says "it's slow," can you answer with a number instead of a shrug? This day is the difference between "it works on my machine" and "it survives contact with reality."

Why backend engineers need it. Every outage you will ever be paged for is, underneath, one of a small number of stories: a dependency got slow and the failure propagated; a deploy went out and nobody could see the error rate climb; a health check lied and traffic kept hitting a broken instance; one abusive client saturated a shared resource. The tools in this chapter — observability, circuit breakers, health probes, rate limits — exist precisely to turn each of those stories into a controlled, visible, recoverable event instead of a 4-hour incident.

🔑 Key idea

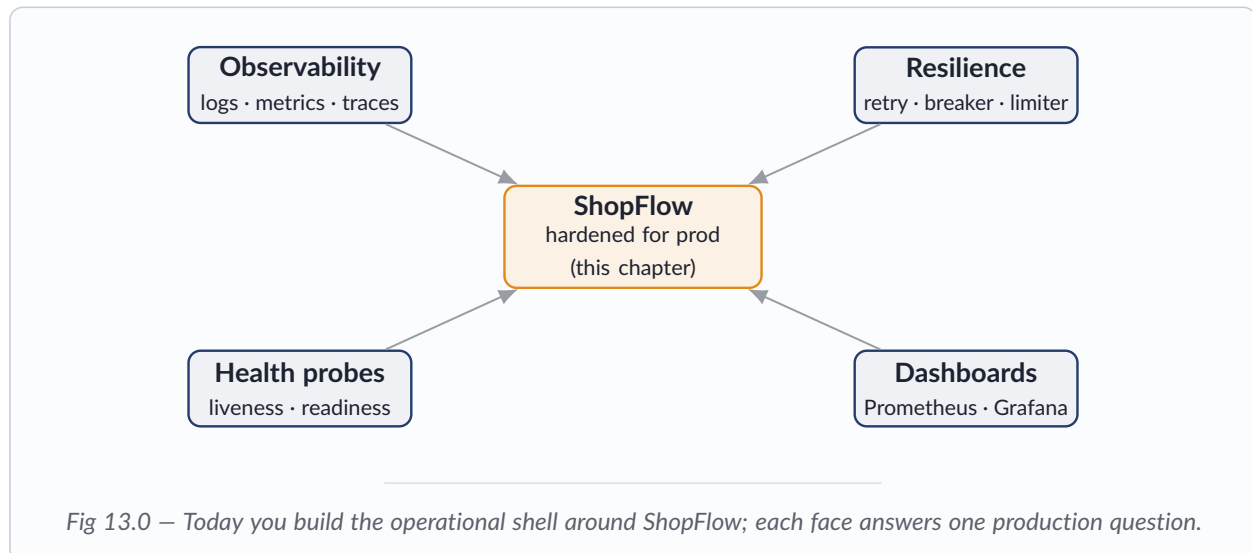
"Production-ready" is not a feeling, it is a checklist of **failure behaviour**. A ready service answers four questions on demand: *Can I see what it is doing?* (observability), *does it degrade instead of collapse when a dependency fails?* (resilience), *does the platform know when to restart vs. divert it?* (health probes), and *can one bad actor starve everyone else?* (rate limiting). Today you make ShopFlow answer all four.

Where it fits in a Spring Boot application. These concerns wrap *around* the code you already wrote.

Actuator and Micrometer sit beside your beans and expose their internal state; Resilience4j attaches proxies (the same AOP mechanism from Day 1) around your outbound calls; a servlet filter in front of your controllers enforces rate limits; and the orchestrator (Day 8's Compose, or Kubernetes later) polls your health endpoints from the outside. Nothing here changes your business logic — it hardens the shell around it.

How it connects to previous chapters. Day 1's proxies are how `@CircuitBreaker` intercepts a method. Day 5's filter chain is where the rate limiter lives. Day 8's Compose file gains a `healthcheck` block. Day 9's Redis becomes the shared token store for distributed rate limiting. Day 11's Micrometer registry is the source Prometheus scrapes. Day 12's pipeline is what promotes the hardened image. Today ties the whole roadmap into an operable system.

Real company examples. The circuit breaker entered the mainstream through Netflix's Hystrix, born from real cascading failures in their streaming backend; Resilience4j is its lightweight, functional successor. Prometheus came out of SoundCloud and is now the CNCF standard that Google, GitLab and thousands of others run. The liveness/readiness split is baked into Kubernetes because Google learned, at scale, that "the process is alive" and "the process can serve traffic" are genuinely different questions.

**2****Concept Map**

Hold the whole territory in one picture. The top half is *seeing* (observability); the bottom half is *surviving* (resilience). Both feed the single verdict at the bottom: is this thing production-ready?

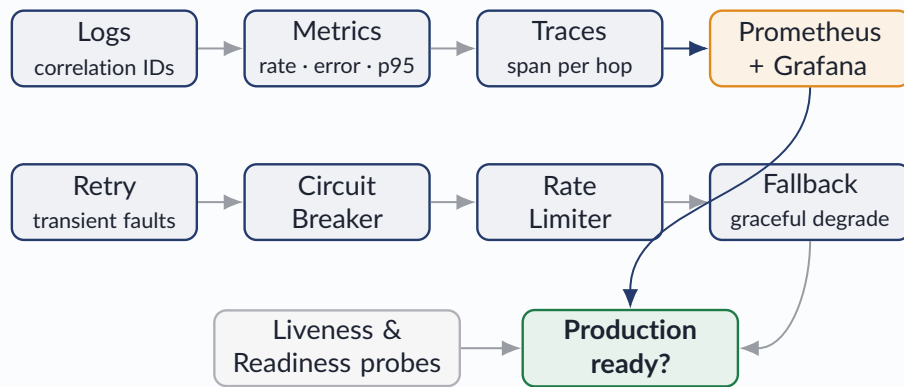


Fig 13.1 — The Day 13 concept map: observability lets you see failure, resilience lets you survive it, health probes let the platform react.

Terminology to anchor: **Observability** → **Logs** → **Metrics** → **Traces** → **Liveness** → **Readiness** → **Circuit Breaker** → **Rate Limiter** → **Dashboard**.

3

Guided Learning

3.1 The Three Pillars of Observability

What is it? **Observability** is the property that you can understand a system's internal state purely from the signals it emits — without shipping a new build to add a `println`. The three complementary signals are **logs** (discrete, timestamped events), **metrics** (numeric time series), and **traces** (the causal path of one request across services).

Why does it exist? "Monitoring" answers questions you knew to ask in advance (CPU > 90%). "Observability" is for the questions you did *not* anticipate: *why are exactly the checkout requests from one region timing out only since 14:05?* You cannot pre-build a dashboard for every such question, so you emit rich, structured, correlated signals and slice them after the fact.

How does it work internally? Each pillar has a different shape and cost, and they are meant to be used *together*, not as substitutes.

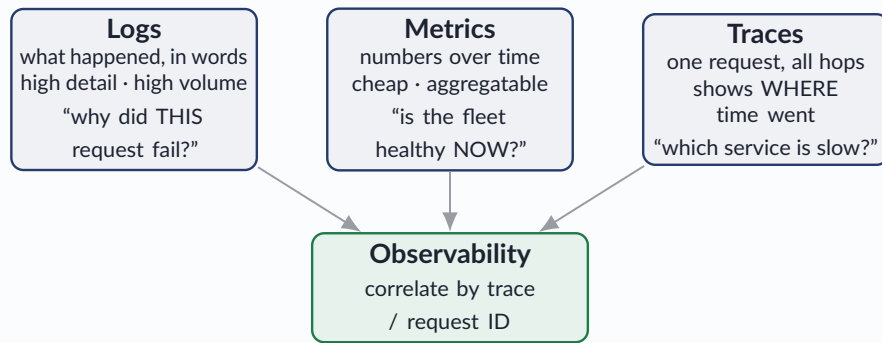


Fig 13.2 — Three pillars, three shapes. Logs tell the story of one request; metrics aggregate millions; traces stitch a request across services.

The glue: a correlation ID. The single most valuable habit is tagging every log line of one request with the same identifier, so you can reconstruct that request's whole story with one query. In Spring Boot 3 this is built on **Micrometer Tracing**: a filter puts a `traceId` into the SLF4J MDC (Mapped Diagnostic Context), and your log pattern prints it. If you also accept an inbound `X-Correlation-Id` you can follow a request all the way from the mobile client.

CorrelationIdFilter.java

```

@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class CorrelationIdFilter extends OncePerRequestFilter {

    static final String HEADER = "X-Correlation-Id";

    @Override
    protected void doFilterInternal(HttpServletRequest req,
        HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String cid = req.getHeader(HEADER);
        if (cid == null || cid.isBlank()) {
            cid = UUID.randomUUID().toString();
        }
        MDC.put("correlationId", cid); // now every log line carries it
        try {
            res.setHeader(HEADER, cid); // echo back so the caller can log it too
            chain.doFilter(req, res);
        } finally {
            MDC.remove("correlationId"); // MUST clean up: threads are pooled + reused
        }
    }
}

```

application.yml – log pattern includes the ID

```
logging.pattern.level=%5p [${spring.application.name},%X{correlationId}]
```

👁 Behind the scenes

The MDC is a `ThreadLocal` map. That is why the `finally` block is not optional: Tomcat hands the same worker thread to the next request, and a leftover `correlationId` would silently mislabel someone else's logs. The same trap bites you the moment you hop threads with `@Async` (Day 11) or reactive code — the MDC does not follow the work unless you propagate it.

💡 Production tip

Log *structured JSON* in production, not human-prose lines. With `logstash-logback-encoder` each event becomes a JSON object with `correlationId`, `level` and any key/value context as first-class fields — so your log platform can filter `correlationId="abc"` instead of grepping substrings. Humans read logs during a demo; machines read them during an incident.

3.2 Metrics that Matter — Micrometer, Prometheus & the RED Method

What is it? A **metric** is a named numeric time series with labels, e.g. `http_server_requests` tagged by `uri`, `method`, `status`. Micrometer (Day 11) is the vendor-neutral facade that records them; **Prometheus** is a server that *scrapes* (pulls) the `/actuator/prometheus` endpoint every 15s and stores the samples; **Grafana** queries Prometheus and draws the graphs.

Why does it exist? You cannot log every request's latency and then eyeball millions of lines. Metrics pre-aggregate: a **histogram** of request durations lets you compute a p95 across the whole fleet in one cheap query. Metrics are how you alert on *symptoms* rather than causes.

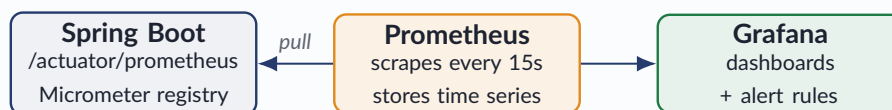


Fig 13.3 — The standard open-source monitoring stack. Prometheus pulls; it is not pushed to. Grafana never touches your app directly.

The RED method. For any request-driven service, three metrics tell you almost everything: **R**ate (requests/second), **E**rrors (fraction failing), **D**uration (latency distribution, watched at p95/p99, never the mean). Everything else is diagnosis; these three are the vital signs. Micrometer already exposes them from the `http_server_requests` timer — you just need the queries.

prometheus.yml — scrape config

```

scrape_configs:
- job_name: 'shopflow'
  metrics_path: '/actuator/prometheus'
  scrape_interval: 15s
  static_configs:
    - targets: ['shopflow:8080'] # service name on the Compose network

```

PromQL for the three RED panels

```

# request rate (req/s), summed across all endpoints
sum(rate(http_server_requests_seconds_count[1m]))

# error rate: fraction of responses that are 5xx
sum(rate(http_server_requests_seconds_count{status=~"5.."}[1m]))

```

```
/ sum(rate(http_server_requests_seconds_count[1m]))  
  
# p95 latency from the histogram buckets  
histogram_quantile(0.95,  
  sum(rate(http_server_requests_seconds_bucket[5m])) by (le))
```

When should I use it? Always, for anything you operate. The RED trio is the minimum viable dashboard; add saturation (pool usage, queue depth) as you learn where your service hurts.

When should I not? Do not turn every string into a metric label. Labels with unbounded values — a user ID, an order ID, a raw URL with path params — explode **cardinality**: Prometheus stores one time series *per unique label combination*, and millions of them will OOM the server.

⚡ Performance note

Micrometer templates URIs for you: `/orders/42` and `/orders/99` both record as `/orders/{id}`, one series. The moment you add your own tag like `.tag("orderId", id)`, you have re-created the cardinality bomb by hand. Rule of thumb: a label's value set should be small and bounded (status codes, endpoints, regions) — never user-generated.

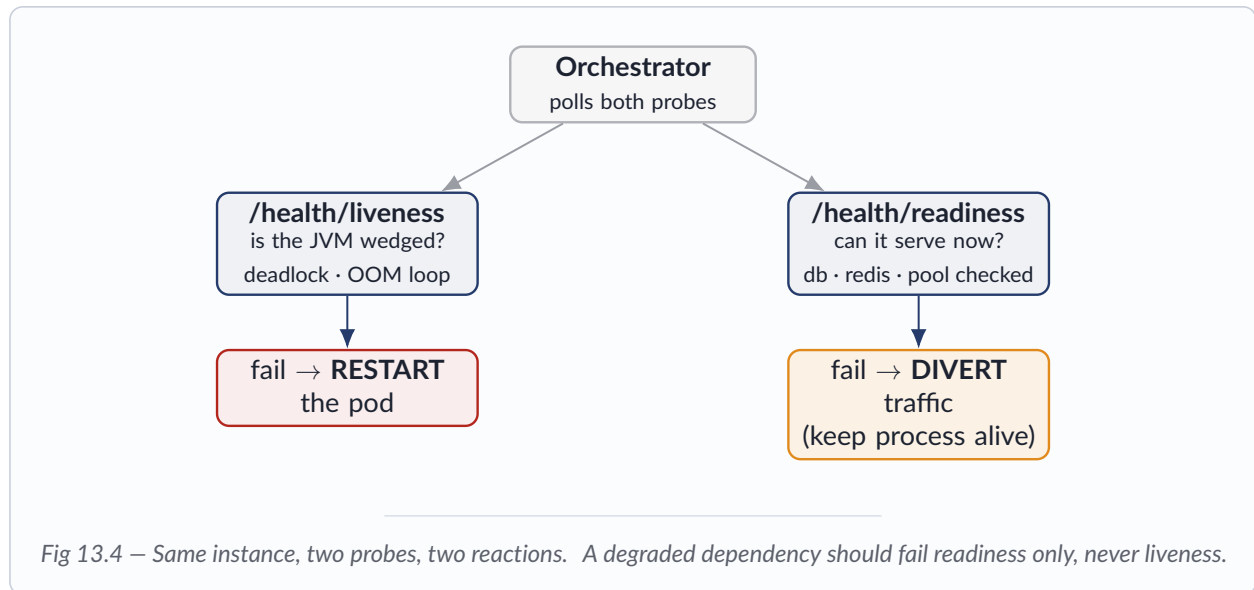
⚠ Common misconception

"p95 latency is roughly the average plus a bit." No. Averages hide the tail — a service can have a 40ms mean and a 3-second p99 because 1% of requests hit a cold cache or a lock. Users feel the tail, and one slow dependency shows up in p99 long before it moves the mean. Alert on percentiles, and read them from a histogram, not by averaging pre-computed averages (which is mathematically meaningless).

3.3 Health Checks — Liveness vs Readiness

What is it? Two *different* yes/no questions the orchestrator asks your process. **Liveness**: "is this process fundamentally broken — should I kill and restart it?" **Readiness**: "can this instance serve traffic *right now* — should it be in the load-balancer rotation?"

Why does it exist? Conflating them causes outages. Suppose the database is briefly unreachable. If your one health check fails and the orchestrator treats it as liveness, it *restarts* every instance — a restart storm that does nothing to bring the database back and drops all in-flight work. The correct behaviour is: stay alive (readiness fails, so you are pulled from the load balancer), and rejoin automatically when the dependency recovers. Same signal, opposite reactions.



How does it work internally? Spring Boot Actuator ships `LivenessStateHealthIndicator` and `ReadinessStateHealthIndicator` out of the box, exposed at `/actuator/health/liveness` and `/actuator/health/readiness` once you enable probes. Readiness automatically flips to `OUT_OF_SERVICE` during graceful shutdown, so the load balancer stops sending new requests *before* the app stops accepting them. You compose additional indicators into the readiness group via config.

application.yml – probes and groups

```

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      probes:
        enabled: true
        show-details: always
      group:
        liveness:
          include: livenessState
        readiness:
          include: readinessState,db,redis
  health:
    circuitbreakers:
      enabled: true      # surface breaker state as a health contributor
  
```

Wire it into Compose (Day 8). The healthcheck must hit *readiness*, and `start_period` must be generous – a JVM can take 20–40s to warm up, and you do not want it declared unhealthy during boot.

docker-compose.yml – healthcheck excerpt

```

shopflow:
  image: shopflow:latest
  healthcheck:
    test: ["CMD", "curl", "-f",
          "http://localhost:8080/actuator/health/readiness"]
    interval: 10s
    timeout: 3s
  
```

```
retries: 3
start_period: 40s    # grace window while the JVM warms up
```

⚠ Common misconception

"Liveness should also check the database." Almost never. If liveness checks the DB, then a DB blip restarts every replica simultaneously — turning a recoverable dependency hiccup into a self-inflicted outage. Liveness should check only things a *restart could fix* (a wedged thread, an unrecoverable state). Dependency health belongs in readiness.

💡 Production tip

Set liveness to be nearly un-failable and cheap; make readiness honest but bounded. A readiness check that itself blocks for 30s on a dead database is as bad as no check — always give dependency health indicators a short timeout, or they become the outage.

3.4 The Circuit Breaker State Machine

What is it? A **circuit breaker** is a proxy that watches the failure rate of calls to one dependency and, once failures cross a threshold, *stops calling it* — returning immediately (fail fast) instead of waiting on timeouts. It is a three-state machine borrowed from electrical engineering: trip the breaker before the wiring melts.

Why does it exist? The enemy is the **cascading failure**. A payment provider slows from 50ms to 30s. Every checkout thread now blocks for 30s waiting. Your fixed-size thread pool fills with threads parked on a dead dependency; new requests — even ones that do not touch payment — queue and time out. One slow dependency has taken down the entire service. The breaker cuts that feedback loop: after enough failures it opens, checkout returns a fast fallback, threads stay free, and the rest of ShopFlow keeps serving.

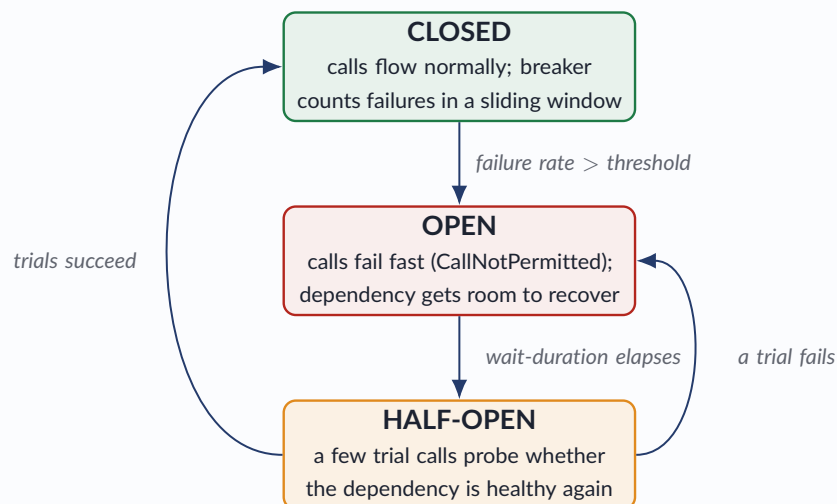


Fig 13.5 — The circuit breaker state machine. **CLOSED** lets calls through; **OPEN** fails fast; **HALF-OPEN** sends trial calls to test recovery.

How does it work internally? Resilience4j records outcomes in a **sliding window** — either the last

N calls (`COUNT_BASED`) or all calls in the last N seconds (`TIME_BASED`). When the failure rate over a full window exceeds `failure-rate-threshold`, it transitions `CLOSED` → `OPEN` and starts a timer. While `OPEN`, every call is rejected instantly with `CallNotPermittedException` — no thread touches the dependency. After `wait-duration-in-open-state`, it moves to `HALF-OPEN` and permits a small number of trial calls. If they succeed, it closes; if any fails, it opens again and the timer restarts.

application.yml – circuit breaker instance

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 20
        minimum-number-of-calls: 10      # don't judge on 1 sample
        failure-rate-threshold: 50       # open at 50% failures
        slow-call-duration-threshold: 2s # a slow call counts as a failure
        slow-call-rate-threshold: 80
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
        automatic-transition-from-open-to-half-open-enabled: true
```

👁 Behind the scenes

`minimum-number-of-calls` is the detail beginners miss. Without it, the very first call failing is a 100% failure rate and the breaker trips on a single fluke. It tells Resilience4j: "collect at least this many samples before you are allowed to compute a rate and open." A breaker that trips on one request is worse than no breaker — it manufactures outages.

👤 Interview insight

Expect: "Walk me through the circuit breaker states." A strong answer names all three and the *transitions with their triggers*: `CLOSED` opens on *failure-rate-over-window*; `OPEN` moves to `HALF-OPEN` after a wait; `HALF-OPEN` closes on successful trials or re-opens on a single failing trial. Then add the *why*: `OPEN` exists to give the dependency breathing room and to keep your threads free.

3.5 Retry, Fallback & Composition Order

What is it? **Retry** re-invokes a failing call a few times with backoff, for **transient** faults (a dropped connection, a one-off timeout). A **fallback** is the graceful answer when everything has failed — here, marking a payment *deferred* so the order survives and a worker can settle it later. Together with the breaker they form a small stack of decorators around one method.

PaymentGateway.java

```
@Service
public class PaymentGateway {

    private static final Logger log =
        LoggerFactory.getLogger(PaymentGateway.class);
    private final PaymentClient client;

    public PaymentGateway(PaymentClient client) {
        this.client = client;
    }

    // Retry is OUTER, circuit breaker is INNER (see composition note).
```

```
// Fallback lives on the outermost annotation only.
@Retry(name = "paymentService", fallbackMethod = "deferPayment")
@CircuitBreaker(name = "paymentService")
public PaymentResult charge(ChargeRequest req) {
    return client.charge(req); // remote HTTP call; may time out
}

// Fallback signature = same args + the Throwable that caused it.
private PaymentResult deferPayment(ChargeRequest req, Throwable ex) {
    log.warn("payment deferred order={} cause={}",
        req.orderId(), ex.toString());
    return PaymentResult.deferred(req.orderId());
}
}
```

How does it work internally? Order matters. Each annotation is a proxy layer, and Resilience4j nests them in a fixed default order (outermost first): `Retry` → `CircuitBreaker` → `RateLimiter` → `TimeLimiter` → `Bulkhead`. Read outside-in: a retry re-drives the *whole* inner stack, so each attempt is recorded by the breaker; the breaker, being inside retry, can open mid-retry and make the remaining attempts fail fast.

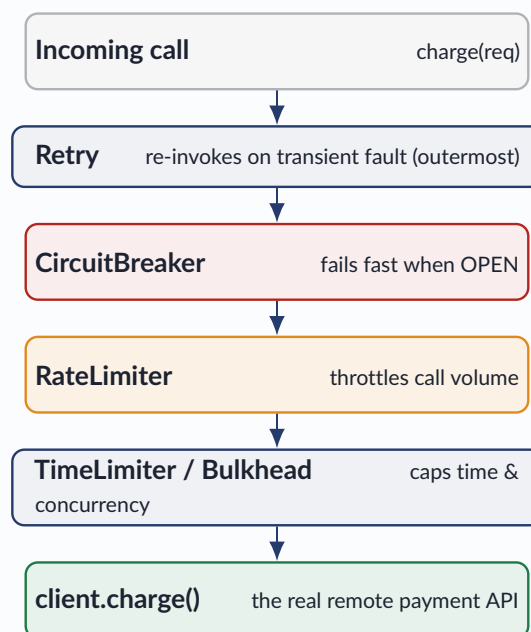


Fig 13.6 — The Resilience4j decorator stack (default order). A call falls through each layer; the breaker and limiter can short-circuit it before the remote call.

When should I use retry? Only for *idempotent* operations and *transient* faults. A payment `charge` is a dangerous thing to retry blindly — retrying a request that actually succeeded but whose *response* was lost can double-charge a customer. Constrain retry to network-level exceptions, and make the operation idempotent with an idempotency key.

application.yml — retry, scoped to transient faults

```
resilience4j:
  retry:
```

```
instances:
  paymentService:
    max-attempts: 3
    wait-duration: 200ms
    enable-exponential-backoff: true
    exponential-backoff-multiplier: 2
    retry-exceptions:
      - java.io.IOException
      - java.util.concurrent.TimeoutException
    ignore-exceptions:
      - io.github.resilience4j.circuitbreaker.CallNotPermittedException
```

⚠ Common misconception

"Put `fallbackMethod` on every annotation." Do not. If the *inner* `@CircuitBreaker` also had a fallback, it would swallow the exception and return a normal value — so the outer `@Retry` would see *success* and never retry. Declare the fallback on the *outermost* decorator only; the inner ones should propagate exceptions upward so the layers above can act on them.

💡 How senior engineers think

The most dangerous line in a resilience config is a naive retry. Three services each retrying three times is a $3 \times 3 \times 3$ amplification: a small downstream blip becomes a $27 \times$ traffic spike that keeps the dependency down — a **retry storm**. That is exactly why retry sits *inside* nothing that prevents it: the circuit breaker is your safety valve. Retry + breaker together; retry alone is a loaded gun. Add jitter to backoff so a thousand clients do not retry in lockstep.

3.6 Rate Limiting — Protecting Yourself and Others

What is it? A **rate limiter** caps how many requests a caller may make in a window, rejecting the excess with HTTP 429 `Too Many Requests` and a `Retry-After` header. It protects *you* from being overwhelmed and protects your *downstreams* from being hammered through you.

Why does it exist for internal services too? Because "trusted" callers cause the worst incidents: a buggy retry loop in a sister service, a batch job that fires 100k requests at once, a mis-configured client polling every 10ms. Without a limit, one internal caller starves every other tenant of a shared pool. Rate limiting is fairness enforcement, not just abuse defence.

How does it work internally? Two dominant algorithms. A **token bucket** holds up to *capacity* tokens and refills at a steady rate; each request spends one token; an empty bucket means 429. It permits short bursts (spare tokens) while bounding the long-run average — usually what you want. A **sliding window** counts requests over the trailing N seconds precisely, at higher storage cost. A naive **fixed window** (counter reset each second) is simplest but allows a double burst across the boundary.

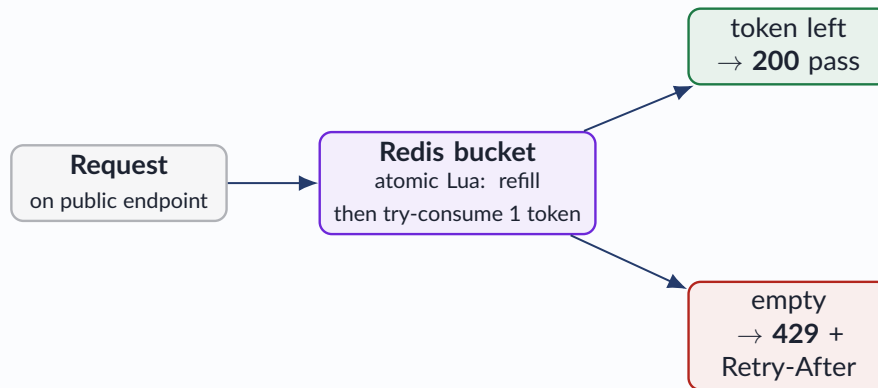


Fig 13.7 – A token-bucket limiter in Redis. Every instance shares one bucket, so the limit holds across the whole fleet.

Why Redis, not in-process? Resilience4j's own `RateLimiter` is per-JVM. With three ShopFlow replicas behind a load balancer, an in-process limit of 100/s becomes 300/s in aggregate — and the number drifts every time you autoscale. A *distributed* limiter keeps the counter in Redis (Day 9), so all replicas share one bucket. The consume-and-check must be *atomic*, which is why it runs as a single Lua script server-side — no read-modify-write race between instances.

RateLimitFilter.java

```

@Component
@Order(10) // after correlation-id, before controllers
public class RateLimitFilter extends OncePerRequestFilter {

    private final TokenBucketService buckets;

    public RateLimitFilter(TokenBucketService buckets) {
        this.buckets = buckets;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req,
        HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {

        String key = "rl:" + clientKey(req); // e.g. API key or client IP
        if (!buckets.tryConsume(key)) { // atomic Redis Lua call
            res.setStatus(HttpStatus.TOO_MANY_REQUESTS.value()); // 429
            res.setHeader("Retry-After", "1");
            res.setContentType("application/json");
            res.getWriter().write("{\"error\":\"rate_limited\"}");
            return; // do NOT continue the chain
        }
        chain.doFilter(req, res);
    }
}
  
```

Security warning • choose the throttle key carefully

The key you rate-limit on is the policy. Limit by client IP and one NAT gateway (a whole office, a mobile carrier) shares one bucket — you punish thousands for one user. Limit by user ID and unauthenticated traffic is unprotected. Limit by API key for partners, per-user for authenticated endpoints, and per-IP

only as a coarse backstop. And never trust a client-supplied `X-Forwarded-For` without a trusted proxy in front, or attackers simply spoof a fresh identity per request.

✓ Best practices — what experienced engineers do

- Separate liveness and readiness; readiness includes dependencies, liveness does not.
- Wrap every external call (payment, third-party API) with a circuit breaker; add retry only where the call is idempotent.
- Alert on *symptoms* — error rate, p95 latency, saturation — not raw counters nobody reads.
- Tag every request's logs with a correlation ID so one incident's full story is one query.
- Keep the fallback on the outermost decorator; keep metric label cardinality bounded.

✗ Common mistakes beginners make

- No breaker on an external call — one flaky dependency exhausts the thread pool and takes the whole service down.
- Treating "the process is up" as the only health signal, ignoring degraded dependencies.
- Retrying non-idempotent operations (double charges) or retrying without a breaker (retry storms).
- Logging without structure or correlation IDs, so incident debugging is grep archaeology.
- No alerting at all — learning about the outage from a customer instead of a dashboard.

4 Visualizations to Internalize

Two of today's diagrams carry most of the load. Reproduce them from memory — the act of redrawing is what converts "I recognise that" into "I can explain that on a whiteboard."

 Redraw: the circuit breaker state machine — 3 states, 4 transitions, each labelled with its trigger

 Redraw: one probe, two reactions — how liveness and readiness diverge when the database is briefly down

5 Guided Coding Lab — Harden the Payment Path

Goal of this lab: take ShopFlow's existing payment call and make it survive a dead provider — observably, resiliently, and without taking the rest of the app down. You will see the breaker trip in a metric, then watch it recover.

1 Add the dependencies and a correlation-ID filter

Add `resilience4j-spring-boot3`, `spring-boot-starter-actuator`, and `micrometer-registry-prometheus`. Register the `CorrelationIdFilter` from 3.1 and add `correlationId` to your log pattern.  **Why:** Before you change behaviour you must be able to see behaviour — every later step is verified by reading a log line or a metric, not by hoping.

2 Wrap `charge()` with a circuit breaker + retry + fallback

Annotate `PaymentGateway.charge` exactly as in 3.5: `@Retry` outer, `@CircuitBreaker` inner, `fallbackMethod` on the retry only, returning `PaymentResult.deferred(...)`.  **Why:** This is the single highest-value change of the day: one flaky dependency can no longer cascade into a full outage.

3 Configure the instances and scope the retry

Add the `paymentService` circuit-breaker and retry blocks from 3.4/3.5. Set `retry-exceptions` to network faults only and `ignore-exceptions` to `CallNotPermittedException`.  **Why:** An unscoped retry double-charges customers and amplifies outages; scoping it is what makes retry safe rather than dangerous.

4 Split liveness and readiness, wire readiness into Compose

Enable probes, put `readinessState`, `db`, `redis` in the readiness group, and point the Compose healthcheck at `/actuator/health/readiness` with a 40s `start_period`.  **Why:** Now a brief DB blip diverts traffic instead of triggering a restart storm — the difference between a blip and an outage.

5 Add a Redis token-bucket rate limiter returning 429

Register `RateLimitFilter`; back `tryConsume` with an atomic Redis Lua script keyed per API key or IP. Return 429 with `Retry-After` when the bucket is empty.  **Why:** This protects the shared payment provider from being hammered and enforces fairness across clients — across all replicas, not just one JVM.

6 Stand up Prometheus + Grafana and build the RED panels

Add Prometheus + Grafana services to Compose, scrape `/actuator/prometheus`, and build three panels using the PromQL from 3.2: request rate, error rate, p95 latency.  **Why:** A metric nobody graphs is invisible; a dashboard is where "the app feels slow" becomes "p95 doubled at 14:05 on the payment endpoint."

7 Simulate the failure and watch the machine work

Point the payment client at a dead host (or a stub that sleeps 5s). Fire load; watch the breaker go OPEN in the `resilience4j_circuitbreaker_state` metric, checkout return deferred instantly, and the breaker close again once the stub recovers. 💡 *Why*: Reading the state transition on your own dashboard is what turns Fig 13.5 from a picture into an instinct.

🔧 Production tip

Do not test resilience only with a config change. Actually kill the dependency — `docker compose stop` the payment stub mid-load — and confirm three things: checkout still responds fast, the error budget is spent on *fallbacks* not timeouts, and no thread pool metric flatlines. Resilience you have not seen fail is resilience you do not have.

6 Thinking Exercises

Reflection prompts, not quiz questions. Write a sentence or two — the goal is to make your reasoning explicit.

🧠 Think it through

Your service calls a payment provider and a (non-critical) recommendations service. You have one thread pool for all outbound calls. Explain, step by step, how a slowdown in the *recommendations* service could cause *payments* to start failing — and what isolation mechanism would stop it.

🧠 Think it through

A teammate proposes a single `/health` endpoint that checks the database and returns 503 if it is down, used for both liveness and readiness. Describe the concrete production failure this invites, and rewrite the design in one or two sentences.

🧠 Think it through

You add a retry (3 attempts) to a call, and during the next incident the outage lasts *longer* than before you added it. Explain how a retry can make an outage worse, and what two settings you would add to make retry safe.

7

Debugging Practice

 Debugging Lab • The circuit breaker that never opens

Symptom. The payment provider is clearly down — every call times out — yet the breaker stays `CLOSED` and threads keep piling up on the dead dependency.

Evidence.

application.yml — the misconfigured instance

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        sliding-window-size: 100
        minimum-number-of-calls: 100
        failure-rate-threshold: 50
        # calls average ~4/min in this environment
```

Work the method:

- ✓ **Observe.** `resilience4j_circuitbreaker_state` shows `closed`; `buffered-calls` count creeps up slowly and never reaches 100.
- ✓ **Hypothesise.** The window needs 100 calls before it will *compute* a rate, but at 4 calls/min it takes 25 minutes to fill — the breaker cannot trip in time.
- ✓ **Verify.** Lower `minimum-number-of-calls` to 10 (or switch to `TIME_BASED`); reproduce and watch the state flip to `open`.
- ✓ **Fix.** Size the window to your real traffic rate: enough samples to avoid flukes, small enough to react within seconds.
- ✓ **Prevent.** Add an alert on `circuitbreaker_state == open` and load-test the breaker's reaction time, not just its config validity.

 Debugging Lab • The restart storm

Symptom. At 02:40 the database has a 30-second network blip. Instead of recovering, every ShopFlow replica restarts, in-flight orders are lost, and the outage lasts 15 minutes.

- ✓ **Observe.** The orchestrator event log shows liveness-probe failures on every pod, followed by kills and cold restarts stacked on top of the DB blip.
- ✓ **Hypothesise.** The liveness probe checks the database, so a dependency blip is being treated as "the process is broken — restart it."
- ✓ **Verify.** Inspect the health group config; confirm `db` is in the *liveness* group, not just readiness.
- ✓ **Fix.** Move dependency checks (`db`, `redis`) into the *readiness* group only; leave liveness as

`livenessState.`

- ✓ **Prevent.** Make it a review rule: liveness may only check things a restart could fix. Add a chaos test that pauses the DB and asserts pods divert, not restart.

8 Mini-Labs

🔧 Mini-Lab • Beginner

25 min

Goal. See a circuit breaker trip with your own eyes.

Context. A trivial `GET /flaky` that calls a stub which throws 70% of the time.

Requirements. Annotate the call with `@CircuitBreaker(name="flaky"); set failure-rate-threshold: 50, minimum-number-of-calls: 10.`

Hints. Hit the endpoint in a loop; scrape `/actuator/prometheus` and `grep circuitbreaker_state.`

Expected outcome. After 10 calls the state flips to `open` and further calls return the fallback instantly.

Common pitfalls. Omitting `minimum-number-of-calls`, so the first failure trips a 100% rate and confuses the demo.

🔧 Mini-Lab • Intermediate

45 min

Goal. Prove your rate limiter holds *across replicas*.

Context. Two ShopFlow instances behind one load balancer, sharing Redis.

Requirements. Implement the token bucket as an atomic Redis Lua script; limit to 100/s per client key; return `429 + Retry-After.`

Hints. Run both instances via Compose; drive load with a tool that reports the 429 ratio.

Expected outcome. Aggregate accepted traffic stays near 100/s regardless of which instance serves each request.

Common pitfalls. Doing a read-then-write in Java instead of one Lua call — two instances race and the real limit drifts above 100/s.

🔧 Mini-Lab • Production-style

70 min

Goal. Ship a RED dashboard and one meaningful alert.

Context. You want to know about a payment degradation before a customer tells you.

Requirements. Prometheus + Grafana in Compose; panels for request rate, 5xx error rate, and p95 latency; one alert rule: p95 on the payment endpoint $> 1s$ for 5 minutes.

Hints. Use `histogram_quantile` over the `_bucket` series; filter by `uri` for the payment route.

Expected outcome. Killing the payment stub visibly moves error rate and p95, and the alert fires after the sustained window.

Common pitfalls. Alerting on a single scrape (flaps constantly) — use a `for: duration` so a blip does not page you.

9 Engineering Notes

Judgment from engineers who have carried a pager. Read these as instinct, not trivia.

 How senior engineers think

The goal of resilience is not "never fail" — it is "fail in a small, predictable, cheap way." A deferred payment that a background worker settles in 30 seconds is a good failure; a checkout page that hangs for 30 seconds and then 500s is a catastrophic one. Design your fallbacks around what the *user* can tolerate, not around what is technically easiest to return.

 Performance note

Every resilience wrapper costs something. A circuit breaker maintains a sliding-window buffer per instance; a rate limiter does a network round-trip to Redis on every request; enabling histogram buckets multiplies the series each timer emits. On a hot path, measure the overhead — and do not put a distributed rate limiter in front of an endpoint that serves a million req/s without checking that Redis can take the load.

 Interview insight

Expect: "Why is rate limiting necessary even for internal services?" The answer that lands: trust does not prevent bugs. A retry loop, a runaway batch job, or a mis-set poll interval in a sibling service can generate more damaging load than any external attacker — and because it is "internal," nobody thought to bound it. Rate limiting is fairness and blast-radius control, not just abuse defence.

 Production tip

Build the production-readiness checklist *into the pipeline*, not into a wiki nobody reads. Fail the Day 12 build if the actuator health endpoint is missing, if an external client has no breaker, or if a new endpoint has no rate limit. The cheapest time to catch an un-hardened service is before it ever ships.

10 Build-Along Project — ShopFlow

ShopFlow already ships through CI (Day 12) and emits metrics (Day 11). Today you make it *operable*: it survives a dead dependency, tells the platform when to divert vs. restart, fends off abusive load, and shows its vital signs on a dashboard.

 Build-Along • Day 13 — production hardening

- Wrap the payment integration in `PaymentGateway.charge` with a `Resilience4j@CircuitBreaker` (inner) + `@Retry` (outer) + a `deferPayment` fallback; scope retry to transient faults only.
- Configure the `paymentService` instances (window, threshold, wait-duration, backoff) in `application.yml`; surface breaker state as a health contributor.
- Enable Actuator probes and split *liveness* (`livenessState`) from *readiness* (`readinessState,db,redis`); point the Compose healthcheck at `/actuator/health/readiness` with a 40s `start_period`.
- Add a distributed token-bucket rate limiter (Redis Lua, from Day 9) on the public endpoints via a `OncePerRequestFilter`, returning 429 with `Retry-After`.
- Add a `CorrelationIdFilter` + structured logging so one request's full story is one query.
- Stand up Prometheus + Grafana in Compose and build the RED dashboard: request rate, 5xx error rate, and p95 latency, with one alert on sustained payment-endpoint p95.

Definition of done: stopping the payment stub mid-load leaves checkout responsive (returning deferred), the breaker visibly trips and recovers on the Grafana dashboard, a DB blip diverts traffic without restarting any container, and abusive load gets 429s while everyone else keeps serving.

11 Chapter Summary

Key takeaways

- ✓ Observability = logs (one request's story) + metrics (the fleet's vital signs) + traces (where time went), glued by a correlation ID.
- ✓ Liveness and readiness are different questions with opposite reactions: restart vs. divert. Dependencies belong in readiness only.
- ✓ A circuit breaker (CLOSED → OPEN → HALF-OPEN) stops a slow dependency from exhausting your threads and cascading into a full outage.
- ✓ Retry is for idempotent, transient faults — and only ever behind a breaker, or it becomes a retry storm.
- ✓ Rate limit on the right key, in Redis for a multi-replica fleet, returning 429 + `Retry-After`.

Things to remember

- Fallback on the outermost decorator only; an inner fallback hides the exception and defeats retry.
- Keep metric label cardinality bounded — never tag by user or order ID.

Common pitfalls (last look)

- DB check in liveness → restart storm. • No `minimum-number-of-calls` → breaker trips on a fluke.
- In-process rate limit across replicas → the real limit is $N \times$ what you configured.

📄 One-Page Cheat Sheet

Three pillars — logs = events, metrics = time series, traces = per-request path; correlate by trace/correlation ID.

RED metrics — Rate, Errors, Duration (p95/p99, from a histogram — never the mean).

Liveness vs readiness — liveness → restart (only restart-fixable state); readiness → divert (includes dependencies).

Breaker states — CLOSED (counting) → OPEN (fail fast) → HALF-OPEN (trial) → CLOSED/OPEN.

Composition order — `Retry(CircuitBreaker(RateLimiter(TimeLimiter(Bulkhead))))`; fallback on the outermost only.

Retry safety — idempotent ops only, transient exceptions only, always behind a breaker, with jittered backoff.

Rate limit — token bucket in Redis (atomic Lua), keyed per client; excess → 429 + `Retry-After`.

 Interview questions to master

- › What is the difference between liveness and readiness checks, and what does each failure trigger?
- › Explain the circuit breaker states and every transition between them.
- › Why is rate limiting necessary even for internal, trusted services?
- › What would you put on a production-readiness checklist before launch?
- › What are the three pillars of observability, and what question does each one answer?
- › Why is retrying a payment call dangerous, and how do you make retry safe?

Suggested further reading

- *Resilience4j Reference* — CircuitBreaker, Retry, RateLimiter, and the aspect order section.
- *Spring Boot Reference* — "Kubernetes Probes" and "Actuator / Metrics" (Micrometer + Prometheus).
- Google SRE Book — "Monitoring Distributed Systems" (the four golden signals) and "Handling Overload."
- Michael Nygard, *Release It!* — the origin of the circuit breaker and bulkhead stability patterns.

 Key idea

What you will learn next (Day 14). ShopFlow now runs, ships, and survives. Tomorrow is the capstone: you assemble the fourteen days into one coherent, reviewable system — and rehearse explaining every decision out loud, because being able to *narrate* why the breaker sits where it does is what turns a working project into a job offer.

DAY 14 • CAPSTONE

Final Project — ShopFlow

14

Assembling, verifying & presenting a production-ready backend — built by you

🕒 Estimated study time: 10 hours

🎯 By the end of this day, you will be able to...

- Read a real-world specification like an engineer: separate functional from non-functional requirements and turn ambiguity into documented decisions.
- Assemble the thirteen prior days into one coherent backend — persistence, REST, security, tests, containers, cache, messaging, resilience and CI.
- Model the order lifecycle as an explicit, validated state machine instead of a free-form status string.
- Diagnose the three incidents every on-call engineer meets: intermittent 500s, an exhausted connection pool, and a bad deploy that must be rolled back.
- Present the project the way interviewers reward: justify every decision, name your trade-offs, and ship a repository that clones green.

1 The Big Picture

What this day solves. For thirteen days you learned mechanisms one at a time — the IoC container, indexes, optimistic locking, JWT, Testcontainers, Docker, Redis, RabbitMQ, circuit breakers. Each was correct in isolation. This day answers the harder question: *do they hold together as one system?* A production-ready backend is not a pile of correct features; it is a set of decisions that stay consistent under load, under failure, and under the gaze of the next engineer who reads your code. ShopFlow is where you prove you can make those decisions.

Why backend engineers need it. Nobody is paid to write a `@Transactional` method. Engineers are paid to ship a *system* that takes an order, charges nobody twice, never oversells stock, keeps answering during a dependency outage, and can be rolled back in ninety seconds when a deploy goes wrong. The capstone is the smallest honest rehearsal of that job — a take-home assignment or a first-sprint project compressed into one domain.

🔑 Key idea

“Production-ready” is not a feeling; it is a checklist you can defend line by line. It means: **correct** under concurrency, **observable** when it misbehaves, **recoverable** when a deploy is bad, **tested** so a stranger can change it safely, and **documented** so the reader understands *why*, not just how to run it. ShopFlow is deliberately a two-entity domain carrying a very large amount of that engineering weight.

Where it fits in a Spring Boot application. ShopFlow is the same layered application you scaffolded on Day 1, now fully populated. The container still sits at the centre; every day since has plugged a component into it. Today you do no new mechanics — you assemble, verify, and rehearse the story

you will tell about it.

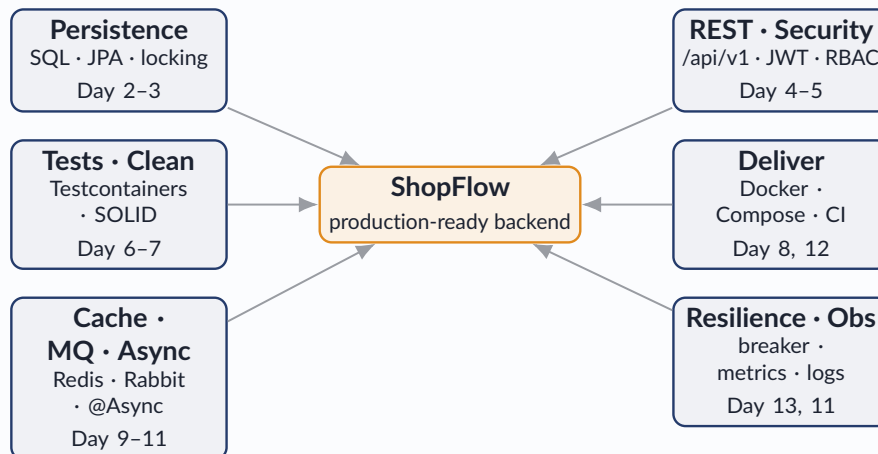


Fig 14.1 — Thirteen days of isolated mechanisms converge into one production-shaped backend.

How it connects to previous chapters. Literally everything. The build order below is the roadmap in miniature: schema (Day 2–3), CRUD and error handling (Day 4), auth (Day 5), tests (Day 6), clean layering (Day 7), containers (Day 8), then the cross-cutting concerns of caching, messaging, async and resilience (Days 9–11, 13), finished by CI (Day 12). If any earlier day is fuzzy, ShopFlow is where the fuzz becomes a compile error.

Real company examples. A senior take-home at Stripe, Monzo, or a typical Series-B startup is exactly this shape: a small domain, a big expectation of engineering rigour. Reviewers do not count features — they clone the repo, run one command, read the README, and look for judgment. ShopFlow is engineered to survive that exact review.

2 Concept Map

Hold the whole system in one picture before you touch a keyboard. Read it top-to-bottom (a request's path) and notice the three side concerns every production service carries: a message path, an observability path, and a delivery path.

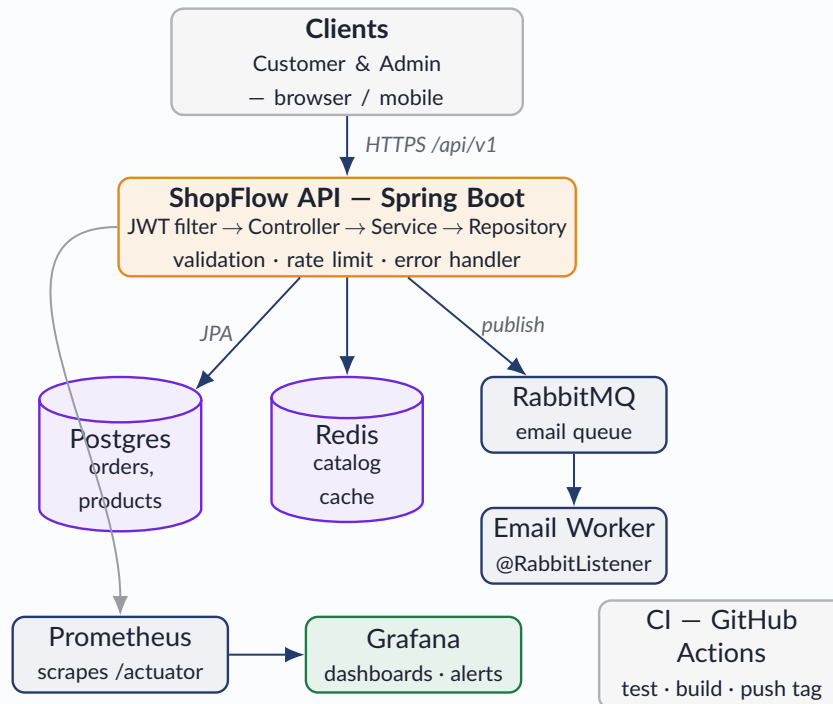


Fig 14.2 — The ShopFlow runtime: one app, three backing services, plus observability and CI around the edges.

Terminology to anchor: **Requirement** → **Domain model** → **State machine** → **Optimistic lock** → **Async event** → **Cache invalidation** → **Observability** → **Immutable artifact** → **Rollback**.

3 Guided Learning

3.1 Reading a spec: functional vs non-functional

What is it? A specification has two halves that beginners blur together. **Functional requirements** describe *what the system does* — observable behaviour a user could click through. **Non-functional requirements** (NFRs) describe *how well it must do it* — the qualities that never appear on a screen but decide whether the system survives production: correctness under concurrency, security, testability, latency, recoverability.

Why does it exist? Juniors ship the functional half and call it done; the demo works. Seniors know the NFRs are where the real work — and the real interview signal — lives. Anyone can write “place an order.” The craft is placing an order that cannot oversell stock, is covered by a test, emits a metric, and can be rolled back.

Why treat ambiguity as a decision? The spec is deliberately underspecified in places (what TTL? keyset or offset pagination? cancel a SHIPPED order?). That is not an oversight to escalate — it is an invitation to decide and *write down why*. “I chose a 60-second catalog TTL because prices change rarely and stale reads are tolerable for a minute” is exactly the sentence an interviewer wants to hear.

3.2 ShopFlow — functional requirements

Feature	What the backend must do	Day
Auth	Registration and JWT login; passwords BCrypt-hashed, never stored in clear.	5
Roles	<i>CUSTOMER</i> manages only their own orders; <i>ADMIN</i> manages the catalog and views all orders.	5
Catalog	CRUD for admins; paginated, filterable browsing for everyone.	2, 4
Orders	Validate stock, decrement inventory with optimistic locking, compute the total server-side.	3
Lifecycle	<i>PENDING</i> → <i>PAID</i> → <i>SHIPPED</i> → <i>DELIVERED</i> (or <i>CANCELLED</i>); every transition validated.	3
Email	Confirmation sent <i>asynchronously</i> via a message queue on placement.	10, 11
Caching	Catalog reads served from Redis; the cache is invalidated on every write.	9

⚠ Common misconception

“The total comes from the request body.” Never. A client that computes its own price can pay one cent for a laptop. The server recomputes the total from the current product prices at placement time; the request only carries product ids and quantities. The same rule kills every “negative quantity” and “someone else’s order id” exploit — trust the server’s data, validate the client’s.

3.3 ShopFlow — non-functional requirements

Concern	Requirement	Day
Architecture	Layered (Controller/Service/Repository), SOLID, thin controllers.	7
Testing	Unit tests for every business rule; Testcontainers integration; MockMvc controller tests.	6
Security	JWT auth, BCrypt passwords, role-based authorization, locked-down CORS.	5
API	Versioned <code>/api/v1</code> , one consistent error format, OpenAPI docs, page- or keyset-based pagination.	4
Resilience	Circuit breaker on any external call; rate limiting on public endpoints.	13
Observability	Actuator health/metrics, structured logs with correlation ids, a Grafana dashboard.	11, 13
Delivery	Multi-stage Dockerfile, Compose (app+db+redis+mq), CI running tests and building the image.	8, 12

💡 How senior engineers think

Read the NFR table as your grading rubric. When you present ShopFlow, an interviewer will silently walk exactly these rows: “show me the concurrency test,” “show me the error format,” “show me the dashboard.” A feature with no test, no metric, and no error contract is, to a reviewer, not finished — it is a prototype wearing a finished feature’s clothes.

3.4 The domain model & optimistic locking

What is it? Two entities carry the whole domain: *Product* (catalog item with a price and a stock count) and *Order* (a customer’s basket with a status and line items). The engineering weight is concentrated in one field on *Product*: the stock count, mutated concurrently whenever two customers race for the last unit.

Product.java

```
@Entity
public class Product {
```

```
@Id @GeneratedValue
private Long id;
private String name;
private BigDecimal price;
private int stock;

@Version          // optimistic-lock token
private long version;
// getters, no public setter for stock -> mutate via domain method
}
```

How does it work internally? `@Version` makes Hibernate append the version to every `UPDATE`'s `WHERE` clause: `UPDATE product SET stock=?, version=version+1 WHERE id=? AND version=?`. If another transaction already bumped the version, zero rows match and Hibernate throws `OptimisticLockException`. No row is ever silently overwritten; the loser retries or fails cleanly.

Database insight

Optimistic locking is the right default for a web backend because conflicts are *rare* — two people wanting the same last unit at the same millisecond is the exception, not the rule. You pay nothing on the happy path (no held row locks, no blocked connections) and only handle contention when it actually happens. Pessimistic `SELECT ... FOR UPDATE` is the opposite trade: it serialises access up front and is worth it only under genuinely high contention on a hot row.

Common misconception

Optimistic locking does not *prevent* the race — it *detects* it. The oversell (two reads both seeing “1 available”) is stopped because the second `UPDATE` matches no row and fails, so inventory never goes to `-1`. Your service must catch `OptimisticLockException` and decide: retry, or return `409 Conflict` to the customer. Detection without a handling policy is just a different crash.

3.5 The order lifecycle as a validated state machine

Why does it exist? A `status` column that any code can set to any string is a bug generator: an order jumps from `PENDING` straight to `DELIVERED`, or a `CANCELLED` order ships. Modelling status as an explicit **state machine** — a fixed set of states and a fixed set of legal transitions — turns “did we validate this?” into “is this edge in the graph?”

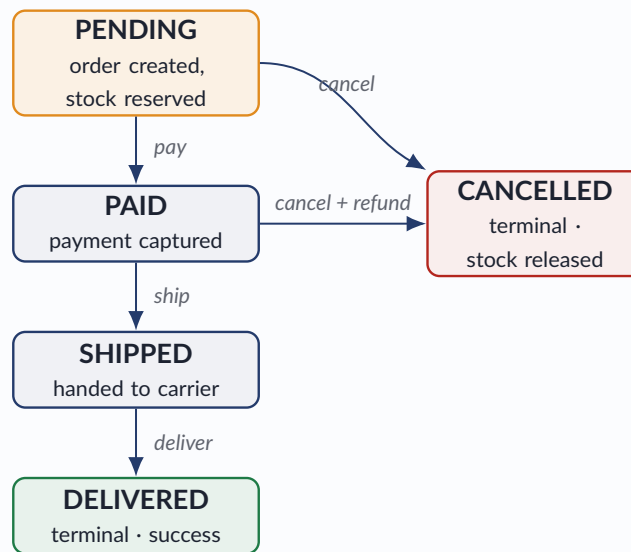


Fig 14.3 – The order state machine. Only the drawn edges are legal; every other transition is a 409.

How does it work internally? Keep the legal edges in one place and refuse everything else. The transition method is the single choke point through which status ever changes.

Order.java

```

public enum OrderStatus {
    PENDING, PAID, SHIPPED, DELIVERED, CANCELLED;

    private static final Map<OrderStatus, Set<OrderStatus>> NEXT = Map.of(
        PENDING, Set.of(PAID, CANCELLED),
        PAID, Set.of(SHIPPED, CANCELLED),
        SHIPPED, Set.of(DELIVERED),
        DELIVERED, Set.of(),
        CANCELLED, Set.of());

    public boolean canGoTo(OrderStatus target) {
        return NEXT.get(this).contains(target);
    }
}

// in Order:
void transitionTo(OrderStatus target) {
    if (!status.canGoTo(target))
        throw new IllegalStateException(status, target); // -> 409
    this.status = target;
}

```

Interview insight

“How do you model an order’s status?” The weak answer is “a string column.” The answer that lands: “an enum with an explicit transition table, so illegal transitions are impossible by construction and map to a 409, not a corrupt row.” Then mention the two side effects that ride on transitions — releasing reserved stock on CANCELLED, and never allowing DELIVERED or CANCELLED to move again because they are terminal.

3.6 One request, every day at once

How does it work internally? The reason ShopFlow is a good capstone is that a single “place order” request touches nearly every day of the roadmap. Trace it and you are reciting the whole book.

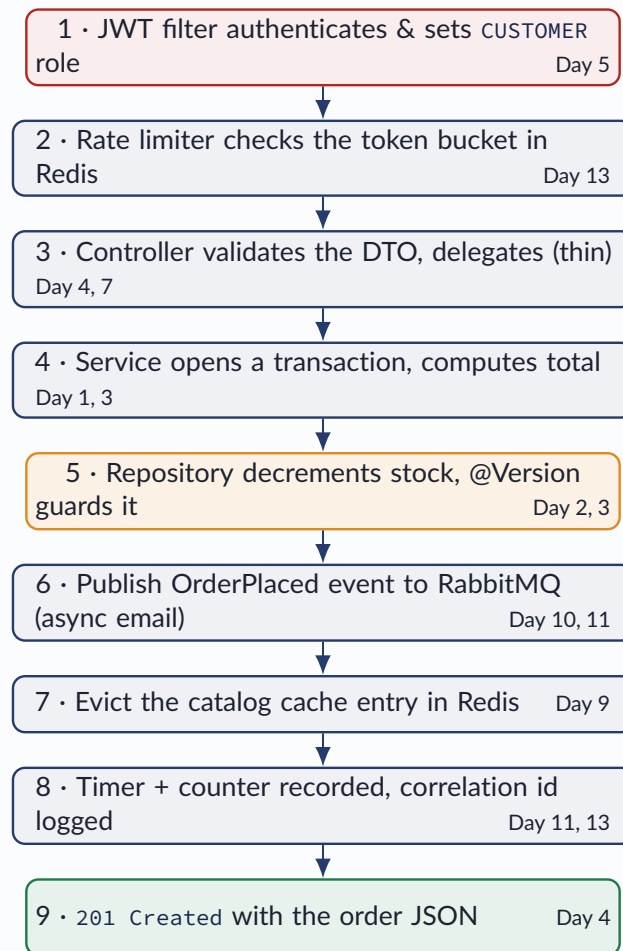


Fig 14.4 — `POST /api/v1/orders`: one request threading through thirteen days of concepts.

⚡ Performance note

Notice what is *inside* the transaction and what is not. Steps 4–5 are transactional and must be fast — they hold a database connection. The email (step 6) must be *outside* the transaction: never do slow I/O (SMTP, HTTP) while holding a connection, or you convert one slow dependency into an exhausted pool. Publishing an event and letting a worker send the mail is the whole reason the message queue exists.

✓ Best practices — what experienced engineers do

- Recompute money server-side; the client sends ids and quantities, never prices or totals.
- Every status change goes through one `transitionTo` choke point — no scattered `setStatus`.
- Do slow I/O (email, external HTTP) outside the DB transaction, via the queue.
- Write the test with the feature, not after; a rule with no test is a rule you will break.
- Document each ambiguous decision (TTL, pagination style, cancel rules) in the README.

✖ Common mistakes beginners make

- Trusting the request body's total or the client's product price.
- A free-text `status` string that any code can set to anything.
- Sending the confirmation email synchronously inside `placeOrder`, blocking the request.
- Catching `OptimisticLockException` nowhere, so a lost race becomes an ugly 500.
- Gold-plating: adding wishlists and coupons instead of hardening the two entities you have.

4 Visualizations to Internalize

Two diagrams carry this whole chapter. Redraw them *from memory* — if you can reproduce the architecture and the state machine on a whiteboard, you can answer most of the capstone interview.

 Redraw: the ShopFlow architecture — app, Postgres, Redis, RabbitMQ + worker, Prometheus/Grafana, CI

 Redraw: the order state machine, and label what each transition validates and its side effect

5 Guided Coding Lab — Assemble ShopFlow

Goal of this lab: walk the spec's *Suggested Build Order*, one deliberate step at a time. Do not skip ahead to the exciting parts (caching, messaging) before the happy path works — the order below exists because it minimises rework. Build depth on a small surface; resist every urge to add a feature.

1 Schema, entities, migrations

Model `Product` and `Order` (with `OrderItem`); add `@Version` to `Product`. Write the tables as versioned migrations, not `ddl-auto=update`. Seed a few products. 💡 *Why*: The schema is the contract every later layer depends on; migrations make it reproducible in CI and in a teammate's clone.

2 Core CRUD, DTOs, validation, error handling

Build product CRUD (admin) and paginated browsing (all), plus order placement returning a DTO. Add `@Valid` on inputs and one `@RestControllerAdvice` that renders every error in a single JSON shape. 💡 *Why*: A consistent error contract and DTO boundary is what makes the API usable by anyone but you — and it is the first thing a reviewer inspects.

3 Auth: registration, login, JWT, role checks

Add registration (BCrypt), login issuing a JWT, a filter that authenticates the token, and method/route rules so `CUSTOMER` sees only their orders and `ADMIN` manages the catalog. Lock CORS to known origins. 💡 *Why*: Security is not a later layer you bolt on; wire it before the endpoints multiply, so every new route is protected by default.

4 Tests as you go — never at the end

For each rule you add, add its test: unit tests for the state machine and total computation, a Testcontainers test for the optimistic-lock oversell, MockMvc tests for the controller contract and auth. 💡 *Why*: Tests written alongside the code pin behaviour while it is fresh; tests deferred to “the end” are the tests that never get written.

5 Refactor toward clean layering

Once the happy path is green, push logic out of controllers into services, keep repositories thin, and make services depend on interfaces where a seam helps. Thin controllers, fat-enough services, dumb repositories. 💡 *Why*: Refactor *after* it works and *with* tests as a safety net — clean layering done up front is guesswork; done now it is informed by real code.

6 Containerize and get docker compose up working end to end

Write a multi-stage Dockerfile and a Compose file bringing up app + Postgres + Redis + RabbitMQ. Externalise config via environment variables and a `prod` profile. 💡 *Why*: “One command brings up the whole stack” is a completion-checklist item and the single best signal to a reviewer that the project is real.

7 Layer in the cross-cutting concerns, then CI

Now add Redis caching (invalidated on write), the async email via RabbitMQ, a circuit breaker on the external call, rate limiting on public routes, Actuator metrics with a Grafana dashboard — then a CI pipeline that runs the tests and builds the image on every push. 💡 *Why*: These are cross-cutting by design; adding them onto a working, tested core is safe, whereas building them first would be building on sand.

🔧 Production tip

After every step, run `docker compose down -v && docker compose up`. The most valuable thing you can prove is that a *cold* clone comes up green with one command. A repo that only works on your laptop because of a hand-run migration or a locally cached image is, to an interviewer, a broken repo — no matter how good the code inside is.

6 Thinking Exercises

These are reflection prompts, not quiz questions. Write a sentence or two — making your reasoning explicit is the point.

Think it through

The spec says “resist the urge to add features.” Why is depth on two entities a *stronger* signal to an interviewer than breadth across ten? What does adding a wishlist actually demonstrate, and what does hardening the oversell path demonstrate instead?

Think it through

You chose optimistic locking for stock. Under what traffic pattern would that choice become the wrong one, and what would you switch to? How would you *know* from your metrics that you had crossed that line, rather than guessing?

Think it through

The confirmation email goes through a queue so it is asynchronous. What new failure mode did you just introduce (that a synchronous send did not have), and what does ShopFlow do if the email worker is down for an hour? Is a lost email acceptable? Defend your answer.

7

Debugging Practice

The three incidents below are the ones every on-call backend engineer meets. They are also the ones interviewers ask as “walk me through your process.” Practise the process, not the guess.

Debugging Lab • The app returns 500s intermittently

Symptom. Roughly one request in twenty fails with a 500; the rest are fine. It started around the last deploy but is not every request.

Evidence.

grep by correlation id

```
# error rate ticked up right after 14:02 deploy
grep 'level=ERROR' app.log | grep 'cid=7f3a'
# -> OptimisticLockException on Product.stock, order path
```

Work the method:

- ✓ **Observe.** Open the dashboard: is the error-rate spike correlated with the deploy or with a traffic change? Note it began at 14:02.
- ✓ **Hypothesise.** Intermittent, not total, points at contention or a bad code path on a fraction of requests — here, lost optimistic-lock races on a hot product left unhandled.
- ✓ **Verify.** Filter logs by `cid` (correlation id) for the failing requests; the stack shows `OptimisticLockException` bubbling to the default 500 handler.
- ✓ **Fix.** Catch it in the service: retry once, or return 409 `Conflict` with a clear message — a lost race is a client outcome, not a server fault.
- ✓ **Prevent.** Add a concurrency test that fires two decrements at the last unit and asserts one wins with 201 and the other gets 409, never a 500.

🔧 Debugging Lab • Connection pool exhausted

Symptom. Under load the app hangs, then throws `Unable to acquire JDBC Connection after a timeout`; requests queue and latency explodes.

Evidence.**HikariCP log**

```
Connection is not available, request timed out after 30000ms
HikariPool-1 - Pool stats (total=10, active=10, idle=0, waiting=27)
```

Work the method:

- ✓ **Observe.** `active=10, waiting=27` — every connection is checked out and a queue is forming. The pool is not leaking idle; it is fully busy.
- ✓ **Hypothesise.** Connections are held too long. The prime suspect in ShopFlow: slow I/O inside a transaction (an email send or external call that should have been async), or a slow un-indexed query.
- ✓ **Verify.** Check whether the email/external call sits inside `@Transactional`; run `EXPLAIN ANALYZE` on the order-path queries for sequential scans.
- ✓ **Fix.** Move the slow I/O out of the transaction (publish to the queue), add the missing index, keep transactions short. Only then consider raising the pool size.
- ✓ **Prevent.** Alert on `hikaricp_connections_pending`; assert in a test that the email publish happens after commit, not inside the transaction.

 Debugging Lab • A bad deploy that must be rolled back

Symptom. The 14:02 deploy raised the error rate and the fix is not obvious. You need the site healthy *now*, not a root cause in an hour.

- ✓ **Observe.** Error rate and latency both stepped up exactly at the deploy; the previous version was healthy for days. This is a deploy correlation, not a slow-burning bug.
- ✓ **Hypothesise.** The new image is bad. The safest path back to health is the last known-good *artifact*, not a forward fix under pressure.
- ✓ **Verify.** Confirm the previous image tag (e.g. `shopflow:1.8.3`) is still in the registry and was the last green CI build.
- ✓ **Fix.** Redeploy the last known-good tag. Never rebuild from source under pressure — that is a new, untested artifact when you least want one.
- ✓ **Prevent.** This is exactly why CI/CD promotes immutable, tagged images: rollback is “point at the old tag,” a ninety-second operation, not a rebuild.

8 Mini-Labs

 Mini-Lab • Beginner

30 min

Goal. Make the API's error contract consistent and self-documenting.

Context. A reviewer hits a validation error, a 404, and a 409 and expects the same JSON shape each time.

Requirements. One `@RestControllerAdvice` rendering `timestamp`, `status`, `error`, `message`, `path`; wire OpenAPI so the docs list every endpoint and status.

Hints. Map `MethodArgumentNotValidException` → 400, `EntityNotFoundException` → 404, `IllegalStateException` → 409.

Expected outcome. Every error path returns the same envelope; `/swagger-ui` reflects reality.

Common pitfalls. Letting Spring's default whitelabel error leak on one path, so the contract is “consistent except when it isn't.”

 Mini-Lab • Intermediate

50 min

Goal. Prove, with a test, that ShopFlow cannot oversell.

Context. One product, `stock=1`, two customers placing an order at the same instant.

Requirements. A Testcontainers test firing two concurrent `placeOrder` calls; assert exactly one 201 and one 409, and that final `stock` is 0 (never `-1`).

Hints. Use an `ExecutorService` + `CountDownLatch` to release both threads together; assert on the DB row afterward.

Expected outcome. A deterministic, repeatable test that would fail if someone removed `@Version`.

Common pitfalls. Testing against H2 instead of real Postgres, so locking semantics differ from production; both threads landing sequentially so no real race occurs.

Mini-Lab • Production-style

90 min

Goal. Make ShopFlow observable and resilient enough to debug the Part 7 incidents.

Context. You want the “500s intermittently” and “pool exhausted” incidents to be visible on a dashboard before a user reports them.

Requirements. Add a correlation-id filter (MDC) so every log line carries a `cid`; expose a `Timer` on order placement and a `Counter` on 409s; build a Grafana panel for p95 latency, error rate, and `hikaricp_connections_pending`; put a circuit breaker on the email publish.

Hints. Micrometer `@Timed + Actuator /prometheus`; a fallback that logs and drops the email when the breaker is open.

Expected outcome. A dashboard on which each Part 7 incident would show up as a visible spike.

Common pitfalls. Instrumenting only the happy path, so the 409s and breaker-opens — the interesting events — are invisible.

9 Engineering Notes

Judgment that turns a working project into a hireable one.

💡 How senior engineers think

Know your project's weak points before the interviewer finds them. “I'd add idempotency keys to order placement next, because a client retry can currently double-order” earns more trust than a claim that the project is perfect. Reviewers hire the candidate who can critique their own code, because that is the candidate who will critique the team's.

👤 Interview insight

Be able to whiteboard two things from memory: the `Order/Product/OrderItem` relationships (and where `@Version` lives), and the JWT flow — register (BCrypt) → login issues a signed token → filter verifies the signature on every request → role drives authorization. If you can draw both without notes, you can carry the whole conversation.

⚡ Performance note

Every cache decision is a staleness trade. A 60-second catalog TTL means a price change can be one minute stale — fine for a catalog, unacceptable for stock. That is why ShopFlow caches catalog *reads* but never the live stock count, and invalidates the cache on every write. Be ready to defend the exact TTL you chose and what a stale read would cost.

🔧 Production tip

Have the test suite runnable and green in a fresh clone. An interviewer who clones your repo and it does not build has learned more about you than any code-style nit could tell them. “`git clone`, one command, green” is the single highest-leverage hour you can spend before a technical screen.

10 Build-Along Project — ShopFlow

This is the last increment: no new mechanics, only assembly, proof, and the document that ties it together. Everything from Days 1–13 already exists in your tree — the layered module (Day 1), the schema and locking (Days 2–3), CRUD and errors (Day 4), auth (Day 5), tests (Day 6), clean layering (Day 7), containers (Day 8), cache/queue/async (Days 9–11), CI (Day 12), resilience and observability (Day 13). Today you make them one system and write the README that explains *why*.

🔗 Build-Along • Day 14 — final assembly & the README that explains decisions

- Wire the cross-cutting concerns end to end: catalog cache invalidated on write (Day 9), async confirmation email via RabbitMQ (Days 10–11), circuit breaker + rate limiter (Day 13), Actuator metrics on a Grafana dashboard (Day 11, 13).
- Confirm `docker compose up` brings the full stack — app + Postgres + Redis + RabbitMQ — to health with one command from a clean checkout.
- Make CI green on every push: it runs the unit, Testcontainers and MockMvc suites, then builds and tags the image.
- Verify each completion-checklist item: all 14 days represented, tests green in CI, OpenAPI accurate, one-command stack.
- Write the README as a *decision log*, not a run book: why layered, why optimistic locking, why this TTL, why keyset vs offset pagination, the state machine, the JWT flow, and one honest “what I’d improve next.”
- Rehearse the whiteboard: the entity relationships and the JWT flow, from memory, in under five minutes.

Definition of done: a stranger clones the repo, runs one command, watches the whole stack come up healthy and the tests pass in CI, opens the README and understands every architectural decision — and you can defend each of them out loud.

11 Chapter Summary

Key takeaways

- ✓ Production-ready is a defensible checklist: correct, observable, recoverable, tested, documented.
- ✓ The non-functional requirements — not the features — are where the engineering and the interview signal live.
- ✓ Optimistic locking *detects* the oversell race; your service must *handle* it (retry or 409).
- ✓ Model status as an explicit state machine so illegal transitions are impossible, not merely discouraged.
- ✓ Roll back by redeploying the last known-good *tagged image*, never by rebuilding under pressure.

Things to remember

- Recompute money server-side; do slow I/O outside the transaction via the queue.
- Correlation ids + metrics are what turn “it’s slow sometimes” into a diagnosable incident.
- Depth on two entities beats breadth across ten — resist scope creep.

Common pitfalls (last look)

- Trusting client-supplied totals. • Free-text `status` with no transition guard.
- Synchronous email inside `placeOrder` → pool exhaustion under load.
- A repo that only builds on your laptop → the worst possible interview signal.

📄 One-Page Cheat Sheet

Race condition (oversell) — two reads see the last unit; fixed by `@Version` optimistic lock or an atomic DB decrement with a check.

500s intermittently — dashboard for the spike → logs by correlation id → downstream health → resource saturation.

Pool exhausted — long transactions, slow downstream query, leaked connections, or an undersized pool — fix the hold time first.

Safe rollback — redeploy the last known-good tagged image; never rebuild from source under pressure.

JWT integrity — token is signed (HMAC/RSA); the server verifies the signature on every request and rejects tampered payloads.

Scale read-heavy — cache (Redis) → read replicas → CDN for public data, before scaling the DB vertically.

Fix N+1 — `JOIN FETCH`, `@EntityGraph`, or a DTO projection — never blanket EAGER fetching.

👤 Interview questions to master

- › Give a backend race condition and explain how optimistic locking fixes it.
- › The app returns 500s intermittently — what is your process?
- › Connection pool exhausted — what are the likely causes and your first fix?
- › How do you safely roll back a bad deploy, and why not rebuild?
- › Walk me through your JWT auth flow and where the token is verified.
- › Why a layered architecture, and what makes a controller “thin”?
- › Justify your cache TTL and how you keep it from serving stale stock.

Suggested further reading

- *Release It!* (Nygard) — stability patterns: circuit breakers, bulkheads, timeouts in the wild.
- *Designing Data-Intensive Applications* (Kleppmann) — the “why” behind locking, replication and consistency.
- Spring Boot Reference — *Production-ready Features* (Actuator) and *Testing* chapters.
- Your own README — reread it as a stranger; every unexplained decision is an interview question you have not yet answered.

🔑 Key idea

What comes after Day 14. You now have one small system you understand completely — top to bottom, request to rollback. That is worth more than ten half-built ones. Next: pick a single NFR and take it further than the spec asked — add idempotency keys, introduce read replicas, or trace a request across services with OpenTelemetry. The habit that carries you from junior to senior is the one this whole notebook rehearsed: understand the *mechanism*, never just the annotation, and always be able to say

why.

APPENDIX A

Backend Engineer Interview Cheat Sheet

A

The judgment behind the terms — review this before every interview

Junior and mid-level interviews test *judgment* as much as syntax recall: the ability to explain *why* a decision was made, not just recite the term. Every answer below states the mechanism, not only the label. Read a question, answer it out loud from memory, then check yourself.

Spring

› BeanFactory vs ApplicationContext?

`ApplicationContext` extends `BeanFactory` with event publishing, AOP integration, i18n, and eager singleton instantiation by default.

› What are the Spring bean scopes?

`singleton` (default, one per context), `prototype` (new instance per request), and `request` / `session` / `application` — the last three only in web contexts.

› Constructor vs field injection?

Constructor injection is preferred: explicit required dependencies, immutability via `final` fields, and testability without reflection.

› What triggers auto-configuration?

Conditional annotations (`@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`) evaluated against the classpath and existing beans.

SQL & Databases

› Clustered vs non-clustered index?

A clustered index determines the physical row order on disk (one per table, usually the primary key); non-clustered indexes are separate structures pointing back to the row.

› What is a covering index?

An index that contains every column a query needs, letting the database answer entirely from the index without touching the table (an index-only scan).

› How do you find a slow query in production?

Enable slow-query logging, run `EXPLAIN ANALYZE`, check for sequential scans, missing indexes, or stale statistics; compare estimated vs actual row counts.

Hibernate / JPA

› What is dirty checking?

Hibernate compares a managed entity's current state to its loaded snapshot at flush time and generates `UPDATE` statements automatically for changed fields.

› How do you fix N+1?

`JOIN FETCH`, `@EntityGraph`, or a DTO projection query — never blanket `EAGER` fetching.

› What does @Version do?

Enables optimistic locking: Hibernate includes the version in the `UPDATE`'s `WHERE` clause and throws if no row matched (someone else updated first).

REST API

› PUT vs PATCH?

`PUT` replaces the full resource and is idempotent; `PATCH` applies a partial update and may or may not be idempotent depending on the patch semantics.

› How do you version an API safely?

URI versioning (`/v1`, `/v2`) for breaking changes only; additive changes (new optional fields) do not require a new version.

› 401 vs 403?

401 Unauthorized — no or invalid credentials, we do not know who you are. 403 Forbidden — known identity, insufficient permission.

Security

› How does JWT prevent tampering?

The token is cryptographically signed (HMAC or RSA); the server verifies the signature on every request, rejecting any token whose payload was altered.

› CORS vs CSRF?

CORS controls which browser origins may call your API via JavaScript. CSRF exploits ambient credentials (cookies) sent automatically by the browser — largely irrelevant for header-based JWT auth.

System Design Basics

› How would you scale a read-heavy API?

Add a cache (Redis) in front of the database, add read replicas, and consider CDN caching for static/public data before scaling the database vertically.

› How would you design a rate limiter?

Token bucket or sliding-window counter stored in Redis, keyed by client ID/IP, checked before the request is processed; return 429 when exceeded.

› Vertical vs horizontal scaling?

Vertical: bigger machine — simple, but has a ceiling and a single point of failure. Horizontal: more machines behind a load balancer — requires statelessness (hence JWT over server sessions).

Concurrency

› Race condition — give a backend example.

Two requests read the same product stock (both see 1 available) and both decrement, resulting in `-1` stock. Fixed with optimistic locking or a DB-level atomic decrement with a check constraint.

› Deadlock — how does it happen?

Two transactions each hold a lock the other needs, both waiting indefinitely. Prevented by consistent lock ordering and keeping transactions short.

Docker

› Image vs container?

An image is a read-only template (layers + metadata); a container is a running (or stopped) instance of that image with its own writable layer.

› Why multi-stage builds?

Keep build-time tools (JDK, Maven cache) out of the final runtime image — a smaller, more secure image shipped to production.

Git

› Merge vs rebase?

Merge preserves full history with a merge commit; rebase rewrites commits onto a new base for a linear history — never rebase shared/public branches.

› How do you undo a pushed commit safely?

`git revert` creates a new commit undoing the change, safe for shared history; `git reset` rewrites history and should stay local/unpushed.

Production Troubleshooting

› The app is returning 500s intermittently — what is your process?

Check dashboards for error-rate / latency spikes correlated with a deploy or traffic change; check logs by correlation ID for the failing requests; check downstream dependency health (DB, cache, external APIs); check resource saturation (CPU, memory, connection pool).

› Connection pool exhausted — likely causes?

Long-running transactions holding connections, a slow downstream query, a leak from not closing resources, or a pool simply undersized for current traffic.

› How do you safely roll back a bad deploy?

Redeploy the last known-good image tag (never rebuild from source under pressure) — exactly why CI/CD promotes immutable, tagged artifacts.

💡 How senior engineers think

The candidates who get offers are not the ones who recite the most terms — they are the ones who can say “I chose X because Y, and its weakness is Z, which I would address by W.” Trade-off fluency reads as seniority. Practise answering every question above with a *because*.

APPENDIX B

The ShopFlow Build-Along Master Tracker B

One backend, fourteen increments — tick each as you finish it

Use this as your single source of truth for the capstone. Each day contributes one increment; by Day 14 every box should be ticked and `docker compose up` should bring the whole system to life.

Phase 1 — Foundations & Data (Days 1–3)

- ☐ **Day 1:** Base module — layered packages, constructor injection, typed `@ConfigurationProperties`, dev/prod profiles, bean-timing `BeanPostProcessor`, `GET /api/v1/ping`.
- ☐ **Day 2:** Schema for `users`, `roles`, `products`, `orders`, `order_items` with PK/FK/UNIQUE/CHECK constraints, FK indexes, a composite index for “list orders by user + status”, and a repeatable migration.
- ☐ **Day 3:** JPA entities for the Order/OrderItem/Product graph, LAZY defaults, owned-child cascade, `@Version` on `Product.stock`, and a one-query `@EntityGraph` + DTO projection listing.

Phase 2 — API, Security & Quality (Days 4–6)

- ☐ **Day 4:** Order API (`POST/GET/GET {id}`) with DTOs, validation, centralized error shape, pagination envelope, and OpenAPI docs.
- ☐ **Day 5:** JWT login + refresh, BCrypt storage, custom JWT filter, stateless `SecurityFilterChain`, role-based access (CUSTOMER vs ADMIN), locked-down CORS.
- ☐ **Day 6:** Mockito unit tests for every business rule, a Testcontainers repository test proving no N+1, and MockMvc tests covering success, validation, not-found, and auth failures.

Phase 3 — Architecture & Delivery (Days 7–8)

- ☐ **Day 7:** Refactor order placement — extract a validator, publish `OrderPlacedEvent`, move email into an async listener decoupled from the transaction; thin controllers, interface-typed services.
- ☐ **Day 8:** Multi-stage Dockerfile (non-root), `.dockerignore`, and a `docker-compose.yml` bringing up app + Postgres + Redis via env vars and named volumes.

Phase 4 — Scale, Resilience & Ship (Days 9–14)

- ☐ **Day 9:** Redis-backed `@Cacheable` product-catalog reads with a TTL, `@CacheEvict` on writes, and a cache-hit-ratio log line.
- ☐ **Day 10:** `OrderPlacedEvent` published to RabbitMQ, an idempotent consumer that emails, retries with backoff, and a Dead Letter Queue after 3 attempts.
- ☐ **Day 11:** A dedicated bounded thread pool for async work, Actuator health/metrics/prometheus endpoints, and a custom “orders per minute” Micrometer counter.
- ☐ **Day 12:** A GitHub Actions pipeline that tests every PR, then builds, tags by commit SHA, and pushes the image on merge to main — secrets from the CI store.
- ☐ **Day 13:** Resilience4j circuit breaker + retry on the payment call, separate liveness/readiness probes, a public-endpoint rate limiter, and a Grafana dashboard.

☐ **Day 14:** Full assembly against the completion checklist below.

 Build-Along • Final completion checklist

- ☐ All 14 days' concepts are represented somewhere in this codebase.
- ☐ `docker compose up` brings up the full stack (app + db + redis + mq) with one command.
- ☐ Tests are green and run in CI on every push.
- ☐ OpenAPI docs are accurate and complete.
- ☐ The README explains architecture *decisions*, not just how to run it.

APPENDIX C

Glossary of Key Terms

The vocabulary of a backend engineer, in plain language

C

IoC / DI — Inversion of Control: a container, not your code, creates and wires objects. Dependency Injection is how it hands collaborators in.

BeanPostProcessor — A hook that runs around every bean's initialization — where proxies are attached.

Proxy (CGLIB/JDK) — A wrapper around your bean that adds behaviour (transactions, caching, async) for external calls.

Auto-configuration — Conditional beans created based on what is on the classpath, unless you defined your own.

DispatcherServlet — Spring MVC's front controller that routes every request to a handler method.

Filter vs Interceptor — Filters run outside Spring on the raw servlet request; interceptors run inside dispatch, around the handler.

Persistence Context — Hibernate's per-transaction first-level cache that tracks managed entities and dirty-checks them.

N+1 problem — One query for parents plus one per parent for children; fixed with `JOIN FETCH` or `@EntityGraph`.

Optimistic locking — Detect concurrent updates via a `@Version` column; cheap, needs retry logic.

Pessimistic locking — Lock the row at the database (`SELECT ... FOR UPDATE`); safe under contention, lower concurrency.

DTO — Data Transfer Object — the intentional shape of your API, decoupled from JPA entities.

Idempotency — The same request applied twice has the same effect as once — essential for retries and `PUT/DELETE`.

BCrypt — A salted, adaptive password hash; deliberately slow to resist brute force.

JWT — A signed, self-contained token proving identity without a server-side session lookup.

CORS — A browser rule for which origins may call your API from JavaScript.

CSRF — An attack abusing ambient cookies; largely irrelevant for header-based JWT auth.

RBAC — Role-Based Access Control — coarse roles bundling fine-grained permissions.

Test pyramid — Many fast unit tests, fewer integration tests, a handful of E2E tests.

Testcontainers — Ephemeral real databases in Docker for integration tests — no H2 dialect surprises.

MockMvc — Tests the web layer (status, JSON, validation) without starting a server.

SOLID — Five design principles for maintainable, changeable object-oriented code.

Hexagonal — Ports-and-adapters architecture isolating domain logic from frameworks.

Multi-stage build — A Dockerfile that builds in one stage and ships only the lean runtime in the next.

Cache-aside — The application checks the cache, falls back to the DB on a miss, and populates the cache.

TTL — Time-to-live: how long a cache entry stays valid before expiring.

DLQ — Dead Letter Queue — where messages go after exhausting retries, so failures are visible.

At-least-once — A delivery guarantee where messages may arrive more than once; consumers must be idempotent.

Circuit breaker — Stops calling a failing dependency (fails fast) to let it recover and avoid thread exhaustion.

Rate limiter — Caps requests per client per window; returns 429 when exceeded.

Liveness vs readiness — Liveness: is the process alive (else restart)? Readiness: can it serve traffic right now?

Prometheus / Micrometer — Micrometer is the metrics facade; Prometheus scrapes and stores the time series.

G1 GC — The default JVM garbage collector since JDK 9, tuned for low pause times.

Thread pool — A bounded set of reusable worker threads; size by CPU- vs I/O-bound workload.

CompletableFuture — Composes asynchronous work; always attach an `exceptionally` handler.

Keyspace pagination — Cursor-based paging (`WHERE id > :last`) that stays fast at any depth.

Isolation levels — `READ UNCOMMITTED` → `SERIALIZABLE`: more correctness, less concurrency.

You reached the end of the workbook. If every checklist in Appendix B is ticked and every term above is one you can explain out loud — you are ready. Go build.